L'editor di rsedit

Architettura della libreria `rsedit_core` e dei suoi due front-end — manuale per lo sviluppatore

Indice

1 Introduzione 4
1.1 Come leggere questo manuale 4
1.2 Comportamento e difetti 4
1.3 Convenzioni 4
2 Panoramica 6
2.1 Gli strati 6
2.2 Dove sta cosa 6
2.3 La superficie pubblica 8
2.4 I thread 8
3 Il ciclo di vita del programma 10
3.1 Dal `main` al ciclo degli eventi 10
3.2 Il ciclo degli eventi della TUI 15
4 Lo stato: `EditorState` e i compartimenti 19
4.1 La facciata 19
4.2 Le regole dei compartimenti 20
4.3 I compartimenti 21
4.4 Le coppie con `editor/` 21
4.5 L'ordine dei lock 22
4.6 Accesso ai buffer 22
4.7 L'editor come contesto dell'interprete 22
4.8 Le decisioni 23
5 Dal tasto al comando 28
5.1 Il vocabolario dei tasti 28
5.2 Le keymap 28
5.3 `handle_key_event` 30
5.4 L'argomento prefisso 31
5.5 La risoluzione: `resolve_key_sequence` 32
5.6 `run_command_form`: l'esecuzione di un comando 33
5.7 La cattura di una sequenza 34
5.8 Le decisioni 35
6 Comandi e argomenti 36
6.1 Che cos'è un comando 36
6.2 Le specifiche degli argomenti 36
6.3 `call-interactively` e la catena degli argomenti 36
6.4 Le decisioni 37
7 Minibuffer e ricerca incrementale 39
7.1 Il minibuffer 39
7.2 La ricerca incrementale 39
7.3 Le decisioni 40

8 I buffer 42
8.1 La struttura `Buffer` 42
8.2 `BufferTrait` e `GapBuffer` 42
8.3 Le due porte 43
8.4 Undo 45
8.5 Mark, regione, overlay e testo virtuale 46
8.6 Le cache del testo 47
8.7 File su disco 47
8.8 Le decisioni 49
9 Il testo: ricerca, sostituzione, kill ring, rettangoli, liste di risultati e riempimento 52
9.1 Posizioni: sempre caratteri 52
9.2 `Pattern`: che cosa cercare 53
9.3 La sessione di ricerca incrementale: `Isearch` 55
9.4 La sessione di sostituzione: `Replace` 56
9.5 Il kill ring 58
9.6 I rettangoli 59
9.7 Le liste di risultati 60
9.8 Il riempimento 61
9.9 Le decisioni 62
10 Modi 64
10.1 Il modo maggiore 64
10.2 I modi minori 64
10.3 Due lessici sullo stesso testo 66
10.4 Lo scanner delle espressioni 67
10.5 Facce e temi 70
10.6 Le decisioni 70
11 Il lavoro in background 73
11.1 Il WorkManager 73
11.2 I lavori 75
11.3 Il nucleo 76
11.4 Gli argomenti 77
11.5 L'esecutore 78
11.6 Il registro 79
11.7 I thread dedicati 79
11.8 Il watchdog 80
11.9 Misure, segnalazioni e opzioni 80
11.10 Gli utenti del WorkManager 82
11.11 Come si prova 85
11.12 Le decisioni 86
12 Lo schermo 89
12.1 Finestre 89
12.2 `snapshot`: dalla memoria alla fotografia 89
12.3 La composizione di una finestra 90
12.4 Le evidenziazioni del momento 91
12.5 Le righe e le loro colonne 93
12.6 Il margine dei numeri 94
12.7 La riga di stato 94
12.8 Il renderer della TUI 94
12.9 La finestra grafica 95
12.10 Il mouse 95

12.11 Le decisioni ... 96
13 Macro di tastiera ... 99
13.1 Le decisioni ... 99
14 Le primitive dell'editor ... 100
14.1 Il registro ... 100
14.2 Le primitive di modifica ... 101
14.3 File, buffer, finestre e domande ... 106
14.4 Comandi, ricerche e modi ... 108
14.5 Le decisioni ... 110
15 I moduli Lisp ... 117
15.1 Caricare una volta: `require` e `provide` ... 118
15.2 Dove si cerca ... 118
15.3 I plugin ... 118
15.4 Come è scritto un modulo ... 119
15.5 I moduli distribuiti ... 119
15.6 Le decisioni ... 128
16 I test ... 131
16.1 I test della documentazione ... 131
17 Ricette ... 132
17.1 Una primitiva o un comando in Rust ... 132
17.2 Un comando in Lisp ... 132
17.3 Un modo maggiore ... 132
17.4 Un lavoro di sfondo ... 132
17.5 Un modulo Lisp nuovo ... 133
17.6 Un nuovo front-end ... 133
18 Appendice ... 134
18.1 Glossario ... 134
18.2 Mantenere aggiornato questo manuale ... 134

1 Introduzione

Questo manuale descrive la libreria `rsedit_core` (`core/src/`), i moduli Lisp che ne fanno l'editor (`core/lisp/`) e i due front-end che la usano (`src/`): il terminale e una finestra grafica. Il linguaggio è descritto a parte, nel manuale dell'interprete del repository `risp`: qui l'interprete è dato per noto, e si spiega come l'editor lo ospita, come trasforma un tasto in un comando, come conserva il testo, come lavora in background e come descrive lo schermo a chi lo disegna. Le regole con cui è scritta tutta la documentazione di rsedit, questo manuale compreso, sono in `documentazione.typ`.

La libreria è un editor *senza testa*. Non apre un terminale, non legge la tastiera e non disegna: riceve eventi già tradotti nel proprio vocabolario (`KeyEvent`, `MouseEvent`, testo incollato, dimensioni) e restituisce, quando le viene chiesto, una fotografia immutabile di ciò che andrebbe mostrato (`FrameSnapshot`). Tutto ciò che sta in mezzo — buffer, finestre, modi, comandi, lavori di sfondo — è in `rsedit_core`, e quasi tutto il comportamento visibile all'utente è scritto in Lisp nei moduli di `core/lisp/`.

> Due confini reggono l'architettura. L'interprete è un crate a sé, `risp`, e non può nominare alcun tipo dell'editor: questo permette di provarlo e riusarlo da solo. E `ui/` non nomina nulla di `editor/`, come verificano i test (`tests::layering_tests`): questo permette di scrivere un renderer contro una fotografia invece che contro gli interni della libreria.

1.1 Come leggere questo manuale

Il Sezione 2 e il Sezione 3 danno il quadro: gli strati, i thread, che cosa succede dall'avvio al ciclo degli eventi. Poi il manuale segue il percorso di un tasto: lo stato che attraversa (Sezione 4), la risoluzione e l'esecuzione del comando (Sezione 5, Sezione 6, Sezione 7), il testo che il comando modifica (Sezione 8, Sezione 9), i modi che lo interpretano (Sezione 10), il lavoro che continua in background (Sezione 11) e la fotografia che il front-end disegna (Sezione 12). Gli ultimi capitoli sono di riferimento: le macro (Sezione 13), le primitive dell'editor (Sezione 14), i moduli Lisp (Sezione 15), i test (Sezione 16) e le ricette per i cambiamenti più comuni (Sezione 17). Ogni capitolo finisce con le sue decisioni: il problema, la scelta, che cosa comporta e l'alternativa scartata.

Gli estratti di codice sono presi dal sorgente e semplificati: commenti tolti, parametri generici abbreviati, rami ripetitivi riassunti. Ogni riga di un estratto corrisponde a una riga del file indicato sopra di esso; il percorso è relativo a `core/src/`. I file sorgente rimandano al manuale con `Design: doc/architettura-editor.typ, <etichetta>`., e le etichette dei capitoli e delle sezioni non cambiano, perché il codice le nomina.

Ogni affermazione è stata letta nel codice; i comportamenti che dipendono dall'esecuzione (avvio, undo, comandi di shell, colorazione) sono stati osservati con piccoli programmi che usano la libreria come farebbe un front-end.

1.2 Comportamento e difetti

Questo manuale descrive il comportamento *attuale* del codice, anche dove quel comportamento è un difetto. I difetti sono descritti, con la riproduzione e una correzione proposta, nel *Rapporto sui problemi del codice* (`rapporto-problemi.typ`). Dove il manuale descrive un punto che il rapporto considera un difetto, un rimando scritto `#rapporto("ID")` indica la scheda corrispondente; quando il difetto viene corretto si aggiorna la descrizione qui, si toglie il rimando, e la scheda passa tra le risolte.

1.3 Convenzioni

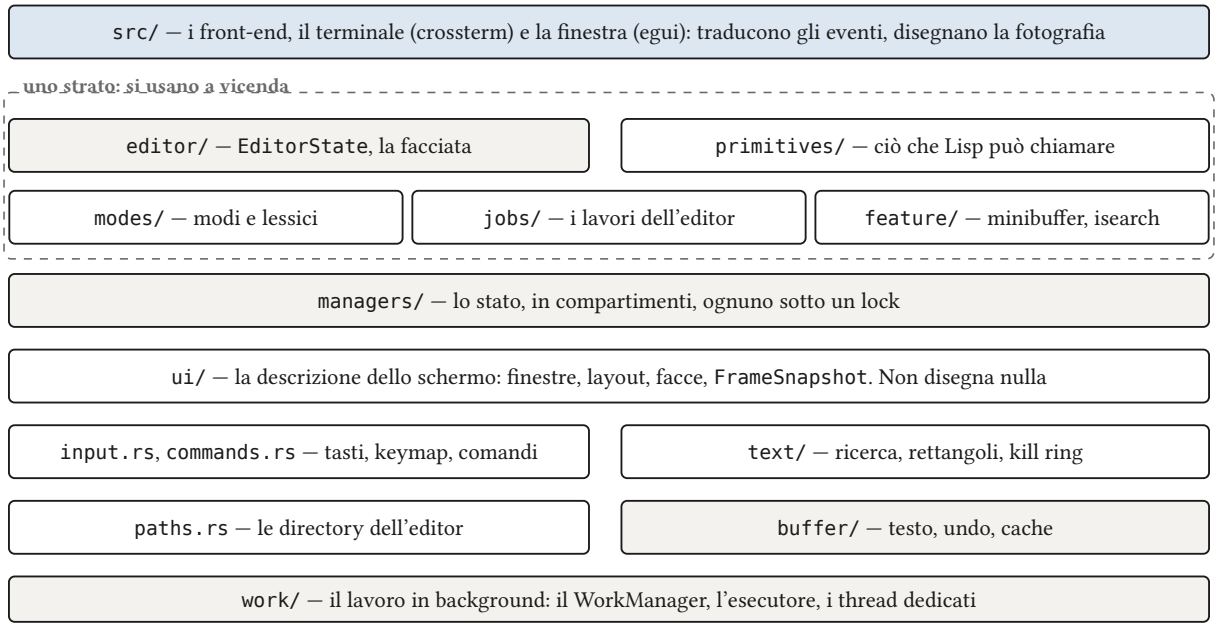
- I nomi in `monospazio` sono identificatori: tipi e funzioni Rust (`run_command_form`), primitive e variabili Lisp (`call-interactively`, `mouse-mode`), file (`editor/keys.rs`, relativo a `core/src/` salvo indicazione diversa).

- Nei diagrammi le caselle grigie sono dati o stato, quelle bianche funzioni o passi, quelle azzurre appartengono a un altro thread o al front-end, quelle rosate sono errori. Nei diagrammi di sequenza le frecce continue sono chiamate e quelle tratteggiate risposte.
- «Primitiva» è una funzione Rust registrata nell'ambiente Lisp; «comando» è una funzione (Rust o Lisp) registrata anche nel registro dei comandi, e quindi raggiungibile con `M-x` e capace di farsi chiedere gli argomenti.

2 Panoramica

2.1 Gli strati

`lib.rs` elenca i moduli in ordine di dipendenza: ognuno usa quelli sopra di sé e, di quelli sotto, solo ciò che la sua voce dichiara. La figura li mostra dal basso verso l'alto, cioè nella direzione in cui si costruisce.



Fino a `managers/` le frecce sono omesse perché la regola è una sola: ogni livello può usare tutti quelli disegnati sotto di lui. `work/` non conosce l'editor: un buffer emette un suo argomento senza sapere chi lo ascolta (Sezione 11.4). I cinque moduli in alto sono uno strato solo, non un ordine: si usano a vicenda — `editor/` installa le primitive e avvia i lavori, che agiscono sull'editor — e la mappa li elenca per ciò che sono. L'interprete non è nella figura: è il crate `risp`, che non conosce l'editor; le sue espressioni compaiono da `input.rs` in su.

La regola ha quattro eccezioni, dichiarate. `buffer/` conserva alcuni tipi che vengono da più in alto, le facce di `ui/` (negli overlay e nel testo virtuale) e gli stati della colorazione e della scansione di `modes/` (nelle due cache del testo, Sezione 8.6). `input.rs` lega i tasti a espressioni Lisp, `ELispExp`, il cui tipo nomina `EditorState` di `editor/` (Sezione 5.2). `ui/` compone le finestre sulla tabella dei buffer, `Buffers` di `managers/`, e usa `ELispExp` per dare a Lisp l'id di una finestra e per leggere lo stato che un modo mette nella riga di stato. E `managers/` conserva i modi maggiori e le tabelle sintattiche di `modes/`, e le espressioni Lisp che i comandi, i legami e gli hook registrano (Sezione 4.3); le finestre, le facce e il tema che conserva vengono da `ui/`, che gli sta sotto. Per ogni modulo da `work/` a `managers/`, `layering_tests` tiene l'elenco di ciò che usa da più in alto: vuoto, tranne per questi quattro. Un uso nuovo fa fallire il test finché non lo si aggiunge lì e nella mappa di `lib.rs`.

2.2 Dove sta cosa

Percorso	Contenuto
`lib.rs`	Ordine dei moduli e superficie pubblica (`pub use`). Tutto il resto è `pub(crate)`.
`paths.rs`	Le directory dell'editor, configurazione, dati, cache e stato, secondo le convenzioni di ogni sistema: `app_dir` (Sezione 3.1.2); quella dei dati installati con l'editor, `installed_data_dir`; e i nomi dei file, resi assoluti e smontati senza chiedere al disco (Sezione 14.3.1).

Percorso	Contenuto
`buffer/`	`Buffer`, `BufferTrait`, `GapBuffer`, undo, mark, overlay, testo virtuale, cache di colorazione (`syntax.rs`) e di scansione (`scan.rs`), timbro del file su disco (`disk.rs`), codifica del file (`coding.rs`, Sezione 8.7.1).
`text/`	Ciò che si può calcolare sul testo conoscendo solo `BufferTrait`: la ricerca (`search.rs`), le sessioni di ricerca incrementale e di sostituzione (`isearch.rs`, `replace.rs`), il kill ring, i rettangoli, le liste di posizioni (`results.rs`), il riempimento (`fill.rs`).
`input.rs`	`KeyEvent`, `KeyCode`, `MouseEvent`, `Keymap`, `TransientKeymap`, la lettura e la scrittura dei nomi dei tasti (`parse_key_sequence`, `describe_keys`), i legami scritti in Rust (`fill_default_keymaps`).
`commands.rs`	`ArgSpec`, `PrefixArg`, `Invocation`, `PendingCommand`, `CommandRegistry`: il vocabolario dei comandi.
`managers/`	I compartimenti dello stato: `Buffers`, `Windows`, `Modes`, `Commands`, `KillYank`, `Macros`, `History`, `Visits`, `Log`, `Runtime`, `Loader`.
`editor/`	`EditorState` e le porte dei compartimenti (`mod.rs`); la metà della facciata di ogni compartimento, nel file con il suo nome (`buffers.rs`, `windows.rs`, `modes.rs`, `commands.rs`, `kill_yank.rs`, `macros.rs`, `log.rs`, `runtime.rs`, `loader.rs`, `visits.rs`); ciò che coordina più cose, nei file che portano il nome di ciò che coordinano (`keys.rs`, `mouse.rs`, `frame.rs`, `boot.rs`, `settings.rs`, `replace.rs`, `results.rs`, `background.rs`, `session.rs`, `work.rs`). Vedi Sezione 4.4.
`primitives/`	Le 327 primitive dell'editor, in 43 moduli, e `install_primitives`.
`modes/`	`MajorMode` (`mod.rs`), la grammatica di colorazione (`grammar.rs`), la tabella sintattica (`syntax_table.rs`) e lo scanner delle espressioni, che la legge (`sexp.rs`) (Sezione 10).
`jobs/`	I lavori di sfondo dell'editor: `Highlighter`, `Prescanner`, `FileWatcher` e `AutoSaver` (Sezione 11.10.1).
`feature/`	Le due funzionalità che non possono dipendere da un file Lisp: il minibuffer e la ricerca incrementale (Sezione 7).
`work/`	Il WorkManager: il nucleo dello scheduler, l'esecutore, il registro, i thread dedicati, il watchdog, gli argomenti e le opzioni (Sezione 11).
`ui/`	La descrizione dello schermo: l'albero delle finestre e le flottanti (`windows.rs`), la composizione di una finestra (`compose.rs`), le righe e le loro colonne (`layout.rs`), il margine dei numeri (`gutter.rs`), la riga di stato (`mode_line.rs`), facce e temi (`faces.rs`), `FrameSnapshot` (`frame.rs`) (Sezione 12).
`tests/`	Circa cento file di test dell'editor, più `perf/` (Sezione 16).
`../lisp/`	I venticinque moduli Lisp distribuiti con l'editor, in tre cartelle: `base/`, `features/`, `languages/` (Sezione 15).
`../build.rs`	Copia ogni cartella dei moduli in `target/<profilo>/data/lisp`, accanto all'eseguibile, e con loro i temi e le pagine del manuale.
`../../src/`	I front-end: `main.rs` (la riga di comando e l'avvio); `tui/`, il terminale, con il ciclo in `mod.rs`, i tasti e il mouse in `input.rs`, i colori in `colour.rs` e il disegno in `render.rs` (Sezione 3.2, Sezione 12.8); `gui/`, una finestra grafica con

Percorso	Contenuto
	egui, con `Core` in `mod.rs`, il ciclo in `app.rs`, i tasti in `keys.rs` e il disegno in `paint.rs` (Sezione 12.9).

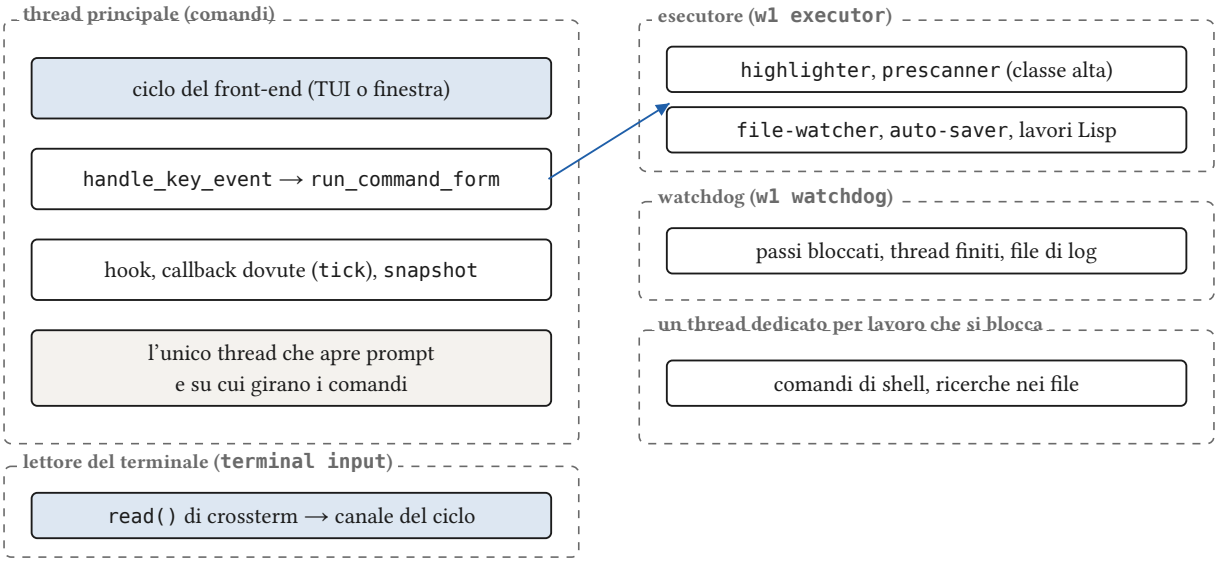
2.3 La superficie pubblica

Un front-end vede soltanto ciò che `lib.rs` riesporta: `EditorState`, `create_global_env`, `BufferTrait`, `GapBuffer`, i tipi di input (`KeyEvent`, `KeyCode`, `KeyModifiers`, `MouseEvent`, `MouseKind`, `MouseButton`), i tipi della fotografia (`FrameSnapshot`, `RenderableWindowView`, `Highlight`, `GutterCell`, `Separator`, `Rect`, `Face`, `Style`, `Color`, `Theme`, `NAMED_COLORS`), dall'interprete `Env`, `LispContext`, `Parser` ed `eval`, i compartimenti `Buffers` e `Windows`, le directory dell'editor (`app_dir`, `AppDir`, `DirOptions`, `DirError` e i nomi delle variabili `CONFIG_DIR`, `CACHE_DIR`, `XDG_CONFIG_HOME`, `XDG_CACHE_HOME`, Sezione 3.1.2), più `mouse_mode`, `MOUSE_MODE`, `isolate_config_for_tests` e l'alias `ELispExp<B>` (`LispExp<EditorState<B>>`). I metodi di `EditorState` che un front-end usa sono pochi:

Metodo	Uso
`create_global_env::<B>()`	Costruisce editor e ambiente Lisp, carica la configurazione.
`handle_key_event(ev, &env)`	Un tasto premuto.
`handle_paste(text, &env)`	Un incolla «bracketed», come un solo comando.
`handle_mouse_event(ev, &env)`	Un evento del mouse; risponde se ha cambiato qualcosa.
`resize(env, w, h)`	Nuove dimensioni del terminale o della finestra, in celle.
`snapshot(&env, w, h)`	La fotografia da disegnare.
`next_redraw_in(&env, &frame)` `drag_scroll_in()`	Quanto si può aspettare prima di ridisegnare anche senza eventi.
`tick(&env)`	Il tempo è scaduto: esegue ciò che era in attesa (callback di sfondo, scorrimento durante il trascinamento).
`on_background_change(sveglia)`	La funzione che il lavoro di sfondo chiama quando ha cambiato lo schermo: sveglia il ciclo del front-end (Sezione 11.10).
`is_running()`	Falso dopo `quit`.
`shutdown(entro)`	All'uscita, anche per un panic: ferma il lavoro di sfondo e scrive nel file le ultime righe del log.
`start_log_file`, `enable_log_file`, `log_diagnostic`, `set_echo_message`, `take_backtrace_suffix`	Diagnostica.

2.4 I thread

L'editor usa più thread, ma uno solo esegue i comandi: il principale, su cui gira il ciclo del front-end, quello del terminale o quello di eframe per la finestra. La figura mostra chi esiste. Gli altri thread dell'editor appartengono al WorkManager (Sezione 11), e portano il suo numero nel nome; la TUI ha in più un thread suo, che legge il terminale e manda gli eventi nel canale del ciclo (Sezione 3.2).



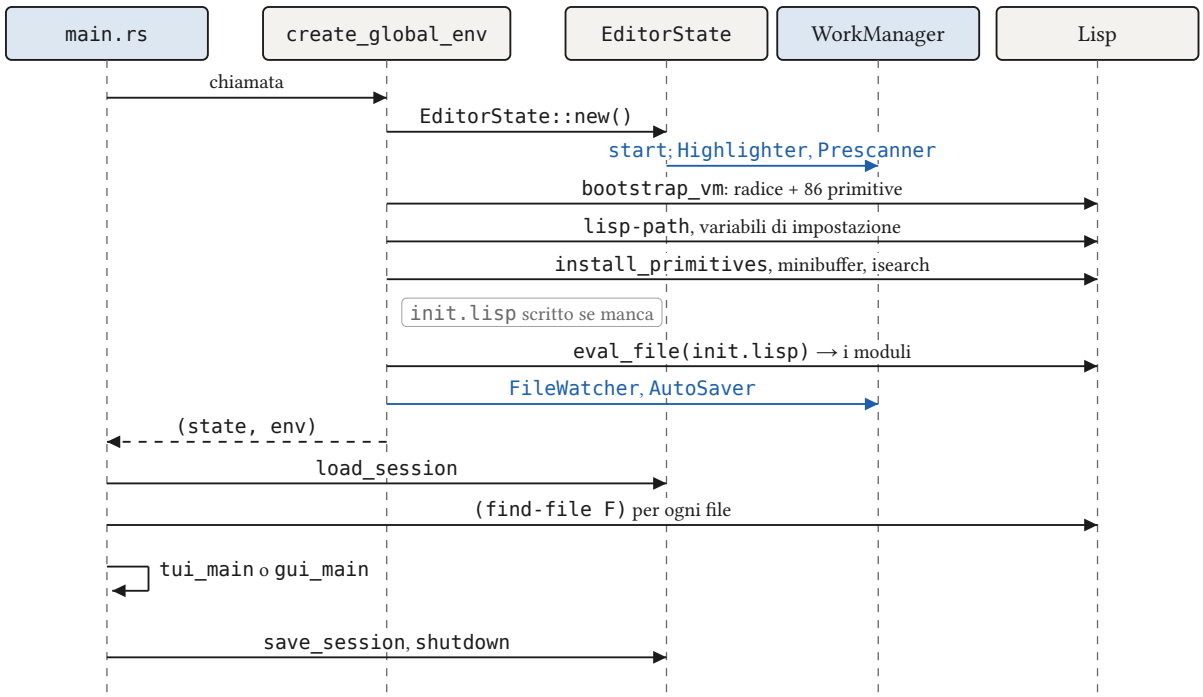
La freccia è la casella con cui il thread dei comandi consegna il lavoro all'esecutore. Tutti condividono lo stesso `EditorState` (clonato: ogni campo è un `Arc`) e, i lavori che lo usano, lo stesso ambiente Lisp. Il carburante, il permesso di `yield`, il contrassegno del comando per l'undo, i dati di `string-match` e il segno di `in_worker` sono invece per thread (variabili `thread_local!`). I lavori di sfondo non chiamano mai Lisp sul proprio thread quando l'effetto deve toccare l'interfaccia: lasciano una *callback dovuta*, che il thread principale esegue tra un comando e l'altro (Sezione 11.10). Fuori dal WorkManager restano i thread di `(spawn ...)`, che l'interprete avvia da sé, e il breve `stop shell` di un comando di shell fermato, che un'uscita non aspetta: a quel punto è `shutdown` a uccidere ciò che resta dei comandi (Sezione 11.10.2).

3 Il ciclo di vita del programma

3.1 Dal `main` al ciclo degli eventi

`src/main.rs` legge la riga di comando — i file da aprire, `--gui`, `--font FILE` e `--help` — e chiama `create_global_env::<GapBuffer>()`; poi attiva il file di log `rsedit.log` nella directory della cache (Sezione 3.1.2) con `start_log_file`, legge la sessione lasciata dall'avvio precedente con `load_session` (Sezione 3.1.3), apre ogni file della riga di comando valutando `(find-file PERCORSO)` ed entra nel ciclo di un front-end: `tui_main`, il terminale, se lo standard input è un terminale e `--gui` non è chiesto, altrimenti `gui_main`, una finestra grafica. Un errore di avvio dell'interprete viene stampato e il programma esce; un errore di `find-file` finisce nell'area dei messaggi. All'uscita `ShutDownOnExit` scrive la sessione con `save_session`, se non si sta srotolando un panic, e chiama `shutdown`, anche per un panic, che ferma il lavoro di sfondo e scrive nel file le ultime righe del log.

Il file di log è di un editor solo. `enable_log_file` lo apre senza troncarlo, ne prende il blocco del sistema operativo (`File::try_lock`) e solo allora lo svuota, e lo scrive in coda: un secondo editor aperto insieme, trovandolo bloccato, scrive in `rsedit-PID.log` accanto, e lo dice nel log. Prima il secondo troncava il file del primo, che continuava a scrivere dalla posizione a cui era arrivato, sopra le righe dell'altro. Il blocco se ne va con il processo, anche dopo un crash, e l'editor che parte dopo riprende `rsedit.log`; dove il filesystem non ha blocchi, il file si usa come prima. Quando non c'è una directory della cache, o il file non si può creare, `start_log_file` lo dice nel log e nell'area messaggi, se lì non c'è già altro da dire, come l'errore di `init.lisp`: l'editor parte con il log in memoria soltanto, e chi cercasse il file dopo un crash saprebbe perché non c'è.



3.1.1 I passi di `create_global_env`

1. `EditorState::new()` riempie la keymap globale con le poche associazioni scritte in Rust (`fill_default_keymaps`), crea ogni compartimento con il suo `Arc<RwLock<…>>`, avvia il WorkManager (`start`) e gli consegna i due lavori che girano sempre, `Highlighter` e `Prescanner`, che si svegliano quando un buffer o una grammatica cambiano.
2. `bootstrap_vm` crea l'ambiente radice con le 86 primitive di base e verifica l'interprete.
3. Ogni impostazione che l'editor legge (`editor/settings.rs`) riceve il suo valore iniziale, così che `(setq ...)` in una configurazione modifichi un valore che esiste già: per esempio `mouse-mode` (`nil`), `echo-message-timeout` (5), `minibuffer-width` (60), `minibuffer-height`

(3), `window-separator` (`|`), `auto-save` e `watch-files` (`t`). `document_settings` dà a ognuna la documentazione che `C-h v` mostra. Vengono creati il modo `fundamental-mode` e il modo minore `read-only-mode` (Sezione 8.8).
4. `install_primitives`, `install_minibuffer` e `install_isearch` registrano le primitive, i comandi, il modo `minibuffer-mode` e il modo `isearch-mode`.
5. `rsedit-path` riceve il percorso dell'eseguibile.
6. `set_search_paths` cerca i plugin nella directory dei dati, li registra in `Loader` e dà il primo valore a `lisp-path` e a `theme-path`: la configurazione, poi i plugin, poi l'editor (Sezione 15.2).
7. Se manca, `init.lisp` viene creato con `DEFAULT_INIT_LISP` (vedi sotto).
8. `load_init` valuta `init.lisp` con `eval_file`, e `init.lisp` carica i moduli Lisp con `require`. Un errore, di sintassi o di valutazione, ferma il resto del file ma non l'avvio: `load_init` lo scrive nel log e nell'area dei messaggi, e l'editor parte con ciò che è stato valutato prima (Sezione 3.1.6). Solo ciò che fallisce prima, nell'ambiente stesso, è un errore di `create_global_env`, e `main` esce con un codice diverso da 0.
9. Il WorkManager riceve `FileWatcher` e `AutoSaver`, che a ogni passo leggono le loro impostazioni (`watch-files`, `watch-file-interval`, `auto-save`, `auto-save-interval`).

3.1.2 Le directory dell'editor

L'editor tiene quattro directory sue: quella della configurazione, con `init.lisp`; quella dei dati, con ciò che si installa per l'editor invece di scriverlo, oggi i plugin; quella della cache, con ciò che può rifare, come il file di log: cancellarla non perde nulla che conti; e quella dello stato, con ciò che ricorda da un avvio all'altro, oggi la sessione (Sezione 3.1.3): cancellarla perde quel ricordo e nient'altro. Le trova `app_dir`, in `paths.rs`: le si chiede una directory per ciò a cui serve (`AppDir::Config`, `AppDir::Data`, `AppDir::Cache`, `AppDir::State`), e risponde con il suo posto su questo sistema, creandola se manca e se `DirOptions::create` lo chiede. La usano `create_global_env`, per `init.lisp` e i plugin, e `main.rs`, per il log e la sessione. Le quattro directory si trovano con la stessa regola:

Fonte, in ordine di precedenza	Directory
`RSEDIT_CONFIG_DIR`, `RSEDIT_DATA_DIR`, `RSEDIT_CACHE_DIR`, `RSEDIT_STATE_DIR`	usata così com'è, su ogni sistema
`XDG_CONFIG_HOME`, `XDG_DATA_HOME`, `XDG_CACHE_HOME`, `XDG_STATE_HOME` (solo Unix, e solo se assoluta)	la variabile, con `rsedit` aggiunto in fondo
il sistema	su Windows `%APPDATA%\rsedit`, `%LOCALAPPDATA%\rsedit\data`, `%LOCALAPPDATA%\rsedit\cache` e `%LOCALAPPDATA%\rsedit\state`; altrove, macOS compreso, `$HOME/.config/rsedit`, `$HOME/.local/share/rsedit`, `$HOME/.cache/rsedit` e `$HOME/.local/state/rsedit`

Una variabile impostata ma vuota conta come non impostata. Quando nessuna fonte risponde, `app_dir` dà `DirError::Undetermined`; gli altri errori sono una directory che manca senza che si chieda di crearla (`NotFound`), un file al suo posto (`NotADirectory`) e una creazione fallita (`CannotCreate`). Che la directory si possa scrivere non viene controllato. La scelta tra le fonti, `choose_dir`, riceve i valori invece di leggerli dall'ambiente: così i test ne provano la precedenza senza scrivere nell'ambiente del processo, che tutti i test vedono. `isolate_config_for_tests` dà invece a ogni processo di test quattro directory sue, attraverso le quattro variabili `RSEDIT_*`.

I dati installati *con* l'editor — moduli, temi, pagine — non sono una di queste: stanno in `data` accanto all'eseguibile, e `installed_data_dir` li cerca lì o un livello sopra, dove `build.rs` li mette per i binari di test in `target/<profilo>/deps/`. La usano `data-directory` e `theme-path`.

Cinque decisioni:

1. **Su macOS, `~/.config` e `~/.cache`.** È lì che guarda chi usa un editor da terminale su un Mac: gli editor da cui arriva, Emacs con XDG, Neovim e Helix, tengono lì configurazione e cache, e `~/.local/state` lo stato. Scartato: `~/Library`, la convenzione delle applicazioni impacchettate.
2. **Su Windows la cache è locale.** `%APPDATA%` segue la persona da una macchina all'altra e viene copiato a ogni accesso. Una cache è proprio ciò che si può rifare, e quindi ciò che non va copiato: sta sotto `%LOCALAPPDATA%`.
3. **Lo stato ha una directory sua.** La storia dei prompt e i file recenti non sono configurazione, che la persona scrive e porta con sé, né cache: un programma che pulisce la cache non deve portarsi via ciò che l'editor ricorda. La specifica freedesktop ha `XDG_STATE_HOME` proprio per questo — storia delle azioni, file usati di recente, stato da riprendere al riavvio — e lì la tiene anche Neovim. Su Windows sta sotto `%LOCALAPPDATA%`, come la cache: è di questa macchina. Scartato: un file nella cache, dove «cancellarla non perde nulla che conti» non sarebbe più vero.
4. **Su Windows le variabili `XDG_*` non contano.** Le imposta solo chi si porta dietro un'abitudine di Unix; preferirle alla risposta del sistema metterebbe la configurazione dove nessun programma Windows la cerca.
5. **Ogni processo di test ha le sue directory.** Costruire un editor crea la directory della configurazione e valuta ciò che contiene: sotto la `HOME` di chi lancia i test scriverebbe lì la sua configurazione, e alcuni test spostano `HOME`, che vale per tutto il processo mentre i test girano in parallelo. `isolate_config_for_tests` punta configurazione, dati, cache e stato a quattro directory temporanee del processo, una volta sola e prima del primo editor, attraverso le quattro variabili `RSEDIT_*`: senza quella dei dati, i plugin di chi lancia i test entrerebbero in `lisp-path`. È la porta di una persona, così i test passano da `app_dir` invece di aggirarla. È pubblica per i test del crate binario, che il `cfg(test)` di una libreria non raggiunge.

3.1.3 La memoria tra un avvio e l'altro

L'editor ricorda tre cose da un avvio al successivo: le risposte date a ogni prompt, i file visitati, dal più recente, e dove era il punto in ognuno. Stanno in due compartimenti — le risposte in `History`, i file e le posizioni in `Visits` — e su disco in un file solo, `session`, nella directory dello stato (Sezione 3.1.2). `load_session` lo legge in `main.rs` dopo `create_global_env` e prima dei file della riga di comando; `save_session` lo scrive all'uscita, da `ShutDownOnExit`. Tutti e due stanno in `editor/session.rs`; ciò che registra una visita e una posizione, in `editor/visits.rs`.

Che cosa	Dove	Quando si registra	Impostazione
Le risposte ai prompt	`History`	`minibuffer-confirm`, con `History::remember`, come in ogni avvio (Sezione 7).	`save-history`
I file recenti	`Visits`	`find-file`, anche per un file già aperto o che non esiste, e `write-file` verso un percorso nuovo: `record_visit` mette il percorso in testa e taglia la lista a `recent-files-length`.	`recent-files-length`
Le posizioni	`Visits`	`close_buffer`, per il buffer che si chiude, e `save_session`, per ogni buffer aperto con un file: `record_place`. Un punto all'inizio toglie la voce invece di scriverla.	`save-place`

Una posizione torna in `new_buffer`: `place_for` dà il punto ricordato per il file, e `goto_offset` ve lo porta, troncato alla fine del testo, prima che il buffer entri nella tabella e che girino i suoi hook. La finestra lo segue al primo disegno, come segue ogni punto (Sezione 12.1).

Il file è di testo, una voce per riga, dopo un'intestazione con la versione:

```
rsedit-session 1
history M-x find-file
history M-x grep
recent  /home/me/notes.txt
place /home/me/notes.txt  1842
```

I campi sono separati da un tab; la barra rovesciata, il tab, l'a capo e il ritorno carrello dentro un campo si scrivono `\\`, `\t`, `\n` e `\r`, e una barra davanti a qualunque altra cosa vale per sé. Le voci di ogni lista vanno dalla più recente, e le chiavi della storia in ordine alfabetico (`History::rings`), così che due scritture dello stesso stato diano lo stesso file. `Session::parse` non legge un file la cui prima riga non sia l'intestazione, e lo dice nel log; salta, contandole, le righe che non capisce, un tipo che non conosce compreso. Un file che manca, o una directory che manca, è il primo avvio, e non dice nulla.

`load_session` restituisce ciò che legge come dicono le impostazioni in quel momento, cioè dopo `init.lisp`: con `save-history` a `nil` la storia letta si scarta, con `save-place` a `nil` le posizioni. `History::restore` e `Visits::restore` mettono ciò che arriva dal file *dietro* a ciò che l'avvio conosce già, e un percorso già noto tiene la voce più nuova; i limiti sono `history-length`, `recent-files-length` e `PLACES_LIMIT`, 400 posizioni. `save_session` registra le posizioni dei buffer aperti, copia le tre liste uscendo da ogni lock, e solo allora scrive, con `write_atomically` (Sezione 8.7). Un errore va nel log: l'editor sta uscendo, e non c'è un'area dei messaggi da leggere.

Le decisioni:

1. **In Rust, non in un modulo Lisp.** La storia sta già in un compartimento, e risp non ha un modo di scrivere dati che si rileggano senza valutarli: `format` con `%S` non mette le virgolette a una stringa, e l'unico lettore è `eval-string`. Un percorso con una virgoletta romperebbe il file, e rileggerlo vorrebbe dire eseguire all'avvio ciò che un altro processo ha scritto. In Lisp resta ciò che è politica: `recent-files`, il modulo del comando che riapre un file (Sezione 15.5.22).
2. **Un formato a righe.** Si scrive e si rilegge senza interprete, una riga rovinata si perde da sola, e una riga di un tipo che una versione futura aggiunga si salta invece di fermare la lettura. Scartato: JSON o un formato binario, che chiederebbero una dipendenza per tre liste di stringhe.
3. **Si scrive all'uscita, una volta.** Scrivere a ogni comando metterebbe il disco sul percorso dei tasti, e un lavoro di sfondo a intervalli scriverebbe di continuo pochi byte che cambiano di rado. Il prezzo è che un editor ucciso, o fermato da un panic, non scrive nulla, e l'avvio dopo legge la sessione di quello prima: è ciò che il manpage `session` dice nei limiti.
4. **Non durante un panic.** Un panic può lasciare un buffer a metà di una modifica, e un secondo panic dentro un `Drop`, durante lo srotolamento, interrompe il processo prima che `shutdown` scriva nel file le ultime righe del log. Il log di un crash vale più della sessione: `ShutDownOnExit` salta `save_session` quando `std::thread::panicking` risponde sì.
5. **La legge `main.rs`, non `create_global_env`.** Dopo `init.lisp`, così che le impostazioni dicano già la loro, e prima dei file della riga di comando, che si aprono dove erano stati lasciati. Fuori da `create_global_env` perché i test costruiscono editor in parallelo nella stessa directory di stato isolata: uno che leggesse la sessione scritta da un altro vedrebbe risposte e file che non sono suoi. I test della sessione chiamano `load_session` e `save_session` con una directory propria.
6. **Vince chi esce per ultimo.** Due editor aperti insieme non fondono le loro sessioni: il file è sempre intero, perché si scrive con `write_atomically`, ma l'editor che esce dopo sostituisce ciò che ha scritto l'altro.

Scartato: rileggere e fondere all'uscita, che per un caso raro chiederebbe regole di fusione diverse per tre liste ordinate.

7. **La posizione prima degli hook.** Un `find-file-hook` vede il punto dove il file era stato lasciato e, se lo sposta, ha l'ultima parola. La posizione è in caratteri, come ogni posizione dell'editor; un file accorciato si apre alla fine.
8. **Leggibile solo da chi lo possiede.** La storia contiene ciò che si è scritto a ogni prompt, i comandi di `M-!` compresi. Un file `session` nuovo nasce vuoto con i permessi `0600`, su Unix, prima della scrittura atomica, che dà al file nuovo i permessi di quello che sostituisce; uno che c'è già tiene i suoi.
9. **Le posizioni hanno un tetto loro.** Una posizione serve anche per un file uscito dalla lista dei recenti, e `recent-files-length` a 0 non deve cancellarle: `PLACES_LIMIT` è fisso, come `save-place-limit` in Emacs.

3.1.4 Il file `init.lisp` predefinito

Il file `init.lisp` predefinito chiede un modulo solo, `default-configuration` (Sezione 15.5.11), che chiede con `require` tutti gli altri, i moduli di base e poi i facoltativi, e imposta `dired-sort` a `'type` e `mouse-mode` a `t`; l'utente scrive le sue impostazioni dopo quella riga. L'ordine dei moduli non conta: ognuno chiede quelli che gli servono (Sezione 15).

Il file viene scritto solo se manca, e da allora è dell'utente. Ciò che l'editor distribuisce sta in `default-configuration`, che arriva con ogni versione: un modulo aggiunto dopo arriva a un `init.lisp` che lo chiede. Uno scritto prima di quel modulo, che chiede ogni modulo per nome, va aggiornato a mano una volta, o sostituito con la riga sola. Un file ancora più vecchio, con `(eval-file "dired")`, funziona ancora: `lisp-path` nomina le tre cartelle dei moduli, ed `eval-file` cerca un nome nudo in ognuna.

3.1.5 `eval_file`

`eval_file(nome, env)`, in `editor/loader.rs`, cerca prima `nome` come percorso. Se non esiste e il nome non contiene `/`, `\` o `.`, cerca `nome.lisp` in ogni directory di `lisp-path`, in ordine. Il contenuto viene valutato da `eval_source` dentro uno scope di carburante (`begin_command`), con il file annotato in `Loader` come quello da cui viene un `provide`, e `eval_forms` lo legge una forma alla volta, come `load` in Emacs: ogni forma è valutata alla radice della catena di ambienti (`top_level`) prima che si legga la successiva. Un file mancante produce una riga di log e il valore `nil`. Lo valuta ogni volta: è `require` a caricare una volta sola, e legge il file con lo stesso `eval_source`.

da `editor/loader.rs` — estratto semplificato

```
loop {
    match parser.next() {
        Ok(form) => {
            // la riga dove finisce la forma letta, dalla mappa del sorgente
            read_to = map.locate(items).end_line;
            last = eval(&form, top_level(env), self)?;
        }
        Err(ParserError::VoidExp) => return Ok(last),   // fine del testo
        Err(why) => return Err(syntax_error(path, read_to, why)),
    }
}
```

Il primo errore, di sintassi o di valutazione, ferma il file da quel punto e risale come errore; le forme prima di lui restano valutate. Un errore di sintassi nomina il file e la riga dopo la quale comincia la forma che non si legge (`… does not parse: a ) that closes nothing, in the form after line 2`): il lettore lo trova solo finendo quella forma, e la riga dove è finita la precedente è quella che la mappa del sorgente conosce. Un commento sull'ultima riga, con o senza a capo, non nasconde nulla. Fino alla correzione di AVV-2 il file veniva avvolto in `(progn …)` e letto come una forma sola: una parentesi di troppo chiudeva la `progn` e il resto del file non veniva letto, una che mancava, o un commento finale senza a capo, lo rendevano illeggibile, e `eval_file` restituiva `nil` senza dire nulla.

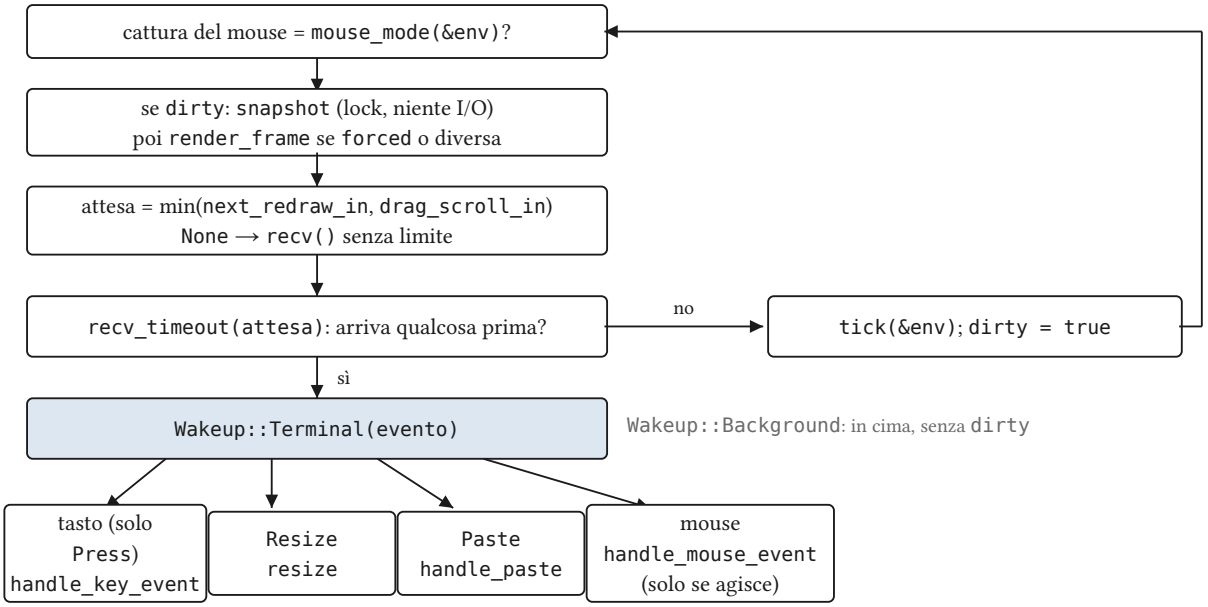
`eval_forms` distingue la fine del testo da un errore con un `match` sulle varianti di `ParserError`, che risp esporta insieme a `Parser`. `syntax_error` dice in parole le parentesi e le stringhe che non si chiudono, o che non chiudono nulla; le altre varianti, più rare, con il loro nome.

3.1.6 Le decisioni

1. **I due lessici partono con l'editor.** L'evidenziatore e la prescansione sono sottomessi in `EditorState::new`, prima che esista un modo: un lavoro che aspetta un cambiamento non costa nulla, e uno mai avviato costerebbe un file senza colori e senza una ragione detta. Sono sottomessi una volta sola, e uno che va in panic si risottomette da sé (Sezione 11.10.1).
2. **I due osservatori partono dopo `init.lisp`.** Il controllo dei file e il salvataggio automatico leggono le loro impostazioni a ogni passo, e sono sottomessi dopo la configurazione: ciò che dice conta dalla prima occhiata.
3. **Un errore in `init.lisp` ferma il file, non l'editor.** Come in Emacs con `--debug-init` spento: le forme prima dell'errore restano valutate, quelle dopo no, e l'editor parte. Un editor che non parte lascia senza lo strumento con cui correggere il file. L'errore va nell'area dei messaggi, dove si vede subito, e nel log con il nome del file e la traccia delle chiamate; il log, ancora in memoria, finisce nel file che `main` apre subito dopo. Scartato: valutare ogni forma anche dopo un errore, che porterebbe avanti una configurazione a metà con errori a catena.
4. **Il file predefinito carica moduli e imposta variabili, niente di più.** I moduli che chiede non sono sempre presenti — un binario di test non li ha accanto, e `require` registra il modulo mancante e risponde nil —: senza di loro `defcommand` non è una macro, una forma che la usa diventa una chiamata qualunque, e il resto di `init.lisp` non viene valutato. Un comando sta in un modulo o in Rust. È una stringa grezza, dove una barra rovesciata è una barra rovesciata; ed è una costante con un nome, perché un test la valuti in un ambiente nudo.
5. **Un file si valuta alla radice dell'ambiente.** Un `defun`, un `defvar` o il `setq` di un nome nuovo legano nell'ambiente in cui sono valutati, e dentro una funzione se ne vanno con lei. `eval_file` valuta alla radice (`top_level`), da dovunque venga la chiamata: un modulo caricato da una funzione, o con `M-:`, definisce ciò che definirebbe `init.lisp`.

3.2 Il ciclo degli eventi della TUI

`tui_main` (`src/tui/mod.rs`) attiva la modalità raw e l'incolla «bracketed», rileva la profondità di colore (`COLORTERM`, `TERM`), comunica le dimensioni iniziali in `frame-width` e `frame-height`, e poi gira finché `is_running()` è vero. Il ciclo aspetta su un canale solo, che porta due generi di risveglio (`Wakeup`): gli eventi del terminale, che un thread lettore («terminal input», avviato da `read_terminal`) legge con `read()` e manda nel canale, e la notizia che un lavoro di sfondo ha cambiato qualcosa, che l'editor dà alla callback registrata con `on_background_change`.



ogni ramo, tranne un mouse ignorato, imposta `dirty` e `forced` e torna in cima

il thread lettore manda ogni evento nel canale; la callback di `on_background_change`, un `Background`

Il cuore del ciclo, senza la gestione della cattura del mouse:

da `../../src/tui/mod.rs` — estratto semplificato

```
while state.is_running() {
    if dirty {
        let (cols, rows) = terminal::size()?;
        let next = state.snapshot(&env, cols as usize, rows as usize); // lock, niente I/O
        if forced || next != frame || next.clipboard.is_some() {
            render_frame(&next, depth)?;                              // I/O, niente lock
        }
        frame = next; dirty = false; forced = false;
    }
    let wake = [state.next_redraw_in(&env, &frame), state.drag_scroll_in()]
        .into_iter().flatten().min();
    let wakeup = match wake {
        Some(remaining) => match inbox.recv_timeout(remaining) {
            Ok(wakeup) => wakeup,
            Err(RecvTimeoutError::Timeout) => {
                state.tick(&env);    // l'attesa è scaduta: callback dovute, scorrimento
                dirty = true;
                continue;
            }
            Err(RecvTimeoutError::Disconnected) => break,
        },
        None => match inbox.recv() { Ok(wakeup) => wakeup, Err(_) => break },
    };
    let event = match wakeup {
        Wakeup::Background => continue,     // in cima: l'editor ridice quanto aspettare
        Wakeup::Terminal(event) => event?,
    };
    match event {
        Event::Key(k) => match translate_key(k) {
            Some(ev) => if k.kind == KeyEventKind::Press {
                state.handle_key_event(ev, &env); dirty = true; forced = true;
            },
            None => state.log_diagnostic(/* "no translation for the key" */),
        },
        Event::Resize(w, h) => {
            state.resize(env.clone(), w as usize, h as usize); dirty = true; forced = true;
        }
        Event::Paste(text) => {
            state.handle_paste(text, &env); dirty = true; forced = true;
        }
        Event::Mouse(m) => if let Some(ev) = translate_mouse(m)
                              && state.handle_mouse_event(ev, &env) {
            dirty = true; forced = true;
        },
        _ => (),
    }
}
```

Quattro dettagli contano. Il frame si compone solo se qualcosa è successo (`dirty`), e si disegna solo se è diverso da quello sullo schermo, o se qualcosa è stato fatto dall'utente (`forced`), o se porta testo per gli appunti. L'attesa è guidata dall'editor: se nulla è in sospeso `next_redraw_in` risponde `None` e il ciclo si blocca sul canale senza consumare CPU. Un `Background` non sporca nulla: riporta il ciclo in cima, dove `next_redraw_in` vede il cambiamento e risponde con il tempo che manca a un turno dall'ultimo frame; poi l'attesa scade, e `tick` e il frame seguono come per il tempo. E `tick` viene chiamato solo quando l'attesa scade, non a ogni giro.

La cattura del mouse segue `mouse-mode` a ogni giro, prima del frame: `mouse-mode-toggle` vale dal clic dopo. `translate_key` traduce anche Canc, Ins, Inizio, Fine, PagSu, PagGiù e i tasti funzione; un tasto che

non sa tradurre, come Stamp, viene soltanto registrato nel log. Shift con un carattere diventa la maiuscola; `BackTab` diventa `Tab` con Shift. Gli altri eventi del terminale, come il focus, si ignorano.

Il terminale è preso da una guardia, TakenTerminal, che entra nello schermo alternativo, come ogni programma a tutto schermo, e il cui Drop disattiva la cattura del mouse e l'incolla, mostra il cursore, torna allo schermo principale, con ciò che il terminale mostrava prima che l'editor partisse, e lascia la modalità raw: all'uscita normale come a un errore di I/O o a un panic. Un panic sul thread che disegna restituisce il terminale già nel hook, prima che il messaggio venga stampato. Il thread lettore non si aspetta: finisce al primo errore, o al primo evento dopo che il ciclo se n'è andato.

3.2.1 Le decisioni

1. **Un thread che legge il terminale, e un canale.** crossterm sa aspettare un evento con un limite, ma nessuno può interrompere quell'attesa da un altro thread: un lavoro di sfondo che cambia un buffer, un file ricaricato, resterebbero fuori dallo schermo fino al tasto dopo. Con il canale il ciclo aspetta una cosa sola, e sia il terminale sia l'editor possono svegliarlo.
2. **Un evento che non cambia nulla non compone un frame.** Un terminale che riporta il mouse manda decine di movimenti al secondo: comporre e disegnare per ognuno costerebbe e farebbe sfarfallare lo schermo.
3. **Dopo un'azione dell'utente si disegna anche un frame uguale.** Un altro programma può aver scritto sul terminale, e un tasto è il modo naturale di ripararlo. Un frame composto perché il tempo è passato si disegna solo se è diverso: il controllo dei file ogni pochi secondi e un lavoro ogni 40 ms non ridisegnano uno schermo fermo.
4. **La cattura del mouse si ricontrolla a ogni giro.** `mouse-mode` è una variabile che il Lisp cambia quando vuole; seguendola a ogni giro il cambiamento vale subito, e un terminale non resta a riportare il mouse a un editor che non lo vuole.
5. **L'incolla è un evento solo.** Con l'incolla «bracketed» il testo arriva tutto insieme e diventa `insert-pasted-text`: un passo di undo, gli hook una volta, nessuna parentesi chiusa da `electric-pair` a ogni apertura. È anche l'unica strada da cui gli appunti di sistema entrano nella TUI.
6. **Gli eventi che l'editor non conosce si ignorano.** Il focus e ciò che un terminale riporta oltre ai tasti, al mouse, all'incolla e alle dimensioni non sono un errore: un ciclo che li rifiutasse si fermerebbe per un evento che nessuno ha chiesto.

4 Lo stato: `EditorState` e i compartimenti

4.1 La facciata

`EditorState<B: BufferTrait>` è una struttura di `Arc`: clonarla costa poco e ogni clone vede lo stesso stato. I suoi campi sono privati, e ognuno è un *compartimento* di `managers/` sotto un `RwLock`, un dato piccolo con un lock o un atomico proprio, o il WorkManager. La tabella dà i tipi senza l'`Arc` che li condivide.

Campo	Tipo	Contenuto
`buffers`	`RwLock<Buffers<B>>`	Tabella nome ⟶ `Arc<RwLock<Buffer>>`, buffer corrente, ordine di recente uso.
`windows`	`RwLock<Windows>`	Albero delle finestre affiancate, finestre flottanti, focus, prossimo id, trascinamento, ultimo clic.
`modes`	`RwLock<Modes<B>>`	Registro dei modi, maggiori e minori, regole dei modi minori, keymap globale, completamenti globali, modi per estensione, keymap transitoria, tasti di ripetizione, hook globali.
`commands`	`RwLock<Commands<B>>`	Registro dei comandi, pila dei comandi in attesa di argomenti, argomento prefisso, ultimo comando e sua forma.
`kill_yank`	`RwLock<KillYank>`	Kill ring, ultimo yank, flag «il comando precedente ha ucciso/incollato», ultimo rettangolo ucciso, testo dovuto agli appunti di sistema.
`macros`	`RwLock<Macros>`	Macro in registrazione, ultima macro, profondità delle riproduzioni, contatore.
`history`	`RwLock<History>`	Storia dei prompt per chiave (100 voci per chiave predefinite).
`visits`	`RwLock<Visits>`	File visitati, dal più recente, e dove era il punto in ognuno (Sezione 3.1.3).
`log`	`RwLock<Log>`	Righe diagnostiche e file a cui sono copiate.
`runtime`	`RwLock<Runtime>`	Colonna obiettivo, isearch e sostituzione in corso, `FuelMeter`, pila delle chiamate per i backtrace, tema in uso e tema di fabbrica, i lavori avviati da Lisp per nome.
`echo_message`	`RwLock<EchoMessage>`	Testo dell'area messaggi con l'istante in cui è stato scritto.
`pending_keys`	`RwLock<Vec<KeyEvent>>`	Tasti di una sequenza non ancora completa.
`owed`	`RwLock<Vec<OwedCallback>>`	Callback chieste dai lavori di sfondo e non ancora eseguite (Sezione 11.10.4).
`current_results`	`RwLock<Option<String>>`	Il buffer di risultati che `next-error` percorre.
`grammar_epoch`	`AtomicU64`	Quante volte è cambiata una grammatica: il timbro di una colorazione.
`syntax_changed`	`Topic`	L'argomento `syntax-changed`, risolto una volta (Sezione 11.4).

Campo	Tipo	Contenuto
`work`	`WorkManager<EditorState<B>>`	Il lavoro asincrono: i lavori dell'esecutore e i thread dedicati (Sezione 11).
`background_change`	`BackgroundChange`	Se un lavoro ha cambiato lo schermo dall'ultima fotografia, e come dirlo al front-end (Sezione 11.10).
`running`	`AtomicBool`	Falso dopo `quit`.
`shell_commands`	`AtomicUsize`	Quanti comandi di shell stanno girando.
`command_errors`	`AtomicUsize`	Quanti comandi sono falliti: la riproduzione di una macro si ferma al primo.

`EditorState` implementa `Debug` e `PartialEq` solo perché `LispContext` li richiede, ma tutti e due rispondono invece di andare in panic: `Debug` stampa i nomi dei buffer, o `<locked>` se la tabella è sotto un lock di scrittura, perché stampare non deve aspettare; `PartialEq` confronta per identità, e due handle sono uguali quando condividono lo stato, come ogni clone di uno.

4.2 Le regole dei compartimenti

Ogni compartimento le segue; la documentazione di `managers/mod.rs` le riassume.

Regola	Perché
Un compartimento non raggiunge un altro compartimento.	Riceve i valori che gli servono e restituisce risposte (`Windows::scroll_focused` riceve un numero di righe, non un buffer). Così solo la facciata tiene due lock insieme.
Non consegna guardie.	Si accede allo stato solo con una closure (`windows(|w| …)`), e il lock non sopravvive alla domanda.
Non valuta mai.	Nessun compartimento riceve un `Env`. Può conservare valori Lisp (il corpo di un hook), mai eseguirli: Lisp può rientrare nell'editor da qualunque primitiva.
Risponde con un verdetto, non con un effetto.	Quando un'operazione ha conseguenze altrove (un buffer da ripuntare, una finestra da rifocalizzare) restituisce un enum come `BufferRemoved` o `WindowRemoved`, e la facciata agisce.
Non fa nulla di lento con il lock preso.	Niente disco, rete o attese. Registrare una diagnostica aggiunge una riga alla lista e basta: il file lo scrive il watchdog con `Log::flush`, che tiene il lock della lista solo per copiare le righe.

Le prime quattro regole servono alla correttezza: ognuna toglie un modo in cui due pezzi dello stato si possono vedere in disaccordo. L'ultima serve alla latenza, ed è quella che si rompe per sbaglio, perché romperla non costa nulla finché il disco è veloce e l'albero piccolo. Il segnale è un lock tenuto attraverso una chiamata di cui non si controlla la durata: una lettura o una scrittura, un'allocazione grande quanto un file, una callback. Se non si sa dire quanto dura il corpo, si copia fuori ciò che serve e si rilascia prima.

La regola che nasce da tutte queste, e che vale anche fuori dai compartimenti, è una sola: *mai tenere un lock attraverso una chiamata all'interprete.* `run_hook` copia la lista degli hook e rilascia `modes` prima di valutarli; `command_specs` restituisce dati posseduti perché `call-interactively` valuta Lisp subito dopo. Un `RwLock` non è rientrante: un hook che chiama `add-hook` mentre lo stesso thread tiene il lock di lettura del registro si blocca per sempre, ogni volta.

4.3 I compartimenti

Ogni compartimento è un tipo di `managers/`, sotto un `RwLock` della facciata, e tiene un'invariante che i suoi campi presi uno per uno non terrebbero.

Compartimento	Che cosa garantisce
`Buffers`	La tabella non è mai vuota, e `current` nomina sempre un buffer della tabella: `make_current` e `bind_current` rifiutano un nome assente, `rename` e `remove` spostano `current` nello stesso passo.
`Windows`	Il focus nomina sempre una finestra che esiste, e l'ultima finestra affiancata non si chiude.
`Modes`	Le metà globali e quelle dei modi — keymap, sorgenti di completamento, hook — si leggono sotto lo stesso lock, nell'ordine in cui la risoluzione le consulta.
`Commands`	L'argomento prefisso appartiene a un comando solo, e i comandi in attesa di argomenti si chiudono in ordine inverso a quello in cui sono cominciati.
`KillYank`	I flag «il comando precedente ha ucciso, ha incollato» valgono rispetto all'anello e si spostano insieme, una volta per comando.
`Macros`	I tasti di una registrazione diventano l'ultima macro in un passo solo, e nessun tasto riprodotto viene registrato, a qualunque profondità.
`History`	Un cammino nella storia vale solo nell'anello e nel prompt in cui è cominciato.
`Visits`	Un percorso compare al più una volta in ogni lista, i file recenti e le posizioni, e ogni lista va dal più recente e resta entro il suo limite.
`Log`	Ogni riga arriva al file una volta e nell'ordine in cui è stata registrata, salvo le scritture che falliscono e le righe uscite dalla lista prima di essere scritte.
`Runtime`	Nessuna: raggruppa ciò che appartiene alla sessione e non a un documento (Sezione 4.8).
`Loader`	Una feature fallita non è mai in vigore, e ciò che è stato fornito durante un `require` fallito se ne va con lui; le funzioni in attesa di una feature si consegnano una volta sola, quando arriva (Sezione 15).

La facciata raggiunge un compartimento con una *porta* che esegue una closure: `windows(|w| …)` per leggere, `windows_mut(|w| …)` per cambiare. Il lock non sopravvive alla domanda, e dentro la closure non si richiama la facciata: un `RwLock` non promette che una seconda lettura sullo stesso thread riesca; si può chiamare ciò che non prende lock, come fa `set_mode`, che sposta il contatore delle grammatiche dentro la porta di `modes`. `Log` è il solo che prende il lock da sé: `Log::append` e `Log::flush` ricevono il `RwLock<Log>`, perché il WorkManager registra senza tenere un editor, che lo contiene.

4.4 Le coppie con `editor/`

`managers/` ed `editor/` si accoppiano per nome, con una regola sola.

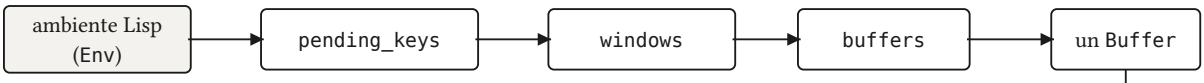
- `managers/X` è il compartimento: i campi sotto un lock e le operazioni che non hanno bisogno d'altro.
- `editor/X` è la sua metà della facciata: i metodi di `EditorState` che danno alle operazioni di X un nome da editor (`command_specs` per `Commands::specs`, perché la facciata ha anche nomi di buffer e di finestre) e ciò che usa X insieme a un altro compartimento, al testo di un buffer o all'interprete.
- Le porte dei lock stanno tutte in `editor/mod.rs`, accanto ai campi: è il solo file che prende i lock dei compartimenti. Una porta sola ne tiene due, `windows_mut_with_buffers`, per `snapshot`, che compone ogni finestra sul buffer che mostra (Sezione 4.5).
- Un metodo che inoltra una chiamata sola al compartimento X sta in `editor/X.rs`, chiunque lo chiami: i cinque del tema in `runtime.rs`, quelli che il mouse chiede alle finestre in `windows.rs`.

- Un compartimento senza metodi di facciata non ha un file in `editor/`: oggi `history`, le cui porte bastano.
- Chi ha bisogno di un compartimento solo, primitive comprese, usa la porta: `ctx.modes(|m| …)` in una primitiva è previsto.

Gli altri file di `editor/` portano il nome di ciò che coordinano — `keys.rs`, `mouse.rs`, `frame.rs`, `boot.rs`, `settings.rs`, `replace.rs`, `results.rs`, `background.rs`, `session.rs` — e tengono solo ciò che mette insieme più cose: un compartimento e il testo di un buffer, l'interprete o un altro compartimento. `work.rs` è la facciata del WorkManager, che non è un compartimento: lo si raggiunge intero (`work()`), perché non ha un lock che si possa prendere fuori ordine o tenere attraverso l'interprete. L'implementazione di `LispContext`, il contratto dell'editor con l'interprete, sta in `mod.rs`.

4.5 L'ordine dei lock

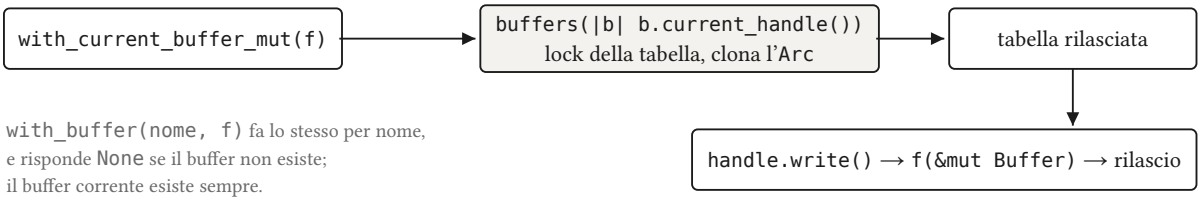
Quando un'operazione deve tenere più lock insieme, li prende in quest'ordine. Sono due a farlo. `snapshot` tiene `windows`, in scrittura, `buffers` e i buffer che compone, attraverso la sola porta che apre due compartimenti, `windows_mut_with_buffers`. La risoluzione di un tasto tiene `pending_keys` mentre legge il buffer corrente e poi interroga `modes`, e lo prende in scrittura per togliere una mappa `Release`. Ogni altro percorso copia fuori ciò che gli serve e rilascia prima di prendere il lock seguente; `echo_message` e gli altri compartimenti si prendono da soli.



le variabili si leggono *prima* di ogni altro lock e mai con uno preso: `snapshot` legge le impostazioni, poi l'area messaggi, poi `windows` e `buffers` insieme; la risoluzione di un tasto tiene `pending_keys`, legge il modo del buffer corrente e solo dopo prende `modes`.

`buffers` ha una particolarità: nel percorso comune non viene tenuto mentre si blocca un buffer. `with_buffer` e `with_current_buffer` clonano l'`Arc` del buffer dalla tabella, *rilasciano* la tabella e solo allora bloccano il buffer. Così chi scrive in un buffer non chiude fuori dalla tabella gli altri thread — l'evidenziatore e lo scanner la percorrono mentre l'utente scrive. Soltanto `snapshot` tiene la tabella e i singoli buffer insieme, nell'ordine indicato.

4.6 Accesso ai buffer



`with_buffer(nome, f)` fa lo stesso per nome, e risponde `None` se il buffer non esiste; il buffer corrente esiste sempre.

`Buffers` non è mai vuoto: nasce con `*scratch*` corrente. Le funzioni che cambiano quale buffer è corrente passano tutte da `make_current` o, per `set-buffer`, che agisce su un buffer senza mostrarlo, da `bind_current`, che non tocca l'ordine dei buffer recenti: tutte e due rifiutano un nome assente dalla tabella (Sezione 4.8).

4.7 L'editor come contesto dell'interprete

`EditorState<B>` è il `T: LispContext` dell'interprete:

Metodo del trait	Implementazione
`consume_fuel(n)`	`FuelMeter` di `Runtime` (clonato fuori dal lock); `Exhausted` diventa `OutOfFuel`.
`log_diagnostic(msg)`	`Log::append`: la riga va nella lista; il file lo scrive il watchdog (`Log::flush`).
`with_unwind(cleanup)`	Esegue la pulizia di un `unwind-protect` con il carburante che resta e, finito quello, con una riserva di 100.000 passi, che non passa alla valutazione ordinaria e non si ricarica fino al prossimo scope esterno.
`begin_thread_evaluation()`	`arm_thread()`: il nuovo thread riceve il budget configurato.
`push_call_frame` / `pop_call_frame` / `call_frame_depth` / `truncate_call_frames`	La pila `call_stack: Vec<String>` di `Runtime`, che `report_error` legge come backtrace.

Lo scope di carburante dell'editor è `begin_command()`: lo aprono `run_command_form`, ogni hook, `report_error`, ogni callback dovuta, ogni funzione di `after-resize-hook` ed `eval_file`. Solo l'apertura più esterna ricarica il budget (10.000.000 di passi), quindi un `eval-file` dentro un comando consuma il budget del comando, e così `pre-command-hook`, che gira dentro lo scope del comando. `post-self-insert-hook` e `post-command-hook`, invece, girano dopo che lo scope si è chiuso: ogni loro funzione riparte con il budget pieno.

4.8 Le decisioni

1. **Lo stato è diviso in compartimenti, ognuno sotto un lock.** Un compartimento raccoglie i campi che sono un fatto in più pezzi. Sotto lock separati si vedrebbero a metà di un cambiamento — il focus che nomina una finestra che il layout ha già tolto — e ogni operazione che ne tocca due dovrebbe prenderli in un ordine scritto altrove. Sotto un lock solo non c'è un ordine da rispettare, e il disaccordo non si può vedere.
2. **`current` non nomina mai un buffer assente.** Nella tabella, `current` è una chiave e `recency` una lista di chiavi: due pezzi su tre valgono solo rispetto al terzo. `current` si cambia solo con `make_current` e `bind_current`, che rifiutano un nome assente, e `rename` e `remove` lo spostano nello stesso passo, non lasciandolo al chiamante. Un nome assente diventato corrente farebbe fallire la ricerca successiva con un panic, e con l'editor andrebbe perso ciò che non era salvato nelle altre finestre.
3. **`Buffers` non legge il testo e non conosce le finestre.** Ciò che un buffer contiene è di `Buffer`, e chi lo mostra sono le finestre, che ne tengono il nome. Il rapporto è a senso unico, e i due compartimenti non hanno bisogno l'uno del lock dell'altro: quando un nome cambia o un buffer sparisce, `BufferRenamed` e `BufferRemoved` lo dicono e la facciata ripunta le finestre. `rename` non tocca neppure il campo `name` del buffer: bloccare un buffer tenendo la tabella metterebbe la tabella in coda a chi sta scrivendo nel buffer.
4. **I campi di `Commands` sono la vita di un comando.** Il registro dice che cosa può eseguire `M-x` e con quali argomenti, l'argomento prefisso è il conteggio scritto prima, la pila è ciò che resta da raccogliere, gli ultimi due campi sono ciò che resta al comando successivo. Si scrivono tutti nel corso di un tasto, e l'ordine in cui lo sono è il protocollo. Che il prefisso sia ancora in lettura è un campo a sé: «nessun argomento» e «un argomento finito» sono stati diversi, e una cifra significa cose diverse nei due.
5. **Gli hook stanno con i modi, non con i comandi.** Sembrano macchinario dei comandi, ma un hook appartiene a un modo: quelli registrati per tutti i modi sono la metà globale di `MajorMode::hooks`, come la keymap globale lo è di `MajorMode::keymaps`.

6. **Guardare indietro nella storia non fa perdere ciò che si scriveva.** Il primo passo indietro mette da parte il testo del prompt, e andare avanti oltre la voce più recente lo restituisce. Altrimenti, per tornare a ciò che si stava scrivendo, bisognerebbe annullare il prompt e ricominciare.
7. **La storia non ricorda tutto.** Non ricorda la stringa vuota, che è come si esce da un prompt, né la ripetizione della voce più recente: lo stesso comando eseguito quattro volte lascia una voce, o tornare indietro di un passo costerebbe quattro tasti. Una ripetizione più vecchia resta dov'è e si aggiunge di nuovo in testa: spostarla riordinerebbe la storia sotto l'utente. Il tetto, cento voci come in Emacs, è per prompt, così un prompt usato di continuo non spinge fuori le voci di un altro. Ai due capi di un anello il prompt resta com'è, invece di ricevere una copia di sé: il tasto deve poter dire «sei alla fine».
8. **Anello e flag di `KillYank` stanno insieme.** L'anello da solo sarebbe un campo. Ne fanno un compartimento le due domande che Emacs fa a ogni kill e a ogni yank: il comando precedente ha ucciso? ha incollato? Una serie di `C-k` deve fare una voce sola, e `yank-pop` deve sapere quale testo ha appena inserito uno yank. Sotto lo stesso lock «accodare o aggiungere» e «flag ed estensione dello yank» sono ciascuno un'operazione sola.
9. **Due flag per domanda.** Un comando alza `killed_this` mentre gira, e `roll_over` lo sposta in `killed_last` quando è finito; lo stesso per lo yank. Con un flag solo, un comando non potrebbe chiedere che cosa ha fatto il precedente senza vedere anche ciò che ha appena fatto lui. Le due coppie si spostano insieme, perché un comando chiede di tutte e due.
10. **Il rettangolo ucciso non sta nell'anello.** Un rettangolo è una forma, e l'anello contiene testo: con un rettangolo nell'anello ogni yank dovrebbe chiedersi che forma ha, e un blocco copiato da un terminale non si distinguerebbe dalle righe di cui è fatto. È un posto e non un anello suo, perché non c'è uno `yank-pop` dei rettangoli; e due kill di rettangolo di seguito sono due rettangoli, non un blocco che sullo schermo non c'è mai stato.
11. **La lista del log e il suo file sono una cosa sola.** Il file è lo specchio della lista: abilitarlo scrive tutto ciò che è stato registrato fino a quel momento, così il file contiene la storia intera e non solo ciò che è venuto dopo. Funziona solo se lista e file si sistemano insieme, sotto lo stesso lock: altrimenti una riga registrata in mezzo da un altro thread arriverebbe troppo tardi per la storia e troppo presto per il file.
12. **Il file del log non lo scrive chi registra.** Registrare aggiunge una riga e la conta come non scritta, e basta: niente disco, quindi registrare costa lo stesso con il disco pieno o con la directory di casa in rete, e nessun comando, nessun passo dello scheduler aspetta una scrittura. Scrive `Log::flush`, che il watchdog chiama a ogni giro, un quarto di secondo per impostazione: su un thread che non è bloccato quando il resto lo è, che è quando un log serve. Un'ultima volta alla chiusura.
13. **`Log` prende il suo lock da sé e non va mai in panic.** È il solo compartimento con funzioni che ricevono il proprio `RwLock`, perché il WorkManager registra senza tenere un editor (Sezione 11). Una scrittura che fallisce si perde, lasciando un buco nel file, e un lock avvelenato si usa così com'è: `flush` gira sul watchdog, che riporta i panic degli altri e non deve averne di suoi — con il disco pieno, salvare un file registra una diagnostica la cui scrittura fallisce a sua volta.
14. **Una macro registra tasti, non comandi.** Non ogni tasto diventa un comando: `C-u` e le cifre li prende il lettore dell'argomento prefisso, e `zap-to-char` legge il suo carattere con una cattura di tasti, senza un secondo comando. Un registratore che guardasse i comandi riprodurrebbe `zap-to-char` come l'attesa di un carattere che non arriva. I tasti seguono poi ciò che significano *ora*: si cambia un legame e la macro fa la cosa nuova, come in Emacs. E si scrivono gratis: `describe_keys` e `parse_key_sequence` sono inversi, quindi una macro è una stringa nella sintassi di `define-key`.
15. **Registrazione e riproduzione stanno insieme.** Finire una registrazione sposta i tasti nel posto dell'ultima macro: separati, un lettore in mezzo vedrebbe due copie o nessuna. E se un tasto si registri è una domanda su registrazione e riproduzione insieme: una macro che ne chiama un'altra registra la chiamata, non ciò in cui si espande. I comandi del registratore (`NOT_RECORDED`) non si registrano, perché i tasti che chiudono una registrazione si premono mentre gira: l'elenco è per nome e sta accanto allo stato, perché quali comandi siano del registratore è una domanda sua.

16. **Le riproduzioni annidate hanno un tetto.** Una riproduzione è una chiamata Rust attraverso il gestore dei tasti, quindi una macro che chiama se stessa fa crescere la pila, che finisce molto prima del carburante — se ne spendono pochi passi per livello — e una pila finita chiude il processo invece di dare un errore. Sedici livelli bastano: l'annidamento legittimo, una macro che chiama una macro con nome che ne chiama un'altra, ne usa due o tre.
17. **Le metà globali stanno con il registro dei modi.** La keymap di un modo e quella globale si consultano in quest'ordine a ogni tasto, e le sorgenti di completamento del modo e quelle globali si fondono a ogni completamento. Separate, chi le legge dovrebbe sapere che i posti sono due e prenderli nell'ordine giusto. La keymap transitoria e i tasti di ripetizione stanno sotto lo stesso lock, così risolvere un tasto prende un lock solo.
18. **Gli hook e le sorgenti si copiano fuori prima di chiamarli.** Le liste di `Modes` contengono espressioni Lisp, e niente in `Modes` le chiama: ogni lettura le copia e rilascia il lock. Un hook che chiama `add-hook`, `define-key`, `make-mode` o `add-syntax-rule` vuole questo lock in scrittura, e con il lock tenuto si bloccherebbe ogni volta, su un thread solo, con un Lisp qualunque.
19. **Il cammino tra le keymap si scrive una volta.** `binding_for`, `is_prefix` e `bindings` rispondono sui legami in effetto accanto alla risoluzione che l'editor esegue, e ne condividono il cammino. Un secondo cammino scritto altrove sarebbe una seconda risposta a «che cosa fa questo tasto», e le due divergerebbero la prima volta che l'ordine cambiasse. Ogni legame porta con sé la mappa da cui viene (`BindingSource`): «`g` esegue `dired-refresh` qui, perché il buffer è in `dired-mode`» è quasi tutto ciò che vuole sapere chi cerca un tasto.
20. **I modi per nome di file sono una lista ordinata.** I modelli si sovrappongono — `\.rs$` e `^Cargo\.` corrispondono entrambi a `Cargo.rs` — e quale vince deve averlo deciso qualcuno, non un hash: vince il primo dichiarato.
21. **I tasti di ripetizione si dichiarano.** La regola che verrebbe in mente, «dopo una sequenza che finisce con K, un K da solo ripete», farebbe di `f` dopo `C-x C-f` un altro `find-file`.
22. **`Runtime` raggruppa per durata, non per un'invariante.** Gli altri compartimenti tengono insieme un fatto in più pezzi; questi campi no: nessuna operazione ne tocca due, e metterli insieme non chiude nessuna finestra di disaccordo. Il guadagno è più piccolo: la facciata non porta otto lock indipendenti, e chi cerca lo stato dell'editor, distinto da quello di un documento, ha un posto solo. Il tema sta qui in mancanza di meglio: è dell'interfaccia, e si sposterà quando ci sarà un compartimento suo.
23. **Il carburante non sta sotto il lock di `Runtime`.** `FuelMeter` si sincronizza da sé: il budget vivo è una variabile del thread, e l'atomico dietro si legge solo quando si apre uno scope. Si tiene come `Arc` e si dà per clone, così chi apre uno scope, che dura un comando intero, non tiene il lock attraverso l'interprete.
24. **Le finestre non leggono i buffer.** Una finestra nomina un buffer e non sa altro. Dove un'operazione ha bisogno di qualcosa del buffer — lo scorrimento il numero di righe, `C-l` la riga del punto — riceve il numero: i due compartimenti non si tengono mai insieme, e le operazioni si provano senza un editor. Per lo stesso motivo un clic risponde con una colonna dello schermo, che `editor/mouse.rs` traduce in una colonna del buffer: una riga con testo virtuale è più larga della sua riga di testo.
25. **I doppi clic li conta l'editor.** Un terminale manda una pressione per volta e nient'altro: «questa è la seconda nello stesso punto» lo sa solo chi tiene la prima, e deve essere l'editor, perché in un front-end sarebbe un calcolo da rifare in ogni front-end. Due clic sono una coppia nella stessa cella entro 400 ms, un po' meno del `double-click-time` di Emacs. La stessa cella e non una vicina, perché un clic subito accanto al precedente sposta il cursore di un carattere, e leggerlo come doppio sarebbe l'errore più fastidioso. Dopo una coppia il primo clic si dimentica: tre pressioni sono una coppia e una singola.
26. **Un trascinamento si ricorda, e scorre a tempo.** È l'unico gesto del mouse che non è un evento solo: dove è cominciato decide che cosa significano gli eventi dopo, ovunque sia andato il puntatore. Un puntatore fermo fuori dalla finestra non manda eventi, ma la selezione deve continuare: per questo lo scorrimento durante il trascinamento è un timer di 60 ms, non una risposta al movimento.
27. **Una striscia divide il frame e non prende il focus.** `open_bottom` apre una finestra alta esattamente quanto chiesto sotto tutte le altre, che cedono ciascuna una parte dello spazio: una striscia è del frame,

non di una finestra tagliata a metà. Il focus resta dov'è, perché la striscia si mostra mentre si scrive altrove — i completamenti sotto un prompt — e il focus che si spostasse renderebbe corrente il suo buffer: il tasto successivo finirebbe nella lista dei suggerimenti.

28. **I float sono nel giro delle finestre, ma non nel loro conto.** `all_ids` mette i float dopo le finestre affiancate, dal più vecchio, perché un float da cui il focus è uscito sarebbe altrimenti irraggiungibile. `count` conta solo le affiancate, perché decide se una finestra si può chiudere e che cosa annulla `C-x 1`, e un prompt disegnato sopra non divide nulla.
29. **Scorrere sposta la vista, e il punto la segue.** `C-v` sposta la vista e il punto la segue solo se ne è uscito, come in Emacs: spostare il punto e lasciare che il disegno lo insegua lo porterebbe al bordo invece di tenergli il posto sullo schermo. Due righe restano in comune tra una schermata e l'altra. La rotella segue la stessa regola nella finestra con il focus, e nelle altre lascia il punto dov'è: il punto è del buffer, e un'altra finestra può mostrare lo stesso buffer. La rotella porta l'ultima riga fino in cima, `C-v` solo fino in fondo: la prima guarda la fine di un file, il secondo lo legge.
30. **La facciata dà ai compartimenti nomi da editor.** `is_command` inoltra a `Commands::is_command`, e sembra un doppione; ma per lo più rinomina: `Commands::specs` è chiaro perché il tipo dice di che cosa parla, mentre `EditorState`, che ha anche nomi di buffer e id di finestre, ha bisogno di `command_specs`. Togliere gli inoltri metterebbe in ogni chiamata quale compartimento tiene che cosa, e spostare un campo tra due compartimenti cambierebbe ogni chiamante invece di un inoltro.
31. **Un inoltro sta con il suo compartimento.** Chi cerca che cosa l'editor offre su un compartimento lo trova in un file, e chi scrive un inoltro nuovo sa dove metterlo, chiunque sia a usarlo. I file di coordinamento restano fatti di coordinamento.
32. **Una porta sola tiene due compartimenti.** `snapshot` compone ogni finestra contro la tabella dei buffer, e ne sposta lo scorrimento: ha bisogno di `windows` in scrittura e di `buffers` insieme. La porta che li apre entrambi li prende nell'ordine dei lock, scritto così in un posto solo (Sezione 4.5).
33. **Il contatore delle grammatiche sta accanto al registro, non dentro.** Una colorazione porta il timbro della grammatica sotto cui è stata calcolata, e aggiungere una regola non cambia il testo: senza il contatore nessuno se ne accorgerebbe. Ogni fotografia chiede a ogni buffer se la sua colorazione è indietro, e un atomico fuori dal registro risponde senza prenderne il lock. Si sposta solo sotto il lock in scrittura del registro, dopo il cambio della grammatica, e lo si legge sotto quel lock insieme alla grammatica: le due cose non si vedono mai in disaccordo. Una tabella sintattica che cambia emette `syntax-changed` e non sposta il contatore: è dell'altro lessico.
34. **Il messaggio e il suo istante stanno sotto un lock solo.** Con due lock, chi legge potrebbe accoppiare un messaggio nuovo all'istante del vecchio e nasconderlo appena scritto. Ogni messaggio fa ripartire il suo tempo, e un messaggio scaduto resta nello stato: è la vista che lo dimentica.
35. **Le callback dovute non sono un compartimento.** Ogni compartimento non sa nulla del Lisp, e quella lista ne è fatta: metterla dietro un compartimento romperebbe la regola, o chiederebbe una seconda rappresentazione di una chiamata, inventata per non dire che cos'è. Sta accanto a `current_results`, come stato che la facciata tiene da sé.
36. **Finestra con il focus e buffer corrente cambiano insieme.** Lo fanno in due modi soli: `show_in_focused_window` cambia il buffer e lascia il focus, `set_focused_window_id` sposta il focus e rende corrente ciò che la finestra mostra, e ogni spostamento del focus finisce in `follow_focus`, anche quando è il compartimento a spostarlo. Se i due andassero alla deriva, l'editor disegnerebbe il cursore in una finestra e scriverebbe in un'altra.
37. **In un buffer in sola lettura si scrive sotto il suo lock.** Una trascrizione — l'uscita di un comando di shell, i risultati di una ricerca — è in sola lettura perché nessuno ci scriva. `append_to_buffer` scrive con `Buffer::writable`, che toglie `read-only-mode` e lo rimette dentro un solo lock in scrittura: non c'è un momento in cui il buffer è visibile e scrivibile. `dired` fa lo stesso dal Lisp, dove non può tenere un lock e deve fidarsi che nulla giri in mezzo.
38. **Un file che non c'è si apre vuoto e non modificato.** `new_buffer` con un percorso legge il file e fallisce se non può; un file che non esiste non è un errore, è il modo comune di crearne uno.

`new_file_buffer` fa un buffer vuoto che ricorda dove andrà, non modificato: lasciarlo non perde nulla, e `C-x C-s` crea il file anche se non si è scritto niente.

39. **Un buffer chiuso con modifiche non salvate lascia la sua copia.** La copia del salvataggio automatico si butta quando ciò che teneva è nel file: a un salvataggio, e alla chiusura di un buffer senza modifiche. `close_buffer` non chiede nulla, e non chiedono `close-buffer` e `kill-buffer-without-saving` che lo chiamano: la copia può essere tutto ciò che resta del lavoro.

5 Dal tasto al comando

5.1 Il vocabolario dei tasti

Un `KeyEvent` è un `KeyCode` (`Char(c)`, `Backspace`, `Enter`, `Esc`, `Tab`, le quattro frecce, `Delete`, `Insert`, `Home`, `End`, `PageUp`, `PageDown`, `F(n)` per i tasti funzione, e `None`, che nessuno costruisce) più i modificatori `ctrl`, `alt`, `shift`. Un `MouseEvent` dice che cosa ha fatto il mouse (`MouseKind`: un pulsante premuto, rilasciato o trascinato, la rotella), in quale cella del frame e con quali modificatori. Sono i tipi che un front-end costruisce e consegna a `handle_key_event` e a `handle_mouse_event`.

Nelle associazioni i tasti si scrivono come in Emacs: `C-x`, `M-x`, `C-M-f`, `S-<tab>`, con i nomi `<ret>`, `<esc>`, `tab`, `<backtab>`, `<backspace>`, `<space>`, `<left>`, `<right>`, `<up>`, `<down>`, e quelli di Emacs per gli altri: `<delete>`, `<insert>`, `<home>`, `<end>`, `<prior>` e `<next>` per le pagine (accettati anche `<pageup>`, `<pagedown>` e `<deletechar>`), da `<f1>` a `<f35>`. I legami di base li legano come i tasti di Emacs che fanno lo stesso (`common-keymaps.lisp`), e vim, nello stato normale, come i suoi. I modificatori `C-`, `M-` e `S-` si leggono in un ciclo, in qualunque ordine e ognuno una volta: `M-C-x` è `C-M-x`, e `C-C-x` non è un tasto. Con un carattere `S-` è la sua maiuscola, perché così arriva un carattere con Shift (`C-S-a` è `C-A`), e con un carattere che non ne ha non è un tasto; con un tasto con nome, `S-<tab>`, è Shift premuto. `define-key` dà un errore Lisp per un tasto che non si legge e per un modo che non c'è, invece di rispondere nil con una riga nel log. Una sequenza è una stringa di tasti separati da spazi: `"C-x C-f"`. `parse_key_sequence` legge queste stringhe e `describe_keys` le riscrive nello stesso modo; stanno in `input.rs`, con i tipi.

La keymap globale scritta in Rust (`fill_default_keymaps`) contiene pochissimo: `M-x` (`command-execute-prompt`), le frecce, e `self-insert` che inserisce ogni carattere premuto da solo, ASCII o no. Tutto il resto — `C-x C-f`, `C-s`, `C-k`, … — viene da `common-keymaps.lisp` e dagli altri moduli.

5.2 Le keymap

Una `Keymap` associa sequenze di tasti a espressioni Lisp: un nome di comando (`forward-char`), una forma che porta già i suoi argomenti (`(self-insert "x")`) o una lambda. Il tipo è `ELispExp`, che `run_command_form` esegue così com'è; ed è l'unica cosa che `input.rs` prende da più in alto, perché il suo tipo nomina `EditorState`.

da `input.rs` — estratto semplificato

```rust
pub struct Keymap<B: BufferTrait> {
    bindings: HashMap<Vec<KeyEvent>, ELispExp<B>>,  // la sequenza intera
    prefixes: HashSet<Vec<KeyEvent>>,               // ogni prefisso proprio
    character_handler: Option<ELispExp<B>>,         // il ripiego dei caratteri
}
pub fn insert(&mut self, keys: Vec<KeyEvent>, ast: ELispExp<B>) {
    for len in 1..keys.len() {
        self.prefixes.insert(keys[..len].to_vec());
    }
    self.bindings.insert(keys, ast);
}
```

Ogni tasto chiede due cose a una keymap: se la sequenza è un legame (`get`) e se è l'inizio di uno (`is_prefix`). Tutte e due sono la ricerca in una tabella hash: `insert` registra i prefissi propri della sequenza insieme al legame, e `remove` ricostruisce l'insieme dai legami che restano. `binding` aggiunge il ripiego: un carattere senza modificatori che non è un legame né l'inizio di uno va al `character_handler`, se c'è; un simbolo diventa `(simbolo "c")`, ogni altra funzione `(funcall 'f "c")`. `set_character_handler` lo installa, ed è ciò che fa `(define-key MAPPA "<char>" ...)`; con `nil` lo toglie.

Le keymap stanno nel compartimento `Modes`: la globale, quella di ogni modo e la *transitoria*, consultata prima delle altre finché resta installata. Una `TransientKeymap` è una `Keymap`, il messaggio da mostrare

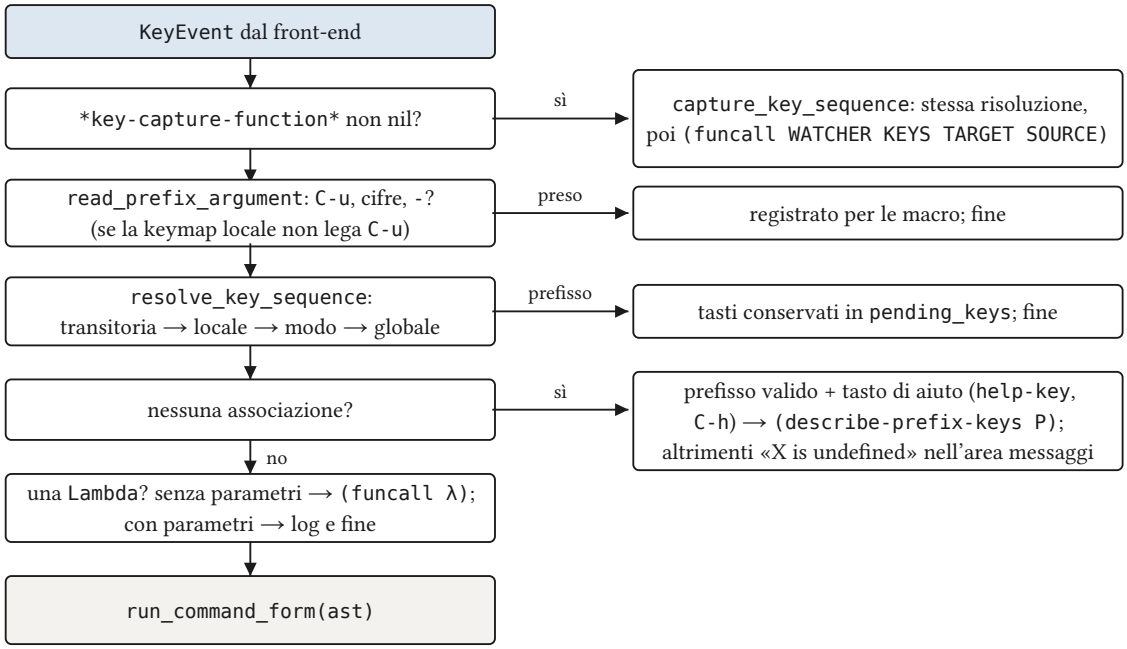
mentre è attiva e che cosa fare di un tasto che non lega (`OnUnbound`, nella tabella della risoluzione). La installano le domande a un tasto (`y-or-n`, `save-some-buffers`), la vista predefinita di `query-replace`, la striscia dei completamenti e la ripetizione: `TransientKeymap::repeating` costruisce la mappa `[o]`, che `install_repeat_keymap` rimette dopo ogni comando con un tasto di ripetizione.

Da Lisp le keymap si cambiano con le primitive di `primitives/keymaps.rs`: `define-key`, `set-local-keymap`, `set-transient-keymap` e `clear-transient-keymap`, `define-repeat-key`. Lì sta anche `repeat`, `C-x z`, che valuta la forma dell'ultimo comando dentro il comando che la ripete: ne eredita il gruppo di undo e gli hook, invece di aprirne altri dentro i suoi.

5.2.1 Le decisioni

1. **I prefissi si conservano, non si cercano.** Ogni tasto chiede se la sequenza è l'inizio di un legame. Rispondere scorrendo i legami vorrebbe dire scorrere tutta la mappa, centinaia di voci per i soli caratteri, sul percorso tra il tasto e il carattere che compare. L'insieme cambia solo quando si definisce un legame, all'avvio o quando l'utente cambia la sua configurazione; la domanda si fa a ogni tasto.
2. **Legame e prefissi si registrano insieme.** Un legame senza i suoi prefissi sarebbe irraggiungibile: `C-x` da solo risulterebbe non definito, e la sequenza verrebbe buttata prima di arrivare a `C-x C-f`. Per questo i campi sono privati e `insert` li scrive tutti e due: lo stato sbagliato non si può costruire, invece di essere solo sconsigliato. Togliere un legame è raro, e `remove` ricostruisce i prefissi invece di contarli.
3. **Domande e offerte sono lo stesso meccanismo.** «Premi `y` o `n`» e «premi ancora `o`» sembrano due funzioni diverse, ma sono una keymap consultata per prima che poi se ne va: cambia solo `OnUnbound`. Essere una keymap, invece di un ciclo che legge le risposte, fa delle risposte comandi normali: si legano come tutto il resto, l'aiuto li descrive, e ognuno è un comando a sé per l'undo.
4. **Una mappa `Release` non ha un'uscita.** Se ne va dentro la risoluzione del tasto, mentre la sequenza è ancora sotto lock, dove non si può chiamare l'interprete. Una mappa con una sessione dietro lo dice in `owner`, e la sessione si chiude dopo il tasto, fuori dalla risoluzione, quando la sua mappa non è più quella in cima (`end_replace_left_behind`): è il caso della domanda di `query-replace`, un'offerta che un altro tasto può lasciare.

5.3 `handle_key_event`



I tasti vengono registrati per la macro di tastiera in corso anche quando non eseguono nulla: una macro deve riprodurre ciò che è stato premuto, compresi `C-u 5` e i tasti non associati.

Il front-end chiama `handle_key_event` solo per gli eventi di pressione; l'incolla arriva da `handle_paste` come un unico comando `(insert-pasted-text TESTO)`, che passa anch'esso da `run_command_form`.

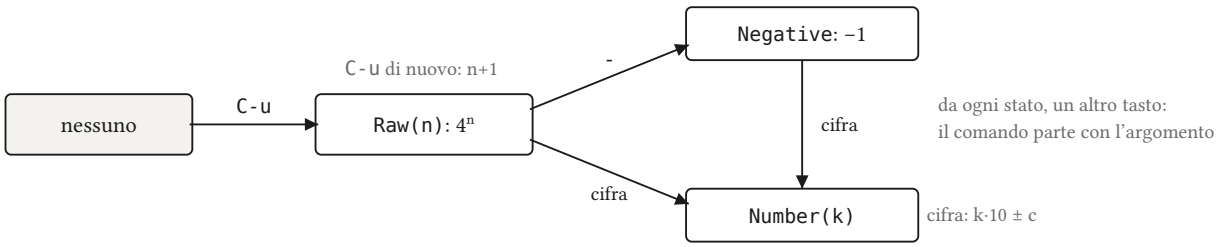
In codice, la funzione è una sequenza di uscite anticipate:

da `editor/keys.rs` — estratto semplificato

```
pub fn handle_key_event(&self, event: KeyEvent, env: &Arc<Env<EditorState<B>>>) {
    if let Some(watcher) = env.get_variable(KEY_CAPTURE_FUNCTION) && watcher.is_truthy() {
        self.record_keys(&[event.clone()], None);
        self.capture_key_sequence(event, watcher, env);     // read-key-sequence
        return;
    }
    if !(Commands::is_prefix_argument_key(&event) && self.local_key_claims(&event))
        && self.read_prefix_argument(&event)                // C-u, cifre, -
    {
        self.record_keys(&[event], None);
        return;
    }
    let mut sequence = self.pending_key_events();
    sequence.push(event.clone());
    let mut ast = match self.resolve_key_sequence(event.clone()) {
        Dispatch::Run(ast) => { self.record_keys(&sequence, command_named_by(&ast)); ast }
        Dispatch::Nothing  => return,                       // un prefisso: si aspetta
        Dispatch::Unbound { described, prefix, last } => {
            /* prefisso + tasto di aiuto → describe-prefix-keys; altrimenti
               "X is undefined" nell'area messaggi */
            return;
        }
    };
    if let ELispExp::Lambda(ref lambda) = ast {             // un tasto legato a una lambda
        if !lambda.params.is_empty() { /* log */ return; }
        ast = ELispExp::form(vec![ELispExp::symbol("funcall"), ast]);
    }
    self.run_command_form(ast, env);
}
```

5.4 L'argomento prefisso

`C-u` e le cifre non arrivano alle keymap: `Commands::read_prefix_argument` li prende per primo, a meno che la keymap locale del buffer (`set-local-keymap`) non leghi `C-u` lei stessa (`local_key_claims`). Lo stato è la coppia `prefix_arg: Option<PrefixArg>` e `reading_prefix: bool`.



Un tasto qualunque che non fa parte dell'argomento chiude la lettura (`reading_prefix = false`) ma lascia l'argomento al comando che segue. Il comando lo legge tramite `call-interactively` (le specifiche `p` e `P`); dopo *ogni* comando, riuscito o fallito, `clear_prefix_argument` lo cancella. Non esistono le forme `M-4` o `C-4` di Emacs. Mentre l'argomento e la sequenza sono a metà, `pending_input()` li descrive (`C-u 4 C-x-`) e la TUI li mostra a destra nella riga dei messaggi.

I caratteri sono legati a forme che portano già il loro argomento, `(self-insert "x")`; una forma così non passa da `call-interactively`, e `run_command_form` le aggiunge l'argomento prefisso come conteggio: `(self-insert "x" 3)`, e `C-u 3 x` inserisce tre «x», in una sola modifica. Solo alla forma che un tasto esegue: un `self-insert` chiamato da un altro comando non è moltiplicato da un conteggio dato a quel comando. Un conteggio negativo è un errore, come in Emacs.

5.5 La risoluzione: `resolve_key_sequence`

Il tasto si aggiunge a `pending_keys`; poi, con un solo lock di lettura su `modes`, la sequenza viene cercata nelle keymap, in ordine:

1. la *keymap transitoria*, se installata;
2. la keymap locale del buffer corrente, se `set-local-keymap` gliene ha data una;
3. le keymap dei modi minori accesi nel buffer, dall'ultimo acceso (Sezione 10.2);
4. la keymap del modo maggiore del buffer corrente;
5. la keymap globale.

da `editor/keys.rs` — estratto semplificato

```
let mut pending = self.pending_keys.write().unwrap();
pending.push(event);
let names = self.keymap_modes();     // la locale, se c'è, i minori, il modo
let current_modes: Vec<&str> = names.iter().map(String::as_ref).collect();
let bound = self.modes(|modes| {
    if let Some(map) = modes.transient() {
        match (
            map.keymap.get(&pending).cloned(),
            map.keymap.is_prefix(&pending),
            map.on_unbound,
        ) {
            (Some(ast), _, _) => return Bound::Command(ast),
            (None, true, _)   => return Bound::Prefix,
            (None, false, OnUnbound::Refuse)  => return Bound::Refused,
            (None, false, OnUnbound::Pass)    => {}
            (None, false, OnUnbound::Release) => release_transient = true,
        }
    }
    // la locale, poi i modi minori, poi il modo, poi la globale
    let hit = modes.binding_below_transient(&current_modes, &pending);
    let prefix = modes.is_prefix_below_transient(&current_modes, &pending);
    match (hit, prefix) {
        (Some(binding), _) => Bound::Command(binding.target),
        (None, true)  => Bound::Prefix,
        (None, false) => Bound::Unbound,
    }
});
if !matches!(bound, Bound::Prefix) { pending.clear(); }
```

Una sequenza è cercata *intera*: `pending_keys` contiene già i tasti precedenti, e la tabella di una keymap associa sequenze di tasti, non tasti singoli. Un risultato `Prefix` lascia la sequenza in `pending_keys`, in attesa del tasto successivo; ogni altro risultato la svuota. Un modo che lega un prefisso tiene viva la sequenza anche se solo la globale la completa. Le stesse due funzioni (`binding_below_transient`, `is_prefix_below_transient`) rispondono anche a `key-binding` e ai comandi di aiuto, così ciò che l'aiuto riporta e ciò che l'editor esegue non possono divergere.

La keymap transitoria (`TransientKeymap`) è installata da una primitiva o dal meccanismo di ripetizione e dice che cosa fare di un tasto che non conosce:

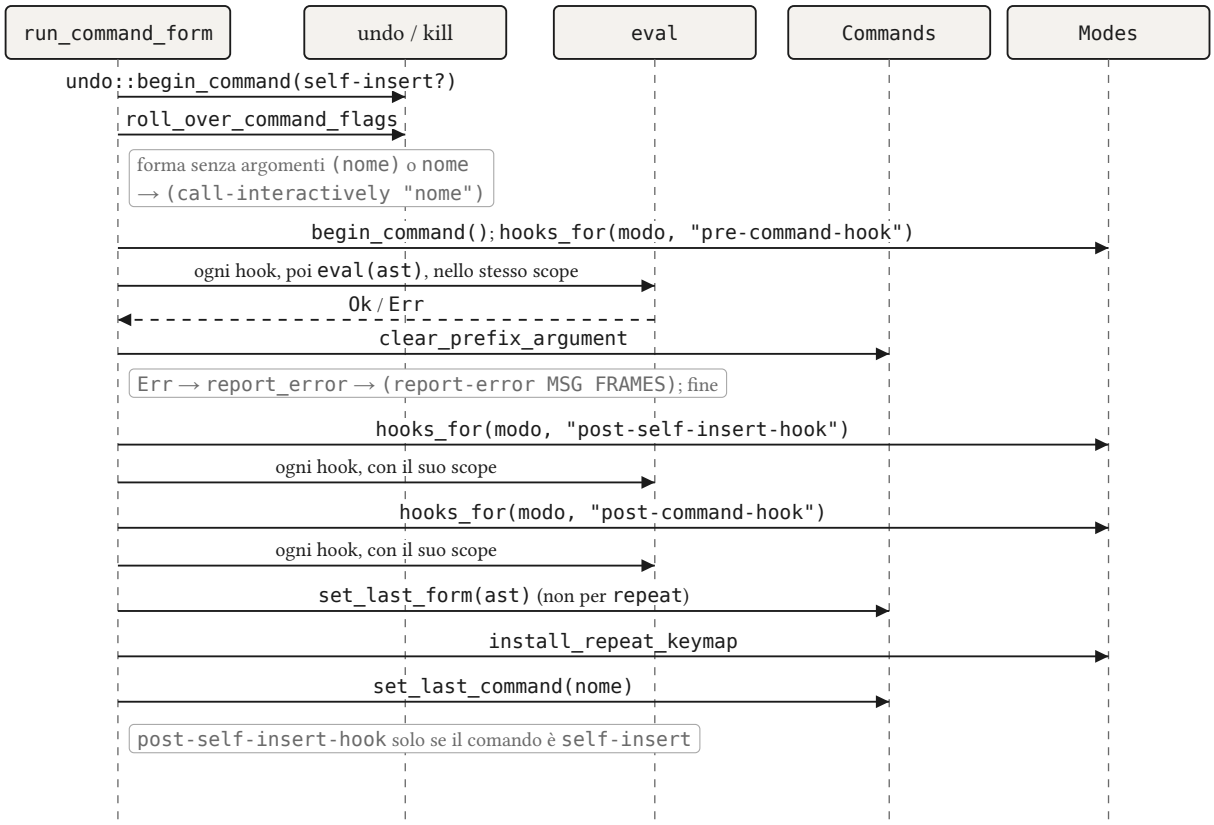
`OnUnbound`	Comportamento	Uso tipico
`Refuse`	Il tasto è rifiutato in silenzio; la mappa resta. Il messaggio della mappa occupa la riga dei messaggi.	Una domanda (`y-or-n`, `save-some-buffers`): si risponde solo con i suoi tasti.
`Pass`	Il tasto prosegue verso le altre keymap; la mappa resta.	La striscia dei completamenti: si può continuare a scrivere.

`OnUnbound`	Comportamento	Uso tipico
`Release`	Il tasto prosegue; la mappa viene tolta.	La ripetizione (`C-x o o o`): un'offerta che si può ignorare. Anche la vista predefinita di `query-replace`, la cui sessione si chiude con lei.

Dopo ogni comando, `install_repeat_keymap` controlla se il comando ha un *tasto di ripetizione* dichiarato (`define-repeat-key`) e in tal caso installa una mappa `Release` che lo associa al comando stesso.

5.6 `run_command_form`: l'esecuzione di un comando

Tutto ciò che il tasto, l'incolla, il mouse, la cattura e la riproduzione di macro fanno eseguire passa da qui.



da `editor/commands.rs` — estratto semplificato

```
pub(super) fn run_command_form(&self, mut ast: ELispExp<B>, env: &Arc<Env<EditorState<B>>>) {
    let bound_command = /* Some(nome) se ast è `nome` o `(nome)`, cioè senza argomenti */;
    let this_command  = /* il simbolo in testa, se c'è */;
    undo::begin_command(this_command == Some("self-insert"));     // nuova epoca
    self.roll_over_command_flags();                                // kill ring
    if let Some(name) = bound_command {
        ast = ELispExp::form(vec![
            ELispExp::symbol("call-interactively"), ELispExp::string(name),
        ]);
    }
    let outcome = {
        let _command = self.begin_command();
        let mode = self.with_current_buffer(|buf| buf.current_mode.clone());
        self.run_hook(&mode, "pre-command-hook", env);
        eval(&ast, env.clone(), self)
    };
    self.clear_prefix_argument();
    if let Err(e) = outcome {
        self.report_error(&format!("{:?} {:?}", ast, e), env); return;
    }
    let mode = self.with_current_buffer(|buf| buf.current_mode.clone());
    if this_command == Some("self-insert") {
        self.run_hook(&mode, "post-self-insert-hook", env);
    }
    self.run_hook(&mode, "post-command-hook", env);
    if this_command != Some("repeat") { self.commands_mut(|c| c.set_last_form(Some(ast))); }
    self.install_repeat_keymap(this_command);
    self.set_last_command(this_command);
}
```

1. *Undo.* `undo::begin_command` assegna al thread una nuova epoca; tutte le modifiche del comando finiranno in un solo gruppo (Sezione 8.4).
2. *Flag del kill ring.* «Il comando precedente ha ucciso» diventa vero o falso *prima* che il comando parta, così una serie di `C-k` si accumula in una sola voce del kill ring e un comando che fallisce non lascia il flag acceso.
3. *Instradamento.* Una forma senza argomenti diventa una chiamata a `call-interactively` con il nome del comando, così l'editor raccoglie gli argomenti dichiarati. Una forma che porta già i suoi argomenti — `(self-insert "a")` — resta com'è: il percorso della digitazione non paga nulla.
4. *Esecuzione.* `pre-command-hook`, poi `eval`, dentro lo stesso scope di carburante: l'hook spende il budget del comando.
5. *Argomento prefisso* cancellato, anche se il comando è fallito.
6. *Errore.* `report_error` incrementa `command_errors`, prende la pila delle chiamate come backtrace, la svuota e chiama la funzione Lisp `report-error` (definita in `debug.lisp`: messaggio nell'area messaggi e, se `debug-on-error`, una finestra `*Backtrace*`). Senza quella funzione scrive nel log. Dopo un errore non girano gli hook e `last-command` non cambia.
7. *Hook.* Prima `post-self-insert-hook` (solo per `self-insert`: qui si aggancia `electric-pair`), poi `post-command-hook`. Ogni lista è la concatenazione degli hook del modo e di quelli globali, copiata con un solo lock; un hook che fallisce viene registrato nel log e non ferma gli altri.
8. *Memoria.* La forma eseguita diventa quella di `repeat`; si offre il tasto di ripetizione; `last-command` riceve il nome.

5.7 La cattura di una sequenza

`read-key-sequence` arma `*key-capture-function*`. Il tasto successivo segue la stessa risoluzione di un tasto normale — la sequenza a metà si vede come sempre — ma al posto di eseguire il comando viene chiamata la funzione con la sequenza, la forma che sarebbe stata eseguita (citata) e il nome della keymap

che la fornisce, o `"prefix-argument"` per `C-u`. È così che `describe-key` e `zap-to-char` leggono un tasto: non esiste in rsedit un «leggi un tasto» che blocchi, perché i tasti arrivano uno per giro del ciclo degli eventi.

5.8 Le decisioni

1. **Un percorso solo esegue un comando.** Il tasto, l'incolla, il mouse, la cattura e la macro passano tutti da `run_command_form`: un secondo percorso andrebbe tenuto in passo con il primo a mano, per l'undo, gli hook e `last-command`.
2. **Un tasto che allunga una sequenza non è un comando.** `resolve_key_sequence` risponde invece di eseguire. `C-x` non deve chiudere il gruppo di undo in cui si sta scrivendo, né spezzare in due voci una serie di `C-k`: sono cose che succedono tutte dopo il punto in cui la risoluzione restituisce.
3. **La sequenza a metà sta nell'editor, non in un modo.** Una keymap di modo si consulta prima della globale, e una sequenza cominciata sotto l'una non deve restare a metà nell'altra. `pending_keys` è un vettore che si riempie e si svuota, così il tasto che è da solo un legame non alloca nulla.
4. **La cattura segue la risoluzione vera.** `read-key-sequence` non intercetta i tasti prima della risoluzione: li accumula nella stessa sequenza, così un `C-x` a metà si vede come sempre, e devia solo al posto dell'esecuzione. Ciò che fa un tasto ha una sola risposta vera, quella su cui l'editor agirebbe; una seconda implementazione delle precedenze riporterebbe quella vecchia il giorno in cui l'ordine cambia.
5. **Armare la cattura è una variabile.** `*key-capture-function*` è un fatto in un posto solo, che il Lisp legge e imposta come ogni variabile, al prezzo di una ricerca per tasto; nil quando nessuno cattura. Si disarma prima di chiamare la funzione: una funzione che apre un buffer o fallisce non deve lasciare catturato anche il tasto dopo.
6. **Il tasto di aiuto dopo un prefisso è una domanda.** Una sequenza non associata, il cui ultimo tasto è l'ultimo di `help-key` e il cui resto è un prefisso vivo, chiama `describe-prefix-keys`; senza un modulo di aiuto è indefinita come ogni altra. La variabile si legge solo su una sequenza già non associata, mai su un tasto digitato.
7. **Un incolla è un comando solo.** Il terminale dice che un incolla è un incolla. Digitato tasto per tasto sarebbe annullato un carattere alla volta, e `electric-pair` accoppierebbe di nuovo ogni parentesi: arriva invece come `insert-pasted-text`, con un gruppo di undo, un `post-command-hook` e nessun `post-self-insert-hook`.
8. **Dopo un comando senza tasto di ripetizione non si toglie nulla.** Una mappa `Release` l'ha già tolta la risoluzione, passando il tasto oltre; una `Refuse`, una domanda, risponde con i suoi comandi, e `install_repeat_keymap` gira dopo ognuno: toglierla lì la toglierebbe appena ha avuto risposta.

6 Comandi e argomenti

6.1 Che cos'è un comando

Per l'interprete un comando non esiste: è una funzione qualunque. Diventa un comando quando ha una voce nel registro di `Commands` (`register_command`), che associa al nome l'elenco degli argomenti da chiedere. Una primitiva diventa comando registrandosi con `into.command(nome, puntatore, specifiche, DOC)` invece che con `into.function`; una funzione Lisp con la macro `defcommand` di `commands.lisp`:

```
(defcommand greet (who) ("sGreet whom: ")
  "Say hello to WHO."
  (message "Hello, %s" who))
```

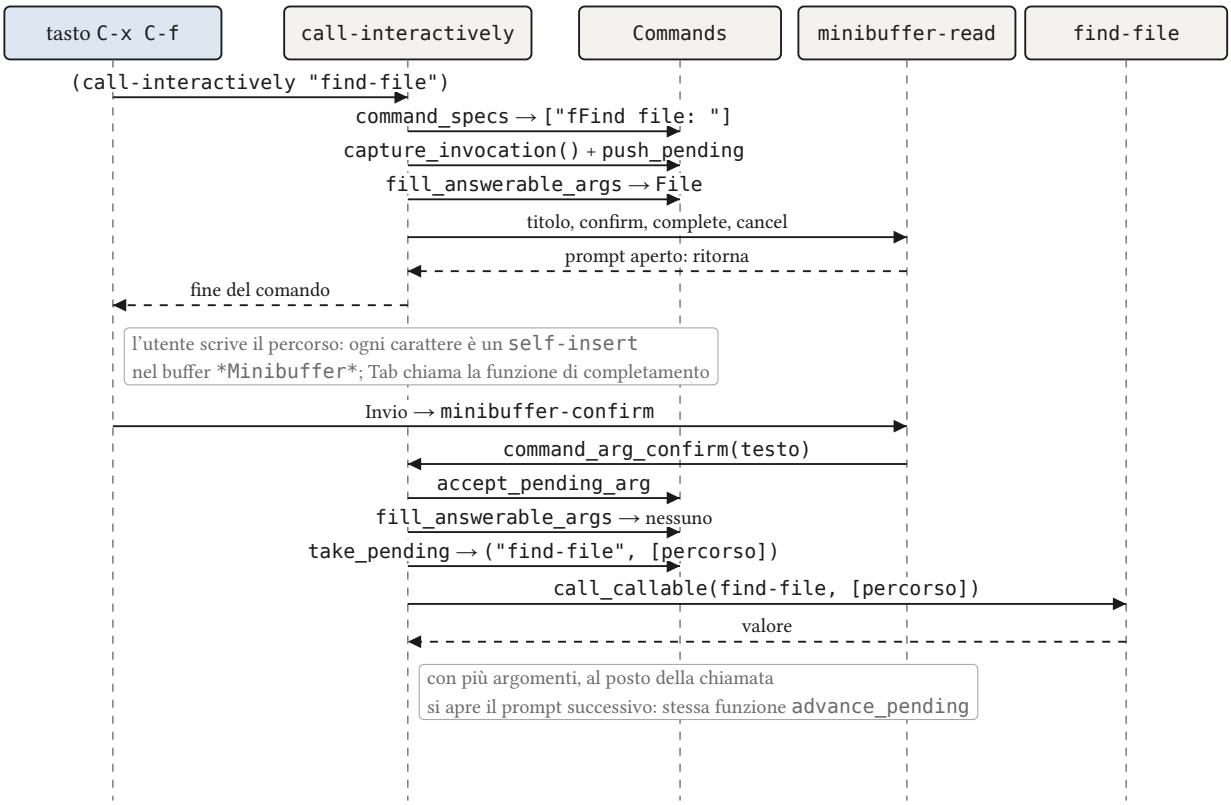
`M-x` completa solo sui nomi registrati; una funzione non registrata si può comunque legare a un tasto, e viene chiamata senza argomenti. Il registro, `CommandRegistry`, è una tabella dal nome del comando alle sue specifiche, già analizzate (`ArgSpec::parse`). Le primitive del registro, `call-interactively` e `M-x` sono in `primitives/commands.rs`.

6.2 Le specifiche degli argomenti

Codice	`ArgSpec`	Valore passato al comando
`s`	`String`	Il testo digitato nel prompt.
`n`	`Number`	Il testo convertito in numero; un testo non numerico viene chiesto di nuovo.
`b`	`Buffer`	Un nome di buffer, con completamento sui buffer vivi (escluso `*Minibuffer*`).
`f`	`File`	Un percorso, completato un componente alla volta.
`p`	`Count`	L'argomento prefisso come numero, 1 se assente. Non chiede nulla.
`P`	`RawCount`	L'argomento grezzo: `nil`, `(4)` per `C-u`, un numero, o il simbolo `-`. Non chiede nulla.
`r`	`Region`	*Due* valori, inizio e fine della regione attiva. Senza regione il comando è rifiutato prima di aprire qualunque prompt.

6.3 `call-interactively` e la catena degli argomenti

Il minibuffer non blocca: apre un prompt e ritorna, e la risposta arriva con un tasto successivo. Raccogliere due argomenti non può quindi essere un ciclo: è una catena, e lo stato tra un anello e l'altro sta nella pila dei `PendingCommand` di `Commands`.



`Invocation` — argomento prefisso e regione — è catturata quando il comando inizia (`capture_invocation`), non quando l'ultimo argomento arriva, e risponde agli argomenti `p`, `P` e `r` di tutta la catena.

Il titolo del prompt dice chi sta chiedendo: `find-file - Find file:` o, con più argomenti, `query-replace (2/2) - With:`. Il prompt viene aperto chiamando `minibuffer-read` per nome, così un minibuffer alternativo serve anche gli argomenti dei comandi. Annullare il prompt (`command_arg_cancel`) toglie il comando dalla pila: un elenco di argomenti incompleto non arriva mai a un comando. Un nuovo `call-interactively` senza minibuffer aperto svuota la pila dagli orfani rimasti da prompt chiusi in altro modo.

Il completamento degli argomenti `b` e `f` è fatto in Rust; una funzione in `*command-arg-completion-function*`, se impostata, lo sostituisce per tutti i tipi ricevendo il tipo come simbolo (`buffer`, `file`, …) e il prefisso.

6.4 Le decisioni

1. **Un comando è un'idea dell'editor, non dell'interprete.** Un comando è qualcosa che si raggiunge con `M-x` o con un tasto, e di cui l'editor sa chiedere gli argomenti: un nome di file con il completamento, un buffer, un numero. Per un interprete che non sa di stare in un editor non significa nulla. Così l'interprete non ne sa niente: un comando è una funzione qualunque con una voce nel registro, `(find-file "/tmp/x")` da Lisp è una chiamata come le altre, e né il tipo `Lambda` né `eval` devono sapere che i comandi esistono.
2. **Le specifiche si analizzano alla registrazione.** Un codice sbagliato, come `"zFind file: "`, diventa un errore all'avvio dell'editor invece di una sorpresa la prima volta che qualcuno esegue il comando. Per la stessa ragione un testo dopo `p`, `P` o `r`, che non chiedono nulla, è un errore e non un dettaglio da ignorare.
3. **Prefisso e regione si fissano quando il comando comincia.** Gli argomenti non arrivano tutti insieme: uno chiesto arriva qualche tasto dopo, da un minibuffer che per aprirsi ha cambiato buffer. Quando arriva l'ultimo, la regione appartiene a un altro buffer e l'argomento prefisso è già stato cancellato per il comando

successivo. All'inizio sono invece ciò che l'utente intendeva: `C-u 4 M-x foo` passa quattro a `foo`, per quanto a lungo resti aperto il prompt.

4. **I comandi in attesa sono una pila.** Un comando può cominciare mentre un altro sta ancora chiedendo i suoi argomenti (`M-x` dentro un prompt), e i due devono chiudersi nell'ordine inverso a quello in cui sono cominciati. Un posto solo perderebbe il primo.

7 Minibuffer e ricerca incrementale

7.1 Il minibuffer

Il meccanismo del minibuffer è in Rust perché troppe cose ne dipendono — `M-x`, `M-:`, gli argomenti dei comandi — per affidarlo a un file Lisp che potrebbe mancare. L'aspetto, invece, è sostituibile: `minibuffer-read` chiama la funzione contenuta in `*minibuffer-read-function*`, che per impostazione predefinita è `default-minibuffer-prompt`. `feature/minibuffer.rs` installa `minibuffer-mode`, con i tasti e gli hook di un prompt, e tiene lo stato del prompt aperto e la storia; le primitive sono in `primitives/minibuffer.rs`.

```
(minibuffer-read PROMPT ON-CONFIRM ON-CHANGE ON-CANCEL [MODE [HISTORY-KEY]])
```

Argomento	Significato
`ON-CONFIRM`	Chiamata con il testo finale quando si preme Invio, *dopo* la chiusura del prompt.
`ON-CHANGE`	Nonostante il nome, è la funzione di completamento: riceve il testo e restituisce i candidati. È chiamata al primo Tab dopo ogni modifica; i Tab successivi scorrono i candidati, o li consegnano a `*completion-read-function*` se impostata (è ciò che fa `completion.lisp`).
`ON-CANCEL`	Chiamata senza argomenti quando si preme Esc.
`MODE`	Il modo del buffer del prompt (predefinito `minibuffer-mode`); `isearch-mode` per la ricerca.
`HISTORY-KEY`	La chiave della storia (`M-p`, `M-n`); per default il testo del prompt, così ogni prompt ha una storia senza chiederla. Fino a `history-length` voci, 100 per default.

`default-minibuffer-prompt` legge MODE e KEY con `optional_name`, per cui nil è assente come un argomento lasciato fuori, salva le callback in variabili globali (`*minibuffer-on-confirm*`, `*minibuffer-on-change*`, `*minibuffer-on-cancel*`, `*minibuffer-previous-buffer*`, …) e apre una finestra flottante con bordo sul buffer `*Minibuffer*`, *al centro* del frame, larga `minibuffer-width` (60) e alta `minibuffer-height` (3) e ridotta se il terminale è più piccolo. Il centro è scelto perché il fondo dello schermo è occupato da altre strisce (i completamenti, la compilazione). Ogni prompt usa lo stesso buffer, `*Minibuffer*` (`PROMPT_BUFFER`): per questo ne è aperto uno alla volta. Un secondo prompt, chiesto mentre il primo è aperto, viene rifiutato con errore, come in Emacs senza enable-recursive-minibuffers. Nel prompt `C-g`, come Esc, è `minibuffer-cancel`: la mappa di `minibuffer-mode` lo lega e vince sul `keyboard-quit` globale, che dei prompt non sa nulla. Mentre la striscia dei completamenti è aperta, la sua mappa transitoria lo prende per prima: il primo `C-g` chiude la striscia, il secondo il prompt. Ciò che Tab, la storia e un completamento scelto mettono nel prompt passa da `set_minibuffer_content`. Scrive nel buffer `*Minibuffer*` per nome, qualunque sia il buffer corrente, e risponde se c'era un prompt in cui scrivere: senza, `minibuffer-choose-completion` risponde nil e non tocca nulla. Sostituisce tutta la risposta, non solo il testo prima del punto, e lo fa dalle due porte, `delete_range` e `insert_text` (Sezione 8.3): la sostituzione è un passo d'annullamento come lo scrivere, e il punto va in fondo alla risposta.

Un worker di sfondo non può aprire un prompt: `minibuffer-read` controlla `in_worker()` e rifiuta con un errore, perché il tasto che risponderebbe arriverebbe in mezzo a ciò che l'utente sta facendo.

7.2 La ricerca incrementale

`C-s` apre un prompt in `isearch-mode` e ricorda dove era il punto. Da lì il lavoro lo fa il `post-command-hook` di quel modo: ogni comando eseguito mentre il prompt è aperto — ogni carattere scritto, ogni cancellazione — provoca un giro di `isearch-update`, che ripete la ricerca e sposta il punto.

Non serve un ciclo «leggi un tasto», che in rsedit non esiste. Le primitive sono in `primitives/isearch.rs`: i quattro comandi che cominciano una ricerca e ciò su cui gira una ricerca aperta. Dove va il punto, che cosa è scritto nel prompt e come si prepara una ricerca sono in `feature/isearch.rs`, con `isearch-mode` e `case_fold`, la sola lettura di `case-fold-search`.

> Mentre il prompt è aperto il buffer corrente è il minibuffer. Tutto ciò che riguarda la ricerca lavora quindi sul buffer cercato *per nome*, preso dallo stato della sessione (`Isearch`) in `Runtime`. Una modifica che usasse il buffer corrente cercherebbe nella riga del prompt; il test `an_isearch_searches_the_file_not_the_prompt` esiste per questo. Lo stato della sessione e l'algoritmo di ricerca sono descritti nel Sezione 9.

7.3 Le decisioni

1. **Il minibuffer e la ricerca incrementale sono in Rust.** La politica dell'editor sta in `core/lisp/`, dove si ridefinisce o si toglie; queste due sono le eccezioni, per la stessa ragione: troppe cose ne dipendono. `M-x`, `M-:`, `find-file`, la ricerca e la sostituzione sono un prompt con una callback diversa, e un file che non si carica lascerebbe un editor che non sa fare una domanda. La keymap di `isearch-mode` è la prima consultata mentre il prompt è aperto, e senza di lei i tasti non vorrebbero dire ciò che il prompt dice. Una terza candidata dovrebbe portare lo stesso argomento: «sarebbe più veloce in Rust» non lo è.
2. **Il meccanismo in Rust, l'aspetto sostituibile.** `minibuffer-read` chiama la funzione che `*minibuffer-read-function*` nomina, e ogni chiamante usa quella che vi si mette: un meccanismo scritto in Rust non lega il prompt che si vede. Scartato: un prompt fisso.
3. **Le primitive stanno in `primitives/`, senza eccezioni.** `feature/` tiene i meccanismi che le primitive guidano e i modi che ognuna installa: una keymap che deve esistere qualunque cosa si carichi è un fatto dello stesso tipo dei meccanismi che lega. Così «dov'è definita questa primitiva» ha sempre la stessa risposta.
4. **Le variabili del prompt si scrivono con `setq`.** `Env::set_variable` crea sempre un legame nuovo nell'ambiente che riceve: da una primitiva chiamata con un ambiente suo, il globale, `*minibuffer-on-confirm*` per esempio, resterebbe nascosto alle letture che seguono. `setq` aggiorna il legame dove sta, e ne crea uno solo se non c'è.
5. **La storia di un prompt sta sotto il suo testo.** Ogni prompt ha una storia senza che un solo chiamante cambi: `M-x`, `find-file`, `switch-to-buffer` e ciò che un modulo chiede sono tutti `minibuffer-read`, e una storia solo per chi la chiede sarebbe una storia su cui non si conta. Costa che due prompt con le stesse parole condividano un anello, che di solito è giusto, e HISTORY-KEY lo evita. Il cammino nella storia si dimentica quando un prompt si apre e quando si chiude; le voci restano.
6. **La storia in tre grafie, senza condizioni.** `C-p` e `C-n`, che in un prompt di una riga non hanno altro da fare; `M-p` e `M-n`, quelli di Emacs; le frecce. La striscia dei completamenti lega `C-n`, `C-p` e le frecce con una keymap transitoria, consultata prima di ogni altra (Sezione 5.2): mentre è aperta li prende lei, e quando si chiude tornano al prompt. Nessuna delle due guarda l'altra, ed è la sola soluzione che resta vera quando una terza vorrà gli stessi tasti.
7. **La ricerca incrementale è un modo del prompt.** Non c'è una lettura bloccante di un tasto: i tasti arrivano uno per giro del ciclo degli eventi, e il ciclo «leggi, estendi, cerca» non si scrive come un ciclo. Il `post-command-hook` scatta una volta per comando, dopo, e solo nel modo del buffer corrente: è «il modello può essere cambiato, guarda di nuovo». Un modo suo perché `minibuffer-mode` è di tutti i prompt, e un hook appeso lì dovrebbe chiedersi ogni volta se il prompt è il suo. Non sostituisce il minibuffer: Invio ed Esc restano i suoi comandi, e la chiusura passa da `minibuffer-cleanup`.
8. **Il buffer cercato si nomina.** Mentre il prompt è aperto il buffer corrente è il prompt, e tutto ciò che riguarda la ricerca lavora sul buffer che la sessione nomina: è la cosa che una modifica sbaglierebbe più facilmente.
9. **`case-fold-search` si legge in un posto solo.** La ricerca incrementale, la sostituzione, occur, grep e `scan-buffer` leggono la stessa variabile con la stessa funzione: una variabile che due comandi leggessero

in modo diverso sarebbe peggio di qualunque valore predefinito. Non legata vale attiva, così un file che non si carica non spegne per omissione la ricerca senza maiuscole.

8 I buffer

8.1 La struttura `Buffer`

Campo	Contenuto
`name`, `file_path`	Nome e file visitato.
`text: B`	Il testo, tramite `BufferTrait` (oggi `GapBuffer`). Contiene anche il punto (il cursore).
`is_modified`	Modificato rispetto al disco.
`current_mode`	Il nome del modo maggiore; decide keymap, hook, grammatica e tabella sintattica.
`undo`	`UndoHistory`: gruppi fatti e disfatti (Sezione 8.4).
`mark: Option<Mark>`	Posizione e flag `active`; la regione è tra mark attivo e punto.
`overlays`	`OverlayTable`: facce su intervalli, per categoria, che seguono le modifiche.
`virtual_text`	Testo mostrato in una posizione senza essere nel buffer (suggerimenti, anteprime).
`version`	Incrementata da ogni modifica: un risultato calcolato su un'altra versione viene rifiutato.
`changed`	L'argomento emesso a ogni modifica del testo: il `buffer-changed` dell'editor, da quando il buffer vi entra (Sezione 11.4).
`syntax`, `scan`	Cache della colorazione e della scansione delle espressioni.
`file_stamp`, `stale`	Timbro (istante di modifica e lunghezza) del file all'ultima lettura o scrittura; flag «cambiato sotto di noi».
`auto_saved_at`	Versione scritta nell'ultimo salvataggio automatico.
`data`	`HashMap<String, Box<dyn Any>>`: dati che altre parti appendono al buffer e che muoiono con lui.
`writer`	Il lavoro che scrive nel buffer da un thread suo (un comando di shell, una ricerca); chiudere il buffer lo ferma.
`minor_modes`	I modi minori accesi, nell'ordine di accensione, ognuno con la sua etichetta (Sezione 10.2). Con `read-only-mode` tra loro il buffer rifiuta le modifiche: `Buffer::read_only`, controllato alle due porte.

8.2 `BufferTrait` e `GapBuffer`

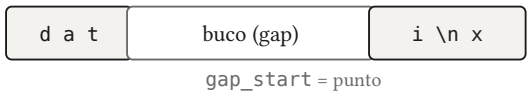
`BufferTrait` è l'unica cosa che il resto dell'editor può assumere sul testo: lunghezza, carattere in una posizione, punto in coordinate lineari e in (riga, colonna), conversioni tra le due, spostamenti del punto, inserimento e cancellazione di un carattere *al punto*, righe, `clear`. Gli spostamenti e le conversioni lavorano in caratteri, non in byte. Ciò che ogni implementazione deve fare è scritto una volta sola, per un `B` qualunque, in `buffer_conformance_tests`: una seconda implementazione è una riga in fondo a quel file, e arriva già provata. Il trait chiede anche `Clone`, che `GapBuffer` deriva: una copia del testo, del buco e dell'indice delle righe.

`GapBuffer` tiene un `Vec<char>` con un buco al punto: inserire e cancellare lì costa O(1), spostare il punto costa la distanza percorsa. Accanto al testo tiene un indice delle posizioni fisiche dei `\n`, diviso in prima e dopo il buco: poiché le posizioni fisiche non cambiano quando il buco cresce o si restringe, scrivere un carattere aggiorna l'indice in O(1).

da `buffer/gap_buffer.rs` — estratto semplificato

```
pub struct GapBuffer {
    data: Vec<char>,                // testo e buco nello stesso vettore
    gap_start: usize,               // primo indice del buco = posizione del punto
    gap_end: usize,                 // primo indice dopo il buco
    newlines_before: Vec<usize>,    // posizioni fisiche dei \n prima del buco
    newlines_after: VecDeque<usize>, // posizioni fisiche dei \n dopo il buco
}
fn insert(&mut self, c: char) {
    if self.gap_start == self.gap_end { self.gap_grow(); }   // buco esaurito: raddoppia
    self.data[self.gap_start] = c;
    if c == '\n' { self.newlines_before.push(self.gap_start); }
    self.gap_start += 1;
}
```

Spostare il punto vuol dire spostare il buco: `move_gap` copia con `copy_within` i caratteri che il buco deve scavalcare dall'altra parte, e sposta tra le due metà dell'indice i ritorni a capo che attraversano il buco, correggendone la posizione fisica della larghezza del buco. Un ritorno a capo non cambia mai posizione fisica per un inserimento o una cancellazione al punto, ed è questo che rende O(1) l'aggiornamento dell'indice durante la digitazione. La posizione *logica* del k-esimo ritorno a capo è `newlines_before[k]` se k cade nella prima metà, altrimenti la posizione fisica nella seconda metà meno la larghezza del buco; `line_start(riga)` è la posizione del ritorno a capo precedente più uno. `cursor_1d_to_2d` trova la riga di una posizione con una ricerca binaria nell'indice. Per il punto non serve nemmeno quella: il punto è il buco, e i ritorni a capo prima del buco sono proprio `newlines_before`, quindi `cursor_pos` legge la riga dalla sua lunghezza e la colonna dalla distanza dal suo ultimo elemento, in tempo costante. Lo chiede ogni fotogramma, per ogni finestra: con un conteggio dei caratteri dall'inizio del buffer costava, in un file di 8 MB con il punto in fondo, circa 17 ms a chiamata.



`gap_start` = punto

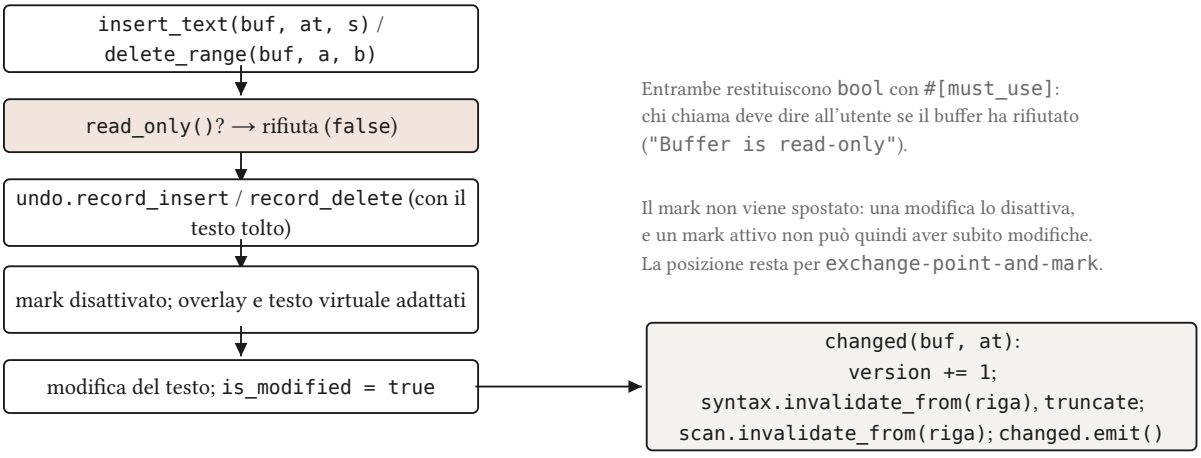
`newlines_before: Vec<usize>`

`newlines_after: VecDeque<usize>` (posizioni fisiche)

un carattere costa 4 byte (`char`); il buco cresce raddoppiando quando si esaurisce

8.3 Le due porte

Ogni primitiva che cambia il testo passa da `edits::insert_text` o da `edits::delete_range`. È la proprietà su cui si reggono undo, overlay, cache e sola lettura: una nuova primitiva di modifica li ottiene tutti per il solo fatto di non poter evitare le porte.



Entrambe restituiscono `bool` con `#[must_use]`: chi chiama deve dire all'utente se il buffer ha rifiutato (`"Buffer is read-only"`).

Il mark non viene spostato: una modifica lo disattiva, e un mark attivo non può quindi aver subito modifiche. La posizione resta per `exchange-point-and-mark`.

da `primitives/edits.rs` — estratto semplificato

```
pub(crate) fn insert_text<B: BufferTrait>(
    buf: &mut Buffer<B>, at: usize, content: &str,
) -> bool {
    if buf.read_only() { return false; }
    if content.is_empty() { return true; }
    let at = at.min(buf.text.len());
    let point = buf.text.cursor_pos_1d();
    let inserted = content.chars().count();
    buf.undo.record_insert(at, inserted, point);          // 1. la storia
    buf.mark = deactivated(buf.mark);                     // 2. la regione
    buf.overlays.adjust_for_insert(at, inserted);         // 3. ciò che è ancorato al testo
    buf.virtual_text.adjust_for_insert(at, inserted);
    undo::apply_insert(&mut buf.text, at, content);       // 4. il testo
    buf.is_modified = true;
    changed(buf, at);                                     // 5. versione e cache
    true
}
pub(crate) fn changed<B: BufferTrait>(buf: &mut Buffer<B>, at: usize) {
    buf.version = buf.version.wrapping_add(1);
    let line = buf.text.cursor_1d_to_2d(at.min(buf.text.len())).0;
    buf.syntax.invalidate_from(buf.version, line);
    buf.syntax.truncate(buf.text.line_count());
    buf.scan.invalidate_from(buf.version, line);
    buf.changed.emit();                                   // sveglia chi aspetta
}
```

`delete_range` ha la stessa forma. Ordina e limita gli estremi, copia il testo che sta per togliere (tutto il buffer con `to_string`, una porzione con `slice`), lo consegna a `record_delete` — è l'unico modo in cui l'undo potrà rimetterlo — e poi adatta overlay e testo virtuale, cancella e chiama `changed` dalla posizione di partenza. L'ordine conta: la storia registra lo stato *prima* della modifica, gli overlay vengono spostati con gli stessi numeri che descrivono la modifica, e la versione cambia per ultima, così nessun lettore può vedere la versione nuova insieme al testo vecchio (tutto avviene sotto lo stesso lock di scrittura del buffer). Le porte non scrivono messaggi: un rifiuto lo riferisce chi le chiama, con `edited`, dopo la closure del buffer (Sezione 14.2).

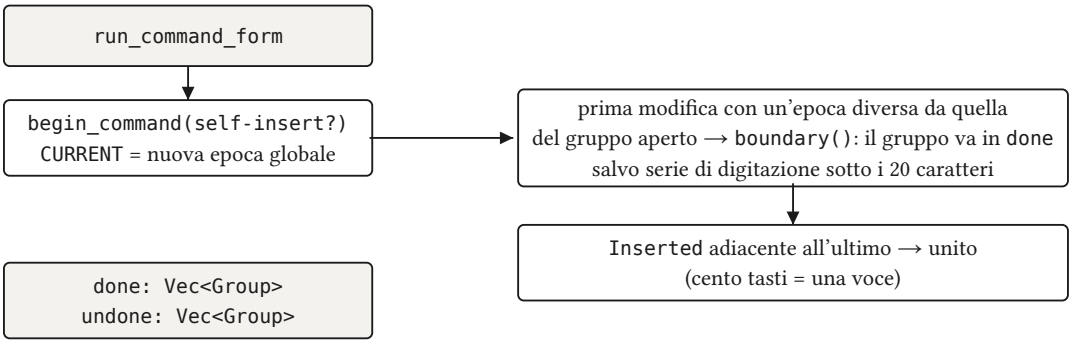
Undo e redo passano da una terza porta, `edits::replay`: rifiuta un buffer in sola lettura (`Replayed::Refused`, che chi chiama riferisce con `edited`), fa applicare a `UndoHistory` l'inverso del gruppo e, per ogni modifica che la storia sta per fare, sposta overlay e testo virtuale come le due porte, con gli stessi `adjust_for_insert` e `adjust_for_delete`; poi disattiva il mark, segna il buffer modificato e chiama `changed` da `Applied::from`. Non registra nulla: l'inverso lo scrive la storia stessa (Sezione 8.4). Ne passano `undo`, `redo` e il passo indietro di `query-replace` (`replace_back`).

Le tre porte dicono ogni modifica, prima che il testo si muova, anche a chi la guarda: `Buffer::touched`, che `track-changes-begin` accende e `track-changes-end` spegne restituendo ciò che è cambiato (`buffer/touched.rs`). Tiene due numeri, quanti caratteri all'inizio e quanti alla fine nessuna modifica ha raggiunto, e il testo che il tratto fra i due aveva quando la guardia è cominciata: una modifica fuori dal tratto lo allarga, e il testo che aggiunge è letto lì, dalla porta, prima della modifica; fuori dal tratto il testo è ancora quello di allora. La fine confronta il tratto di prima con quello di adesso come `string-diff` confronterebbe i due testi interi, ma costa quanto il tratto, non quanto il file. La terza porta non può leggere il testo che la storia sta scrivendo: un undo dentro il tratto non costa nulla, uno che ne esce fa del tratto intero il cambiamento. È ciò che Vim usa per sapere che cosa ha scritto una sessione di Insert (Sezione 15.5.28).

Un percorso solo oggi scavalca le porte, e di proposito: `adopt_text` (lettura di un file, `revert-buffer`, ricarica dal watcher) sostituisce tutto il testo e fa a mano ciò che le porte farebbero: incrementa la versione, invalida entrambe le cache dalla riga 0, svuota overlay, testo virtuale, mark e storia dell'undo (conservandone il limite), riporta il punto alla stessa riga e colonna. Un testo che arriva dal disco non è una modifica da annullare; per chi guarda le modifiche, invece, è cambiato tutto.

8.4 Undo

La storia registra *modifiche*, non fotografie: un inserimento è `Inserted { at, len }`, una cancellazione `Deleted { at, text }`. Un gruppo è l'insieme delle modifiche di un comando, con la posizione del punto prima della prima modifica. Disfare applica l'inverso del gruppo e produce il gruppo che lo rifarebbe; rifare è la stessa operazione sull'altra pila.



una nuova modifica svuota `undone`: nessun albero dei rami

Le due funzioni che decidono i gruppi sono queste:

da `buffer/undo.rs` — estratto semplificato

```rust
pub(crate) fn begin_command(amalgamating_kind: bool) {      // chiamata da run_command_form
    if PINNED.get() > 0 { return; }      // dentro una macro: un gruppo solo
    CURRENT.with(|current| {
        let previous = current.get();
        current.set(Command {
            epoch: NEXT_EPOCH.fetch_add(1, Ordering::Relaxed),
            amalgamating_kind,
            joins_previous: amalgamating_kind && previous.amalgamating_kind,
        });
    });
}

fn push(&mut self, change: Change, point: usize) {            // ogni modifica passa da qui
    self.undone.clear();                                       // niente albero dei rami
    let command = current_command();
    if self.open.is_some() && command.epoch != self.open_epoch
        && !(command.joins_previous && self.open_insert_len() < AMALGAMATION_LIMIT)
    {
        self.boundary();                // il gruppo aperto va in `done`
    }
    self.open_epoch = command.epoch;
    self.bytes += change.weight();                             // byte di testo cancellato
    let group = self.open.get_or_insert_with(|| Group { changes: Vec::new(), point });
    if let Change::Inserted { at, len } = change
        && let Some(Change::Inserted { at: prev_at, len: prev_len }) =
            group.changes.last_mut()
        && *prev_at + *prev_len == at
    {
        *prev_len += len;               // inserimento contiguo: si allunga
        return;
    }
    group.changes.push(change);
}
```

Disfare un gruppo vuol dire applicare le sue modifiche al contrario, dall'ultima alla prima, e costruire intanto il gruppo che rifarebbe ciò che si è appena disfatto. Ogni modifica è annunciata a `follow`, come

`Step::Insert` o `Step::Delete`, prima di toccare il testo: è così che la terza porta sposta ciò che è ancorato agli offset (Sezione 8.3).

da `buffer/undo.rs` — estratto semplificato

```
fn apply_inverse<B: BufferTrait>(
    &mut self, text: &mut B, group: Group, follow: &mut dyn FnMut(Step),
) -> Group {
    let mut inverse = Vec::with_capacity(group.changes.len());
    for change in group.changes.iter().rev() {
        match change {
            Change::Inserted { at, len } => {
                // si toglie, ricordando che cosa
                let removed = text.slice(*at, at + len);
                follow(Step::Delete { from: *at, to: at + len });  // prima che il testo si
muova
                apply_delete(text, *at, at + len);
                inverse.push(Change::Deleted { at: *at, text: removed });
            }
            Change::Deleted { at, text: content } => {
                let len = content.chars().count();
                follow(Step::Insert { at: *at, len });
                apply_insert(text, *at, content);              // si rimette il testo salvato
                inverse.push(Change::Inserted { at: *at, len });
            }
        }
    }
    Group { changes: inverse, point: /* posizione della prima modifica */ }
}
```

`undo` chiude il gruppo aperto, toglie l'ultimo da `done`, lo applica al contrario e mette l'inverso in `undone`; `redo` fa l'operazione simmetrica. Restituiscono la posizione in cui riportare il punto.

- *Il confine è pigro.* Chiamare `boundary` a ogni tasto costava un lock di scrittura sul buffer anche per i tasti che non modificano nulla (circa 120 dei 890 ns di un tasto). Ora un comando timbra un'epoca in una variabile del thread (`CURRENT`), e il gruppo aperto si chiude alla prima modifica che arriva con un'epoca diversa. Le epoche vengono da un contatore globale, così due thread non possono confondere le loro.
- *Digitazione.* `joins_previous` è vero se il comando e il precedente sono entrambi `self-insert`; la serie continua nello stesso gruppo finché il gruppo non ha inserito `AMALGAMATION_LIMIT` (20) caratteri.
- *Macro.* `OneGroup` (RAII, annidabile) blocca il timbro: tutti i comandi di una riproduzione finiscono in un gruppo solo.
- *Fuori dai comandi.* Una modifica fatta da Lisp senza un tasto premuto usa l'epoca rimasta nel thread, quindi si unisce al gruppo dell'ultimo comando; `undo-boundary` chiude il gruppo a mano.
- *Limite.* `DEFAULT_UNDO_LIMIT` è 1 MiB di *testo cancellato* (solo le cancellazioni portano testo); oltre, i gruppi più vecchi vengono scartati. `set-undo-limit` lo cambia per buffer. Il gruppo più recente non viene mai scartato, anche se da solo supera il limite.

8.5 Mark, regione, overlay e testo virtuale

Il mark è una posizione con un flag `active`. La regione esiste solo se il mark è attivo (`region_bounds` la ordina e la limita alla lunghezza del testo), e ogni modifica la disattiva, come in Emacs. Il flag `rectangle` dice che la regione si disegna come un rettangolo; il rettangolo tra due posizioni lo calcola `Rectangle::between_offsets`, in `text/rectangle.rs` (Sezione 9). Gli *overlay* sono intervalli con una faccia e una categoria (le corrispondenze di una ricerca, una diagnostica, l'enfasi di una pagina di manuale); sono, con il testo virtuale, le sole posizioni conservate che seguono le modifiche: le porte li adattano con `adjust_for_insert` e `adjust_for_delete`. Nessuna delle due estremità cresce, e un overlay che si riduce a lunghezza zero viene cancellato. Il *testo virtuale* è ancorato a un punto, non a un intervallo, ed è per questo una tabella a parte: il layout delle righe lo inserisce nel testo mostrato (`ui/layout.rs`), e un clic

su di esso viene riportato alla posizione vera del buffer. Le primitive sono in `primitives/overlays.rs` e `primitives/virtual_text.rs`.

8.6 Le cache del testo

Cache	Che cosa conserva	Chi la riempie
`SyntaxCache`	Gli intervalli colorati di ogni riga e lo stato del lessico (la pila delle regioni aperte) all'inizio di ogni riga, per versione, modo ed edizione delle grammatiche.	L'evidenziatore, `LINES_PER_TURN` (500) righe per passo, svegliato da `buffer-changed` e `syntax-changed`.
`ScanCache`	Un punto di ripresa ogni 64 righe per la scansione delle espressioni bilanciate, per versione, modo ed edizione della tabella sintattica.	Lo scanner preventivo, 500 righe per passo, svegliato dagli stessi argomenti. `forward-sexp`, l'indentazione e `syntax-ppss` la leggono e partono dal punto di ripresa più vicino invece che dall'inizio.

Entrambe sono invalidate dalla riga modificata in giù, dalla stessa chiamata (`changed`); la colorazione vecchia resta disegnata finché la nuova non la sostituisce. Una cache che non corrisponde a versione, modo ed edizione correnti non viene usata: il caso peggiore è una scansione dall'inizio, la stessa che si farebbe senza cache.

8.7 File su disco

`find-file` legge il file, imposta `file_path` e il timbro (`FileStamp`: istante di modifica e lunghezza, un solo `stat`), e sceglie il modo secondo le espressioni regolari di `add-auto-mode`. Il file si legge con `read_text`, che riconosce la sua codifica (Sezione 8.7.1) e non fallisce per ciò che contiene; un file che non si può leggere affatto lo dice nell'area messaggi e nel log. Le primitive sono in `primitives/files.rs`, e `save-some-buffers` e l'uscita, che salvano con lo stesso `write_buffer`, in `primitives/quit.rs` (Sezione 14.3.5). Salvare scrive l'intero testo, nella codifica del buffer, con `disk::write_checked`, e aggiorna il timbro, altrimenti ogni salvataggio sembrerebbe la scrittura di qualcun altro. `write_atomically` scrive il testo in un file nuovo accanto al vecchio (`create_new`, con un nome nascosto che nessun altro ha), gli dà i permessi del vecchio, lo porta sul disco con `sync_all` e lo rinomina sul vecchio, un passo che il sistema fa tutto insieme; poi porta sul disco la directory. Una scrittura che si ferma a metà lascia dunque il file vecchio, e un errore prima del rename toglie il file nuovo. Dove un rename cambierebbe che cosa è il file, scrive sul posto, come `std::fs::write`: attraverso un collegamento simbolico scrive il file a cui punta, e il collegamento resta; un file con altri nomi (collegamenti fisici), uno di un altro proprietario o gruppo, e uno in una directory in cui l'editor non può scrivere si scrivono sul posto, perché un file nuovo non avrebbe gli altri nomi, sarebbe di chi scrive, o non si potrebbe creare. Ogni salvataggio, che venga da `save-buffer`, da `save-some-buffers` o da `write-file`, scrive l'esito, riuscito o fallito, nell'area messaggi e nel log. Il testo si copia sotto il lock con la sua `version`, si scrive senza lock, e `mark_written` toglie «modificato» solo se la versione è ancora quella: una modifica arrivata nel frattempo, da un lavoro o da un callback, lascia il buffer modificato e la copia di ripristino al suo posto, e il messaggio lo dice.

Prima di copiare il testo, tutti i percorsi di salvataggio manuale chiamano `files::before_save`: `write_buffer_expected` (anche per le domande di uscita), `write-file` e `file_io::save_named`. Si prendono dal buffer modo maggiore, modi minori e percorso; `Modes::hooks_for` copia i callback di `before-save-hook` in quell'ordine, poi i globali. Ogni funzione gira sul thread dei comandi con il buffer destinazione corrente, senza lock di buffer, finestre o registro. Una funzione che cambia il buffer corrente non cambia il contesto del callback seguente. Il chiamante torna corrente alla fine, se esiste ancora.

Un attachment `file-io:before-save`, tolto dal distruttore di `SaveHookGuard`, rifiuta la ricorsione sullo stesso buffer anche prima della prenotazione asincrona. Dopo ogni callback si ricontrollano identità, nome

e percorso: eliminare, rinominare o cambiare file interrompe il salvataggio. Il primo errore Lisp interrompe anche gli hook rimasti e viene mostrato e registrato; le modifiche già fatte restano nel buffer. I normali hook possono continuare dopo un errore, ma un controllo prima della scrittura deve poterla impedire. Gli hook si eseguono prima dello snapshot, così anche la pulizia degli spazi è una modifica visibile e annullabile. Le copie periodiche di recupero non eseguono Lisp e conservano il testo effettivo.

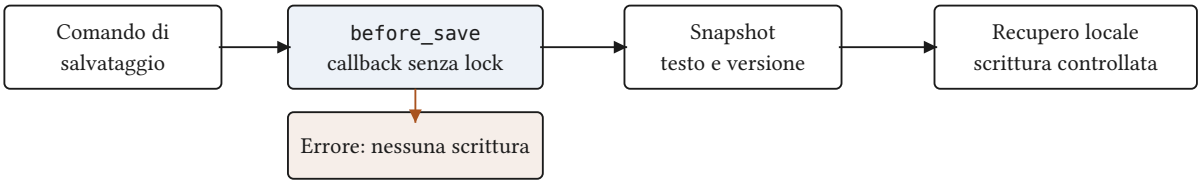


Figura 1: Preparare un salvataggio. L'ultimo passo gira in un lavoratore per i comandi interattivi; una conferma asincrona di conflitto riprende lo snapshot esistente, senza rieseguire gli hook.

Il *watcher* (`FileWatcher`, ogni `watch-file-interval` secondi, 3 per default, al massimo 2 controlli contemporanei; spento da `(setq watch-files nil)`) confronta i timbri: un buffer pulito il cui file è cambiato viene ricaricato con `adopt_text`, nella codifica che il file ha ora; uno modificato, o il cui file è sparito, viene marcato `stale`, e il salvataggio successivo chiede conferma. L'*autosalvataggio* (`AutoSaver`, ogni `auto-save-interval`, 30 secondi per default, al massimo 4 buffer per turno; spento da `(setq auto-save nil)`) scrive i buffer modificati la cui versione è cambiata dall'ultima copia in `#nome#`, nella directory del cache (`auto-save-directory`); `recover-file` riporta la copia nel buffer. La versione si registra solo dopo una scrittura riuscita: una fallita si riprova al turno dopo, e il log lo sente una volta, quando le scritture cominciano a fallire (`auto_save_failing`). La copia è sempre in UTF-8, la codifica dell'editor: `recover-file` la legge così, e il buffer tiene la codifica del suo file.

I comandi interattivi usano le varianti asincrone in `primitives/file_io.rs`; la proprietà Lisp `interactive-function` cambia solo `call-interactively`, lasciando sincrone le funzioni pubbliche esistenti. `editor/file_io.rs` riserva una chiave per percorso, prima di avviare un lavoro dedicato: due letture, quattro scritture, due controlli del watcher e un lavoro di recupero. Nessuna coda illimitata. Il timeout `file-io-timeout` (10 secondi) segnala, senza cancellare la syscall o liberare la riserva. `tick` applica gli esiti sul thread dei comandi; confronta identità del buffer, percorso e versione. La risposta tardiva non pulisce nuove modifiche, non sostituisce testo digitato durante una lettura e non seleziona una finestra che l'utente ha lasciato. Chiudere un buffer in scrittura, o uscire normalmente, aspetta l'esito.

`FileStamp::try_of` distingue solo `NotFound` dagli altri errori. La scrittura ricontrolla il timbro prima di preparare il temporaneo e subito prima del rename; la conferma vale per quel timbro e quella versione, non per modifiche successive. Il controllo usa mtime e lunghezza e non è un lock distribuito: un altro processo e la cache del filesystem possono ancora correre fra stat e rename. Gli esiti non verificabili conservano «modificato» e il recupero.

Per default `auto-save-directory` è `app_dir(Cache)/autosave`, attraverso `paths.rs`. Non si ripiega sulla rete quando la cache non funziona; `nil` resta una scelta esplicita per le copie accanto al file. Le copie sono in UTF-8 e precedono anche una scrittura manuale. Un lock fra soli lavoratori serializza scritture e cancellazioni delle copie; un indice per versione impedisce che una copia vecchia sovrascriva o cancelli una più recente. Il watcher e l'autosalvataggio fanno solo snapshot nel turno dell'esecutore: le operazioni sul disco sono nei loro lavoratori separati. Il rendering legge le informazioni di I/O da attachment tipizzati del buffer (`buffer/disk.rs`), senza dipendere dallo stato dell'editor. Restano sincroni esplorazione directory, completamento nomi file, primitive filesystem e hook Lisp che li chiamano.

`buffer/io_status.rs` conserva cause tipizzate (`FileIssue`: categoria, `ErrorKind`, codice OS e messaggio) e il resoconto `AutoSaveReport` negli attachment. Il writer locale registra data UTC, versione, destinazione ed esito appena termina, conservando l'ultimo successo anche dopo un errore. Saltare una vecchia versione non aggiorna la data; eliminare la copia dopo un salvataggio riuscito conserva lo storico e

ne annota la rimozione. I callback tardivi non riscrivono lo stato di recuperi più recenti. `file-io-status` e `auto-save-status` leggono solo questi dati; `%i` mostra causa di indisponibilità ed esito/ora dell'ultimo recupero. Uno stallo resta pendente: non diventa una diagnosi di rete sulla sola base del timeout.

8.7.1 Le codifiche

`buffer/coding.rs` dice come i byte di un file sono il suo testo. `Coding` ha tre valori: `Utf8`; `Utf8WithSignature`, per un file che comincia con i tre byte `EF BB BF`, che la lettura toglie, perché il buffer non cominci con un carattere invisibile, e la scrittura rimette; e `Windows1252`, la codifica a un byte di Windows per le lingue occidentali, in cui si legge ogni file che non è UTF-8 valido.

Windows-1252 è Latin-1 con caratteri tipografici al posto di quasi tutti i controlli C1, da `0x80` a `0x9F`: `€`, le virgolette, i trattini. La tabella è scritta a mano, `HIGH`, perché è solo questo che la distingue da Latin-1. I cinque byte che la codifica non definisce tengono il controllo C1 che hanno in Latin-1, come nella tabella WHATWG: così ogni byte ha un carattere suo, la lettura non fallisce mai, nemmeno su un file binario, e un file letto e riscritto senza modifiche torna identico byte per byte.

`decode` sceglie la codifica e `read_text` legge un file con essa; ne passano `find-file` (in `new_buffer`), `revert-buffer`, il watcher e `read-file-to-string`, e `grep--scan` decodifica allo stesso modo, saltando i file binari (Sezione 14.4.3). Il buffer ricorda la codifica in `Buffer::coding`, e `write_buffer` e `write-file` scrivono con `encode`. Un carattere che la codifica non ha, come `ā` in Windows-1252, fa rifiutare il salvataggio con un messaggio che lo nomina, e il file resta com'era: scritto come altro, non tornerebbe. `find-file` dice nell'area messaggi un file letto in Windows-1252; `buffer-file-coding` dice la codifica del buffer, e `set-buffer-file-coding` la cambia per il prossimo salvataggio, segnando il buffer come modificato.

Decisioni:

1. **Windows-1252, e non un rilevatore.** Indovinare la codifica tra molte (`chardetng`) può sbagliare, e un errore cambia i byte salvati. Una codifica a un byte in cui ogni byte ha il suo carattere non perde nulla anche quando non è quella giusta: i caratteri si vedono male, ma il file torna com'era. Per chi scrive in una lingua occidentale è quasi sempre quella giusta.
2. **Rifiutare, non convertire.** Un salvataggio che passasse in silenzio a UTF-8 cambierebbe la codifica di un file che altri programmi leggono nella sua. Convertire è una scelta, `set-buffer-file-coding`.

8.8 Le decisioni

1. **La sola lettura sta sul buffer e si controlla alle porte.** Un buffer che *presenta* qualcosa — un elenco di directory, l'elenco delle facce, un backtrace — è la vista di uno stato che sta altrove: scriverci non cambia quello stato, fa solo dire allo schermo una cosa falsa. I tasti da soli non bastano a proteggerlo: ogni tasto stampabile è legato a `self-insert`, e `M-x kill-line` o una riga di Lisp passerebbero comunque. Dalle porte invece passa ogni modifica, `undo` e `redo` compresi, dalla terza. Il punto si muove liberamente: leggere è lo scopo di un buffer in sola lettura.
2. **La sola lettura è un modo minore, e del nucleo.** `Buffer::read_only` chiede se `read-only-mode` è tra i modi minori accesi (Sezione 10.2), e `Buffer::set_read_only` lo accende e lo spegne sotto il lock del buffer. Lo stato è uno solo: `C-x C-q`, `minor-mode-p`, `C-h M` e l'etichetta `RO` della riga di stato dicono la stessa cosa di un file bloccato a mano e di un elenco. Il modo lo crea `create_global_env` e non un modulo, perché il nucleo protegge trascrizioni e liste di risultati anche senza moduli. In due cose non è un modo minore come gli altri. Resta acceso quando cambia il modo maggiore (`change_major_mode` lo conserva): un buffer protetto resta protetto. E `minor-mode-hook` gira per il comando `read-only-mode` e per `minor-mode-on` e `minor-mode-off`, non per `set-buffer-read-only` né per il Rust, che lo tolgono e lo rimettono attorno a ogni riscrittura di un elenco, spesso con un lock in mano. Scartato: un flag nel buffer accanto al modo, due stati da tenere allineati.

3. **I dati appesi vivono quanto il buffer.** Ciò che una funzionalità ricorda *di un buffer* — i risultati di una ricerca mostrati lì, una tabella, lo stato di una vista — sta in `data`, sotto un nome, come `Box<dyn Any>`. Uccidere il buffer distrugge la mappa, e ogni valore con il suo `Drop`; un `downcast` del tipo sbagliato risponde `None`. Due funzionalità appendono i loro dati allo stesso buffer senza sapere l'una dell'altra. Scartati: una tabella globale per nome di buffer, sbagliata dopo un cambio di nome e ancora lì dopo la chiusura; il testo del buffer, che va bene solo per ciò che si mostrerebbe comunque; un puntatore grezzo, che non ha distruttore e che con un cast sbagliato è un comportamento indefinito invece di un rifiuto.
4. **`adopt_text` sostituisce il testo in un metodo solo.** Sei istruzioni sparse nei chiamanti sono sei occasioni di dimenticarne una, e quelle facili da dimenticare non si vedono subito: senza la versione l'evidenziatore salverebbe i colori del testo vecchio sul testo nuovo, senza svuotare gli overlay una diagnostica resterebbe su una riga che si è spostata. Butta via undo, mark, overlay e testo virtuale, che descrivono posizioni di un testo che non c'è più: disfare verso uno stato che il file non ha mai avuto è peggio che non poter disfare. Tiene il modo, il limite dell'undo, e la riga e la colonna del punto, perché tornare circa dove si era è lo scopo di ricaricare un file che si sta leggendo.
5. **Il flag del rettangolo sta sul mark.** È una proprietà della selezione, non del buffer: ogni modifica disattiva il mark, e così chiude anche il modo rettangolo, senza una seconda cosa da ricordare di spegnere. Cambia solo ciò che si disegna: i comandi dei rettangoli lavorano da punto e mark in ogni caso.
6. **Gli overlay seguono le modifiche, alle porte.** Le regole di un modo sono espressioni regolari sul testo: dicono «una parola di questa lista è una parola chiave», non «*questo* tratto, per qualcosa che è successo». Per dire la seconda cosa ci sono gli overlay, che quindi devono seguire il testo: le porte chiamano `adjust_for_insert` e `adjust_for_delete`, come per l'undo e la versione. Una posizione che si sposta senza i suoi overlay è un'evidenziazione sulle parole sbagliate, minuti dopo, senza niente che dica perché.
7. **La tabella degli overlay è ordinata per inizio.** Una modifica sposta gli overlay che stanno dopo di lei: con l'ordine sono una ricerca binaria e una coda, invece di tutta la tabella. Le due regole di aggiustamento sono monotone, quindi l'ordine sopravvive e non si ricostruisce mai. Il caso più comune, una modifica dopo l'ultimo overlay, costa un confronto con `furthest_end`.
8. **Nessuna estremità di un overlay cresce.** `start` si sposta quando l'inserimento è prima o proprio lì, `end` solo quando è strettamente prima: un testo inserito all'inizio o alla fine resta fuori, uno inserito dentro entra a farne parte. Una corrispondenza evidenziata resta esattamente il testo che corrispondeva, qualunque cosa si scriva accanto.
9. **Il testo virtuale è un punto, in una tabella sua.** Non è un overlay: un overlay colora caratteri che ci sono, e uno senza larghezza viene cancellato. Non è nemmeno testo inserito davvero: il buffer risulterebbe modificato, l'undo si riempirebbe di cose mai scritte e il salvataggio le scriverebbe nel file. È una posizione che si sposta con il testo e non sparisce mai. La tabella non sa a che cosa serve: suggerimenti, anteprime e il resto sono categorie scelte da chi li produce.
10. **Due cache, perché due lessici.** La grammatica decide che aspetto ha il testo, e il suo stato è la pila delle regioni aperte; la tabella sintattica decide che cosa significa — dove sono stringhe, commenti e liste — e il suo stato è un `Resume`. `forward-sexp`, l'indentazione, `syntax-ppss` ed electric-pair leggono la seconda, e nessuno di loro si può servire dalla prima (Sezione 10).
11. **La colorazione tiene uno stato per riga, la scansione un punto ogni 64 righe.** Quasi ogni riga di quasi ogni file comincia nello stesso stato vuoto, e gli stati della colorazione, internati, costano poco. Uno stato della scansione contiene invece le *posizioni* delle liste aperte, così due righe alla stessa profondità hanno stati diversi e internarli non risparmia nulla. Un punto di ripresa ogni 64 righe costa un sessantaquattresimo della memoria, e una risposta al più 64 righe di scansione.
12. **Le cache stanno sul buffer.** Ciò che le invalida è una modifica, e le modifiche passano dalle porte; ciò che descrivono è un testo, a cui non devono sopravvivere. Una tabella a parte nell'editor andrebbe ripulita a ogni chiusura di un buffer, e sarebbe sbagliata, invece che assente, il giorno in cui non lo fosse.
13. **Chi legge la cache della scansione non la scrive.** La riempie solo lo scanner preventivo, e i movimenti la leggono con il lock di lettura che hanno già. Riempirla leggendo vorrebbe dire prendere il lock di

scrittura del buffer sul percorso di un tasto, nel mezzo della valutazione di Lisp, per far funzionare una cache. Il prezzo: un file appena aperto si scandisce dall'inizio finché lo scanner preventivo non l'ha percorso.

14. **Una modifica tiene ciò che sta sopra.** Una modifica alla riga L non cambia il testo prima di L: i punti di ripresa e gli stati che entrano nelle righe fino a L restano esatti, e si scarta solo da L in giù, senza dover contare le righe aggiunte o tolte.
15. **Il colore vecchio resta disegnato.** Dopo una modifica nulla di ciò che sta sotto è più affidabile, ma gli intervalli restano e si disegnano finché l'evidenziatore non li sostituisce. Cancellarli farebbe passare il resto del file al testo semplice a ogni tasto, che è molto peggio di qualche millisecondo di colore un po' sbagliato.
16. **La versione non basta.** Un cambio di modo, o una regola aggiunta alla grammatica di un modo, non tocca il testo. Per questo le due cache ricordano anche il modo e l'edizione della grammatica o della tabella con cui sono state calcolate, e una cache che ne descrive un'altra ricomincia dall'inizio. Così resta fuori dal lessico anche lo stato di un'altra grammatica, che nomina le regioni con il loro posto in quella.
17. **La colorazione non si risincronizza.** Dopo una modifica si ricalcola fino alla fine del file, invece di fermarsi dove lo stato calcolato torna uguale a quello di prima. Fermarsi lì costerebbe le righe fino alla convergenza invece del resto del file, ma richiede di tenere gli stati della generazione precedente e di contare di quante righe la modifica ha spostato il resto: senza quel conto si confronterebbero le righe sbagliate. Il lavoro a passi limitati tiene comunque l'editor reattivo, e la risincronizzazione riguarda quanto lavoro c'è in tutto: si aggiunge se le misure la chiedono.
18. **Undo ha una porta sua.** Rifare la storia non deve registrarsi come storia nuova, quindi `undo` e `redo` non passano dalle due porte, che registrano: la storia applica le modifiche con `apply_insert` e `apply_delete`, sotto lo strato che registra, e non c'è un flag da ricordare. Tutto il resto delle porte lo fa la terza, `edits::replay`: la sola lettura, overlay e testo virtuale annunciati passo per passo con `Step`, il mark, `changed`.
19. **Il testo da rifare si cattura disfacendo.** Disfare un inserimento toglie il testo e lo ricorda nel gruppo inverso, invece di conservarlo prima per un `redo` che può non venire mai.
20. **Una macro è un gruppo solo.** Rieseguire una macro passa ogni comando per la via normale, e lascerebbe cinquanta gruppi dietro una macro di cinquanta comandi: chi la esegue non vuole disfarne un cinquantesimo. `OneGroup` tiene fermo il timbro per tutta la riproduzione, e la regola che c'è già — un gruppo si chiude alla prima modifica sotto un altro timbro — fa il resto.
21. **Il file si riconosce da un timbro, non da un hash.** La domanda «il file è cambiato?» si fa per ogni buffer aperto, ogni pochi secondi. Un hash risponde esattamente ma legge tutto il file; un timbro — istante di modifica e lunghezza — risponde con un solo `stat`. Sfugge una scrittura che cada nella granularità del filesystem e lasci identica la lunghezza: il buffer resta vecchio fino al cambiamento successivo, senza danni.
22. **Il timbro si confronta per differenza, non per «più recente».** Gli istanti tornano indietro nell'uso normale: `git checkout` rimette istanti vecchi, `touch -d` scrive quello che gli si dice, un filesystem di rete non è d'accordo con l'orologio locale. «Diverso da ciò che ho registrato» è una domanda a cui il filesystem risponde; «più recente del mio» è una domanda su due orologi. Il timbro si registra anche a ogni scrittura dell'editor, altrimenti ogni salvataggio sembrerebbe la scrittura di qualcun altro.
23. **`stale` è un flag, non una domanda.** Il momento in cui fermare la sovrascrittura del lavoro di un altro è il salvataggio, non l'istante in cui il cambiamento si nota: il watcher marca il buffer, e il salvataggio successivo chiede.
24. **L'autosalvataggio confronta versioni.** «Modificato» resta vero dal primo tasto fino al salvataggio, quindi un lavoro che scrivesse a ogni turno ogni buffer modificato riscriverebbe per sempre un buffer fermo. La versione si muove solo quando si muove il testo: `auto_saved_at` la ricorda, e la domanda «c'è qualcosa da scrivere?» diventa un confronto tra due numeri. Un editor lasciato aperto di notte non costa nulla.

9 Il testo: ricerca, sostituzione, kill ring, rettangoli, liste di risultati e riempimento

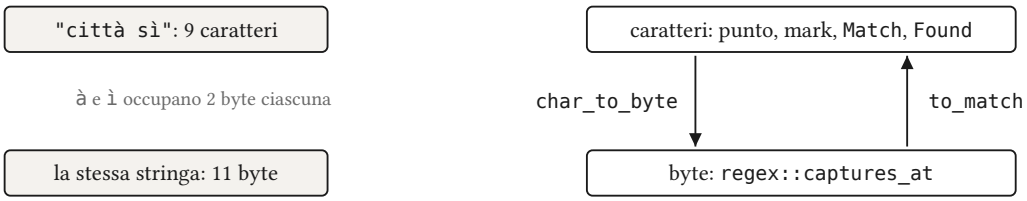
La directory `core/src/text/` raccoglie ciò che si può calcolare su un testo senza sapere che cosa sia un editor. Il criterio di ammissione è scritto in `text/mod.rs`: un file sta qui se lo potrebbe usare un programma che non è un editor. In pratica nessun file della directory importa `editor` o `lisp`, e nessuno riceve un `Buffer`: ricevono un `BufferTrait` (o una `&str`), cioè qualcosa che sa dare caratteri per posizione, e nient'altro. Niente punto, mark, undo, lock o finestre.

File	Che cosa contiene	Chi lo guida
`search.rs`	`Pattern` (che cosa cercare) con `search_forward`, `search_backward` e `scan`; le espressioni compilate di `string-match` (`compiled`).	`feature/isearch.rs`, `editor/replace.rs`, `primitives/isearch.rs` `primitives/occur.rs`, `primitives/scan.rs`, `primitives/match_data.rs`
`isearch.rs`	La sessione di ricerca incrementale: `Isearch` e `Direction`.	`feature/isearch.rs`, `primitives/isearch.rs`
`replace.rs`	La sessione di sostituzione, `Replace`, con `Casing` ed `expand_replacement`.	`editor/replace.rs`
`kill_ring.rs`	`KillRing`: un anello di stringhe con la regola di rotazione e di accodamento.	`managers/kill_yank.rs` (`KillYank`)
`rectangle.rs`	`Rectangle` e `Span`: l'aritmetica di due angoli e delle righe tra loro.	`primitives/rectangle.rs` `ui/compose.rs`
`results.rs`	`Entry` e `Results`: una lista di posti e un cursore che la percorre.	`primitives/occur.rs`, `primitives/results.rs` `primitives/scan.rs`, `editor/results.rs`
`fill.rs`	Il riempimento: `Measure`, `lay_out`, `comment_prefix`, `break_at` (Sezione 9.8).	`primitives/fill.rs`

Le sessioni (`Isearch`, `Replace`) tengono lo *stato* di una ricerca o di una sostituzione tra un tasto e l'altro. Chi le guida le conserva in `Runtime` e chiama un metodo per ogni risposta dell'utente; la sessione conosce il buffer solo per nome (Sezione 9.9).

9.1 Posizioni: sempre caratteri

Tutte le posizioni del modulo sono offset in *caratteri*, come il punto, il mark e la storia dell'undo. Il motore delle espressioni regolari (`regex`) lavora invece in *byte* su una `&str`. La conversione avviene in un posto solo: `char_to_byte` all'ingresso e `to_match` (che usa `byte_to_char`) all'uscita. Un errore di conversione non si vede su testo ASCII e rompe il primo carattere accentato, per questo non è ripetuta nei chiamanti.



9.2 `Pattern`: che cosa cercare

da `text/search.rs` — estratto semplificato

```
pub enum Pattern {
    Literal { chars: Vec<char>, fold: bool },   // confronto carattere per carattere
    Regex(Regex),                               // compilata una volta sola
}
pub fn new(source: &str, regexp: bool, fold: bool) -> Result<Self, String> {
    if source.is_empty() {
        return Err("Searching for the empty string would never finish".into());
    }
    if regexp {
        RegexBuilder::new(source).case_insensitive(fold).multi_line(true)
            .build().map(Pattern::Regex).map_err(|why| format!("Invalid regexp: {why}"))
    } else {
        Ok(Pattern::Literal { chars: source.chars().collect(), fold })
    }
}
```

Tre decisioni stanno in questa funzione:

- *Il modello vuoto è rifiutato.* Troverebbe una corrispondenza in ogni posizione: una ricerca non si muoverebbe mai e una sostituzione non finirebbe. Rifiutandolo qui nessun chiamante deve ricordarsi il controllo. Un'espressione che *può* corrispondere al vuoto (`x*`) resta invece ammessa, ed è compito di chi cicla scavalcare le corrispondenze vuote (`Match::is_empty`): la sostituzione riprende un carattere dopo, come in Emacs, e `x*` sostituito da `-` in `abc` dà `-a-b-c-`.
- *`multi_line(true)`*: `^` e `$` indicano inizio e fine di *riga*, come si aspetta chi scrive un'espressione in un editor. Senza, `^;;` troverebbe al massimo una corrispondenza in tutto il buffer.
- *`fold`* arriva dal chiamante, che lo legge dalla variabile Lisp `case-fold-search` (non legata = vero). Per i letterali il confronto ignora le maiuscole con `chars_equal`, che confronta le forme minuscole dei due caratteri.

9.2.1 *`search_forward` e `search_backward`*

`search_forward(text, from, limit)` restituisce la prima corrispondenza che *comincia* in `from` o dopo e *finisce* entro `limit`. Il limite è ciò che confina una sostituzione alla regione: il chiamante passa la fine della regione invece di quella del buffer.

- *Letterale*: prova ogni inizio da `from` a `limit - len` con `matches_at`, che legge i caratteri uno per uno con `BufferTrait::at`. Non alloca nulla: un passaggio, nessuna copia.
- *Espressione*: copia il buffer *fino al limite* in una `String` (il motore vuole una `&str`), converte `from` in byte e chiama `captures_at`. Si usa `captures_at` e non una fetta che comincia in `from` perché la ricerca deve vedere ciò che precede il punto di partenza: altrimenti ogni ripresa sembrerebbe l'inizio di una riga per `^`. La fetta finisce invece al limite, come in Emacs una ricerca con BOUND: `a+` trova l'`aa` di una regione che finisce dentro `aaaa`, e lì `$` e `\b` vedono il limite come la fine del testo.

`search_backward(text, to, floor)` cerca l'ultima corrispondenza che finisce entro `to` e comincia da `floor` in poi. Per i letterali prova gli inizi all'indietro. Per le espressioni il motore sa cercare solo in avanti, quindi cammina tutte le corrispondenze da `floor` e tiene l'ultima, su una sola copia del testo fino a `to` e in un solo passaggio; gli offset di quella tenuta diventano caratteri alla fine. Prima ogni passo era

una `search_forward`, che ricopiava il buffer, e su 120 000 caratteri con 40 000 corrispondenze la ricerca chiedeva 123 secondi nella build di debug; ora 0,03.

da `text/search.rs` — estratto semplificato

```
let haystack = text.slice(0, to);
let mut at = char_to_byte(&haystack, floor)?;
let mut best = None;
while at <= haystack.len() {
    let Some(captures) = regex.captures_at(&haystack, at) else { break };
    let whole = captures.get(0).expect("a capture set has a group zero");
    at = if whole.is_empty() { /* un carattere dopo */ } else { whole.end() };
    best = Some(captures);
}
best.map(|captures| to_match(&haystack, &captures))
```

9.2.2 `scan`: tutte le corrispondenze in un passaggio

Occur, grep e la lettura dell'output di un compilatore vogliono *tutte* le corrispondenze, con riga, colonna e testo della riga. Chiamare `search_forward` in un ciclo costerebbe una copia del buffer per corrispondenza. `Pattern::scan(haystack, limit)` lavora su una `&str` già pronta e fa un passaggio solo, grazie a un `Cursor` che avanza sempre in avanti e conta caratteri e righe man mano:

da `text/search.rs` — estratto semplificato

```
struct Cursor<'a> {
    haystack: &'a str, byte: usize, chars: usize, line: usize, line_start: usize,
    column: usize,
}
fn advance_to(&mut self, byte: usize) {          // byte >= self.byte, sempre
    for (at, c) in self.haystack[self.byte..byte].char_indices() {
        self.chars += 1;
        if c == '\n' {
            self.line += 1; self.line_start = self.byte + at + c.len_utf8(); self.column = 0;
        } else {
            self.column += 1;
        }
    }
    self.byte = byte;
}
```

Per ogni corrispondenza (in byte, da `next_match`) `scan` porta il cursore al suo inizio, legge riga e colonna, prende il testo della riga (memorizzato, così venti corrispondenze sulla stessa riga non la ricostruiscono venti volte) e converte inizio e fine in caratteri. Poi riprende dalla fine della corrispondenza, o un carattere dopo se era vuota. Il risultato è una `Scan`: il vettore dei `Found` e `truncated`, vero se si è fermata per `limit`. Anche la colonna è contata man mano che il cursore avanza, e non dall'inizio della riga a ogni corrispondenza. Resta però che ogni `Found` porta una copia della sua riga (`line_text`): una riga lunghissima con molte corrispondenze costa la sua lunghezza per ognuna, in tempo e in memoria.

Campo di `Found`	Significato
`start`, `end`	Offset in caratteri nell'intero testo.
`line`	Riga di inizio, contata da 1 (la numerazione di `goto-line`).
`column`	Colonna in caratteri, contata da 0 (come `current-column`).
`line_text`	La riga intera senza `\n`: mostrarla non richiede di rileggere il testo.
`groups`	Ciò che ha catturato ogni gruppo, 0 = l'intera corrispondenza; vuoto per un letterale.

`in_line()` ed `end_in_line()` danno la posizione della corrispondenza dentro `line_text`, che è ciò che serve a una vista per evidenziarla.

9.2.3 Le espressioni di `string-match`

`string-match` e `string-match-p` non costruiscono un `Pattern`: chiedono a `compiled(pattern)` l'espressione già compilata. Ogni thread tiene la sua tabella, dalla sorgente all'espressione (`Arc<Regex>`), e la svuota tutta quando arriva a `CACHE_LIMIT`, 512 voci.

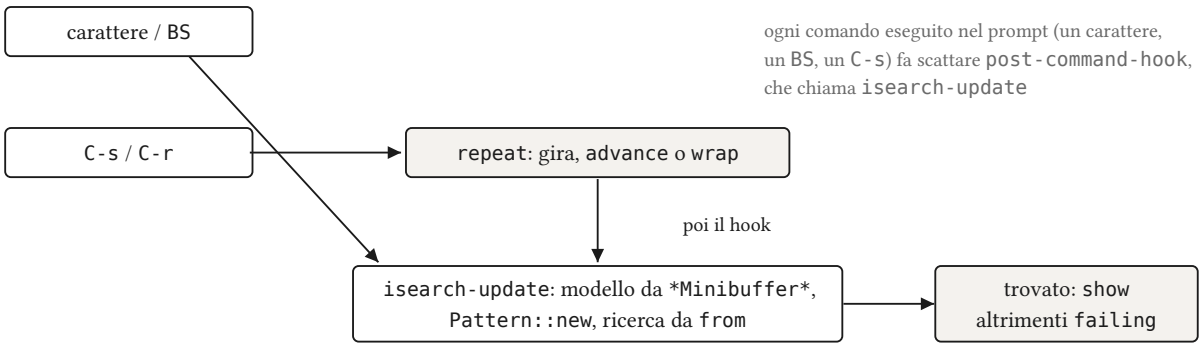
9.3 La sessione di ricerca incrementale: `Isearch`

La parte di interfaccia è descritta nel Sezione 7: `C-s` apre il prompt in `isearch-mode` e il `post-command-hook` di quel modo chiama `isearch-update` dopo ogni comando. Qui interessa lo stato che passa da un tasto all'altro.

da `text/isearch.rs` — estratto semplificato

```
pub struct Isearch {
    pub buffer: String,          // il buffer cercato (il corrente è il minibuffer)
    pub origin: usize,           // dove era il punto: ripristinato da C-g
    pub direction: Direction,    // Forward | Backward
    pub regexp: bool,
    pub from: usize,             // da dove parte la prossima scansione
    pub found: Option<Match>,    // la corrispondenza mostrata
    pub pattern: Option<Pattern>, // il modello compilato: le altre corrispondenze
    pub failing: bool,           // l'ultima scansione non ha trovato nulla
    pub wrapped: bool,           // la ricerca ha fatto il giro
    pub counting: Option<Counting>, // il conteggio delle corrispondenze
}
```

Il campo chiave è `from`. Scrivere un carattere *non* lo sposta: allungare il modello rifà la ricerca dallo stesso punto, quindi la corrispondenza cresce dove l'utente sta guardando. Solo la ripetizione lo sposta.



Le regole dei metodi, tutte prese dal comportamento di Emacs:

- `advance()` sposta `from` a `found.start + 1` andando avanti (a `found.end - 1` andando indietro), non alla fine della corrispondenza: così anche le corrispondenze sovrapposte sono raggiungibili. Ripetere la ricerca di `aa` in `aaa` trova anche quella che comincia in 1.
- `wrap(len)` riparte dall'altro capo del buffer e imposta `wrapped`. Viene chiamato solo quando si ripete una ricerca che sta *già* fallendo: il primo `C-s` a vuoto mostra «Failing I-search», il secondo è la decisione di fare il giro.
- Cambiare direzione non avanza: `from` diventa l'inizio (o la fine) della corrispondenza corrente, perché `C-s C-r` deve ripercorrere le stesse corrispondenze senza saltarne una.
- `report(pattern)` compone il testo per l'area dei messaggi: il conteggio (`count_label`, sotto), `[Failing ][Wrapped ]` seguito da `I-search`, `I-search backward`, `Regexp I-search` o `Regexp I-search backward`, poi `: ` e il modello.

`isearch-update` gestisce anche i casi limite. Con il prompt vuoto riporta il punto a `origin` e toglie il mark. Un'espressione che non si compila (`[a-` è a metà di `[a-z]`) è trattata come una ricerca fallita, non

come un errore: il carattere successivo la renderà valida. Quando trova, `show` mette il punto sull'estremità lontana della corrispondenza e il mark su quella vicina, così la corrispondenza è la regione e la disegna lo stesso codice che disegna una selezione. `isearch-exit` (Invio) lascia il punto sulla corrispondenza e mette il mark, disattivato, dove la ricerca è cominciata, come Emacs, così `C-x C-x` ci torna; `isearch-abort` (`C-g`, `ESC`) riporta il punto a `origin`.

9.3.1 Il conteggio delle corrispondenze

Il prompt dice a quale corrispondenza è la ricerca e quante sono nel buffer, come `3/17 I-search: foo`. Un `Counting` dice di quale modello, così come è stato scritto, e di quale versione del buffer è il conteggio, e lo contiene, quando è fatto, in un `Arc<OnceLock<MatchCount>>`: l'inizio di ogni corrispondenza, in ordine, fino a `COUNT_LIMIT` (100 000), e se ce n'erano altre. `count_label` ne ricava l'etichetta: l'indice è quante corrispondenze cominciano prima di quella mostrata o con lei (`partition_point`), `?` quando non ne è mostrata nessuna o quando sta oltre l'ultima contata di un conteggio troncato, `+` dopo il totale di un conteggio troncato, `0/0` quando non ce n'è nessuna; nulla finché il conteggio non è fatto.

`isearch-update` chiede un conteggio nuovo (`count`, in `primitives/isearch.rs`) quando il modello o la versione del buffer non sono più quelli del `Counting`; una ripetizione con `C-s` non conta di nuovo, e cambia solo l'indice. Le corrispondenze le trova `Pattern::starts`, che scorre il testo una volta come `Pattern::scan` e tiene solo gli inizi, in caratteri. Il buffer si copia sotto il suo lock e si conta a lock reso.

- In un buffer di `COUNT_AT_ONCE` caratteri o meno, un milione, si conta subito, nel comando, e il conteggio è nel messaggio che il tasto scrive.
- In uno più grande si conta su un thread dedicato, `isearch-count` (`spawn_work`, Sezione 11.7). Il thread copia il testo, rinuncia se la versione non è più quella chiesta, conta controllando a ogni corrispondenza lo `Stop`, riempie l'`OnceLock` e chiede una callback dovuta (Sezione 11.10.4): `isearch--count-arrived`, che sul thread dei comandi riscrive il messaggio della ricerca con il conteggio. Un conteggio nuovo avviato con lo stesso nome ferma il precedente, e la callback di quello si scarta perché il suo lavoro non parla più per il nome (`is_latest`); la chiusura della ricerca lo ferma con `cancel_named`.

Le decisioni:

1. **Il risultato nella sessione, il messaggio su chiamata.** `isearch-update` prende la sessione da `Runtime` e la rimette alla fine: un thread che scrivesse lì il suo risultato in quel momento non la troverebbe. Con l'`OnceLock` condiviso il risultato non si perde, chiunque tenga la sessione, e il messaggio lo riscrive una callback, che gira tra un comando e l'altro e trova la sessione al suo posto. Scartato: scrivere il messaggio dal thread, che lo farebbe con una sessione che può non esserci, e con un modello che nel frattempo può essere cambiato.
2. **Subito nei buffer piccoli.** Un thread per ogni tasto in un file di poche righe costerebbe più del conteggio, e il prompt mostrerebbe a ogni tasto un messaggio senza conteggio seguito, un attimo dopo, da quello con: il testo del prompt salterebbe avanti e indietro. Un milione di caratteri si conta in pochi millisecondi.
3. **Le corrispondenze contate non si sovrappongono.** Il conteggio è quello di `scan`, come `occur`; la ricerca invece si ferma anche sulle sovrapposizioni (`advance`). Nei rari casi in cui le due cose divergono, l'indice è quello dell'ultima corrispondenza contata che comincia prima; lo dice il manpage `searching`.
4. **Un tetto al conteggio.** Un modello che corrisponde a ogni carattere di un file grande terrebbe in memoria un inizio per carattere; 100 000 è più di quanto un numero nel prompt possa dire a chi lo legge.

9.4 La sessione di sostituzione: `Replace`

`replace-string`, `replace-regexp` e `query-replace` creano una `Replace` con `begin_replace` (`editor/replace.rs`). Il campo d'azione è la regione se il mark è attivo, altrimenti dal punto alla fine del buffer: `origin` e `limit` sono i due estremi. Poi `seek` cerca la prima corrispondenza e un overlay della categoria `replace-match` la evidenzia.

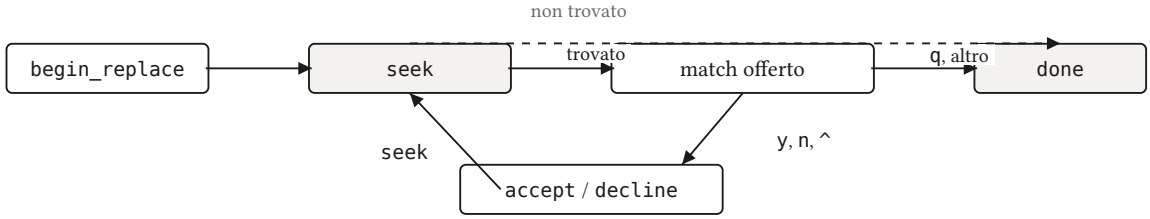
da `text/replace.rs` — estratto semplificato

```
pub struct Replace {
    pub buffer: String, pub pattern: Pattern,
    pub replacement: String,     // il modello, con \1 \& ancora dentro
    pub origin: usize, pub from: usize,
    pub limit: usize,            // spostato da ogni sostituzione
    pub found: Option<Match>, pub replaced: usize,
    pub history: Vec<Step>,      // da dove partiva ogni sostituzione accettata
    pub preserve_case: bool, pub done: bool,
}
```

Chi la guida chiama un verbo per ogni risposta. La vista predefinita (`default_view` in `primitives/replace.rs`) installa una keymap transitoria con cinque tasti: `y` (`replace-this`), `n` (`replace-skip`), `!` (`replace-rest`), `^` (`replace-back`, che usa `unaccept` e l'undo del buffer) e `q` (`replace-done`). La keymap è di tipo `OnUnbound::Release`: qualunque altro tasto toglie la domanda e poi fa ciò che fa di solito, e dopo di lui la sessione si chiude e la corrispondenza non è più evidenziata: la mappa ha come `owner` `REPLACE_QUESTION`, e `handle_key_event`, dopo ogni tasto, chiude una sostituzione che l'ha chiesta e non la vede più. Una vista scritta in Lisp non usa quella mappa, e chiude la sessione da sé. Un'interfaccia grafica userebbe pulsanti; nessuna delle due si esprime come un ciclo che legge una risposta.

Verbo	Effetto sulla sessione
`seek(text)`	`found = search_forward(text, from, limit)`; se non trova nulla, `done = true`.
`accept(new_len)`	La corrispondenza è stata sostituita da `new_len` caratteri. Registra `Step { from: start, limit }` in `history`; sposta `limit` della differenza di lunghezza; riprende da `start + new_len`, o da `start + new_len + 1` se la corrispondenza era vuota, come in Emacs. Incrementa `replaced`.
`decline()`	Salta la corrispondenza: riprende dalla sua fine, o un carattere dopo se era vuota.
`unaccept()`	Riprende l'ultimo `Step`: `from` e `limit` tornano com'erano, `replaced` cala. Il testo lo rimette a posto il chiamante con l'undo del buffer, perché la sessione non sa nulla dei buffer.

Il limite si sposta perché una sostituzione più lunga o più corta sposta tutto ciò che segue, compresa la fine della regione. Senza questo aggiornamento una sostituzione confinata alla regione ne uscirebbe, o si fermerebbe prima. La `history` serve a tornare indietro: dopo una sostituzione di lunghezza diversa il testo da solo non dice più dove era cominciata.



Il testo da inserire lo calcola, per ogni corrispondenza, `expand_replacement`, che riceve il modello di sostituzione, la corrispondenza, il testo effettivamente trovato e `preserve_case`:

- `\1` … `\9` sono i gruppi catturati; un gruppo che non ha partecipato, o qualsiasi gruppo con un modello letterale (che non ne ha), diventa la stringa vuota;
- `\0` e `\&` sono l'intera corrispondenza, presa dal testo (`matched`) e non dal gruppo 0, così funzionano anche con i letterali;
- `\\` è una barra rovesciata; una barra seguita da qualunque altra cosa resta com'è, così un percorso Windows si può scrivere senza raddoppiare le barre.

Con `preserve_case` (il valore di `case-replace` quando la ricerca ignora le maiuscole) il risultato viene poi riscritto secondo `Casing::of(matched)`:

Corrispondenza	`Casing`	`foo` diventa
`BAR`	`Upper`: tutte le lettere maiuscole (almeno due)	`FOO`
`Bar`, `B`	`Capitalised`: la prima maiuscola, le altre minuscole	`Foo`
`bar`, `bAr`, `fooBar`, `42`	`AsWritten`: minuscole, miste o nessuna lettera	`foo`

Sul lato dell'editor, `replace_this` cancella la corrispondenza e inserisce l'espansione con le due porte (`delete_range`, `insert_text`), chiama `accept` e poi cerca la successiva. Con `alone`, quando risponde a una domanda (`replace-this`), chiude un gruppo di undo: `replace-back`, che è un undo, toglie esattamente quella sostituzione. `replace_rest` (il `!` di `query-replace`, e tutto `replace-string`) ripete `replace_this` senza chiudere gruppi, così un undo toglie l'intera serie, e conta le sostituzioni fatte. Il tetto di `len + 1` giri è controllato prima di ogni sostituzione; nessuna sostituzione lo raggiunge davvero, perché ognuna fa avanzare la scansione di almeno un carattere del testo originale.

9.5 Il kill ring

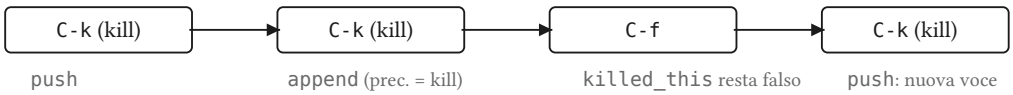
da `text/kill_ring.rs` — estratto semplificato

```
pub struct KillRing {
    entries: VecDeque<String>,  // la più recente in testa
    max: usize,                 // DEFAULT_KILL_RING_MAX = 60
    index: usize,               // da dove legge yank; azzerato da ogni kill
}
pub fn append(&mut self, text: String, direction: Direction) {
    if text.is_empty() { return; }
    match self.entries.front_mut() {
        Some(front) => {
            match direction {
                Direction::Forward => front.push_str(&text),
                Direction::Backward => front.insert_str(0, &text),
            }
            self.index = 0;
        }
        None => self.push(text),
    }
}
```

Un anello invece di un appunto singolo: uccidere del testo non distrugge quello che si stava per incollare. Le operazioni:

- `push(text)` aggiunge una voce in testa e azzera `index`; il testo vuoto è ignorato, così una kill di niente non allontana ciò che si voleva incollare; poi `trim` scarta le voci oltre `max`.
- `append(text, direction)` unisce il testo alla voce più recente: in coda per una kill in avanti, in testa per una all'indietro. Una serie di `C-k` diventa così un'unica voce che si legge nell'ordine giusto, e una serie di `M-DEL` non viene assemblata al contrario.
- `current()` è ciò che inserisce `yank`; `rotate()` avanza `index` di uno (con giro) ed è ciò che usa `yank-pop`; `nth(n)` legge senza muovere l'anello.
- `set_max` (`set-kill-ring-max`) riduce subito l'anello; se `index` finisce oltre la fine torna a 0.

La decisione tra `push` e `append` non è dell'anello ma del compartimento `KillYank` (`managers/kill_yank.rs`), che tiene due coppie di flag: `killed_last` / `killed_this` e `yanked_last` / `yanked_this`. Una kill accoda se anche il comando precedente era una kill; `roll_over`, chiamato all'inizio di ogni comando, sposta «questo» in «precedente» e azzera «questo». Allo stesso modo `yank-pop` è permesso solo se il comando precedente era uno yank, e `last_yank` dice quale tratto di testo sostituire.



tra un comando e l'altro `roll_over`: `killed_last = killed_this; killed_this = false`

Se la variabile `clipboard-sync` è vera (la imposta `clipboard.lisp`), ogni kill lascia anche la voce corrente in `pending_clipboard`. La `snapshot` successiva la prende (`take_pending_clipboard`) e il TUI la manda al terminale con una sequenza OSC 52. Prenderla invece di leggerla evita di rimandarla a ogni ridisegno.

L'ultimo rettangolo ucciso o copiato non va nell'anello: sta in un campo a parte di `KillYank`, una riga per stringa, e `yank-rectangle` lo legge da lì.

9.6 I rettangoli

Il punto e il mark sono due angoli; il rettangolo è il blocco di colonne che le righe tra loro hanno in comune. Non è la regione, che dalla metà di una riga alla metà di un'altra prende anche le righe intere in mezzo.

da `text/rectangle.rs` — estratto semplificato

```
pub struct Rectangle { pub top: usize, pub bottom: usize, pub left: usize, pub right: usize }
pub fn between(a: (usize, usize), b: (usize, usize)) -> Self {   // (riga, colonna)
    Self { top: a.0.min(b.0), bottom: a.0.max(b.0), left: a.1.min(b.1), right: a.1.max(b.1) }
}
```

`bottom` è incluso e `right` no: le righe sono un insieme di *cose*, le colonne un intervallo di *posizioni*. Un rettangolo largo zero è ammesso ed è utile: è una posizione su ogni riga, che è ciò che serve a `string-rectangle` per anteporre un prefisso a un gruppo di righe. Gli angoli possono essere dati in qualsiasi ordine.

Le colonne sono caratteri, e un tab è una colonna, come in tutto l'editor. Sullo schermo un tab arriva fino al prossimo multiplo di `tab-width` (Sezione 12.5), quindi su una riga con un tab il blocco disegnato non è quello che il rettangolo prende: Emacs conta le colonne dello schermo, e qui la scelta resta quella del modello.

`spans(text, rect)` traduce il rettangolo in un `Span` per riga, dall'alto:

da `text/rectangle.rs` — estratto semplificato

```
(rect.top..=rect.bottom.min(last_line)).map(|line| {
    let line_start = text.cursor_2d_to_1d(line, 0);
    let start = text.cursor_2d_to_1d(line, rect.left);    // limitato alla fine della riga
    let end = text.cursor_2d_to_1d(line, rect.right);
    let reached = start - line_start;
    Span { line, start, end, padding: rect.left - reached.min(rect.left) }
})
```

Tutto si regge sul fatto che `cursor_2d_to_1d` limita una colonna oltre la fine della riga alla fine della riga. Su una riga corta entrambi gli estremi cadono sulla fine e lo span è vuoto; `padding` dice quanti spazi mancano per raggiungere il bordo sinistro. Esempio con `left = 4`, `right = 7`:

Riga	Span	`padding`	Testo coperto
`0123456789`	4..7	0	`456`
`01234`	4..5	0	`4` (la riga finisce prima del bordo destro)
`01`	2..2	2	vuoto
(riga vuota)	0..0	4	vuoto

Che cosa fare delle righe corte lo decide l'operazione. Togliere testo (`kill-rectangle`, `copy-rectangle-as-kill`, `clear-rectangle`) prende ciò che c'è e non allunga la riga. Mettere testo (`open-rectangle`, `yank-rectangle`, `string-rectangle`) prima aggiunge il padding, perché una colonna che non esiste è proprio ciò che quelle operazioni devono riempire. Gli span sono dall'alto perché è l'ordine di lettura; chi modifica li percorre dal basso, perché una modifica a una riga sposta tutti gli offset successivi. `text_of` restituisce il testo di ogni span, con una stringa vuota per le righe che non raggiungono il rettangolo, così il blocco mantiene la sua forma.

9.7 Le liste di risultati

Occur, grep e la lettura dell'output di un compilatore producono la stessa cosa: una lista di posti dove andare. `results.rs` la definisce una volta, così produrre la lista e percorrerla sono lavori separati. Così anche le primitive: `primitives/occur.rs` produce le liste di occur e di grep (Sezione 14.4.3), e `primitives/results.rs` le legge, le percorre con `next-error`, ne attacca una da Lisp e la mostra.

Campo di `Entry`	Significato
`kind`	`"buffer"` o `"file"`: altrimenti `*scratch*` e un percorso si distinguerebbero solo indovinando.
`source`	Nome del buffer o percorso del file.
`line`, `column`	Riga da 1, colonna da 0.
`offset`	Offset in caratteri, per `goto-char`.
`text`, `start`, `end`	La riga intera e la posizione della corrispondenza al suo interno.
`at`	La riga dell'elenco su cui la voce è mostrata, da 1, quando il produttore la dice. La lista di una ricerca ha un'intestazione e poi una voce per riga, e non serve; l'uscita di un compilatore nomina i posti sulle righe dove il compilatore li ha scritti, e serve.

`Entry::new(kind, source, found)` copia i campi da un `Found` di `scan`; `rendered()` produce la forma `file:riga: testo` di ogni strumento alla grep. `Results` aggiunge il modello cercato, su che cosa (`over`), le voci, `truncated`, `done` (falso mentre una scansione di sfondo è in corso), il nome del buffer dove è stato messo l'ultimo evidenziatore (`marked`, per toglierlo: per una voce in un file, il buffer in cui la visita è finita, non il percorso che la voce dà) e la voce corrente.

da `text/results.rs` — estratto semplificato

```
pub fn step(&self, step: isize) -> Option<usize> {
    if self.entries.is_empty() { return None; }
    let next = match self.current {
        None if step >= 0 => 0,                    // prima visita in avanti: la prima
        None => self.entries.len() - 1,            // prima visita indietro: l'ultima
        Some(current) => {
            let next = current as isize + step;
            if next < 0 || next as usize >= self.entries.len() { return None; }
            next as usize
        }
    };
    Some(next)
}
```

Il passo non fa il giro, di proposito: la fine della lista è un'informazione («era l'ultimo»), e ricominciare in silenzio farebbe correggere due volte il primo errore. Un insieme di risultati vive nel buffer che lo mostra, in `Buffer::data` sotto la chiave `"results"` (`RESULTS_KEY`): quando il buffer viene chiuso i risultati spariscono con lui, e rinominarlo non rompe nulla. Le voci conservano riga, colonna e offset, e alla prima visita

di un posto ricevono un *marcatore* nel buffer che lo mostra, loro e tutte quelle della stessa sorgente, come fa Emacs con gli errori di una compilazione: una voce della tabella `Markers` del buffer (`buffer/markers.rs`), un offset che le tre porte spostano come gli overlay, e che un inserimento proprio lì spinge avanti, così resta sulla corrispondenza. Da lì le visite vanno al marcatore, e righe aggiunte o tolte sopra non le spostano. I marcatori stanno in `Results::markers`, per voce; una lista rifatta, come quella di una compilazione prima di ogni passo, li passa alle voci che sono lo stesso posto (`Entry::place`) e toglie gli altri, e una lista chiusa li toglie tutti dai buffer in cui erano.

Dove una lista si mostra lo decide `editor/results.rs`. Accanto a ogni lista, nei dati del suo buffer, sta una `Presentation`: chi l'ha prodotta, da quale finestra e da quale buffer è stata chiesta, quale buffer ha preso il posto e in quale finestra si vede. `prepare_results_buffer` sceglie il buffer di un'esecuzione nuova: con `results-reuse-buffer`, la lista più recente dello stesso produttore e dello stesso modo, fermando il lavoro che vi scriveva; altrimenti un buffer nuovo. `display_results` la mostra nella finestra con il focus, se `results-display-location` è `same-window`, o in una striscia in basso, riusando quella che mostra una lista nello stesso modo: occur e grep, due produttori le cui liste sono in `occur-mode`, si passano la striscia, mentre una compilazione ha la sua. `select_results_target` sceglie la finestra in cui una visita apre il posto (la lista stessa con `results-visit-same-window`), e `quit_results` chiude la striscia o rimette il buffer che la lista aveva sostituito.

Il percorso di una voce in un file si legge nella directory della lista, `directory_of` del suo buffer (Sezione 14.3.2), e passa a `find-file` già assoluto. Una lista che ha una directory sua, come l'uscita di `compile`, legge dunque i nomi relativi dove la build è girata, anche dopo un `cd` e anche con `M-g n` chiesto da un altro buffer; una lista senza, come quella di `grep`, i cui percorsi sono già assoluti, li legge nella directory di lavoro.

9.8 Il riempimento

Riempire vuol dire disporre di nuovo le parole di un paragrafo, quante ne stanno su ogni riga prima di una colonna, dietro un prefisso. Le parti sono due, come per la ricerca: `text/fill.rs` sa di prefissi e di colonne e nulla di buffer e di modi; `primitives/fill.rs` trova nel buffer il paragrafo, o il commento, e i suoi prefissi, e fa la modifica.

In `text/fill.rs`	Che cosa fa
`Measure`	La colonna da non superare, la larghezza di un Tab e quante celle prende un carattere: una funzione ricevuta, `ui::width::char_cells` dall'editor, perché `text/` non usa `ui/`. `column_after` dice dove finisce un testo che comincia a una colonna.
`lay_out`	Le parole in righe, la prima dietro un prefisso e le altre dietro un secondo, finché `colonna + 1 + parola` non supera la larghezza; una parola troppo lunga ha una riga sua.
`comment_prefix`	Il prefisso di commento di una riga: rientro, l'apertura più lunga tra quelle del modo, altri caratteri uguali all'ultimo dell'apertura o un `!` (`///`, `//!`, `;;;`), e gli spazi dopo.
`break_at`	Dove spezzare una riga che si sta scrivendo: l'ultima serie di spazi la cui parola precedente finisce entro la colonna, o la prima dopo una prima parola troppo lunga; nulla se il testo, senza gli spazi finali, sta nella colonna.

In `primitives/fill.rs` ogni riga ha un `Kind`: vuota (anche un commento senza testo dopo l'apertura), codice, testo dietro il suo rientro, commento dietro il suo prefisso. In un modo con commenti di riga ogni riga che non è un commento è codice; in uno senza, è testo. `paragraph_at` allarga la riga del punto in su e in giù finché le righe sono dello stesso gruppo (`Kind::joins`): testo con lo stesso rientro, o commenti con lo stesso prefisso a meno degli spazi dopo l'apertura. `filled` prende le parole dopo il prefisso di ogni

riga e le dispone dietro il prefisso della prima riga e quello della seconda. `refill` sostituisce il testo solo se cambia, con `delete_range` e `insert_text` nello stesso gruppo di undo, e rimette il punto dopo lo stesso numero di caratteri di parola, prefissi esclusi (`words_before`, `after_words`). `fill-region` raccoglie i paragrafi che toccano le righe della regione e li riempie dall'ultimo, così le righe di ognuno sono ancora dove sono state trovate, spostando il punto della differenza di lunghezza di quelli prima di lui.

La colonna la legge `setup`, un lock alla volta: il modo e il valore del buffer (la chiave `fill-column` dei dati del buffer, quella che `buffer-get` legge e `set-fill-column` scrive), poi la proprietà `fill-column` del modo e la variabile, poi le aperture dei commenti di riga dalla tabella sintattica. `do-auto-fill` usa `break_at` sulla parte della riga prima del punto, e mette al posto degli spazi un a capo e lo stesso prefisso; `auto-fill-mode`, nel modulo `fill` (Sezione 15.5.15), lo chiama dopo uno spazio scritto.

Le decisioni:

1. **Un paragrafo ha un solo rientro.** In Emacs un paragrafo finisce solo a una riga vuota, e `M-q` sotto un titolo attaccato al testo, come nelle pagine di questo manuale dell'utente, unisce il titolo al paragrafo. Qui una riga con un altro rientro comincia un altro paragrafo. Il prezzo è che un rientro sospeso nel testo (una prima riga meno rientrata delle altre) non si riempie come un paragrafo solo; nei commenti sì, perché il gruppo ignora gli spazi dopo l'apertura.
2. **Nel codice solo i commenti.** Riempire righe di codice le romperebbe, e il modo non ha regole per spezzare un'espressione: `M-q` su codice lo dice e non fa nulla, e `do-auto-fill` non spezza. Così `auto-fill-mode` si può accendere anche in un modo di programmazione.
3. **Solo i commenti di riga.** Le righe interne di un `/* */` non hanno un prefisso dichiarato dal modo (` * ` è una convenzione, non una sintassi): si trattano come codice, e il manpage `filling` lo dice nei limiti.
4. **La colonna del buffer nei suoi dati.** `set-fill-column` vale per un buffer, come in Emacs, dove `fill-column` è locale al buffer; qui non ci sono variabili locali, e i dati del buffer, sotto una chiave che anche Lisp legge, ne fanno le veci.
5. **Uno spazio solo tra le parole.** Due spazi dopo un punto diventano uno: riconoscere la fine di una frase chiederebbe regole per lingua.

9.9 Le decisioni

1. **`text/` è una directory, con una regola d'ammissione.** Un file sta qui se lo potrebbe usare un programma che non è un editor. Messi alla radice del crate, accanto a `editor/` e `ui/`, questi file farebbero una lista di moduli senza un principio, e lo strato sarebbe soltanto descritto. Una directory lo rende visibile e controllabile: `layering_tests` fallisce se un file di qui nomina l'editor o l'interprete.
2. **Ricerca e sessioni stanno sotto l'editor.** La ricerca ha bisogno solo di un testo che dia i caratteri per posizione: né punto, né mark, né undo, né lock. Così si prova su un buffer qualunque, senza un editor intorno, e la ricerca incrementale, la sostituzione e occur non possono avere due idee diverse di che cosa sia una corrispondenza. Lo stesso vale per l'aritmetica di una sostituzione: da dove riparte, di quanto sposta ciò che segue, quando si ferma.
3. **Un'espressione cerca su una copia del testo.** Il motore (`regex`) lavora su una `&str`, quindi la ricerca con un'espressione copia il buffer a ogni chiamata, mentre quella letterale non copia nulla. La ricerca incrementale ricerca a ogni tasto, e su un buffer grande una ricerca con un'espressione costa una copia per tasto. È la versione semplice; quella che scala darebbe il testo a pezzi, senza copiarlo, alla ricerca a flusso di `regex_automata`.
4. **Le espressioni di `string-match` si compilano una volta.** Compilarne una costa circa sette decimi di millisecondo: niente quando un modo dichiara la sua grammatica, un blocco quando succede in un ciclo. I moduli chiamano `string-match` per ogni elemento: la sorgente dei completamenti del manuale toglie l'estensione a ogni pagina che trova, e senza la cache quattromila pagine sarebbero quattromila compilazioni, 2,7 secondi a ogni Tab sul thread che disegna. Con la cache le stesse chiamate costano venti millisecondi.

5. **La cache è per thread, e quando è piena si svuota tutta.** Per thread, perché trovare una corrispondenza non prenda un lock: Lisp gira sul thread dei comandi e su quello dell'esecutore, e una tabella condivisa li metterebbe l'uno sulla strada dell'altro nella chiamata più frequente che c'è. Svuotarla tutta invece di scartare le voci vecchie basta: le espressioni vengono dal sorgente dei moduli e sono qualche decina, e il limite serve solo a un programma che le *genera* dal suo input. Buttare tutto costa una compilazione per ognuna ancora in uso, cioè, nel caso peggiore, il comportamento senza la cache.
6. **Ricerca incrementale e sostituzione sono sessioni, non cicli.** Una ricerca incrementale è un ciclo sui tasti, e una sostituzione un ciclo sulle risposte; ma in rsedit non c'è una lettura bloccante di un tasto, perché i tasti arrivano uno per giro del ciclo degli eventi, e un pulsante di un'interfaccia grafica non è una lettura affatto. Il ciclo è quindi rovesciato, come la raccolta degli argomenti con `PendingCommand` (Sezione 6): la sessione tiene ciò che il ciclo terrebbe nelle sue variabili, e chi la guida chiama un metodo per ogni giro. La sessione non nomina buffer, finestre o lock, solo il nome del buffer e posizioni nel suo testo: si legge e si scrive senza tenere nulla, e descrive una ricerca in un buffer che non è il corrente, come è sempre mentre il prompt è aperto.
7. **La sostituzione segue le maiuscole di ciò che sostituisce.** Una ricerca che ignora le maiuscole e una sostituzione letterale trovano il posto giusto e ci scrivono la cosa sbagliata: sostituire `colour` con `color` in un documento trasforma il `Colour` a inizio frase in `color`, e ognuno va corretto a mano. È ciò che più si nota quando manca.
8. **Un anello solo per tutto l'editor.** Uccidere in un buffer e incollare in un altro è lo scopo del kill ring: l'anello sta sull'editor, in `KillYank`, e non sul buffer, così il testo passa da uno all'altro.
9. **Una riga corta resta nel rettangolo.** Una riga che finisce prima del bordo sinistro è comunque una delle righe del rettangolo, con uno span vuoto e il suo `padding`. Toglierla farebbe dipendere l'altezza del blocco da ciò che contiene.
10. **Il punto e l'evidenziazione della sostituzione stanno nell'editor, la domanda no.** Spostare il punto sulla corrispondenza serve a ogni vista, e una vista che dovesse farlo da sé potrebbe sbagliarlo; segnare la corrispondenza pure, perché una corrispondenza che non si vede è inutile in ogni vista. Ciò che resta alla vista è chiedere: nulla in `editor/replace.rs` mette una domanda sullo schermo o legge un tasto. Una vista che disegna da sé toglie l'overlay `replace-match`.

10 Modi

10.1 Il modo maggiore

Un buffer ha un solo modo maggiore, nominato in `current_mode`. Il modo è un `MajorMode` (`modes/mod.rs`) nel registro di `Modes`:

Campo	Contenuto
`keymaps`	Le associazioni del modo, consultate prima di quelle globali.
`grammar`	Regole e regioni di colorazione (`add-syntax-rule`, `add-syntax-region`).
`hooks`	Hook per nome (`post-command-hook`, `post-self-insert-hook`, `after-close-hook`, …), aggiunti con `(add-hook MODO NOME FUNZIONE)`.
`completion_functions`	Le fonti di `completion-at-point` per questo modo.
`syntax_table`	Delimitatori, stringhe, commenti, quote dei caratteri: ciò che legge lo scanner delle espressioni; `None` per la tabella predefinita.
`lighter`	Per un modo minore, ciò che la riga di stato mostra mentre è acceso; `None` per un modo maggiore (Sezione 10.2).

`Modes` contiene inoltre la keymap globale, i completamenti globali, la lista `auto_modes` (espressione regolare sul nome del file → modo, da `add-auto-mode`), la keymap transitoria, i tasti di ripetizione, gli hook globali, che valgono per ogni modo e si concatenano a quelli del modo, e le regole dei modi minori.

`make-mode` registra un `MajorMode` nuovo sotto il nome dato. Se il nome esiste già (`EditorState::make_mode`), il modo resta con i suoi tasti, i suoi hook e i suoi completamenti, chiunque li abbia dati, e ricominciano da capo soltanto la grammatica e la tabella sintattica: il file di un modo le costruisce aggiungendo, e ricaricato dopo una modifica le avrebbe doppie, mentre `define-key` sostituisce un legame e `add-hook` non ripete una funzione. Come `set_mode`, sposta l'edizione delle grammatiche, così ciò che era colorato con la vecchia si colora di nuovo.

Le proprietà dei simboli completano il modo senza toccare la struttura Rust; le primitive Rust dell'indentazione e `indent.lisp` leggono queste:

```
(put 'rust-mode 'tab-width 4)
(put 'risp-mode 'indent-function 'lisp-indent-line)
(put 'python-mode 'indent-levels-function 'python-indent-levels)
(put 'rust-mode 'indent-region-style 'shift)
```

10.2 I modi minori

Un modo minore è un `MajorMode` con un'etichetta (`lighter: Some(...)`, anche vuota): sta nello stesso registro dei modi maggiori, così `define-key` e `add-hook` gli danno tasti e hook con le primitive di sempre, ma un buffer non lo può avere come modo maggiore (`set-major-mode` lo rifiuta). Lo si accende e spegne in un buffer alla volta: `Buffer::minor_modes` è la lista di quelli accesi, nell'ordine in cui lo sono stati, ognuno con il nome e l'etichetta copiata all'accensione (`ActiveMinor`).

Dove conta	Come
I tasti	`keymap_modes` mette dopo la keymap locale quelle dei modi minori accesi, dall'ultimo acceso, e poi quella del modo maggiore; `source_of` dà a un legame trovato in un modo minore la sorgente `Minor`, che `key-binding` e `C-h b` chiamano `"minor"`.

Dove conta	Come
Gli hook	`before-save-hook` usa il percorso che interrompe il salvataggio al primo errore (Sezione 8.7); gli altri passano da `run_hook`, che prende i modi minori del buffer corrente e chiama `run_hook_of`: prima gli hook del modo maggiore, poi quelli di ogni modo minore acceso, nell'ordine di accensione, poi i globali (`Modes::hooks_for`). `after-close-hook` riceve i modi minori del buffer chiuso, presi prima che se ne vada.
L'accensione	`minor_mode_on` e `minor_mode_off` cambiano la lista sotto il lock del buffer e poi, rilasciato, fanno girare `minor-mode-hook` del modo, e quelli globali con quel nome, con il buffer corrente (`run_minor_mode_hook`).
Le regole	`minor-mode-enable` registra in `Modes::minor_rules` una coppia (modo maggiore, o nessuno per tutti; modo minore). `apply_minor_rules` accende quelli che valgono per il modo del buffer, dopo `major-mode-hook` e prima di `after-change-major-mode-hook`, in `initialize_buffer` e in `change_major_mode`. `minor-mode-disable` toglie la coppia registrata con gli stessi due argomenti (`Modes::remove_minor_rule`), e lascia i buffer come sono.
Il Rust	Dove l'effetto di un modo è in Rust, il codice chiede a `Buffer::minor_mode_enabled` un nome che conosce, sotto il lock del buffer che ha già: `visual_lines::MODE` (Sezione 12.5.1), `SHOW_PAREN_MODE`, `WHITESPACE_MODE` e `hl-line-mode` (Sezione 12.4), `subword-mode` (Sezione 14.2.1), `gutter::MODE` (Sezione 12.6), `OVERWRITE_MODE` (Sezione 14.2). Il modo lo fa un modulo Lisp; il solo che fa il nucleo è `READ_ONLY_MODE` (Sezione 8.8).
Il cambio di modo maggiore	`change_major_mode` svuota la lista sotto lo stesso lock con cui cambia il modo, senza far girare hook, lasciandovi solo `read-only-mode`, e poi applica le regole del modo nuovo.
La riga di stato	`%n` scrive le etichette non vuote, ognuna dopo uno spazio, dalla lista del buffer: `expand_mode_line` non deve chiedere il registro.

In Lisp, `define-minor-mode` (modulo `commands`, Sezione 15.5.1) fa un modo minore e un comando con lo stesso nome e la specifica `"P"`, che legge il prefisso con `minor-mode-wanted-p` come in Emacs: niente per invertire, un numero positivo o `C-u` per accendere, zero, un negativo o `-` per spegnere.

Le decisioni:

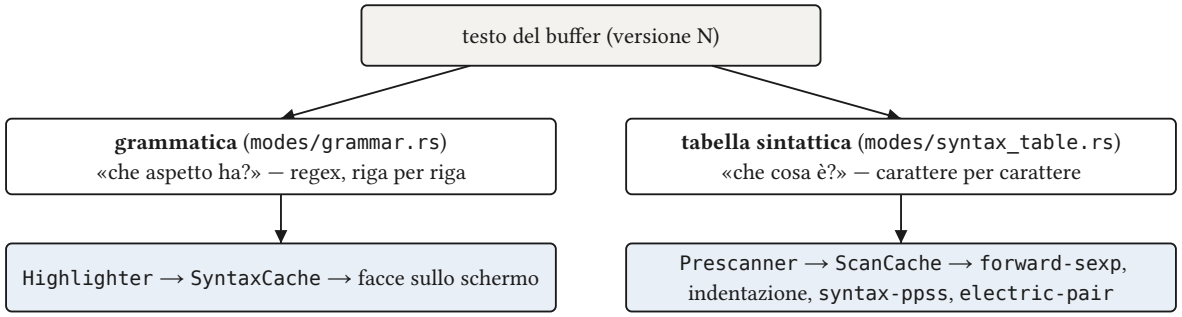
1. **Nello stesso registro.** Un modo minore ha keymap e hook come un modo maggiore, e le primitive che li danno chiedono un nome di modo: con un registro solo funzionano senza cambiare. Lo distingue l'etichetta, che solo un modo minore ha. Scartato: un registro a parte, che avrebbe raddoppiato `define-key`, `add-hook` e il loro aiuto.
2. **Dopo la keymap locale.** La keymap locale è quella di vim, che prende tutto il buffer per uno stato: come le mappe di emulazione di Emacs, sta sopra i modi minori. Tra due modi minori vince l'ultimo acceso, che è quello che chi scrive ha voluto più di recente.
3. **Spenti, senza hook, quando cambia il modo maggiore.** Come Emacs, che li perde con le variabili locali del buffer: un `auto-fill-mode` acceso per un file di testo non resta acceso se il buffer diventa Rust. Le regole del modo nuovo li riaccendono, con i loro hook. Far girare gli hook di spegnimento vorrebbe dire eseguire Lisp mentre il buffer sta cambiando modo, prima dei suoi `major-mode-hook`. Fa eccezione `read-only-mode`, che resta (Sezione 8.8).
4. **L'etichetta copiata nel buffer.** La riga di stato si compone con il buffer e i lock del fotogramma: leggere il registro lì vorrebbe dire prendere `modes` sotto `windows` e `buffers`, che l'ordine dei lock non consente

(Sezione 4.5). Un'etichetta cambiata con `make-minor-mode` vale per i buffer dove il modo si accende da allora.

5. **Solo per buffer, e ovunque con una regola.** Non c'è un modo minore globale. Un modo che vale ovunque — `show-paren-mode`, `whitespace-mode`, `electric-indent-mode` — è un modo minore che il suo modulo chiede per ogni modo maggiore con `minor-mode-enable`, e accende nei buffer già aperti. Un buffer che lo spegne da solo lo ritrova acceso quando cambia modo maggiore, come ogni modo chiesto da una regola; `minor-mode-disable` toglie la regola, per i buffer che verranno. Scartato: una variabile globale con una proprietà per modo maggiore, un secondo meccanismo per la stessa domanda, che non sa rispondere per un buffer solo. Lo dice il manpage `modes` nei limiti.
6. **Il Rust legge il nome nel buffer, non l'ambiente.** La fotografia e il movimento hanno già il buffer sotto lock: chiedergli se un modo è acceso non prende altri lock e non chiama l'interprete. La pulizia al salvataggio usa invece l'hook del modo minore, eseguito senza lock (Sezione 8.7). Senza il modulo che lo fa, il modo non c'è e non è acceso in nessun buffer: il nucleo da solo non disegna coppie né spazi, e non pulisce.

10.3 Due lessici sullo stesso testo

Il testo di un buffer viene letto in due modi diversi, per due domande diverse.



Le due descrizioni sono indipendenti: aggiungere una forma di stringa a un modo significa aggiungerla in entrambe. I file sono divisi allo stesso modo: le strutture in `modes/grammar.rs` e in `modes/syntax_table.rs`, e le primitive in `primitives/grammar.rs`, che dichiara la grammatica (`add-syntax-rule`, `add-syntax-region`), e in `primitives/syntax_table.rs`, che dichiara la tabella e dà le risposte dello scanner (`syntax-ppss`, `balance-point`, `bounds-of-enclosing-list`); `make-mode`, gli hook e `add-auto-mode` sono in `primitives/modes.rs`.

10.3.1 La grammatica

La *grammatica* colora una riga alla volta portando uno stato da una riga all'altra: lo stato è la pila delle regioni aperte (un commento a blocchi che continua, una stringa su più righe). Colorare la riga 5000 richiede solo lo stato all'inizio della riga 5000, che la cache conserva; dopo una modifica si ricolora dalla riga modificata in giù. In ogni riga tutti i modelli attivi sono provati insieme e *vince la corrispondenza più a sinistra* a parità di posizione le regioni vengono prima delle regole. Così `// this /* is not` è un commento di riga e non apre un commento a blocchi. Le espressioni regolari sono quelle della crate `regex`, senza lookahead, lookbehind né backreference: per questo una regola può colorare solo un gruppo (`SyntaxRule::group`) e una regione può dichiarare una sequenza di escape. Se il gruppo non prende parte alla corrispondenza non si colora nulla, e la scansione scavalca comunque la corrispondenza intera; un gruppo che la regex non ha è rifiutato da `add-syntax-rule`, come una regex sbagliata. `highlight_line` colora una riga a partire dallo stato con cui vi si entra e restituisce quello con cui se ne esce; la applica l'evidenziatore (Sezione 11.10.1).

Ogni passo del ciclo di `highlight_line` avanza o chiude una regione: un inizio di regione e un escape contano solo dove corrispondono a qualcosa (`find_nonempty`), una regola avanza almeno di un byte, e una fine vuota chiude una delle regioni aperte, che sono in numero finito. Così il ciclo finisce qualunque siano i pattern; `add-syntax-region` rifiuta comunque, con una riga nel log, un inizio o un escape che corrisponde

alla stringa vuota, perché non è ciò che si voleva dire. Una fine vuota invece è ammessa: `$` chiude la regione alla fine della riga.

`Ahead` ricorda, per ogni pattern della grammatica, la prossima corrispondenza trovata e la posizione da cui la si è cercata. La scansione va solo in avanti, quindi una corrispondenza che comincia dopo la posizione corrente è ancora la prossima, e un pattern che non ha trovato nulla non troverà nulla più avanti: un pattern è interrogato di nuovo solo quando la scansione ha superato la sua corrispondenza. Allo stesso modo `Columns` converte i byte in colonne contando dall'ultima conversione, non dall'inizio della riga. Una riga costa così la sua lunghezza per il numero dei pattern, e non per il numero degli eventi: una riga di 260 KB, che prima chiedeva 85 secondi nella build di debug, ne chiede 0,13.

10.3.2 La tabella sintattica

La *tabella sintattica* dice quali caratteri aprono e chiudono liste, quali delimitano stringhe (`set-string-syntax`, anche multi-carattere come `r#"`), quali commenti (`set-comment-syntax`) e quale carattere quota un carattere (`set-char-quote`). Una stringa multi-carattere è uno *stile*: l'apertura, la chiusura e se una barra rovesciata vi fa da escape, come nelle triple virgolette di Python e non in una stringa grezza. Ogni cambiamento dà alla tabella un'edizione nuova (`SyntaxTable::edition`), con cui le fotografie dello scanner dicono con quale tabella sono state prese (Sezione 10.4.1).

10.4 Lo scanner delle espressioni

`modes/sexp.rs` risponde alle domande sulla struttura del testo: dove finisce l'espressione dopo il punto (`forward-sexp`), dove comincia quella prima (`backward-sexp`), in quale lista, stringa o commento si trova una posizione (`syntax-ppss`, che l'indentazione e `electric-pair` usano). Contare le parentesi non basta: in `(message "close it with )")` tre parentesi su quattro sono testo. Lo scanner quindi fa un'analisi lessicale con quattro stati, e conta un delimitatore solo quando lo sta leggendo come codice. Gli stati e le transizioni:

Stato	Ci si entra da `Code` quando…	Si torna a `Code` quando…
`Code`	— (stato iniziale)	—
`Str { quote, start }`	si legge un carattere di classe `StringQuote`.	si rilegge la stessa virgoletta; un carattere `Escape` fa saltare anche il successivo.
`StyledStr { style, start }`	comincia un'apertura multi-carattere dichiarata con `set-string-syntax` (`r#"`, `"""`).	si legge la chiusura dello stile. Una barra rovesciata fa da escape solo se lo stile lo dice: in una stringa grezza `\` è solo `\`.
`Comment { style, depth, start }`	comincia un'apertura di commento di `set-comment-syntax` (vince la più lunga).	riga: al `\n`. Blocco: alla chiusura con `depth` 0; se lo stile è `nestable` un'altra apertura incrementa `depth`.

Lo stato completo di una scansione è lo stato lessicale più una pila di `Frame`, uno per ogni lista aperta. Il fondo della pila è il buffer stesso e non viene mai tolto: una chiusura in più non svuota mai la pila.

da `modes/sexp.rs` — estratto semplificato

```
struct Frame {
    start: usize,              // inizio dell'espressione, prefisso incluso
    delimiter: usize,          // dove sta la parentesi
    children: VecDeque<Sexp>,  // le ultime `keep` espressioni completate qui
}
pub struct Scan<'a, B: BufferTrait> {
    text: &'a B, table: &'a SyntaxTable,
    pos: usize, state: State, stack: Vec<Frame>,
    prefix: Option<usize>,     // un carattere prefisso (' ` # ,) in attesa
    keep: usize,
}
```

`step()` fa un passo e dice se ha completato un'espressione (`Completed(depth, Sexp)`), se ha solo consumato caratteri (`Advanced`) o se il testo è finito (`End`). In stato `Code` l'ordine dei controlli conta:

1. Un'apertura di commento (la più lunga che corrisponde) entra in `Comment` e scarta un prefisso in attesa. Un commento, a blocchi o di riga, non è un'espressione: si passa come uno spazio, come in Emacs con `parse-sexp-ignore-comments`. Fino alla correzione di SIN-9 un commento a blocchi completato contava come espressione, e `forward-sexp` si fermava alla sua fine, mentre uno di riga si passava.
2. Un'apertura di stringa multi-carattere viene prima dei caratteri singoli, perché contiene virgolette che altrimenti aprirebbero una stringa normale.
3. Un letterale di carattere (`'a'`, `'"'`) è un'unica espressione. Il `'` non ha una classe fissa: apre un carattere in `'a'`, una lifetime in `'static`, un apostrofo nella prosa; lo decide `SyntaxTable::char_literal_at` guardando ciò che segue.
4. Poi la classe del carattere: `Open` mette un `Frame` sulla pila; `Close` toglie quello in cima e completa la lista. Una chiusura che non corrisponde all'apertura chiude lo stesso, e una chiusura senza niente di aperto è testo: un file è sbilanciato per tutto il tempo in cui lo si scrive, e uno scanner che si fermasse lì sarebbe inutile proprio quando serve. `StringQuote` entra in `Str`; `Escape` salta due caratteri; `Prefix` ricorda la posizione del primo prefisso di una serie (`#'foo` comincia al `#`); `Symbol` consuma l'intera serie di costituenti e completa un'espressione; `Punctuation` avanza, e uno spazio stacca un prefisso in attesa.

Ogni espressione completata entra nei `children` del frame in cima, che ne tiene al massimo `keep`. È il limite che rende la scansione economica: `backward-sexp` con un conteggio N ha bisogno solo delle ultime N.

Funzione	Come usa la scansione
`forward(from, n)`	Scansione fino a `from`, poi passi finché non si completano `n` espressioni *alla profondità di* `from` che finiscono dopo `from`; se il testo, o la lista di `from`, finisce prima, il punto non si muove: la fine di una lista non è un passaggio verso la successiva. Da dentro un simbolo il primo è il simbolo stesso: il passo che porta la scansione a `from` lo completa, e `run_to` lo restituisce.
`backward(from, n)`	Scansione fino a `from` con `keep = n`; il frame rimasto aperto è la lista in cui si trova il punto, e i suoi `children` sono i fratelli prima del punto. Restituisce l'inizio dell'N-esimo dal fondo, prefisso incluso.
`down(from)`	Scansione finché la profondità non supera quella di partenza: il punto è appena entrato in una lista. Si ferma con `None` quando scende sotto quella di partenza: oltre la fine della lista di `from` una lista più profonda sarebbe annidata in una che segue.

Funzione	Come usa la scansione
`context_at(pos)`	Scansione fino a `pos`, poi `context()`: profondità, inizio e delimitatore della lista più interna, inizio della stringa o del commento in corso. È ciò che restituisce `syntax-ppss`.
`balance_point(pos)`	La prima posizione dopo `pos` in cui tutte le liste aperte in `pos` sono chiuse. Non chiede se il buffer è bilanciato alla fine: una funzione chiusa resta chiusa anche se quella sotto è ancora da finire.
`enclosing(pos)`	La lista più interna che contiene `pos`, per intero (fino alla fine del testo se non è chiusa); `None` al livello zero.
`pairing(point, reach)`	Il delimitatore accanto al punto, se è codice, con quello che gli corrisponde e se sono dello stesso tipo: per un'apertura, passi finché la pila non torna alla sua profondità; per una chiusura, il frame in cima. Non legge più di `reach` caratteri per parte (Sezione 12.4).

10.4.1 Riprendere da metà: `Resume` e il `Prescanner`

Andare all'indietro non si può fare localmente: se una `)` prima del punto sia codice o testo dipende da tutto ciò che la precede, perché una stringa aperta duecento righe sopra la rende testo. Quindi ogni scansione va in avanti, e può cominciare solo in una posizione di cui si conosce lo stato: l'inizio del buffer, o un `Resume`, una fotografia dello stato presa prima, che `Scan::begin` accetta:

da `modes/sexp.rs` — estratto semplificato

```
pub struct Resume { pos: usize, state: State, stack: Vec<Frame>, prefix: Option<usize> }
pub fn begin(text, table, before: usize, resume: Option<&Resume>, keep: usize) -> Self {
    if let Some(from) = resume.filter(|from| from.pos <= before) {
        return Self { pos: from.pos, state: from.state.clone(),
                      stack: from.stack.clone(), .. };          // riprende da lì
    }
    Self { pos: 0, state: State::Code,
           stack: vec![Frame { start: 0, delimiter: 0, .. }], .. }  // dall'inizio
}
```

Il controllo `from.pos <= before` è l'intero argomento di sicurezza: una fotografia è solo una scorciatoia per arrivare a una posizione, mai un sostituto. Una fotografia sbagliata o troppo avanti costa una scansione completa, non una risposta sbagliata. Le fotografie sono prese senza i `children` dei frame, che dipendono da `keep`: per questo `backward` non riprende mai e scansiona sempre dall'inizio del buffer.

Le fotografie le produce il `Prescanner`, un lavoro di sfondo come l'evidenziatore, in `jobs/prescan.rs` (Sezione 11.10.1). Ne prende una ogni `LINES_PER_CHECKPOINT` (64) righe, all'inizio delle righe 64, 128, 192… (`checkpoint_line`), e le salva nella `ScanCache` del buffer insieme alla versione del testo, al nome del modo e all'edizione della tabella con cui le ha prese. Una fotografia presa con un'altra tabella sarebbe una risposta sbagliata che sembra giusta, e per questo modo ed edizione sono controllati. Dopo una modifica alla riga L, `invalidate_from` tiene le fotografie delle righe fino a L (lo stato all'ingresso di una riga dipende solo dal testo prima) e scarta le altre; `record` accetta solo la fotografia successiva all'ultima valida. `Buffer::scan_resume(pos, table)` sceglie quella della riga multipla di 64 più vicina sotto `pos`, se versione, modo ed edizione corrispondono. A differenza dell'evidenziatore, il `Prescanner` scandisce con il lock di lettura del buffer preso.

Lo scanner è una funzione pura del testo, della tabella e di una posizione: `sexp_tests` lo prova con tabelle costruite in Rust, senza buffer né registro dei modi, e poi che un modo sa dire tutto ciò da Lisp; `balance_point_tests`, `char_quote_tests` e `raw_string_tests` lo provano come lo incontra un utente, con `electric-pair`, e `scan_cache_tests` prova le fotografie.

10.5 Facce e temi

Una faccia è un `Face(u32)`: un indice in un registro globale. Le prime 18 sono predefinite (`default`, `region`, `mode-line`, `mode-line-inactive`, `window-separator`, `keyword`, `type`, `string`, `comment`, `function`, `builtin`, `line-number`, `line-number-current`, `replace-match`, `lazy-highlight`, `show-paren-match`, `show-paren-mismatch`, `trailing-whitespace`); un modulo crea una faccia nuova semplicemente nominandola (`'doc-comment` in `rust-mode.lisp`) e le dà un aspetto con `set-face`. Il `Theme` in `Runtime` associa a ogni faccia uno `Style` (colori di primo piano e sfondo, grassetto, corsivo, sottolineato, inverso), in un vettore indicizzato dalla faccia: una faccia oltre la fine si disegna semplice. I colori sono richieste — un nome o `#rrggbb` (`Color::parse`, che accetta solo cifre esadecimali prima di tagliare la stringa) — e il front-end decide come approssimarli (Sezione 12.8); i sedici nomi convenzionali stanno in `NAMED_COLORS`, nell'ordine della tavolozza dei terminali. Il tema predefinito usa quei colori, il video inverso dove una cosa deve vedersi su ogni sfondo (la regione, la riga di stato, `replace-match`), numeri di riga attenuati e una riga di stato inattiva grigia; le corrispondenze di una ricerca su fondo ciano, le parentesi su fondo blu, o magenta quando non corrispondono, gli spazi a fine riga su fondo rosso (Sezione 12.4). Tutto questo sta in `ui/faces.rs`; le primitive sono in `primitives/theme.rs`.

10.6 Le decisioni

1. **Una riga alla volta, con uno stato portato da una riga all'altra.** Un commento aperto alla riga 10 e chiuso alla 40 colora tutto ciò che sta in mezzo, quindi una riga non si colora da sola. I modelli sul buffer intero sarebbero corretti, ma non incrementali: una corrispondenza può cominciare arbitrariamente lontano. Con lo stato all'ingresso di ogni riga in una cache, ricolorare dopo una modifica costa quanto la modifica, non quanto il file.
2. **Vince la corrispondenza più a sinistra.** Tutti i modelli attivi si provano insieme, in un solo passaggio da sinistra a destra. Scartato: prima le regioni, poi le regole, che è l'implementazione ovvia ed è sbagliata: in `// this /* is not` aprirebbe un commento a blocchi che inghiotte il resto del file.
3. **Niente lookahead: gruppi ed escape.** `regex` non ha lookahead, lookbehind né backreference, ed è così che garantisce un tempo lineare. Ciò che servirebbero a fare lo fanno `SyntaxRule::group`, che colora il nome di una chiamata senza la parentesi che lo segue, e `SyntaxRegion::escape`, che dice alla regione che cosa, simile alla fine, non lo è.
4. **Due lessici, ciascuno per la sua domanda.** La grammatica dice che aspetto ha il testo, a regex e riga per riga; la tabella che cosa è, carattere per carattere. Un modo dichiara una forma in tutti e due.
5. **Le forme del linguaggio si dichiarano nella tabella, non in chi la legge.** Commenti a blocchi, letterali di carattere e stringhe grezze hanno avuto lo stesso difetto: la grammatica conosceva una costruzione che la tabella ignorava, e un delimitatore dentro di essa contava. Dichiarata nella tabella, la forma vale per chiunque la legga: `forward-sexp`, l'indentazione, `syntax-ppss`, `electric-pair`. Scartato: correggere `electric-pair`.
6. **Le stringhe multi-carattere sono stili, non classi.** La classe `StringQuote` dice «questo carattere apre una stringa e lo stesso la chiude», che vale per le virgolette doppie e per nient'altro: una stringa grezza di Rust apre con tre caratteri e chiude con due, una di Python apre con tre, e il primo è già una virgoletta. Vince l'apertura più lunga, così quella di una stringa grezza vince sulla virgoletta con cui comincia, e lo stile si prova prima delle classi.
7. **Il quote dei caratteri è una regola che guarda avanti, non una classe.** `'` quota un carattere in `'a'`, apre una lifetime in `'static` ed è un apostrofo nella prosa, mentre una classe dice che cosa un carattere è una volta per tutte. Come `StringQuote` inghiottirebbe il file dalla prima lifetime; come `Escape` mangerebbe la virgoletta che chiude `"it's"`, perché gli escape valgono anche nelle stringhe. Un letterale è tale solo se si chiude dopo un carattere, con o senza escape.
8. **Si conta un delimitatore solo quando lo si legge come codice.** In `(message "close it with )")` tre parentesi su quattro sono testo, e una risposta che non sa dove stanno stringhe e commenti non fallisce: sposta il cursore in un punto strano, in silenzio. Per questo lo scanner fa un'analisi lessicale invece di contare.

9. **Sempre in avanti, da uno stato noto.** Andare avanti si potrebbe fare localmente, all'indietro no: se una `)` prima del punto sia codice dipende da tutto ciò che la precede, e gli stessi caratteri letti da destra sono ambigui. Le due direzioni fanno lo stesso lessico in avanti, dall'inizio del buffer o da una fotografia.
10. **Una sola porta per l'inizio di una scansione.** `Scan::begin` è il solo punto che sceglie dove una scansione comincia, e la cache arriva lì: i movimenti sono la stessa scansione con domande diverse, e nessuno è cambiato per diventare economico. Una fotografia oltre la domanda si ignora: è una scorciatoia, mai un sostituto, e una che non va bene costa una scansione intera, non una risposta sbagliata.
11. **La fotografia è lo stato intero.** Ricordare solo la profondità sarebbe tentante, ma una `)` è una chiusura o un carattere di una stringa secondo ciò che la scansione sta leggendo, e quanto arriva una lista dipende da dove era la sua apertura: una scansione ripresa da una profondità risponderebbe a `syntax-ppss` con posizioni inventate. Lo stato è piccolo, e tenerlo intero è la sola versione giusta.
12. **Senza i figli dei frame, e `backward` non riprende.** I figli sono ciò che legge `backward-sexp`, e quanti se ne tengono dipende dal `keep` di chi ha avviato la scansione: una fotografia presa con un `keep` risponderebbe in silenzio a un'altra domanda con un altro. Una scansione ripresa conosce la forma delle liste in cui è, non ciò che vi ha passato.
13. **Il punto di equilibrio, non il bilancio del buffer.** `electric-pair`, visto un `{`, deve sapere se ha già una chiusura. Chiedere se la lista più interna è chiusa sbaglia: un `{` nuovo in un blocco già chiuso sembra aver preso la chiusura di fuori, e la coppia si rifiuta dove serve. Chiedere se il buffer intero è bilanciato sbaglia: ciò che si sta scrivendo sotto tiene la profondità sopra zero alla fine, e un `{` con la chiusura alla riga dopo risulta aperto. `balance_point` chiede se la profondità torna a zero più avanti, e si ferma al primo punto di equilibrio: da dentro una funzione, decine di righe e non migliaia.
14. **Un file sbilanciato si scandisce lo stesso.** Una chiusura che non corrisponde all'apertura la chiude lo stesso, e una chiusura senza niente di aperto è testo: un file è sbilanciato per tutto il tempo in cui lo si scrive, e uno scanner che si fermasse lì sarebbe inutile proprio quando serve. Il frame più esterno è il buffer, e non si toglie mai.
15. **La prescansione riempie la cache, non i movimenti.** Riempirla dai movimenti vorrebbe il lock di scrittura del buffer sul percorso dei tasti, in mezzo al Lisp, per far funzionare una cache. Il lavoro di sfondo fa un pezzo limitato per passo, svegliato da una modifica o da una tabella che cambia, e chi legge prende ciò che è pronto e scandisce il resto da sé: senza una fotografia non sbaglia, va solo più piano.
16. **La prescansione legge sotto il lock di lettura.** L'evidenziatore copia le righe e lavora senza lock; lo scanner no, perché lavora in offset assoluti, che le fotografie registrano, e non gli si può dare una lista di righe staccate. Dargli il testo vuol dire dargli il buffer, che non si copia: una copia a ogni turno costerebbe quanto la cache risparmia. Lo rende accettabile la misura del lavoro: un turno di `LINES_PER_TURN` righe di lessico carattere per carattere, senza allocazioni né regex, sono microsecondi, e un lock di lettura blocca solo chi scrive.
17. **Le fotografie portano la versione, il modo e l'edizione della tabella.** Il lock si rilascia tra la scansione e il salvataggio, e nel frattempo il testo o il modo possono cambiare: si salva solo se sono ancora quelli letti. Una fotografia presa con un'altra tabella non è una scorciatoia ma una risposta sbagliata dall'aria giusta, e chi scandisce con un'altra tabella non la riceve.
18. **Gli hook e le fonti di completamento stanno nel modo.** Una variabile sarebbe una lista per tutti i buffer, e ciò che si completa in un buffer Lisp non è ciò che si completa in un elenco di directory.
19. **Una faccia è un nome, non una variante di un enum.** Ogni linguaggio vuole facce sue, `doc-comment` o `attribute`: con un enum chiuso, ognuna sarebbe una modifica qui e la ricompilazione di ogni front-end. Il nome si interna in un intero e non resta una stringa perché si copia in ogni evidenziazione e si cerca nel tema per ognuna: un indice di vettore costa meno di un confronto di stringhe.
20. **Il registro delle facce è globale.** Un nome significa la stessa cosa in ogni buffer e in ogni tema, e un registro per contesto andrebbe portato da `Highlight`, da `SyntaxRule` e da `Theme` senza dire nulla di più. Cresce con i soli nomi che il Lisp usa, e non dimentica: un tema e un'evidenziazione tengono una faccia per il suo id, e un nome dimenticato non si distinguerebbe da uno mai detto. L'id è a 32 bit: a 16, il nome nuovo dopo i primi 65 536 prendeva l'id di uno vecchio, e oltre l'ultimo id `intern` risponde con

la faccia predefinita invece di riusarne uno. I nomi hanno una mappa accanto al vettore, così trovarne uno non scorre il registro. `intern` prende il lock in scrittura anche per un nome già noto: si chiama mentre si configura, mai mentre si disegna.

21. **Un colore è una richiesta, non un indice ANSI.** Il core non sa quanti colori ha il terminale, e una finestra grafica non ne ha una tavolozza: dice `"blue"` o `"#3a5fcd"`, e ogni front-end risponde con ciò che può mostrare.

11 Il lavoro in background

Tutto il lavoro che l'editor fa lontano dal thread dei comandi passa da un solo oggetto, il *WorkManager* di `work/`: la colorazione, la prescansione, il controllo dei file, il salvataggio automatico, i lavori Lisp, i comandi di shell e le ricerche nei file. Il WorkManager li esegue, li elenca, li ferma e ne misura i tempi. Non conosce l'editor: lo stato su cui un lavoro lavora è un parametro di tipo, `C`, che nell'editor è `EditorState<B>` e nei test di `work/` è `()`.

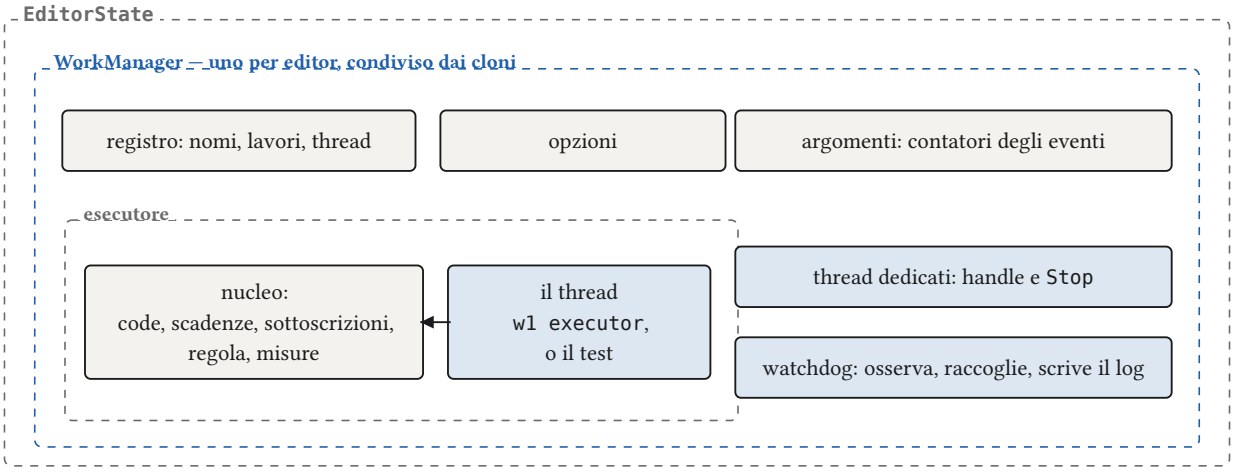
Le regole, in sei righe.

1. **Un solo punto d'ingresso.** Il lavoro asincrono passa dal WorkManager, ha un nome e un `WorkId`, e si può elencare e fermare.
2. **Due forme di lavoro.** I lavori a passi girano sull'esecutore; ciò che può bloccarsi, un processo o il disco, ha un thread dedicato.
3. **Un passo non aspetta mai.** Fa una quantità limitata di lavoro e dice come proseguire.
4. **Lo scheduler misura e segnala, non interrompe.** Un passo troppo lungo finisce nel log; la correzione si fa a mano.
5. **Gli argomenti non portano dati.** Dicono «vai a guardare»: la fonte della verità è lo stato.
6. **Tutto si prova senza thread.** Il nucleo riceve l'ora da un orologio che i test muovono a mano.

Le parole: nel codice un *job* (lavoro) è ciò che gira a passi, e *work* è un lavoro o un thread dedicato; *worker* resta solo nei nomi del Lisp, come `define-worker` e `worker-fuel`. Restano fuori dalla prima regola i thread che `(spawn ...)` avvia nell'interprete, e quello breve con cui un comando di shell fermato riceve `KILL` (Sezione 11.10).

11.1 Il WorkManager

Ce n'è uno per editor, condiviso da ogni clone di `EditorState`: clonarlo costa un `Arc`. La figura mostra che cosa possiede; in azzurro i thread, il resto sono dati che qualunque thread raggiunge attraverso il WorkManager.



I file di `work/` seguono le parti:

File	Che cosa contiene
`manager.rs`	`WorkManager`, la facciata: costruzione, `start`, `submit`, `spawn`, `cancel`, le domande al registro, gli hook, `shutdown` e la modalità manuale; `Shared`, ciò che i suoi cloni condividono.
`registry.rs`	`Registry`: che cosa esiste, per numero e per nome (Sezione 11.6).

File	Che cosa contiene
`executor.rs`	`Msg`, `Executor` ed `executor_main`: la casella dei messaggi e il ciclo che guida il nucleo (Sezione 11.5).
`watchdog.rs`	Il thread del watchdog e il suo turno (Sezione 11.8).
`core.rs`	`Core`, il nucleo: i lavori, le code, la regola, le scadenze e le misure (Sezione 11.3).
`job.rs`	Il contratto di un lavoro: `Job`, `Step`, `Exit`, `Wake`, `StepContext`, `Spec` e `WorkId` (Sezione 11.2).
`topic.rs`	`Topics` e `Topic`, gli argomenti (Sezione 11.4).
`threads.rs`	I thread dedicati: `Stop`, `Lives` e `start_thread` (Sezione 11.7).
`options.rs`, `report.rs`	Le opzioni e le righe `[BKG]` del log (Sezione 11.9).
`clock.rs`	`Clock`, `SystemClock` e `ManualClock` (Sezione 11.11).

11.1.1 Due modalità

`WorkManager::threaded` è quello dell'editor: un thread esegue il nucleo sull'orologio vero, da `start` a `shutdown`. `WorkManager::manual` è quello dei test, e `editor/boot.rs` lo sceglie quando la libreria è compilata per i test (`cfg!(test)`): nulla gira da solo, il nucleo legge un `ManualClock`, e il lavoro avanza quando il test chiama `run_until_idle` o `run_for` (Sezione 11.11). Il resto è uguale nelle due modalità, compreso il modo in cui entra un lavoro.

Una chiamata al WorkManager non tocca mai il nucleo. Cambia il registro, che risponde subito, e lascia un messaggio nella casella dell'esecutore, che lo legge tra un passo e l'altro. Così `submit` dall'interno di un passo è sicuro come da qualunque altro punto, anche in modalità manuale, dove l'esecutore è il thread del test.

11.1.2 Il ciclo di vita

1. **Nasce** in `EditorState::new`, prima dei compartimenti, perché i buffer ricevono da lui l'argomento `buffer-changed` (Sezione 11.4). Il `Log` dell'editor nasce ancora prima: le segnalazioni del WorkManager vanno lì.
2. **Parte** con `start(stato)`, appena lo stato esiste: il thread dell'esecutore riceve la sua copia di `EditorState`, e con lui parte il watchdog. Ciò che è stato sottomesso prima aspetta nella casella. In modalità manuale `start` non fa nulla.
3. **Riceve i lavori dell'editor**: evidenziatore e prescansione in `EditorState::new`, controllo dei file e salvataggio automatico in `create_global_env`, dopo `init.lisp`, perché il loro intervallo è un'impostazione (Sezione 11.10).
4. **Termina** con `shutdown`, che l'editor espone come `EditorState::shutdown(entro)`. Il front-end lo chiama all'uscita, con una guardia che lo chiama anche quando un panic attraversa `main` (`ShutDownOnExit` in `src/main.rs`, con mezzo secondo di attesa).

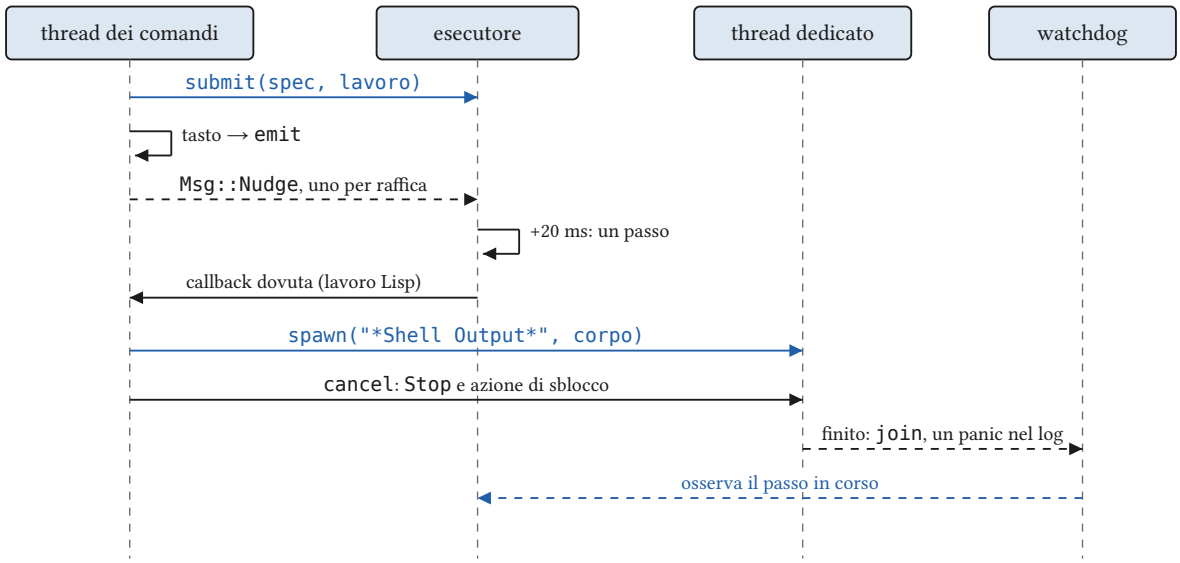
`shutdown` procede così:

1. chiude il registro: da qui `submit` e `spawn` rispondono `None`;
2. mette `Msg::Shutdown` nella casella con il lock del registro preso, così ogni lavoro accettato prima è già nella casella;
3. alza lo `Stop` di ogni thread dedicato ed esegue la sua azione di sblocco;
4. se nessun thread ha preso l'esecutore, in modalità manuale o prima di `start`, lo chiude sul thread del chiamante, e le `exit` dei lavori ricevono lo stato passato a `shutdown`; altrimenti lo chiude il thread dell'esecutore quando legge il messaggio. Ogni lavoro esce con `Exit::Shutdown`;
5. sveglia il watchdog, che si ferma, e aspetta che tutti i thread siano finiti, al massimo per `entro`;
6. fa `join` di ciò che è finito e abbandona il resto, nominandolo nel log: un thread che non si ferma, un passo che non finisce;

7. chiama un'ultima volta l'hook del watchdog, che porta sul file le righe appena scritte.

Una seconda chiamata non fa nulla.

11.1.3 Chi parla con chi



- **Il thread dei comandi** chiama il WorkManager: sottomette lavori, avvia thread dedicati, ferma, cambia opzioni. Le porte delle modifiche emettono gli argomenti, e una raffica di emissioni manda all'esecutore una sola sveglia. Le chiamate non aspettano: girano sul thread di chi chiama.
- **L'esecutore** esegue i passi uno alla volta. Un lavoro Lisp non chiama il suo modo: lascia una callback dovuta, che il thread dei comandi esegue tra un comando e l'altro (Sezione 11.10).
- **Un thread dedicato** lavora per conto suo, e si ferma con lo `Stop` e con l'azione che lo sblocca.
- **Il watchdog** osserva il passo in corso, raccoglie i thread finiti e scrive il file di log, così una segnalazione arriva sul file anche quando ciò che segnala è bloccato.

11.2 I lavori

Un lavoro implementa `Job<C>`:

da `work/job.rs` — estratto semplificato

```
pub trait Job<C>: Send + 'static {
    fn step(&mut self, cx: &mut StepContext<'_, C>) -> Result<Step, JobError>;
    fn exit(&mut self, _state: &C, _why: Exit) {}
}
```

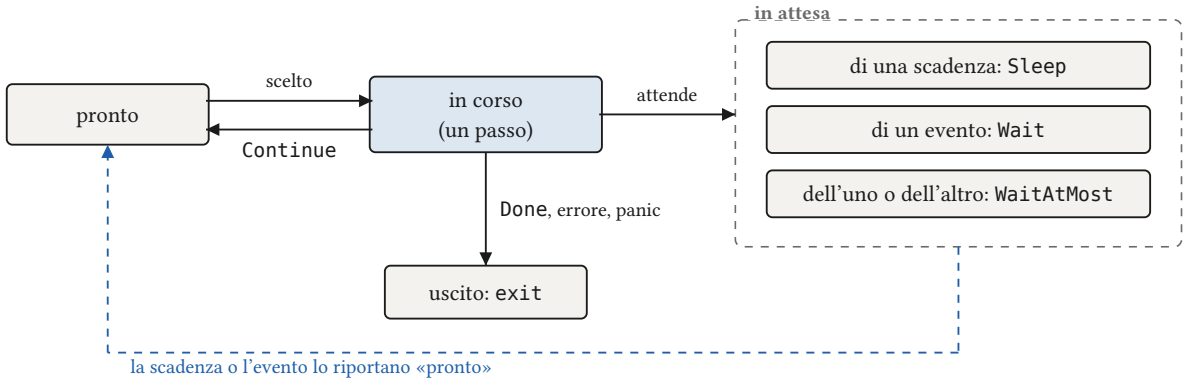
Lo `StepContext` porta lo stato (`state`), il motivo del passo (`woken_by`: `Start`, `Turn`, `Event` o `Timer`) e il numero del lavoro (`id`). Con `cx.worked(unità, su_che_cosa)` il passo dichiara quanto ha fatto, e con `cx.subscribe(argomenti)` cambia gli argomenti che lo svegliano. Il passo risponde con un `Step`:

`Step`	Quando viene il passo successivo
`Continue`	c'è altro da fare: il lavoro torna in fondo alla sua coda, e se nessun altro è pronto riparte subito
`Sleep(d)`	dopo `d`; gli eventi nel frattempo non lo svegliano, ma il passo dopo vede lo stato che annunciavano
`Wait`	al prossimo evento di uno dei suoi argomenti
`WaitAtMost(d)`	al prossimo evento, o dopo `d` al più tardi
`Done`	mai: il lavoro ha finito

Le regole di un passo:

- **Non aspetta mai.** Niente I/O che può bloccarsi, che va su un thread dedicato, e niente `sleep`: chi deve aspettare lo dice nell'esito.
- **Non tiene lock da un passo all'altro.** Copia sotto lock, lavora senza, e salva solo se la versione non è cambiata.
- **Non guarda l'orologio per decidere quanto fare.** Divide il lavoro in unità dei suoi dati (righe, file, carburante) e le dichiara con `cx.worked`; il tempo lo misura lo scheduler.
- **Non si interrompe.** `cancel` ha effetto alla fine del passo in corso.
- **Gli errori passano dal `Result`.** Lo scheduler li scrive nel log con il nome del lavoro, e il lavoro esce.

Un lavoro sottomesso è pronto, o in attesa della sua scadenza se la `Spec` dice `Start::After`. Poi passa da uno stato all'altro così:



`cancel` e `shutdown` portano a «uscito» da qualunque stato; un lavoro in corso ci arriva alla fine del passo. Il motivo dell'uscita è un `Exit`:

`Exit`	Quando
`Finished`	un passo ha risposto `Done`
`Failed`	un passo ha restituito un errore, o ha risposto `Wait` senza un argomento che possa svegliarlo
`Panicked`	un passo è andato in panic: il nucleo lo intercetta con `catch_unwind`
`Cancelled`	qualcuno l'ha fermato, o un lavoro con lo stesso nome l'ha sostituito
`Shutdown`	il WorkManager si chiude

`exit` arriva una volta sola, per qualunque motivo, anche a un lavoro fermato prima del primo passo: è il posto della pulizia, per esempio la callback `ON-DONE` di un lavoro Lisp. Gira sull'esecutore come un passo, ed è protetta e osservata come un passo.

Un lavoro si sottomette con una `Spec`, che ne dà il nome e il modo di eseguirlo: `Spec::new(nome)`, poi `class`, `unit` (l'unità di `cx.worked`, come le segnalazioni la stampano), `on` (un argomento) e `start`. Il nome è unico: sottomettere un lavoro con un nome già preso ferma ciò che lo porta, lavoro o thread dedicato. Il `WorkId` è un numero di tutto il processo e non si riusa: un numero tenuto dopo la fine del suo lavoro non può nominarne un altro.

11.3 Il nucleo

`Core` tiene i lavori e decide che cosa gira. Non ha un thread né un orologio suo: chi lo guida chiama `run_once` un turno alla volta, e il nucleo legge l'ora dal `Clock` che ha ricevuto. Un lavoro è in un solo posto alla volta (`Place`): pronto, nella coda della sua classe; in corso; in attesa di una scadenza; in attesa di un argomento, e forse anche di una scadenza. Un turno:

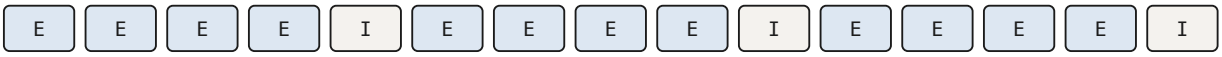
1. chiama `exit` sui lavori usciti, con lo stato che il turno porta;

2. rende pronti i lavori la cui scadenza è passata o i cui eventi sono maturi (`poll`), nell'ordine in cui lo sono diventati;
3. chiude le finestre di segnalazione scadute e controlla il carico dell'ultimo secondo (`tick`, Sezione 11.9);
4. sceglie un lavoro pronto (`choose`) ed esegue un suo passo, misurato (`run_step`);
5. chiama `exit` sui lavori che il passo ha fatto uscire.

Mentre fa il passo, il lavoro è fuori dal nucleo: il suo record non lo contiene, e un lavoro non può girare due volte insieme. Un lavoro fermato durante il passo trova il record sparito quando il passo finisce, ed esce allora. Mentre un passo o una `exit` è in corso, il nucleo lo scrive in `Underway` (il lavoro, l'inizio e un numero d'ordine), che altri thread leggono: il watchdog, per segnalare un passo che non finisce, e `WorkManager::jobs`, per elencare il lavoro come in corso. È proprio quando un passo non finisce che il nucleo non può rispondere da sé.

11.3.1 Chi passa prima

Le classi sono due. `Class::High` è per ciò che l'utente aspetta di vedere, colore e prescansione; `Class::Normal` è il default, e vi stanno il controllo dei file, il salvataggio automatico e tutti i lavori Lisp. Ogni classe ha la sua coda, e un lavoro entra in fondo alla sua ogni volta che diventa pronto. Passa prima la classe alta, ma dopo `high-steps` passi alti di fila (4 per default) tocca a un lavoro normale pronto. La regola si conta in passi e non in tempo, così è deterministica.



`E` = l'evidenziatore su un file enorme, che risponde sempre `Continue`; `I` = un lavoro Lisp con `(yield 'continue)`: un passo ogni cinque.

Quando l'evidenziatore ha finito, il lavoro Lisp ha l'esecutore tutto per sé.

11.3.2 Le scadenze

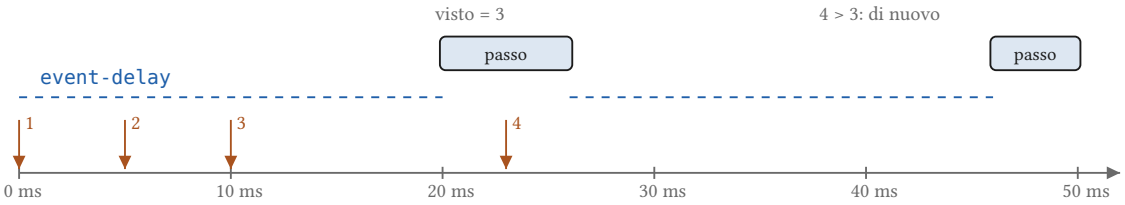
Un lavoro ha al più una scadenza, messa da `Sleep`, da `WaitAtMost` o da `Start::After`, e contata dalla fine del passo: un lavoro lento non si accoda a sé stesso, e un periodo saltato non si recupera. Nessuna scadenza va oltre un anno (`FURTHEST`): un `Instant` molto più lontano può non esistere, e sommarlo andrebbe in panic sull'esecutore, fuori da ogni passo.

11.4 Gli argomenti

Un argomento (`Topic`) è un evento con un nome, e non porta dati: è un contatore. Emetterlo (`emit`) costa un incremento atomico e, se l'esecutore non ha già una sveglia in arrivo, un `Msg::Nudge` nella sua casella; si può fare anche con il lock di un buffer preso. Un nome si risolve una volta (`WorkManager::topic`), e da lì l'argomento si emette senza toccare lock.

- **La sottoscrizione è persistente.** Un lavoro ha eventi in sospeso quando il contatore di un suo argomento ha superato l'ultimo valore che ha visto. Il nucleo aggiorna quel valore *prima* del passo: un evento che arriva durante il passo resta non visto, e risveglia il lavoro.
- **Il ritardo raccoglie le raffiche.** Un lavoro in attesa diventa pronto `event-delay` (20 ms) dopo che il nucleo ha notato il primo evento non visto, e gli eventi che arrivano nel frattempo si sommano a quello. Gli eventi arrivati durante un passo maturano dalla fine del passo.
- **Un `Wait` senza argomenti è un errore.** Nessuno potrebbe svegliare il lavoro: il nucleo lo segnala, e il lavoro esce con `Exit::Failed`.
- **Un passo può cambiare i suoi argomenti** con `cx.subscribe`: è ciò a cui si riduce un `(yield '(wait buffer-changed))` del Lisp. Un argomento nuovo conta come cambiato, perché il passo che lo nomina può aver guardato lo stato appena prima di un'emissione.
- **Un lavoro che dorme ignora gli eventi**, senza perderli: il passo successivo vede lo stato che annunciavano.

Un esempio: tre tasti in 15 ms producono un solo passo, e un quarto tasto che arriva durante il passo ne produce un secondo. Le frecce sono le emissioni, con il valore del contatore.



L'editor emette due argomenti, dichiarati in `editor/work.rs`; il Lisp ne emette di suoi con `emit-event`.

Argomento	Chi lo emette
`buffer-changed`	`edits::changed` (le tre porte: le modifiche, undo, redo, `replace-back`), `Buffer::adopt_text`, il cambio di modo maggiore, un buffer che entra nella tabella
`syntax-changed`	`set_mode`, `edit_grammar`, `edit_syntax_table`

Il modulo dei buffer non conosce lo scheduler. Il compartimento `Buffers` riceve l'argomento `buffer-changed` già risolto e lo dà a ogni buffer che entra nella tabella; fino ad allora un buffer ha un argomento staccato (`Topic::detached`), che conta e non sveglia nessuno.

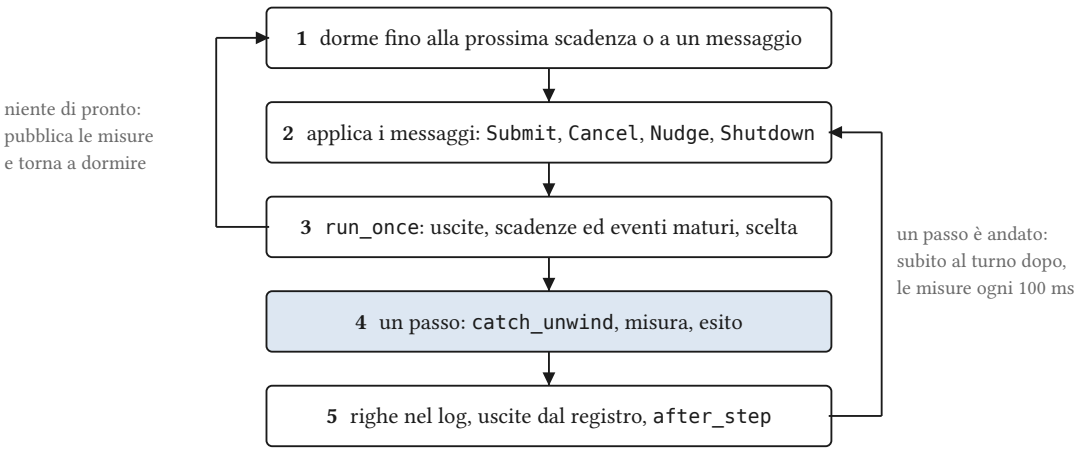
11.5 L'esecutore

L'esecutore (`Executor`) è il nucleo con l'altro capo della casella. Lo guida il thread `w1 executor` dell'editor, da `start` a `shutdown` (`executor_main`), o il test in modalità manuale; il numero nel nome dei thread è quello del WorkManager nel processo. Un turno applica i messaggi in attesa e poi esegue al più un passo:

da `work/executor.rs` — estratto semplificato

```
pub(super) fn turn(&mut self, shared: &Shared<C>, state: &C) -> bool {
    self.drain();
    if self.shut {
        return false;
    }
    shared.nudged.swap(false, Ordering::AcqRel);
    let stepped = self.core.run_once(state);
    self.settle(shared);
    if stepped {
        shared.stepped();
    }
    stepped
}
```

Il ciclo di `executor_main` ripete i turni finché un passo va, e dorme quando non c'è niente di pronto:



- **Le sveglie non si perdono.** Il flag `nudged` dice che un `Msg::Nudge` è già nella casella, e fa di una raffica di emissioni una sola sveglia. Il turno lo abbassa con uno scambio atomico *prima* che il nucleo guardi gli argomenti: un'emissione che il nucleo non vedrà trova il flag basso e manda una sveglia, e una che lo trova alto è avvenuta prima, e il nucleo la vede.
- **Dopo ogni passo, l'hook.** Quando il registro ha già tolto i lavori usciti, il turno chiama l'hook di `after_step`: l'editor lo usa per dire al front-end che lo schermo può essere cambiato (Sezione 11.10). L'hook gira tra due passi, quindi è breve e non aspetta; un suo panic va nel log.
- **Le misure si pubblicano.** Le misure dei lavori passano dal nucleo al registro ogni `PUBLISH_EVERY` (100 ms) mentre l'esecutore è occupato, e prima di dormire quando non lo è; in modalità manuale, alla fine di ogni guida.
- **Dorme fino alla prossima cosa da fare**: la scadenza più vicina, un evento che matura o la chiusura di una finestra di segnalazione (`next_wakeup`), oppure un messaggio, se arriva prima.

11.6 Il registro

Il registro (`Registry`, dietro il lock `registry` di `Shared`) è la risposta del WorkManager a «che cosa esiste», data subito e senza chiedere all'esecutore:

Campo	Che cosa tiene
`closed`	vero dopo `shutdown`: da lì non entra più nulla
`names`	un solo spazio di nomi per lavori e thread: un nome indica una cosa sola, e fermarlo per nome ferma quella, qualunque sia
`latest`	il lavoro più recente sottomesso con ogni nome, finché qualcuno non lo ferma; resta anche dopo la sua fine
`jobs`	i lavori, con ciò che l'esecutore ha pubblicato di loro (`WorkInfo`)
`threads`	i thread dedicati (`Dedicated`): nome, `Stop`, handle, avvio, richiesta di stop

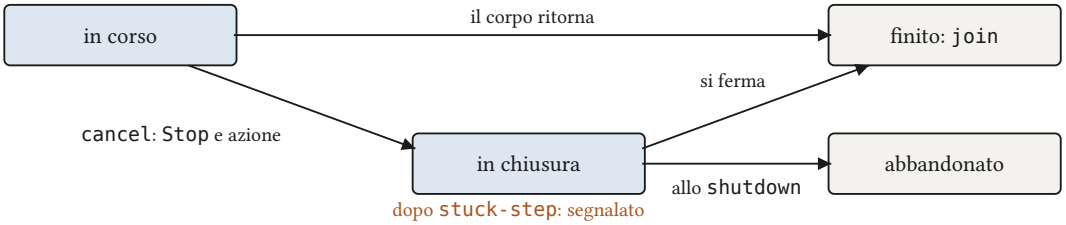
I messaggi per l'esecutore, tranne `Msg::Nudge`, partono con il lock del registro preso: la casella li tiene nell'ordine in cui il registro è cambiato, e dopo `Msg::Shutdown` non ne arriva un altro. Togliere un lavoro dal registro (`withdraw`) lascia un `Withdrawn` da completare senza lock: l'azione di sblocco di un thread gira dopo, sul thread di chi ha chiamato `cancel`, protetta come un passo.

Le domande che il registro risponde:

- `jobs()` elenca i lavori nell'ordine di sottomissione, con le misure dell'ultima pubblicazione; un lavoro che l'esecutore non ha ancora visto appare come è stato sottomesso. Il lavoro nel mezzo di un passo è elencato come in corso da ciò che dice `Underway`, perché dopo un passo che non finisce non si pubblica più nulla.
- `threads()` elenca i thread dedicati ancora vivi, dopo aver raccolto quelli finiti.
- `is_named(nome)` dice se un nome è preso. Lo usa `free_name_for_work`: un comando di shell prende il nome del buffer in cui scrive, e quel nome deve essere libero sia tra i buffer sia qui.
- `is_latest(nome, id)` dice se `id` parla ancora per il suo nome. È la domanda delle callback dovute (Sezione 11.10): un lavoro finito ha ancora diritto alla sua `ON-DONE`, uno sostituito o fermato no.

11.7 I thread dedicati

Un thread dedicato serve a ciò che può bloccarsi e non si divide in passi: leggere l'output di un processo, percorrere un albero di directory. `WorkManager::spawn(nome, corpo)` lo avvia con il nome `w1 nome` (Linux ne mostra 15 byte), lo mette nel registro accanto ai lavori e restituisce il suo `WorkId`; il corpo riceve lo `Stop` e il proprio numero.



- **Un thread non si uccide: gli si chiede di fermarsi.** `cancel` alza lo `Stop`, che il corpo controlla nei punti da cui può uscire, ed esegue l'azione registrata con `Stop::on_cancel`, che sblocca ciò su cui il corpo aspetta: per un comando di shell, fermare il processo chiude la pipe che il corpo legge. L'azione gira una volta sola, anche se è registrata dopo la richiesta, e gira sul thread che chiede lo stop: deve essere breve.
- **Il `join` non blocca nessuno.** Il watchdog, e chi chiede l'elenco dei thread, lo fa solo sui thread che risultano finiti (`is_finished`); un panic nel corpo torna dal `join` e va nel log.
- **Un thread che non si ferma**, per esempio una `stat` su un disco di rete appeso, resta elencato come in chiusura, viene segnalato dopo `stuck-step` e, allo `shutdown`, abbandonato.
- **`shutdown` sa quanto aspettare.** Ogni thread del WorkManager porta una quota di `Lives` dall'avvio alla fine; `shutdown` aspetta che il conto torni a zero, al più per il tempo che gli è dato.

L'editor non chiama `spawn` direttamente. `EditorState::spawn_work` passa al corpo l'editor, tiene il thread fuori dal thread dei comandi per tutta la sua vita (`WorkerTurn`, Sezione 11.10) e, quando il corpo ritorna, dice al front-end di ridisegnare. `spawn_writer` aggiunge un buffer di uscita: il thread prende il nome del buffer, il buffer sa chi scrive in lui prima che il corpo possa scrivere una riga, e chiudere il buffer ferma il thread.

11.8 Il watchdog

Il watchdog è un thread, `wl watchdog`, che parte con l'esecutore, o con il primo thread dedicato se viene prima, e si sveglia ogni `watchdog-every` (250 ms). Guarda ciò che nessun altro può guardare: il passo in corso, che l'esecutore non può segnalare finché ci sta dentro. Un turno (`watch_turn`):

1. raccoglie i thread dedicati finiti (`reap`), e segnala quelli ancora vivi `stuck-step` dopo la richiesta di stop;
2. segnala il passo in corso se dura da `stuck-step` (1 s), e di nuovo ogni volta che la sua durata raddoppia: 1 s, 2 s, 4 s… Un turno in ritardo segnala una volta sola, con la durata vera, e il segno successivo è il primo raddoppio oltre quella. Il numero d'ordine di `Underway` distingue un passo dal successivo, anche dello stesso lavoro; una `exit` che non finisce è segnalata allo stesso modo, «in its exit»;
3. chiama per ultimo l'hook di `after_watch`, così ciò che il turno ha segnalato è già in ciò che l'hook scrive.

L'editor passa ad `after_watch` la funzione `Log::flush`: il file di log si scrive qui, e non da chi scrive una riga. In modalità manuale il watchdog non c'è: i thread finiti si raccolgono quando il test guida il WorkManager, e il test fa un turno del watchdog con `watch_once`.

11.9 Misure, segnalazioni e opzioni

Il nucleo misura ogni passo: quanto è durato, quanto il lavoro ha aspettato in coda e le unità dichiarate con `cx.worked`, da cui viene il costo per unità. Per ogni lavoro tiene i passi, il tempo totale e il passo più lungo; per l'esecutore, l'occupazione dell'ultimo secondo, misurata esattamente su una finestra che scorre. Quando una misura supera la sua soglia, una riga `[BKG]` va nel log. Nessuna segnalazione cambia ciò che un lavoro fa: un lavoro lento si corregge a mano, con le segnalazioni come prova.

Riga	Quando (opzione)	Che cosa riporta
`long step`	un passo dura più di `long-step`	lavoro, durata, su che cosa, unità e costo per unità

Riga	Quando (opzione)	Che cosa riporta
`late start`	un lavoro pronto aspetta più di `late-high` o di `late-normal`	di quanto, e dietro quale lavoro
`load`	l'esecutore è occupato per più di `busy-share` dell'ultimo secondo	la quota, e i tre lavori che la causano
`summary`	si chiude una finestra di `summary-every` che ha contato qualcosa	quante altre occorrenze, e la peggiore
`stuck step`	un passo o una `exit` è in corso da `stuck-step`, poi dal doppio	lavoro e durata; la scrive il watchdog
`thread`	un thread dedicato è vivo `stuck-step` dopo la richiesta di stop, o viene abbandonato allo `shutdown`	nome e tempo trascorso
`error`, `panic`	sempre	lavoro e messaggio; il lavoro esce

La soglia del ritardo cambia con la classe perché un lavoro normale, per progetto, aspetta fino a `high-steps` passi alti. Per ogni coppia di lavoro e regola la prima occorrenza va subito nel log e apre una finestra di `summary-every`; le successive nella finestra si contano soltanto, e alla chiusura un riepilogo dice quante erano e la peggiore. Una finestra che si chiude vuota finisce, e la prima occorrenza dopo un periodo tranquillo va di nuovo subito nel log. Errori e panic non hanno limite: il lavoro esce, e non può ripeterli.

```
[BKG] long step: highlighter, 912 ms on app.min.js (3 lines, 304 ms each)
[BKG] late start: file-watcher started 230 ms late, behind highlighter (6 steps)
[BKG] load: 64% of the last second (highlighter 41%, manpage-index 20%)
[BKG] summary: highlighter, 12 more long steps in the last 10 s (worst 912 ms)
[BKG] stuck step: manpage-index running for 2.0 s
[BKG] thread: *Shell Output* still running 1000 ms after it was asked to stop
```

Le opzioni stanno nel WorkManager, in valori atomici che il nucleo legge a ogni scelta: una modifica vale dal passo successivo, e chi la fa non aspetta lo scheduler. Il Lisp le legge con `work-option` e `work-options` e le cambia con `set-work-option`, che rifiuta un valore fuori dall'intervallo. I tempi sono in millisecondi.

Opzione	Default	Valori	Che cosa regola
`high-steps`	4	1–1000	passi alti di fila prima di uno normale
`event-delay`	20	0–60 000	il ritardo dopo il primo evento non visto
`long-step`	40	1–3 600 000	la soglia del passo lungo
`late-high`	40	1–3 600 000	la soglia del ritardo, classe alta
`late-normal`	200	1–3 600 000	la soglia del ritardo, classe normale
`stuck-step`	1000	1–3 600 000	la soglia del passo bloccato e del thread che non si ferma
`busy-share`	50 %	1–100	la soglia del carico
`summary-every`	10 000	1–3 600 000	la durata di una finestra di riepilogo
`watchdog-every`	250	1–60 000	ogni quanto si sveglia il watchdog

`work-list` elenca ciò che è in background, lavori e thread, con stato e misure. La pagina `background` del manuale dell'utente spiega le une e le altre a chi scrive in Lisp.

11.10 Gli utenti del WorkManager

11.10.1 I lavori dell'editor

Lavoro	Classe, unità	Quando gira	Che cosa fa a ogni passo
`highlighter`	alta, righe	a `buffer-changed` e `syntax-changed`: `Continue` finché un buffer è indietro, poi `Wait`	colora fino a 500 righe del primo buffer indietro, partendo da quello con il focus
`prescanner`	alta, righe	come l'evidenziatore	porta avanti di fino a 500 righe la `ScanCache` di un buffer, con una fotografia ogni 64 (Sezione 10)
`file-watcher`	normale, buffer	`watch-file-interval` (3 s) dopo l'avvio e dopo ogni passo	confronta con il disco al più 8 buffer; ricarica o segna
`auto-saver`	normale, file	`auto-save-interval` (30 s) dopo l'avvio e dopo ogni passo	scrive la copia di al più 4 buffer modificati

Il codice è in `jobs/`: `highlighter.rs`, `prescan.rs`, `watcher.rs` e `autosave.rs`.

Evidenziatore e prescansione sono sottomessi in `EditorState::new`: un lavoro che aspetta un cambiamento non costa nulla. Controllo dei file e salvataggio automatico sono sottomessi in `create_global_env`, dopo `init.lisp`, perché l'interruttore che li spegne e il loro intervallo sono impostazioni del Lisp, che rileggono a ogni passo. Ognuno è sottomesso una volta sola. Uno che va in panic, nel suo `exit` con `Exit::Panicked`, chiede all'editor di sottometterne uno nuovo (`restart_after_panic`): al più `RESTARTS` volte, 3, perché un panic che si ripete a ogni passo non diventi un ciclo, e ogni volta l'area messaggi dice quale lavoro si è fermato ed è ripartito, o che è spento fino al prossimo avvio. Uno nuovo e non lo stesso: il lavoro che ha fatto il panic può essere rimasto a metà. Un panic avvenuto con un lock dell'editor preso lo lascia avvelenato; le porte dei compartimenti lo riprendono com'è (`PoisonError::into_inner`) invece di far cadere anche il thread dei comandi, che così può ancora salvare i buffer: il panic resta nel log.

Il passo dell'evidenziatore è il modello per scriverne altri:

da `jobs/highlighter.rs` — estratto semplificato

```
impl<B: BufferTrait> Job<EditorState<B>> for Highlighter {
    fn step(&mut self, cx: &mut StepContext<'_, EditorState<B>>) -> Result<Step, JobError> {
        match cx.state.highlight_one_turn() {
            Some((buffer, lines)) => {
                cx.worked(lines as u64, &buffer);
                Ok(Step::Continue)
            }
            None => Ok(Step::Wait),
        }
    }
}
```

`highlight_one_turn` copia sotto lock le righe da colorare, lo stato del lessico all'inizio della prima e la versione del buffer; colora senza lock; riprende il lock e salva solo se la versione, il modo e l'edizione delle grammatiche sono ancora quelli copiati, e una cache di un altro modo o di un'altra grammatica si ricolora dall'inizio. Un carattere digitato nel frattempo rende inutile il turno, che il passo successivo rifà dalla riga cambiata. Lo stesso schema regge la prescansione, che però legge le righe sotto il lock di lettura perché la scansione è veloce (Sezione 10.4.1), il controllo dei file e il salvataggio automatico. Una riga non si divide tra

due turni, ma il suo costo cresce linearmente con la sua lunghezza (Sezione 10.3.1): anche quella di un file minificato occupa l'esecutore per un tempo breve.

Il controllo dei file confronta il timbro del file su disco (`FileStamp`, l'istante della modifica e la lunghezza) con quello registrato quando l'editor lo ha letto o scritto l'ultima volta. Un buffer senza modifiche si ricarica; uno con modifiche, o che cambia mentre il file si legge, si segna come superato (`stale`), e il salvataggio successivo chiede. Il salvataggio automatico scrive una copia del buffer, `#nome#` accanto al file o in `auto-save-directory` con il percorso appiattito nel nome, quando la versione è cambiata dall'ultima copia; la scrive con `write_atomically`, come un salvataggio (Sezione 8.7), e un tentativo fallito si riprova al turno dopo. È la copia che `recover-file` rilegge.

11.10.2 I comandi di shell e le ricerche

`shell-command-start` (`primitives/shell.rs`) e la ricerca nei file (`grep--scan`, in `primitives/occur.rs`) girano su un thread dedicato avviato con `spawn_writer`, con il nome del buffer in cui scrivono: `*Shell Output*`, o `<2>`, `<3>`… se il nome è preso tra i buffer o tra i lavori (`free_name_for_work`). Il comando passa dalla shell della piattaforma (`/bin/sh -c`, o `cmd /C`), nella directory di lavoro del buffer corrente (Sezione 14.3.2), con l'input vuoto e stdout e stderr in una sola pipe, che il thread legge come byte: ciò che arriva va nel buffer subito, anche una riga non ancora finita, con ciò che non è UTF-8 sostituito e un carattere diviso tra due letture tenuto per la successiva. Alla fine il thread aspetta il processo e aggiunge una riga `--- exited N ---`, o `--- killed ---`.

Lo ferma `stop-worker` con il nome del suo buffer, o la chiusura del buffer. Il comando gira in un gruppo di processi suo, e la richiesta raggiunge tutto ciò che ha avviato: su Unix l'azione di sblocco manda `TERM` e `CONT` al gruppo, con il `kill` della shell, perché la libreria standard segnala un processo e non un gruppo, e un thread breve, `stop shell`, avviato fuori dal WorkManager, manda `KILL` due secondi dopo a ciò che resta. All'uscita quel thread se ne va con l'editor prima del `KILL`, e per questo l'editor tiene, per riferimento debole, i processi dei suoi comandi (`remember_shell_process`): `shutdown`, finita la sua attesa di mezzo secondo, manda `KILL` ai gruppi di quelli ancora vivi (`kill_left_over`), e un comando che ignora `TERM` non sopravvive all'editor. La lista è dell'editor e non del processo, così che un editor non uccida i comandi di un altro, come in un processo di test. Ciò che ha lasciato il gruppo, come un demone, non viene raggiunto, e se tiene l'output che ha ricevuto la lettura resta aperta: la riga finale aspetta che lo lasci. `shell-command-to-string`, invece, aspetta il comando sul thread che lo chiama e ne restituisce lo stdout. Il buffer dell'output tiene come `Buffer::directory` quella in cui il comando è partito, anche quando è un buffer riusato: un nome relativo che il comando stampa, o che si scrive in quel buffer, si legge lì, e `g` in un'uscita di `compile` rilancia il comando dove era girato. Una directory di lavoro sparita dopo `cd` è detta come tale, non con il «No such file or directory» del sistema, che sembrerebbe parlare della shell. La pagina `shell` del manuale dell'utente dice che cosa vede chi li usa.

Il conteggio delle corrispondenze di una ricerca in un buffer grande gira su un thread dedicato avviato con `spawn_work`, `isearch-count`, che non scrive in un buffer e consegna il risultato con una callback dovuta (Sezione 9.3.1).

11.10.3 I lavori Lisp

`define-worker` e `background-call` (`primitives/workers.rs`) sottomettono un `LispJob` (`primitives/lisp_job.rs`): classe normale, unità il carburante, nessun argomento finché un'attesa non ne nomina. Un passo riprende la fiber una volta, o chiama una volta la funzione di `background-call`, e ciò che la fiber passa a `yield` dice quando viene il passo dopo: `(yield)` dopo l'intervallo del lavoro, 40 ms se non è dato; `(yield 'continue)` appena tocca di nuovo a lui; `(yield '(sleep MS))`; `(yield '(wait TOPIC...))`; `(yield '(wait-at-most MS TOPIC...))`. La tabella completa, con gli esempi, è nella pagina `background`. Gli argomenti di un'attesa diventano del lavoro, e restano finché un'attesa dopo non ne nomina altri: `(wait)` e `(wait-at-most 500)` li tengono. Un altro valore vale l'attesa predefinita, così un lavoro che passa a `yield` il suo progresso continua a funzionare; una lista che comincia come una delle forme e non lo è, come `(sleep "soon")`, è un errore, e il lavoro fallisce con un messaggio che dice la forma.

Ogni passo ha il suo carburante, `worker-fuel` (200 000 per default), letto quando il lavoro parte, impostato sul thread dell'esecutore all'inizio di ogni passo e rimesso com'era alla fine. Il carburante è per thread: un comando non accorcia il passo di un lavoro, e un lavoro non spende quello di un comando. Un passo che lo esaurisce finisce, e il lavoro esce come fallito; altrimenti un `(while t)` senza `yield` fermerebbe l'esecutore, e la colorazione con lui. Durante il passo `WorkerTurn` segna il thread come «in background» (`in_worker`), e le primitive che hanno senso solo sul thread dei comandi lo controllano: `minibuffer-read` rifiuta di aprire un prompt.

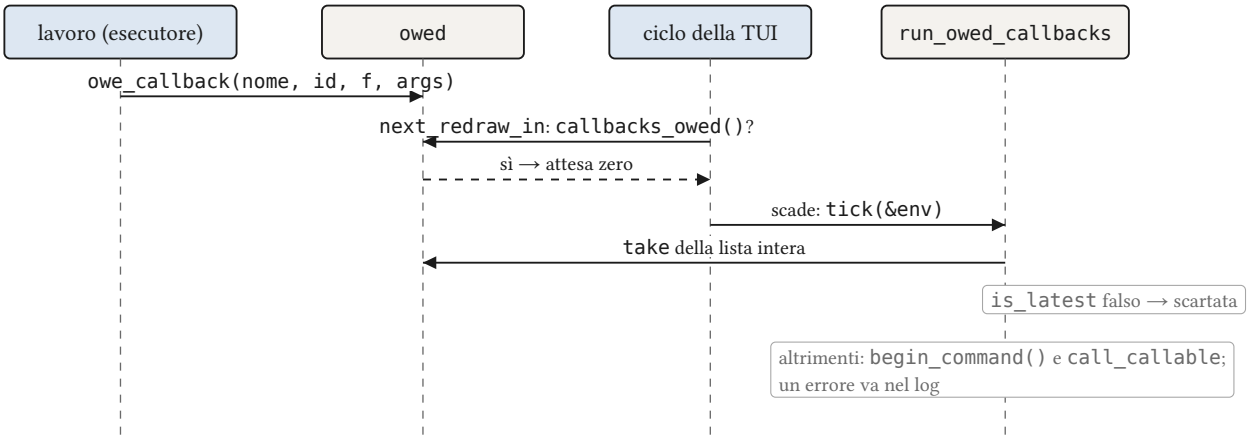
`ON-PROGRESS` e `ON-DONE` non girano sull'esecutore: il lavoro le lascia come callback dovute. `ON-PROGRESS` riceve il nome dopo ogni passo tranne l'ultimo. `ON-DONE` riceve il nome e `t` per `Exit::Finished`, `nil` per `Failed` e `Panicked`; dopo `Cancelled` e `Shutdown` non è chiamata, perché chi ha fermato il lavoro lo sa già.

Le regole, per tutto ciò che un programma Lisp fa in background:

1. **Ciò che può durare più di un frame è un lavoro in background**, avviato da una chiamata che dice che cosa fare e che cosa fare alla fine; non una funzione che avvia qualcosa e lascia al chiamante di scoprire come è andata.
2. **Un lavoro non disegna.** Cambia lo stato, e il renderer se ne accorge: la fine di ogni passo gli dice di guardare.
3. **Un lavoro non suppone che l'editor sia stato fermo.** Il buffer per cui è partito può non esserci più, la variabile può essere cambiata, la versione letta può essere vecchia di due modifiche: ricontrolla prima di salvare, e ciò che non può ricontrollare non lo tiene.
4. **Il progresso è un segnale, non un ridisegno.** Un lavoro dice «c'è dell'altro»; che cosa voglia dire lo decide il modo.
5. **Il modo si tiene coerente da sé**, e chiederlo è giusto solo se il segnale arriva dove un modo può rispondere senza rischi: per questo le callback girano sul thread dei comandi, tra un comando e l'altro (Sezione 11.10.4).
6. **Gli errori vanno nel log, non si sollevano.** Il comando che ha avviato il lavoro è finito da tempo, e non c'è nessuno a cui sollevarli. Un lavoro fallito dice comunque che è finito, perché un modo che non lo sa aspetta per sempre.
7. **Un lavoro si sostituisce, non si duplica.** Avviarne uno con un nome in uso ferma il vecchio, così rivalutare un modulo o premere due volte un tasto non fa danni; le callback in coda del vecchio si scartano, e il suo `ON-DONE` non viene chiamato, perché chi l'ha sostituito lo sa già.

11.10.4 Le callback dovute

Una callback chiesta da un lavoro va nella lista `owed` dell'editor (`owe_callback`), con il nome e il numero del lavoro. Il front-end lo sa da `next_redraw_in`, che risponde zero finché la lista non è vuota, e la esegue chiamando `tick`, che chiama `run_owed_callbacks`:



La lista si prende intera con un solo `take` prima di eseguire qualunque callback: una callback che ne chiede altre le aggiunge a una lista nuova, che girerà al prossimo `tick`, e il lock non resta mai preso mentre gira il Lisp. Una callback il cui lavoro non parla più per il suo nome (`is_latest`) si scarta. Le altre girano con un budget di carburante proprio (`begin_command`), sul thread che possiede l'interfaccia, tra un comando e l'altro: possono aprire buffer, scrivere messaggi o cambiare finestra come un comando, ma non sono comandi, e non cambiano `last-command`.

11.10.5 Quando ridisegnare

Quasi tutto ciò che è sullo schermo cambia perché l'utente ha fatto qualcosa; ciò che cambia un lavoro no. Per questo l'editor passa al WorkManager, come hook di `after_step`, `BackgroundChange::raise`. Alla fine di ogni passo, e quando un thread dedicato scrive nel suo buffer o finisce (`changed_in_background`), `raise` chiama una volta la funzione che il front-end ha registrato con `on_background_change`, e non di nuovo finché il front-end non prende una fotografia. La TUI la usa per svegliare il suo ciclo.

`next_redraw_in(&env, &frame)` è poi il minimo tra: la scadenza del messaggio nell'area messaggi (`echo-message-timeout`, 5 s; `nil` non scade mai); `TURN_INTERVAL` (40 ms, in `editor/frame.rs`) se la colorazione è in corso; il tempo che manca perché sia passato un `TURN_INTERVAL` dall'ultima fotografia, se un lavoro ha cambiato qualcosa da allora (`BackgroundChange::due_in`), così un lavoro che fa un passo al millisecondo non ridisegna il terminale a ogni passo; zero se ci sono callback dovute. `None` vuol dire che nulla è in sospeso e il front-end può bloccarsi sull'input. `drag_scroll_in()` aggiunge i 60 ms dello scorrimento automatico mentre si trascina una selezione oltre il bordo di una finestra.

11.11 Come si prova

Livello	Come
Il nucleo	`work_core_tests`: un `Core` guidato a mano su un `ManualClock`, senza thread. Un lavoro che deve durare 30 ms sposta l'orologio dall'interno del suo passo, e ogni test è esatto; un minuto di scheduler simulato richiede microsecondi.
Il WorkManager	`work_manager_tests`, `work_threads_tests` e `work_watchdog_tests`, con gli aiuti comuni di `work_support`. Quasi tutti usano la modalità manuale; i pochi con thread veri aspettano una condizione con una scadenza, mai un tempo fisso, e chiudono il WorkManager anche quando falliscono.
L'editor e il Lisp	L'editor dei test nasce in modalità manuale: i lavori girano solo con `ctx.work().run_until_idle(ctx)` o `run_for(ctx, durata)`, e i thread veri solo dove un test li vuole.

Gli altri strumenti:

- **Il watchdog in modalità manuale.** Il thread del test è l'esecutore, quindi un test che vuole osservare un passo lo fa eseguire su un altro thread (`drive_elsewhere`), sposta l'orologio e chiama `watch_once`.
- **Invarianti su sequenze casuali.** `random_sequences_keep_the_invariants` genera sottomissioni, `cancel`, emissioni ed esiti con un generatore scritto nel test, a partire da semi fissi, e controlla dopo ogni operazione che ogni lavoro stia in un solo posto, che `exit` arrivi una volta sola e che la regola di `high-steps` valga.
- **Nessun thread lasciato indietro.** Su Linux, `shutdown_leaves_no_thread_behind` conta in `/proc` i thread che portano il nome del WorkManager, e dopo `shutdown` non ne trova più.
- **Gli argomenti alla fonte.** `topic_source_tests` conta ciò che ogni fonte emette, e segue una modifica fino al lavoro che sveglia.
- **I lavori dell'editor, un turno per chiamata.** `syntax_tests`, `scan_cache_tests`, `watcher_tests` e `autosave_tests` chiamano il turno di un lavoro e guardano ciò che ha deciso; il calcolo e il salvataggio sono funzioni separate (`store_turn`, `store_prescan`, `store_file_check`) perché un test possa cambiare il buffer tra i due. I tempi dei file si impostano invece di aspettarli: un test che scrivesse due volte

un file sperando che l'orologio si sia mosso passerebbe su un filesystem lento e fallirebbe su uno veloce. I guasti del salvataggio automatico sono silenziosi — una copia mai scritta, una riscritta a ogni giro, due file con una copia sola — e i test provano il nome della copia come aritmetica, la decisione di scriverla come una decisione, e la sua vita ai due capi.

11.12 Le decisioni

1. **Un solo WorkManager per tutto il lavoro asincrono.** Ogni lavoro, a passi o su un thread, entra dalla stessa porta, ha un nome e un numero, e si può elencare con `work-list` e fermare con `stop-worker`. Scartato: lasciare a ogni sottosistema i suoi thread, che nessuno elenca né ferma e che nessuno aspetta all'uscita.
2. **Lavori a passi su un esecutore, e un thread dedicato per ciò che si blocca.** Un passo non aspetta mai; ciò che deve aspettare ha un thread suo, che il WorkManager avvia e conosce. Comporta che un lavoro a passi si scriva come una macchina che riprende da dove era rimasta. Scartato: un thread per lavoro, che costa uno stack ciascuno e fa girare insieme lavori che prendono gli stessi lock.
3. **Lo scheduler misura e segnala, non interrompe.** Un passo lungo finisce nel log con il suo costo per unità, e si corregge a mano. Scartati: un budget di tempo per passo, che renderebbe il lavoro dipendente dalla macchina e i test non deterministici, e l'interruzione di un passo, che Rust non sa fare in sicurezza. Il carburante dei lavori Lisp non è un budget di tempo: conta i passi del valutatore, uguali su ogni macchina.
4. **Gli argomenti non portano dati.** Un argomento dice «vai a guardare», e ciò che serve sta nello stato, dove nasce. Un messaggio con dati va consegnato uno per uno, invecchia, e quando una coda si riempie va perso: non può essere la fonte della verità. Un contatore costa un incremento, si emette anche sotto il lock di un buffer, e di una raffica fa un numero cresciuto.
5. **Il nucleo non ha thread né orologio.** Chi lo guida gli dà i turni, e l'ora gli arriva da un `Clock`: così si prova tutto senza thread, e in modo esatto. Scartato: un nucleo con il suo thread, che i test potrebbero solo aspettare.
6. **La regola delle classi si conta in passi.** Un passo normale ogni `high-steps` passi alti di fila, e non una quota di tempo: la regola è deterministica e si prova senza orologio.
7. **Il nucleo scandisce i suoi lavori invece di tenere un heap delle scadenze.** I lavori sono decine, non migliaia, e una scansione non può andare fuori passo con i lavori come un heap con voci superate. Un heap si aggiunge se le misure un giorno lo chiedono.
8. **Una chiamata non tocca il nucleo.** Il registro risponde subito, e il resto arriva all'esecutore come messaggio: così `submit` dall'interno di un passo non aspetta l'esecutore, che è il thread stesso che chiama. Comporta che registro e nucleo tengano due viste dei lavori, e che `jobs()` mostri le misure dell'ultima pubblicazione.
9. **Nomi unici, e il nuovo sostituisce il vecchio.** Rivalutare un modulo, o premere due volte lo stesso tasto, non duplica un lavoro. Le callback del lavoro sostituito si scartano con `is_latest`, che il registro cambia insieme al lavoro: due sottomissioni con lo stesso nome, o una sottomissione e uno stop, non possono lasciarlo a indicare il lavoro sbagliato.
10. **Un thread non si uccide: gli si chiede di fermarsi.** Rust non ha un modo sicuro di uccidere un thread, e uno ucciso mentre tiene il lock di un buffer bloccherebbe l'editor al primo accesso. Lo `Stop` e l'azione di sblocco coprono ogni caso tranne il thread che non controlla e non si sblocca, che viene segnalato e abbandonato all'uscita.
11. **`shutdown` è esplicito.** L'esecutore tiene una copia di `EditorState`, che contiene il WorkManager: è un ciclo di riferimenti, e `Drop` non lo rompe. Per questo il front-end chiama `shutdown` all'uscita, con una guardia che lo fa anche dopo un panic, e lo chiama anche ogni test che avvia dei thread.
12. **Il watchdog scrive il file di log.** Una scrittura che si blocca, per un disco pieno o una home di rete, ferma il watchdog e nessun altro, e una segnalazione arriva sul file anche quando ciò che segnala è bloccato. Chi scrive una riga nel log non scrive mai il file.
13. **Le callback dei lavori girano sul thread dei comandi.** Un lavoro non chiama il suo modo: lascia una nota, che il thread dei comandi legge dove leggerebbe un tasto. Se un lavoro chiamasse il modo, del

Lisp scritto per il thread dei comandi girerebbe in un istante qualunque su un altro thread: a metà di un inserimento, dentro un hook che scorre una lista. Nessuna cura nella callback lo ripara, perché a essere sbagliata non è la callback.

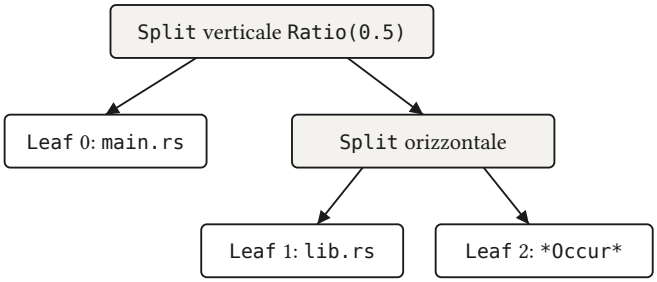
14. **Un comando passa dalla shell.** Ciò che si scrive dopo `M-!` deve voler dire ciò che vorrebbe a un prompt, con pipe, redirezioni e glob. Spezzare sugli spazi cambierebbe `grep -r "due parole" .`, e riscrivere le regole di quoting di una shell darebbe una shell sottilmente diversa da quella che l'utente conosce.
15. **Un comando dice nel suo buffer come è finito.** Il thread ha l'`EditorState` e non l'ambiente Lisp: non può chiamare una callback quando il comando finisce, né mettere un messaggio che scade con un timer che non gira. Scrive nel buffer in cui scrive già una riga `--- exited 0 ---`, che è anche la sola traccia che resta a chi ha guardato altrove.
16. **Stdout e stderr in una pipe sola.** Arrivano nell'ordine in cui il comando li scrive, come a un prompt, e un errore sta accanto all'output di cui parla. Letti uno dopo l'altro, un comando che scrive su stderr più di quanto una pipe tiene prima di chiudere stdout, una build piena di avvisi, aspetterebbe un lettore che aspetta lui.
17. **L'output si legge come byte.** Ciò che arriva si mostra subito, anche una barra di avanzamento o una domanda, che non finiscono una riga. Letto come testo, un byte Latin-1 in un nome di file chiuderebbe la lettura, e il comando morirebbe della pipe chiusa sotto di lui.
18. **Un comando ha un gruppo di processi suo, e niente da leggere.** Fermarlo deve raggiungere ciò che ha avviato, non solo la shell: i figli terrebbero aperta la pipe e sopravviverebbero all'editor. Un gruppo suo è anche un gruppo a cui il terminale non appartiene: un comando che apre il terminale, come una richiesta di password, viene fermato dal sistema invece di prendere i tasti dell'editor, e il `CONT` dopo il `TERM` fa sì che lo senta.
19. **Un lavoro solo per la colorazione, svegliato dai cambiamenti.** Un lavoro per ogni modifica dovrebbe essere breve, o l'editor si inceppa, e «breve» è una speranza, perché dipende dal file. Un lavoro che a ogni passo fa una quantità limitata di lavoro è reattivo per costruzione: ciò che non ha finito è ciò con cui comincia il passo dopo. E accorpa da sé: venti tasti in un secondo non mettono in coda venti lavori, spostano venti volte lo stesso confine. Dorme finché un buffer o una grammatica cambiano, e un editor che non ha niente da colorare non lavora.
20. **Nessun lock mentre si colora.** Le righe si copiano sotto il lock, il lock si rilascia, si colora, e il lock si riprende per salvare. Un lavoro che colorasse sotto il lock bloccherebbe ogni tasto per tutto il tempo.
21. **Un risultato superato non si salva.** Il buffer può cambiare mentre il lavoro è in corso, e gli intervalli calcolati sul testo vecchio descrivono colonne che non vogliono più dire nulla: la versione si ricontrolla al momento di salvare, non solo all'inizio. Né un cambio di modo né una regola aggiunta toccano il testo, quindi la cache ricorda anche il modo e l'edizione delle grammatiche con cui è stata calcolata.
22. **Una sola risposta a «è indietro?».** Il lavoro lo chiede per sapere se fare un turno, la fotografia per sapere se il renderer può dormire: due risposte separate potrebbero non concordare, e il renderer dormirebbe proprio durante il colore che aspetta. La metà economica prima, un lock di buffer; il registro dei modi solo per un buffer davvero indietro, per non metterlo sul percorso di ogni fotografia. Un modo senza grammatica non è mai indietro, o un buffer di testo semplice sveglierebbe il renderer per sempre.
23. **Il controllo dei file ricarica un buffer pulito, mai uno modificato.** L'asimmetria è la funzionalità. Ricaricare un buffer senza modifiche non perde nulla, e si cambia ramo trovando le finestre già aggiornate; ricaricarne uno modificato perderebbe proprio ciò che questo lavoro protegge, e non lo decide un lavoro di sfondo: il buffer si segna, e il salvataggio chiede. Un file sparito non svuota il buffer, che è ora la sola copia.
24. **Pochi file per giro.** `stat` è una chiamata a un filesystem, che può bloccarsi, e l'esecutore che fa girare il controllo colora anche: un giro limitato mette tra l'utente e il suo colore al più `FILES_PER_TURN` attese. Non limita il danno di un solo file bloccato, su un mount di rete morto, e qui niente può: i rimedi sono spegnere il controllo (`watch-files`) o dare ai file un thread loro, che è un progetto diverso, da scegliere apposta.

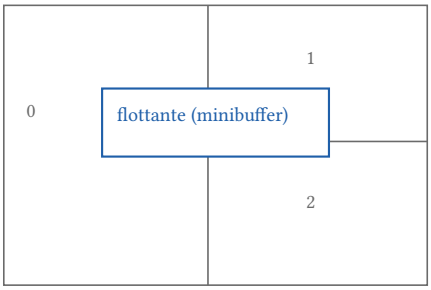
25. **Il giro è un indice nei nomi.** Il cursore indica una posizione nell'elenco dei buffer com'è a ogni giro, quindi un buffer creato o chiuso sposta ciò che viene dopo. Basta a un giro la cui sola promessa è che nessuno resti indietro: ogni buffer si raggiunge in pochi giri, e ricordare un nome costerebbe una ricerca a ogni giro per un'ingiustizia che nessuno vede.
26. **Le impostazioni si leggono a ogni passo.** Gli interruttori e gli intervalli del controllo dei file e del salvataggio automatico si leggono a ogni passo, come l'attesa che segue: valgono anche quando li cambia `init.lisp`, e spenti parcheggiano il lavoro invece di chiuderlo, così si riaccende senza riavvio. Non legati valgono accesi: ciò che protegge dal perdere lavoro non va chiesto. I due lavori portano con sé l'ambiente, come un lavoro Lisp, perché le impostazioni stanno lì.
27. **Una copia, non un backup.** Un backup è il contenuto precedente del file, scritto una volta al primo salvataggio, e serve a chi ha rotto qualcosa e l'ha salvato; una copia del salvataggio automatico è il contenuto attuale del buffer, riscritto mentre si scrive, e serve quando l'editor si ferma senza che glielo si chieda, per un crash o un terminale perso. File diversi, vite diverse.
28. **`#nome#`, o il percorso intero in una directory.** `#nome#` accanto al file è la convenzione nota. Con `auto-save-directory` tutte le copie vanno lì, con il percorso nel nome, perché due `main.rs` di directory diverse non si contendano un nome: i separatori diventano `!`, come in Emacs, e `!`, `%`, `:` (e `\` dove non separa) si scrivono come in un URL, `%21`, `%25`, `%3A`, `%5C`. Emacs raddoppia il `!`, e un `!` accanto a un separatore si legge come quello di un altro percorso: `/a!/b` e `/a/!b` davano tutti e due `!a!!b`. Così ogni carattere si scrive in un modo solo, e due percorsi non hanno lo stesso nome. Il nome non si rovescia, e non serve: il ripristino parte sempre da un percorso che si ha, quello del buffer o uno scritto.
29. **La versione, non il segno di modifica.** «Modificato» resta vero dal primo tasto al salvataggio, e un lavoro che scrivesse ogni buffer modificato a ogni giro riscriverebbe per sempre uno che nessuno tocca. La versione si muove solo con il testo: ricordare l'ultima scritta fa di «c'è qualcosa da salvare» un confronto tra interi.
30. **Al più quattro copie per giro.** Una scrittura costa più di uno `stat`, e l'esecutore colora anche. Il limite è sulle scritture, non sui buffer guardati: una copia dovuta non aspetta quanti altri buffer sono aperti.

12 Lo schermo

12.1 Finestre

Le finestre affiancate formano un albero (`LayoutNode`): una foglia è una `Window` (id, nome del buffer, scorrimento orizzontale e verticale, punto proprio quando non ha il focus, larghezza del margine), un nodo interno è una divisione orizzontale o verticale con una `Division` (`Ratio`, `FirstFixed`, `SecondFixed`). Sopra l'albero stanno le finestre *flottanti* (`FloatingWindow`: una finestra più un rettangolo, un bordo facoltativo, un titolo e la finestra a cui restituire il focus quando si chiude): il minibuffer, il `*Backtrace*`, i popup dei moduli.





Le foglie si dividono lo spazio; le flottanti sono disegnate sopra, con il loro rettangolo.

Il compartimento `Windows` tiene l'albero, le flottanti, l'id con il focus, il prossimo id, lo stato di un trascinamento del mouse e l'ultimo clic (per il doppio clic entro 400 ms). Le operazioni che tolgono una finestra rispondono con un `WindowRemoved` (`No`, `Yes`, `Refocus(id)`) che dice alla facciata se e verso quale finestra spostare il focus.

L'albero, le flottanti e la loro geometria stanno in `ui/windows.rs`: `Rect`, `split_rects`, che dice come una divisione taglia un rettangolo, la stessa aritmetica per comporre e per sapere che cosa c'è sotto il puntatore, e `SplitPath`, la strada dalla radice a una divisione, che non ha un id suo. Una `Window` ricorda anche ciò che l'ultimo frame ha deciso per lei: il rettangolo in cui è stata disegnata (`rect`, riga di stato compresa; vuoto se non è stata disegnata, e allora un clic non la trova), le righe di testo (`text_height`) e la larghezza del margine (`gutter_width`). Li leggono i comandi di scorrimento e il mouse, tra un frame e l'altro. `point` è il punto della finestra quando non ha il focus, `None` per una finestra nuova, che adotta quello del buffer; `show_mode_line` è falso per le strisce, che mostrano le righe chieste e non una di meno.

12.2 `snapshot`: dalla memoria alla fotografia

`snapshot(&env, larghezza, altezza)` costruisce il `FrameSnapshot`. Prende i lock in un ordine fisso e non fa I/O; il disegno avviene dopo, senza lock.

1. Abbassa il segnale del lavoro di sfondo (`BackgroundChange::taken`): ciò che i lavori hanno cambiato fin qui è in questa fotografia.
2. Legge dalle variabili Lisp e dai campi piccoli tutto ciò che serve *prima* di toccare finestre e buffer: `echo-message-timeout`, il tema, l'input in sospeso (`pending_input`), il messaggio della keymap transitoria, `mode-line-format`, `window-separator`, la configurazione del margine (`display-line-numbers-type`, `display-line-numbers-width`), se la colorazione è in corso; e la tabella sintattica della finestra con il focus, se il suo buffer ha `show-paren-mode` acceso, e il modello della ricerca o della sostituzione in corso, se `highlight-all-matches` lo chiede (Sezione 12.4).
3. Legge l'area messaggi, vuota se il messaggio è scaduto.
4. Con `windows` in scrittura (lo scorrimento può dover seguire il punto) e `buffers` in lettura, attraverso `windows_mut_with_buffers`, cerca la coppia di parentesi della finestra con il focus, compone ogni foglia dell'albero nel rettangolo del frame meno l'ultima riga, raccogliendo i separatori; poi compone le flottanti.

5. Consuma il testo destinato agli appunti (`take_pending_clipboard`).

Ogni foglia si compone come dice Sezione 12.3. Le finestre flottanti si compongono dopo, ognuna nel suo rettangolo, senza margine né riga di stato.

Campo di `FrameSnapshot`	Contenuto
`views`	Un `RenderableWindowView` per finestra: rettangolo, nome del buffer, titolo, focus, cursore relativo, righe di testo, evidenziazioni (riga, colonne, faccia), riga di stato, bordo, margine e sua larghezza.
`separators`	Rettangoli e carattere delle linee tra finestre affiancate.
`echo_message` , `prompt`	Il messaggio e l'eventuale offerta della keymap transitoria, che lo sostituisce.
`pending_input`	`C-u 4 C-x-` : ciò che l'editor sta leggendo.
`theme`	Gli stili delle facce (`Arc<Theme>`).
`focused_window_id` , `width` , `height`	Come dice il nome.
`colouring_pending`	La colorazione non ha finito: ridisegnare tra 40 ms.
`clipboard`	Testo che il front-end deve mettere negli appunti di sistema, se presente.

12.3 La composizione di una finestra

`compute_tiled_views` (`ui/compose.rs`) percorre l'albero delle finestre: una divisione taglia il suo rettangolo con `split_rects` e registra la linea tra i due figli, una foglia si compone in un `RenderableWindowView` . Le impostazioni lette dal Lisp prima dei lock — il formato della riga di stato e il margine — viaggiano insieme, in un `ComposeSettings` , con le evidenziazioni del momento (Sezione 12.4). Per ogni foglia, in quest'ordine:

1. registra sulla finestra il rettangolo intero, riga di stato compresa: è ciò contro cui si prova un clic;
2. toglie l'ultima riga per la riga di stato (Sezione 12.7), se la finestra ne ha almeno due, e registra le righe che restano (`text_height`);
3. legge il punto e il numero di righe del buffer sotto un'acquisizione sola del suo lock;
4. calcola il margine (Sezione 12.6) e lo toglie dal rettangolo, a sinistra: da qui in poi scorrimento, cursore ed evidenziazioni conoscono solo le colonne del testo;
5. compone le righe visibili (Sezione 12.5), segue il punto se si è mosso o se la finestra ha cambiato forma, e ricompone le righe se lo scorrimento verticale è cambiato; le righe servono prima dello scorrimento perché ciò che deve restare visibile è la colonna del cursore sullo schermo, che il testo virtuale può spostare;
6. raccoglie le evidenziazioni in colonne del buffer con `window_highlights` , nell'ordine *sintassi* ⟶ *corrispondenze* ⟶ *overlay* ⟶ *regione* ⟶ *parentesi*, e le porta sullo schermo con `place` ; aggiunge per ultime le facce del testo virtuale, perché le ultime vincono;
7. dopo lo scorrimento, che decide quali righe si numerano, calcola le celle del margine.

Segue il punto la finestra con il focus, e ogni finestra affiancata mentre la tastiera è in una flottante, come durante una ricerca incrementale; il cursore lo disegna solo la finestra con il focus. I colori della sintassi vengono dalla cache della colorazione (`syntax_highlights`), che la composizione legge e non riempie mai. Gli overlay si chiedono alla loro tabella per le sole righe visibili (`overlay_highlights`); la regione (`region_highlights`) è un intervallo per riga, o le stesse colonne su ogni riga per un rettangolo. Un buffer che non c'è più dà una finestra vuota, non un errore (`compose_layout`). Le finestre flottanti ricevono le stesse evidenziazioni, nello stesso ordine, con `window_highlights` , e poi il testo virtuale, così un buffer si vede uguale in una finestra affiancata e in una flottante.

12.4 Le evidenziazioni del momento

Quattro cose si disegnano per un fotogramma solo, ricalcolate a ogni `snapshot` e mai conservate nel buffer: la parentesi accanto al punto con quella che le corrisponde, ogni corrispondenza di una ricerca o sostituzione, gli spazi a fine riga e la riga del punto. Parentesi, spazi e riga sono di `show-paren-mode`, `whitespace-mode` e `hl-line-mode` (Sezione 10.2), e si disegnano dove il rispettivo modo è acceso. Le corrispondenze le spegne `highlight-all-matches`, `t` in partenza.

	Le parentesi	**Le corrispondenze**
Dove	La sola finestra con il focus, affiancata o flottante.	Ogni finestra che mostra il buffer cercato.
Che cosa si legge prima dei lock	Il buffer della finestra con il focus, il suo modo e la tabella sintattica del modo (`paren_table`), se il buffer ha `show-paren-mode` acceso.	Il `Pattern` della ricerca in corso, o quello della sostituzione con il suo intervallo, da `Runtime` (`search_marks`).
Dove si calcola	In `paren_spots`, dentro `windows_mut_with_buffers`, con il buffer letto lì, se è ancora in quel modo e ha ancora `show-paren-mode` acceso: `sexp::pairing` dà un `Pair`, e due `SpotMarks` in posizioni del buffer.	In `match_highlights`, durante la composizione di ogni foglia, dopo lo scorrimento: `Pattern::scan` sulle sole righe visibili.
Faccia	`show-paren-match`; `show-paren-mismatch` per due delimitatori di tipo diverso o per uno senza compagno.	`lazy-highlight`, sotto gli overlay e la regione.
Costo massimo	`SHOW_PAREN_REACH` caratteri (102 400) per parte del punto.	Le righe visibili, e al più `MATCHES_PER_WINDOW` (2000) corrispondenze per finestra.

Gli spazi a fine riga li trova `trailing_highlights`, durante la composizione di ogni foglia e di ogni flottante, sulle righe visibili: una serie di spazi e Tab che chiude la riga, o la riga intera se è fatta solo di quelli, nella faccia `trailing-whitespace`. Si disegnano solo nei buffer che visitano un file e hanno `whitespace-mode` acceso (`WHITESPACE_MODE`, in `ui/compose.rs`): lo dice il buffer che si sta componendo, sotto i lock del fotogramma, e prima non si legge nulla. Nella finestra con il focus la riga del punto non li disegna se il punto è tra loro o alla loro fine.

`hl_line_highlights` legge il modo e la riga logica del punto sotto il lock del buffer, durante la composizione della finestra selezionata. Ignora i prompt e lavora sul `Layout` già pronto: ogni `Row` con quel numero di riga produce un `Highlight` nella faccia `hl-line`, da zero alla larghezza del rettangolo del testo. Con il wrap ne produce uno per ogni continuazione visibile; comprende lo spazio dopo il testo, ma non margine, bordo o riga di stato. Foglie e flottanti lo aggiungono prima delle altre facce, senza creare overlay né modificare testo o versione. La GUI fonde gli stili cella per cella; la TUI ridisegna i tratti e conserva il fondo `hl-line` per una faccia successiva senza fondo proprio e senza inversione. Un fondo esplicito o la regione in video inverso prevalgono.

La riga si espande in coordinate di schermo perché il suo spazio vuoto non ha colonne nel buffer: così non serve un valore sentinella per la fine riga, e lo stesso percorso copre testo virtuale, scorrimento e wrap. Il costo è limitato alle righe del `Layout` visibile, senza dipendere dalla lunghezza del file.

`ComposeSettings` porta le prime due cose alla composizione, accanto al formato della riga di stato e al margine: `matches`, una `SearchMarks` con il buffer, il modello e i caratteri entro cui una corrispondenza conta (tutto il buffer per una ricerca, `from..limit` per una sostituzione, cioè ciò che resta da offrire), e

`spots`, una `SpotMarks` con l'id della finestra e le posizioni. `window_highlights` mette tutto insieme nell'ordine di disegno — colorazione, spazi a fine riga, corrispondenze, overlay, regione, parentesi — per le foglie e per le flottanti allo stesso modo, e riceve se la finestra ha il focus.

`pairing` (Sezione 10.4) guarda prima il carattere dopo il punto, se è un'apertura, poi quello prima, se è una chiusura, come `show-paren-mode` in Emacs. Ferma una scansione sul delimitatore e controlla che lo stia leggendo come codice: la scansione deve fermarsi esattamente lì, nello stato `Code`, e il passo seguente deve aprire una lista o chiuderla. Così una parentesi in una stringa, in un commento o in un letterale di carattere non è un delimitatore, e una che apre un commento (`(*`) nemmeno. Per un'apertura si fanno passi finché la pila non torna alla profondità di prima: la chiusura è l'ultimo carattere letto. Per una chiusura il compagno è il `delimiter` del frame in cima alla pila, che è quello che la chiusura toglierà, come fa lo scanner con ogni chiusura. Il tipo si confronta con i caratteri che `SyntaxClass::Open` e `Close` portano.

La sessione di ricerca tiene il modello compilato (`Isearch::pattern`), che `isearch-update` scrive a ogni tasto: `None` quando non c'è nulla di scritto, o quando una regexp non è ancora valida, e allora non si disegna nulla. La sostituzione ha già il suo `pattern`; una sostituzione finita (`done`) non disegna più nulla.

Le decisioni:

1. **Sullo schermo, non nel buffer.** Un overlay per ogni corrispondenza andrebbe ricalcolato su tutto il buffer a ogni tasto, e tolto a ogni uscita dalla ricerca, anche quando la si lascia con un tasto qualunque; e un overlay per la parentesi a ogni movimento del punto. Calcolate durante la fotografia, non c'è nulla da togliere, e ciò che si vede è sempre lo stato di quel momento. Scartato: gli overlay di Emacs (`isearch-lazy-highlight` con un timer), che servono a un ridisegno incrementale che qui non c'è.
2. **Le corrispondenze solo nelle righe visibili.** Il costo dipende dalla finestra, non dal file, e non c'è un contatore da tenere aggiornato. Il prezzo è che una corrispondenza che comincia sopra la prima riga, o finisce sotto l'ultima, non si disegna; il manpage `searching` lo dice nei limiti. Le righe visibili si cercano intere e in un pezzo solo, così `^`, `$` e `\b` vedono gli stessi bordi di riga che vede la ricerca.
3. **La parentesi sotto i lock del fotogramma.** La coppia si cerca con il buffer letto dentro `windows_mut_with_buffers`, lo stesso che si compone, così la fotografia resta uno stato in cui l'editor è stato. La tabella sintattica non si può leggere lì, perché `modes` non ha posto nell'ordine dei lock (Sezione 4.5): si legge prima, con il modo, e se nel frattempo il buffer ha cambiato modo, o ha spento `show-paren-mode`, la coppia non si disegna per quel fotogramma.
4. **Un tetto alla scansione.** Il calcolo si ripete a ogni fotogramma: senza un limite, una parentesi aperta in cima a un file grande costerebbe il file intero a ogni tasto. Il limite vale anche all'indietro, dalla fotografia dello scanner più vicina (`scan_resume`): appena dopo una modifica, prima che il `Prescanner` la raggiunga, una parentesi lontana dall'ultima fotografia valida non si disegna. È la stessa distanza di `blink-matching-paren-distance` in Emacs.
5. **Solo la finestra con il focus, per le parentesi.** Le altre finestre non disegnano il cursore, e una coppia accesa in ognuna, senza il cursore che dice perché, distrarrebbe da quella in cui si scrive. Le corrispondenze si disegnano invece in tutte: si cerca guardando, anche nell'altra metà dello schermo.
6. **Gli spazi a fine riga solo nei file.** Un elenco, l'aiuto, l'uscita di un comando o il `*scratch*` non si salvano, e spazi in fondo a una riga lì non sono un errore di nessuno: disegnarli sarebbe rumore. `whitespace-mode` può essere acceso anche lì, e non disegna nulla. Scartato: un elenco di modi esclusi, che ogni modulo nuovo dovrebbe allungare.
7. **Non dove si sta scrivendo.** Uno spazio appena scritto prima della parola successiva è uno spazio a fine riga per un istante: colorarlo farebbe lampeggiare il rosso a ogni parola, come Emacs evita allo stesso modo. Lo si salta solo nella finestra con il focus, la sola che disegna il cursore.
8. **Nulla senza i moduli.** `show-paren` e `whitespace` (Sezione 15.5.25, Sezione 15.5.29) accendono i loro modi in ogni buffer; `hl-line` (Sezione 15.5.16) lo lascia spento finché non è scelto: un editor senza moduli resta neutro, e quello che arriva con `default-configuration` mostra le coppie e ciò che un file non dovrebbe avere.

9. **Una regexp vuota non si disegna.** Una corrispondenza vuota, come quella di `x*` tra due caratteri, non ha un carattere su cui stare.

12.5 Le righe e le loro colonne

`Layout::compose` (`ui/layout.rs`) compone le righe di una finestra: una `Row` per riga dello schermo, con la linea del buffer che mostra, il testo che si disegna, e per ogni colonna della linea la cella in cui comincia e quante ne prende. Senza `visual-line-mode` una riga mostra una linea intera, e lo scorrimento orizzontale la taglia (`visible_text`), per celle. Il testo virtuale si chiede alla sua tabella una volta per riga, non per carattere.

Una colonna del buffer è un carattere; una *cella* è un posto dello schermo. `ui/width.rs` dice quante celle prende un carattere, con la crate `unicode-width`: due per un ideogramma o un'emoji, nessuna per un accento che si combina con la lettera prima, una per il resto, i caratteri di controllo compresi, che il renderer disegna come figure. Un tab si espande nella composizione, in spazi fino al prossimo multiplo di `tab-width`, contando le celle dall'inizio della riga, testo virtuale compreso: la larghezza è la proprietà `tab-width` del modo del buffer, o la variabile, o 4, tra 1 e 64. `snapshot` le legge dall'ambiente prima di prendere i lock dell'editor, una per modo registrato (`TabWidths`); il mouse legge quella del modo del buffer cliccato. La stessa misura serve ai due front-end: `width::cut` e `width::fit` tagliano e riempiono per celle le righe, la riga di stato, il margine e i titoli, e un carattere largo tagliato da un bordo diventa uno spazio per ogni sua cella dentro, così ciò che segue resta dove è stato contato.

La mappa fa la corrispondenza tra colonne e celle. `to_screen` dice in quale cella comincia una colonna del buffer, `to_buffer` il contrario: ogni cella di un tab o di un carattere largo risponde con la sua colonna. Il testo virtuale ancorato a una colonna si disegna prima di lei, così il punto non cade mai dentro il testo virtuale; una cella dentro il testo virtuale risponde con la colonna dell'ancora, e un clic su un suggerimento mette il punto sul carattere davanti a cui sta. Oltre la fine della linea le due funzioni vanno una colonna per cella, senza fermarsi: una selezione che continua sulla linea dopo mostra selezionato l'a capo. Lo scorrimento orizzontale segue il punto in modo che tutte le celle del suo carattere si vedano.

`place` è il solo punto in cui un'evidenziazione in colonne del buffer (`BufferHighlight`) diventa una in colonne dello schermo (`Highlight`): la mappa attraverso la riga, la sposta dello scorrimento orizzontale e la taglia alla larghezza. Sintassi, overlay e regione passano tutti di lì, e coprono le celle di ciò che coprono: tutte quelle di un tab o di un carattere largo. I caratteri di controllo arrivano così come sono fino al renderer, che li disegna come figure larghe una cella ( ,  …) invece di mandarli al terminale; la GUI li salta. Un tab che arriva al renderer fuori dal testo, in un messaggio o nel nome di un buffer, è  .

12.5.1 Le righe visive

`features/visual-line.lisp` definisce `visual-line-mode` con il sistema dei modi minori (Sezione 10.2): stato nel buffer, etichetta `Wrap`, comando che alterna lo stato o legge un argomento esplicito, keymap consultata solo quando il modo è acceso. Il modulo predefinito lo carica senza accenderlo. Non ci sono una variabile globale e un secondo flag da tenere d'accordo.

`Row::wrap` divide la linea già composta in intervalli di celle: preferisce gli spazi e spezza una parola più larga della finestra al confine di un carattere. Le tabulazioni sono già espanse e i caratteri larghi già misurati; gli accenti che non prendono celle restano con il carattere precedente. Il testo del buffer resta identico, come la cronologia di undo e il flag delle modifiche. Le righe di continuazione condividono la mappa delle colonne attraverso un `Arc`, senza copiarla una volta per continuazione. Una fine linea che cade esattamente sul bordo ha una riga visiva vuota per il punto.

Ogni `Row` conserva il numero della linea del buffer, l'indice della sua continuazione e gli estremi del proprio intervallo di celle nella linea composta. `row_of_position` sceglie la continuazione che contiene una posizione; `to_screen` e `to_buffer` la traducono dentro quella riga. `place` distribuisce ogni evidenziazione su tutte le continuazioni della sua linea, tagliandola ai loro estremi. Anche le facce del testo virtuale si dividono agli stessi confini.

`ui/visual_lines.rs` dà a una riga l'indirizzo `VisualPoint { line, row }` e permette di avanzare, arretrare e comporre una finestra da quell'indirizzo. Compone le linee visibili o attraversate da un movimento, senza scandire l'intero buffer a ogni frame. Una cache locale conserva al massimo l'ultima linea composta durante l'operazione.

`Window::scroll_y` resta una linea del file. `Window::wrap` aggiunge la continuazione in cima e il layout appena disegnato, con larghezza, tabulazione e versione del testo. Due finestre sullo stesso buffer hanno quindi larghezze e posizioni indipendenti. La composizione toglie prima il margine, azzera lo scorrimento orizzontale e segue il punto in righe visive; il margine numera soltanto la prima continuazione. Il mouse legge le righe registrate sulla finestra per risalire alla linea e alla colonna originali.

`primitives/visual_lines.rs` registra i quattro comandi della keymap del modo. I movimenti verticali conservano la colonna dello schermo tra comandi consecutivi e attraversano le continuazioni composte soltanto da testo virtuale o da una parte di tabulazione, che non contengono posizioni proprie del buffer. I comandi di movimento logico restano disponibili al Lisp. `editor/visual_lines.rs` coordina lo scroll di pagina, la rotella e `recenter`, componendo e registrando il nuovo layout anche tra due frame.

`tests/visual_line_tests.rs` verifica stato per buffer, testo e undo, coordinate in entrambe le direzioni, tasti, scroll, ridimensionamento, finestre divise, margine, selezioni, overlay, Unicode e testo virtuale.

12.6 Il margine dei numeri

`display-line-numbers-mode`, un modo minore (Sezione 10.2), accende il margine a sinistra del testo nelle finestre affiancate che mostrano il buffer: la composizione lo chiede al buffer (`gutter::MODE`) con lo stesso lock con cui ne legge il punto e il numero di righe. `display-line-numbers-type` dice quali numeri (`LineNumbers`): quelli delle righe, contati da uno, o con `relative` la distanza dalla riga del punto, che mostra il proprio numero. `gutter_columns` (`ui/gutter.rs`) dice quante colonne prende: le cifre del numero più grande, almeno `display-line-numbers-width` (3 per default, al massimo 20), più lo spazio prima del testo; zero se al testo resterebbero meno di `MIN_TEXT_WIDTH_FOR_GUTTER` (8) colonne. `gutter_cells` ne dà una cella per riga dello schermo, larga esattamente quanto il margine, vuota oltre la fine del buffer, con la faccia `line-number`, o `line-number-current` sulla riga del punto. Un numero troppo largo per il margine perde le cifre di sinistra. Il tipo e la larghezza si leggono in un `GutterSpec` prima che la composizione prenda un lock; se il margine c'è lo decide ogni finestra, dal suo buffer.

12.7 La riga di stato

La riga di stato prende l'ultima riga del rettangolo di una finestra affiancata alta almeno due righe: una finestra di una riga la tiene per il testo, e le strisce e le flottanti non ne hanno. Sta fuori dal rettangolo del testo come il margine, ed è larga quanto la finestra, margine compreso.

`expand_mode_line` (`ui/mode_line.rs`) la scrive seguendo `mode-line-format` (predefinito `" %* %b  %m%n %s  L%l C%c  %p "`): `%b` nome del buffer, `%f` file (o nome), `%m` modo, `%n` le etichette dei modi minori accesi, ognuna dopo uno spazio (Sezione 10.2), `%s` lo stato che il modo tiene nel buffer (`mode-line-state`, come il `NORMAL` di vim), `%l` riga (da 1), `%c` colonna (da 0), `%p` posizione (`All`, `Top`, `Bot` o percentuale), `%*` `**` se modificato e `--` altrimenti, `%%`. `%p` dice `All` quando tutto il buffer sta nella finestra, le cui righe di testo `expand_mode_line` riceve, come in Emacs; altrimenti dice dove è il punto. Una sequenza `%` che non conosce resta com'è scritta, e un formato che non è una stringa ricade su quello predefinito.

12.8 Il renderer della TUI

`render_frame` (`src/tui/render.rs`) disegna una fotografia sullo standard output attraverso `render_to`, che scrive in qualunque `Write` e disegna in quest'ordine: nasconde il cursore e pulisce lo schermo; disegna i separatori (prima delle finestre, perché una flottante deve poterli coprire); per ogni vista disegna bordo e titolo, ogni riga del rettangolo riempita fino alla larghezza, contata in celle (così una flottante è opaca), il margine cella per cella, le evidenziazioni sopra il testo, la riga di stato sotto il rettangolo; poi l'area

messaggi nell'ultima riga, l'input in sospeso allineato a destra nella stessa riga, il cursore *per ultimo* e infine, se c'è, la sequenza OSC 52 per gli appunti.

I colori sono risolti dal front-end secondo la profondità del terminale: 24 bit con `COLORTERM` che contiene `truecolor` o `24bit`; il cubo a 256 colori (o la scala di grigi, se più vicina) con `TERM` che contiene `256color`; altrimenti la voce più vicina dei 16 colori della tavolozza, nell'ordine di `NAMED_COLORS`.

Gli appunti in uscita passano da OSC 52 perché l'editor non sa di avere un terminale e un programma esterno (`xclip`, `pbcopy`) non funzionerebbe via SSH. In entrata non c'è nulla da leggere: il terminale consegna gli appunti con l'incolla «bracketed», che diventa `insert-pasted-text`.

12.9 La finestra grafica

`gui_main` (`src/gui/mod.rs`) apre una finestra con eframe, la libreria di applicazioni di egui, quando lo standard input non è un terminale o quando la riga di comando chiede `--gui`; `--font` le dà un file di caratteri a larghezza fissa. La finestra raggiunge l'editor attraverso `Core`, che passa da dove passa il terminale: i tasti per `handle_key_event`, il resto per il Lisp. Così vim, le macro e gli hook vedono ciò che si fa nella finestra come ciò che si fa nel terminale.

A ogni aggiornamento (`EditorApp::update`, in `src/gui/app.rs`) la finestra legge gli eventi una volta per frame, chiama `tick`, disegna la barra dei comandi, adatta la griglia alla finestra e chiama `resize` se è cambiata, compone la fotografia con `snapshot` e la disegna (`src/gui/paint.rs`); poi chiede il prossimo aggiornamento solo quando `next_redraw_in` o `drag_scroll_in` danno una durata. Il lavoro di sfondo la sveglia come sveglia il ciclo della TUI: la callback registrata con `on_background_change` chiama `request_repaint` sul contesto di egui. Ferma, la finestra non si aggiorna: prima lo faceva ogni 250 ms anche quando nulla cambiava. Chiudere la finestra chiede all'editor di uscire come `C-x C-c`, con le domande sui buffer modificati.

I comandi della barra (Open, Save, Save as, Copy, Cut, Undo, Vim) passano per `M-x`, con i loro hook, il loro passo di undo e la macro in registrazione, e si rifiutano mentre un prompt è aperto. I tasti si traducono in `src/gui/keys.rs`. Con Ctrl o Alt vince il testo che la tastiera riporta, se c'è, perché la punteggiatura dipende dalla disposizione dei tasti; il testo di AltGr, e quello non ASCII di Option, resta testo da scrivere. egui prende per sé Ctrl-C, Ctrl-X e Ctrl-V e li consegna come Copy, Cut e Paste: Copy e Cut tornano `C-c` e `C-x`, e sono `kill-ring-save` e `kill-region` solo con Shift o su macOS; Paste incolla con Shift o su macOS, altrimenti è `C-v`. Canc, Ins, Inizio, Fine, PagSu, PagGiù e i tasti da F1 a F12 arrivano al core con il loro nome (`special_key`), come dal terminale, e fanno ciò che le mappe dicono.

Il disegno usa una griglia di celle come il terminale, un carattere per cella; gli stili delle evidenziazioni si fondono con quelli del testo sotto, dove la TUI ridisegna sopra. Il testo che un metodo di input sta componendo si disegna al cursore, e il rettangolo del cursore passa a egui, che vi apre la finestra del metodo di input.

12.10 Il mouse

Con `mouse-mode` attivo la TUI chiede al terminale di riportare il mouse. `handle_mouse_event` traduce l'evento in un `Hit` (testo di una finestra, riga di stato, separatore, flottante, niente), e da lì in un comando: clic per spostare il punto e il focus, doppio clic, trascinamento per selezionare (con scorrimento automatico ogni 60 ms oltre il bordo), trascinamento di un separatore o di una riga di stato per ridimensionare, rotella per scorrere la finestra sotto il puntatore. Risponde `true` solo se ha cambiato qualcosa. Le domande sulle finestre — che cosa c'è sotto il puntatore, il trascinamento in corso, l'ultimo clic — le fa a `windows.rs`; il punto di un clic diventa una colonna del buffer in `buffer_column`, che compone la sola riga cliccata, perché il testo virtuale fa di una colonna sullo schermo un'altra colonna nel buffer. I comandi sono in `primitives/mouse.rs`, e si registrano senza specifiche degli argomenti: i loro argomenti li passa `handle_mouse_event`, non un prompt.

12.11 Le decisioni

1. **Una fotografia sola, presa tutta insieme.** I lavori, i thread dei comandi di shell e quelli di `spawn` cambiano lo stato mentre il front-end disegna: un renderer che leggesse sei lock in sei istanti comporrebbe un frame mai esistito. `snapshot` legge tutto sotto i lock, senza I/O, e il disegno viene dopo, senza lock.
2. **`snapshot` prende le finestre in scrittura.** Comporre una finestra ne sposta lo scorrimento per tenere visibile il punto: disegnare cambia lo stato. Va bene finché disegna un thread solo; per comporre più frame in parallelo, quel raccordo dovrebbe diventare un passo dopo ogni comando.
3. **L'editor dice al front-end quando ridisegnare.** Quasi tutto cambia per un evento, ma un messaggio scade, la colorazione arriva, un lavoro riempie un buffer: il front-end chiede `next_redraw_in` e riceve una durata sola, e un secondo front-end ha lo stesso comportamento chiedendo la stessa cosa. `None` è la risposta comune, e un editor fermo non costa nulla.
4. **Le flottanti non hanno margine né riga di stato.** Una flottante è un prompt o una striscia di scelte, poche righe lette e chiuse: numerarle non dice nulla, e il bordo dice già che cosa sono.
5. **Il mouse calcola dove, un comando decide che cosa.** Dove cade un clic dipende dal layout e dallo scorrimento di ogni finestra, e lo calcola l'editor; che cosa significa passa dai comandi come un tasto: un passo di undo, un `post-command-hook`, un nome che `M-x` e il log conoscono, un legame che il Lisp può cambiare.
6. **Un clic chiude la selezione.** Una regione va dal mark al punto, e spostare il punto ne sposta un capo: senza questo, ogni clic con una selezione aperta ne trascinerebbe il capo dove si è cliccato, anche lontano, e lascerebbe una selezione su mezzo file che nessuno ha chiesto. Il mark si disattiva e non si cancella: `exchange-point-and-mark` ci torna.
7. **Un trascinamento continua nella finestra in cui è cominciato.** Il seguito e la fine di un trascinamento si risolvono prima di chiedere che cosa c'è sotto il puntatore: dopo poche righe il puntatore è spesso su un'altra finestra o fuori dal frame, e una selezione che cambiasse buffer a metà non sarebbe una selezione.
8. **`mouse-mode` vale per ogni front-end.** `handle_mouse_event` lo controlla da sé, non solo dove il terminale accende la cattura: una finestra grafica, che riceve sempre il mouse, gli ubbidisce allo stesso modo.
9. **La fotografia cresce di un campo, mai di un lock del renderer.** Ciò che un front-end deve sapere entra in `FrameSnapshot`: un renderer che leggesse lo stato da sé tornerebbe a vedere un frame mai esistito. Il tema è nella fotografia per la stessa ragione: disegnare non tocca nulla di condiviso, e la fotografia sa i suoi colori.
10. **L'input in sospeso e l'offerta della keymap transitoria sono campi propri.** Un messaggio dice che cosa è successo, e scade; questi dicono che cosa è vero ora, e restano finché lo è. L'offerta prende il posto del messaggio, e non entra nell'input in sospeso, che finisce con il trattino della sequenza.
11. **La colorazione in corso e gli appunti viaggiano nella fotografia.** Il colore arriva da solo, a passi: un renderer fermo su un evento dormirebbe con il file ancora in bianco e nero, e `colouring_pending` gli dice di tornare. Gli appunti di sistema li raggiunge solo il front-end, che ha un terminale o una finestra: il testo si prende una volta, con la fotografia, e il front-end lo consegna.
12. **Righe, non linee.** Una `Row` ricorda la linea che mostra, ma non c'è un genere di riga che nessuno crea: una decorazione su una riga sua, o una linea spezzata su più righe, saranno un produttore nuovo di righe, non un caso in più di questa.
13. **Nessuna mappa per le righe semplici.** Il caso comune è una riga senza testo virtuale, senza tab e senza caratteri larghi: la colonna è la cella, e la riga non tiene nulla (`columns: None`); per una riga ASCII lo dice un controllo sui byte. Solo le altre tengono, per ogni colonna, la cella in cui comincia e quante ne prende.
14. **Un tipo per le evidenziazioni in colonne del buffer.** `BufferHighlight` e `Highlight` hanno gli stessi campi e un significato diverso: con un tipo solo, un'evidenziazione non portata sullo schermo si disegnerebbe sfasata del testo virtuale e dello scorrimento, senza errori. E `place` è il solo punto che le porta: mappare, scorrere e tagliare in un posto solo, per le tre fonti.

15. **La finestra ricorda ciò che l'ultimo frame ha deciso.** Il rettangolo, le righe di testo, il margine, come lo scorrimento: i comandi e il mouse li leggono tra un frame e l'altro, e un clic e il disegno non possono non concordare, perché leggono la stessa cosa. Un rettangolo vuoto è una finestra non ancora disegnata, che nessun clic trova.
16. **Un punto per finestra.** Due finestre sullo stesso buffer sono due viste, e ognuna resta dove la si è lasciata. Il punto si scrive nella composizione, per le finestre che lo seguono, e non si salva all'uscita da ogni comando, che avrebbe troppi posti da cui uscire. Mostrare un altro buffer dimentica punto e scorrimento: l'offset 4000 di un file non dice nulla di un altro.
17. **Le divisioni fisse.** `FirstFixed` tiene larga la lista di dired mentre la finestra cresce; `SecondFixed` tiene la striscia aggiunta in basso, come `split-window-right` con una dimensione in Emacs. `SplitPath` esiste perché le divisioni non hanno un id: si trovano per la strada dalla radice.
18. **L'albero non ha misure.** Le misure le dà `split_rects` a ogni composizione: nessuna finestra è troppo piccola da dividere, e l'ultima finestra non si chiude. Trascinare una riga di stato o un separatore cambia la divisione senza cambiarne il genere, con un minimo di 2 righe e 4 colonne; la riga di stato fa da confine tra due finestre impilate, e l'ultima in basso non è un confine, perché sotto c'è l'area messaggi. Un frame piccolo può comunque dividere una finestra più piccola di così.
19. **Il margine ha una larghezza minima, non adatta.** Adattato al numero di righe, il testo scivolerebbe di una colonna appena un file supera le 99 righe o scende sotto le 100; con un minimo resta fermo, e si allarga solo se serve. Sotto le 8 colonne di testo i numeri non si mostrano: sarebbero quasi tutto ciò che la finestra mostra.
20. **Una cella del margine per riga, anche oltre la fine.** Celle e righe vanno a coppie per indice, e nessun renderer disegna i numeri di righe che non ci sono più. `gutter_columns` risponde con una larghezza, zero per nessuna, non con un `Option`: chi la usa fa la stessa cosa nei due casi.
21. **Il margine e la riga di stato stanno fuori dal rettangolo del testo.** Lo scorrimento, il cursore e le evidenziazioni non sanno del margine, e un renderer che non conosce il margine disegna tutto dov'era. La larghezza del margine è dichiarata perché la riga di stato lo attraversa, e una finestra senza righe non ha celle da cui misurarlo.
22. **Seguire il punto solo quando si muove.** Se lo scorrimento seguisse il punto a ogni frame, la rotella sarebbe ricondotta subito dov'era: come in Emacs, scorrere è guardare altrove finché non si dice dove si è. Mentre la tastiera è in una flottante seguono il punto anche le finestre senza il focus: durante una ricerca incrementale il punto si muove nel buffer sotto, e un `C-s` che portasse la corrispondenza fuori dallo schermo sembrerebbe fermo.
23. **Ricomporre, non aggiustare.** Quando lo scorrimento cambia le righe si compongono di nuovo: correggere le righe già composte sarebbe una seconda implementazione del layout, da tenere d'accordo con la prima.
24. **L'ordine delle evidenziazioni è una precedenza.** Un overlay batte il colore, perché una diagnostica su una parola chiave deve vedersi; la regione batte tutto, perché ciò che si è selezionato si deve vedere. Le evidenziazioni sono una lista di eccezioni accanto alle righe, non celle con uno stile: una riga senza evidenziazioni è una stringa.
25. **Le impostazioni si leggono prima dei lock, e viaggiano insieme.** Il Lisp ha lock suoi, e la composizione gira con quelli delle finestre e dei buffer già presi; `ComposeSettings` le porta come un gruppo, al posto di una firma di otto argomenti.
26. **La composizione legge la cache dei colori, non colora.** Un file grande si apre senza colore e si colora mentre lo si guarda: colorare nella composizione bloccherebbe il primo frame sul file intero.
27. **Il renderer prende la fotografia, mai l'editor.** Così l'I/O avviene senza lock, e il disegno potrà passare su un thread suo senza rivedere ogni funzione. `render_to` scrive in qualunque `Write` perché i test leggano le sequenze che produce, per esempio dove finisce il cursore.
28. **Il cursore per ultimo, i separatori per primi.** Ogni riga scritta sposta il cursore del terminale, e l'area messaggi lo lascerebbe alla fine del suo testo. Una linea tra le finestre occupa una colonna che nessuna finestra affiancata usa, e una flottante deve poterla coprire.

29. **Ogni riga del rettangolo si riempie fino alla larghezza.** Così una flottante è opaca, e il buffer sotto non si vede tra le sue righe corte; costa nulla, perché gli spazi stanno nella stessa stringa. Le evidenziazioni si ridisegnano sopra le righe: una riga semplice è una sola scrittura. La riga di stato è larga anche sul margine: una barra con una tacca non è una barra.
30. **Gli appunti escono con OSC 52.** L'editor non sa quale programma degli appunti ci sia (`pbcopy`, `xclip`), e via SSH un programma scriverebbe gli appunti della macchina remota. La sequenza finisce con ST e non con BEL perché tmux lo passa senza trattamenti. Gli appunti non si leggono con OSC 52: molti terminali lo disattivano, e la risposta arriverebbe mescolata ai tasti; entrano con l'incolla. Il base64 è scritto a mano: venti righe, dove un crate sarebbe una dipendenza in più per un alfabeto.
31. **Nel dubbio, meno colori.** La profondità si legge da `COLORTERM` e `TERM`, e si indovina in basso: un terminale a 24 bit che riceve un indice della tavolozza mostra un colore vicino, mentre uno a 16 colori che riceve una sequenza a 24 bit non mostra nulla di utile. La tavolozza è la tabella `NAMED_COLORS` del core, una sola: il colore di una voce è quello che l'utente ha dato al suo terminale. Con 256 colori vince il più vicino tra il cubo e la scala di grigi, che rende meglio i grigi.
32. **La finestra grafica passa da dove passa il terminale.** Tasti e Lisp, non le primitive chiamate da Rust: vim, le macro e gli hook vedono tutto, e la barra dei comandi è `M-x` con i suoi hook, il suo undo e la sua macro.
33. **Ctrl-V resta dell'editor.** È la pagina in giù di Emacs e il blocco visivo di vim: incollare con Ctrl-V li toglierebbe a tutti e due. Incolla Ctrl+Shift+V su Linux e Windows, e Cmd+V su macOS. Con gli appunti vuoti egui-winit non manda la pressione di Ctrl-V, e allora il rilascio vale `C-v`. egui-winit manda un evento di tasto per ogni pressione, e il testo solo quando Ctrl non è premuto: lo stesso tasto non si scrive due volte.
34. **La griglia di celle.** La finestra disegna come un terminale, un carattere per cella: la fotografia è fatta di righe e colonne, e il mouse e le evidenziazioni parlano in colonne. Dove la TUI ridisegna sopra, la finestra fonde gli stili, perché non può riscrivere una cella.
35. **Un frame legge l'input una volta.** `update` può essere chiamato di nuovo per lo stesso frame, e gli eventi letti una seconda volta premerebbero due volte lo stesso tasto: `last_input_frame` ricorda il frame già letto.

13 Macro di tastiera

Una macro registra *tasti*, non comandi: `C-u 5`, un tasto letto da `read-key-sequence` e perfino un tasto non associato fanno parte di ciò che l'utente ha premuto. `handle_key_event` chiama `record_keys` nei punti in cui un tasto viene consumato. La riproduzione (`replay_keys`) rimanda i tasti a `handle_key_event`, con un `OneGroup` per ripetizione, e si ferma al primo comando fallito, confrontando `command_errors` prima e dopo ogni tasto. Una macro che richiama macro può annidarsi fino a 16 livelli (`MAX_DEPTH`).

Le primitive stanno in `primitives/macros.rs`. `define-kbd-macro` fa di una stringa di tasti un comando con un nome: una funzione Lisp il cui corpo chiama `kmacro-call-keys` con la stringa, e la cui docstring dice quali tasti preme. Una stringa che non si legge come sequenza di tasti è rifiutata subito. Il contatore (`kmacro-insert-counter`, `kmacro-set-counter`, `kmacro-counter`) sta nel compartimento accanto ai tasti. `kmacro-insert-counter` legge la sua larghezza con `args::width` (Sezione 14.3.9), prima che il contatore vada avanti, e mette gli zeri a mano: `format!` va in panic con una larghezza oltre 65 535.

13.1 Le decisioni

Tre decisioni sulla macro stanno con il suo compartimento (Sezione 4.8): una macro registra tasti, registrazione e riproduzione stanno insieme, le riproduzioni annidate hanno un tetto. Le altre sono della facciata, e l'ultima delle primitive.

1. **La riproduzione passa dalla porta dei tasti.** Ogni tasto registrato torna a `handle_key_event`, la stessa porta di un tasto del terminale: un secondo dispatcher sarebbe un secondo insieme di regole su argomenti prefisso, keymap transitorie, catture e prompt, tenuto in passo a mano. Il prezzo è il rientro, ed è già pagato: il carburante conta la profondità e solo lo scope più esterno ricarica, così i comandi di una riproduzione spendono il budget della riproduzione.
2. **Una riproduzione si ferma al primo comando fallito.** Ogni tasto dopo un errore sarebbe premuto in uno stato che la registrazione non ha mai visto. `run_command_form` non restituisce l'errore — un tasto non ha un chiamante a cui restituirlo —: `report_error` incrementa `command_errors`, che la riproduzione confronta prima e dopo ogni tasto. Un conteggio, che nessuno deve azzerare.
3. **Nessun tasto interrompe una riproduzione.** Il ciclo non legge tasti: è dentro un comando, e il thread che consegnerebbe `C-g` è quello che lo esegue. Una macro che richiama sé stessa la ferma il tetto di profondità, non il budget: ogni livello è una vera chiamata attraverso il gestore dei tasti, e lo stack finirebbe molto prima, interrompendo il processo. Per lo stesso motivo una riproduzione disegna un frame solo, quando il comando restituisce.
4. **Un gruppo di undo per ripetizione.** Ogni ripetizione ha il suo timbro e il suo `OneGroup`: venti ripetizioni sono venti passi, e una macro andata storta alla diciannovesima si riporta indietro fino lì.
5. **Una macro con un nome è una funzione Lisp.** Un comando qui è una funzione Lisp, e una macro costruita come chiusura Rust sarebbe invisibile a `describe-function` e a tutto ciò che legge una definizione; la sua docstring dice i tasti, che sono tutto ciò che c'è da sapere di lei. Una stringa sbagliata si rifiuta quando si definisce la macro, perché quando la si esegue l'unica cosa da dire sarebbe che una macro che nessuno vede è vuota.

14 Le primitive dell'editor

14.1 Il registro

Ogni modulo di `primitives/` ha una funzione `install(into: &Registry)` che registra le sue primitive con `into.function(nome, puntatore, DOC)` o, per i comandi, `into.command(nome, puntatore, specifiche, DOC)`. `install_primitives` chiama tutti i moduli. La macro `primitive!` genera la firma richiesta da `LispPrimitive`; `args.rs` contiene i lettori degli argomenti (Sezione 14.3.9).

Ogni docstring comincia con la firma, nella forma `(nome ARG):` seguita da una frase: è ciò che mostrano i comandi di aiuto. `doc_faithfulness_tests` confronta ciò che l'aiuto racconta con l'editor reale; `primitive_shape_tests` verifica le forme condivise dalle primitive, come il controllo dell'arità.

Modulo	N.	Contenuto
`movement`	20	Movimenti per carattere, riga, parola e paragrafo, ai bordi della riga e del buffer, `goto-line`; il punto come numero (`point`, `goto-char`, `current-column`).
`buffers`	18	Creare, cambiare, chiudere buffer; contenuto; sola lettura; dati appesi (`buffer-put`, `buffer-get`).
`windows`	15	Dividere, chiudere, scegliere, scorrere finestre; `recenter`, `display-buffer-at-bottom`, finestre flottanti.
`killing`	14	Le kill, `kill-ring-save`, `yank`, `yank-pop`, `kill-new` e il resto del kill ring.
`loading`	9	`require`, `provide`, ciò che è caricato e da dove, `eval-after-load`, i plugin (Sezione 15).
`verbs`	14	Maiuscole, spazi, righe vuote, unione e duplicazione di righe, trasposizioni, `zap-to-char`.
`macros`	12	Registrazione e riproduzione delle macro, contatore.
`minibuffer`	12	`minibuffer-read`, conferma, annullamento, completamento, storia.
`replace`	12	`query-replace`, `replace-string`, le loro versioni con le espressioni e i verbi di una vista.
`results`	11	Le liste di posti: leggerle, percorrerle con `next-error`, attaccarne una da Lisp, mostrarle.
`commands`	10	`call-interactively`, `M-x`, `register-command` e il registro dei comandi.
`isearch`	10	La ricerca incrementale: i quattro comandi che la cominciano e ciò su cui gira.
`rectangle`	10	Operazioni sui rettangoli.
`syntax_table`	10	La tabella sintattica, `syntax-class`, `matching-delimiter` e le domande allo scanner: `syntax-ppss`, `balance-point`, `bounds-of-enclosing-list`.
`region`	9	Mark e regione, `keyboard-quit`.
`workers`	9	`define-worker`, `background-call`, `stop-worker`, `running-workers`, `emit-event`, le opzioni dello scheduler, `work-list`.
`files`	8	Aprire, salvare, rileggere e recuperare un file; `write-file`.
`match_data`	8	`string-match`, i dati della corrispondenza, `regexp-opt`.
`completion`	7	`completion-at-point`, le sue fonti, `buffer-words`, `bounds-of-thing-at-point`.
`edits`	7	Le due porte; `self-insert`, `insert`, l'incolla, `insert-newline`, `delete-region`, `delete-char`, `delete-backward-char`.

Modulo	N.	Contenuto
`file_names`	7	Percorsi resi assoluti e smontati, `list-dir`, `match-list`, il completamento dei nomi dei file.
`filesystem`	7	Esistenza, cancellazione e rinomina di file, `directory-files-recursive`, `read-file-to-string`.
`help`	7	Ciò che serve all'aiuto: `key-binding`, `where-is`, `keymap-bindings`, `read-key-sequence`. I comandi `describe-*` sono in `help.lisp`.
`indentation`	7	Lettura e scrittura del rientro, `indent-width`, `indent-line`, Tab con cycling e `newline-and-indent`.
`sexp`	7	Movimenti e kill per espressioni bilanciate, `up-list`, `down-list`.
`comments`	6	Commentare e decommentare.
`keymaps`	6	`define-key`, `set-local-keymap`, le keymap transitorie, `define-repeat-key`, `repeat`.
`mouse`	6	Clic, trascinamenti e rotella.
`theme`	6	Le facce e i loro stili, da `set-face` a `list-colors`.
`overlays`	5	Overlay: farli, cercarli, toglierli.
`quit`	5	`quit`, `save-some-buffers` e le loro risposte.
`undo`	5	`undo`, `redo`, `undo-boundary`, `undo-join`, `set-undo-limit`.
`ask`	4	Domande sì/no con keymap transitoria.
`diagnostics`	4	`log`, `all-logs`, `backtrace`, `set-echo-message`.
`environment`	4	`getenv`, `setenv`, `path-separator`, `data-directory`.
`modes`	4	`make-mode`, `add-hook`, `remove-hook`, `add-auto-mode`.
`shell`	4	Comandi di shell, in background o fino alla fine; `parse-overstrike`.
`virtual_text`	4	Testo virtuale: metterlo, cercarlo, toglierlo.
`evaluation`	2	`eval-file`, `eval-string-top-level`.
`grammar`	2	`add-syntax-rule`, `add-syntax-region`.
`occur`	2	`occur--scan`, `grep--scan`: le ricerche che fanno una lista.
`fuzzy`	1	`fuzzy-filter`.
`scan`	1	`scan-buffer`: tutte le corrispondenze di un modello in un buffer, come voci di una lista di risultati.

Il totale è 327, a cui si aggiungono le 86 primitive di base dell'interprete. I nomi li elenca l'editor stesso: `C-h a` cerca fra tutti, e `C-h f` ne descrive uno.

14.2 Le primitive di modifica

Le primitive che cambiano il testo o spostano il punto stanno in tredici file, uno per concetto. Tutte cambiano il testo soltanto dalle due porte, e `undo` e `redo` dalla terza (Sezione 8.3), e nessuna prende il lock di un altro compartimento dentro la closure di un buffer: chiede prima ciò che le serve, lavora nella closure e riferisce dopo (Sezione 4.5).

File	Contenuto
`edits.rs`	Le due porte (`insert_text`, `delete_range`) e la terza di undo e redo (`replay`), `changed`, `edited`; le primitive che inseriscono e cancellano caratteri: `self-insert`, `insert`, `insert-pasted-text`, `insert-newline`, `delete-region`, `delete-char`, `delete-backward-char`.

File	Contenuto
`undo.rs`	`undo`, `redo`, `undo-boundary`, `undo-join`, `set-undo-limit` (Sezione 8.4).
`movement.rs`	Il movimento per carattere, riga, parola e paragrafo, ai bordi della riga e del buffer, `goto-line`; il punto come numero; le funzioni che dicono dove finiscono una parola, una riga, un paragrafo.
`killing.rs`	Le kill, da `kill-line` a `kill-region`; `kill-ring-save`, `yank`, `yank-pop`, `kill-new` e il resto dell'anello; `cut_out`.
`sexp.rs`	Il movimento per espressioni bilanciate, dentro e fuori dalle liste, e le due kill di espressioni.
`indentation.rs`	Lettura e scrittura del rientro; consultazione delle regole, Tab con cycling e `newline-and-indent`.
`verbs.rs`	Maiuscole, spazi, righe, trasposizioni, `zap-to-char`.
`region.rs`	Il mark e la regione, `keyboard-quit`.
`rectangle.rs`	I comandi sui rettangoli, la cui geometria è in `text/rectangle.rs` (Sezione 9.6).
`comments.rs`	Commentare e decommentare.
`macros.rs`	Registrare e riprodurre i tasti, e il contatore (Sezione 13).
`match_data.rs`	`string-match` e i dati della corrispondenza.
`scan.rs`	`scan-buffer`.

I conteggi si leggono con `movement_count`, in `args.rs`: un numero con il segno, per i movimenti e per le cancellazioni, dove un conteggio negativo fa la stessa cosa nell'altra direzione, come in Emacs: `(kill-word -1)` è `backward-kill-word`, `(delete-char -1)` cancella il carattere prima del punto. Non ha un limite, perché non serve: ogni passo si ferma al bordo del buffer, e un conteggio enorme vuol dire «fin dove si può». Chi lo somma a una posizione lo fa saturando, come `delete-char`.

Dove il buffer ha `overwrite-mode` acceso (`OVERWRITE_MODE`, Sezione 15.5.20), `self-insert` non inserisce: `overwrite_at_point` toglie dopo il punto tanti caratteri quanti ne scrive, senza passare la fine della riga, e poi inserisce il testo, con le due porte e nello stesso comando, quindi in un solo passo di undo. Un a capo si inserisce sempre; `insert`, l'incolla e `insert-newline` non guardano il modo.

Le due porte restituiscono `false` in un buffer in sola lettura, e chi le chiama lo deve dire: `edited(ctx, happened)` scrive «Buffer is read-only» nell'area messaggi e risponde nil, chiamata dopo la closure, perché l'area messaggi è un compartimento a sé. La usano le primitive di `edits.rs`, `kill-region`, `yank`, `yank-pop` e `indent-line-to`. Per tutti gli altri un rifiuto delle porte alza un segno del thread (`take_refusal`), che `run_command_form` legge alla fine del comando e dice con lo stesso messaggio: le kill, i verbi e i commenti, e ogni comando scritto in Lisp, lo dicono senza doverselo ricordare. I rettangoli e `kmacro-insert-counter` chiedono prima se il buffer è in sola lettura, e in quel caso non fanno nulla: niente blocco conservato, punto fermo, contatore che non avanza.

14.2.1 Muovere il punto

`movement.rs` ha due parti. Le funzioni sul testo conoscono solo `BufferTrait`: `goto_offset`, `line_length`, `line_bounds`, `is_blank_line`, `is_word_char`, `word_forward`, `word_backward`, `paragraph_forward` e `paragraph_backward`. Le usano le primitive del file, e con loro le kill, i verbi, i commenti, i rettangoli, il mouse e le liste di risultati. Una parola è una sequenza di lettere, cifre e trattini bassi, senza una tabella sintattica da interrogare; un paragrafo finisce alla riga vuota che lo segue.

`primitives/subword.rs` condivide tra movimento e cancellazione i confini alternativi degli identificatori. `forward` e `backward` ricevono il buffer: con `subword-mode` spento delegano a `word_forward` e `word_backward`; acceso, separano minuscola/maiuscola, lettere/cifre e acronimi (`HTTP|Server`). Leggono caratteri e restituiscono offset in caratteri, anche per Unicode. I separatori sono una stringa letterale, senza

espressioni regolari: `_` per default, `-` soltanto se scelto; le altre punteggiature separano le parole. Il valore globale `subword-separators` si legge prima del lock; un attachment Lisp `lisp:subword-separators` di tipo stringa lo sostituisce nel buffer. Le scansioni saltano le serie di separatori e si fermano a inizio/fine testo anche con conteggi enormi. Le primitive gestiscono segno e zero con lo stesso `movement_count` di prima; le kill passano da `cut_out`, mantenendo undo e kill ring. I verbi di maiuscole e trasposizione usano ancora le parole intere: condividere i confini dei quattro comandi di movimento/cancellazione evita che una kill prenda più di quanto attraversa il movimento corrispondente, senza cambiare il significato degli altri verbi.

`forward-char` e `backward-char` si muovono per offset, quindi attraversano un fine riga come `C-f`. `next-line` e `previous-line` tengono una *colonna obiettivo*:

da `primitives/movement.rs` — estratto semplificato

```
// Chiesti prima del lock del buffer: sono altri compartimenti.
let continuing = ctx.last_command_is("next-line") || ctx.last_command_is("previous-line");
let remembered = ctx.goal_column();
let goal = ctx.with_current_buffer_mut(|buf| {
    let (line, col) = buf.text.cursor_pos();
    let goal = match (continuing, remembered) {
        (true, Some(goal)) => goal,      // in una serie: la colonna di partenza
        _ => col,                        // altrimenti quella del punto
    };
    // ... la riga di arrivo, limitata al buffer
    buf.text.cursor_move(target, goal);  // limita la colonna alla riga
    goal
});
ctx.set_goal_column(Some(goal));
```

Il punto come numero — `point`, `point-min`, `point-max`, `goto-char`, `line-number-at-point`, `current-line`, `current-column` — serve al codice che ha calcolato una posizione invece di descrivere un movimento: un risultato di ricerca, un posto salvato. Conta in caratteri, come il mark, la regione e la storia dell'undo, e una colonna è un carattere, anche un tab, che sullo schermo può prendere più celle (Sezione 12.5).

14.2.2 Uccidere e incollare

Ogni kill toglie il testo che il movimento con lo stesso nome attraversa, chiedendo alla stessa funzione dove finisce: `kill-word` chiede `word_forward`, `kill-paragraph` `paragraph_forward`, `kill-sexp` lo scanner. Una kill mette nell'anello ciò che toglie, una cancellazione (`delete-char`, `delete-backward-char`, `delete-region`) lo scarta, come in Emacs.

Le kill di testo passano da `cut_out`, che cancella con `delete_range` e restituisce ciò che ha tolto, o niente se il buffer ha rifiutato. L'anello lo riceve dopo la closure del buffer:

da `primitives/killing.rs` — estratto semplificato

```
primitive!(kill_word, args, env, ctx, {
    let count = repeat_count(args)?;          // fuori: un `?` nella closure
    let killed = ctx.with_current_buffer_mut(|buf| {
        let from = buf.text.cursor_pos_1d();
        let to = word_forward(&buf.text, from, count);
        cut_out(buf, from, to)                // il testo tolto, o ""
    });
    ctx.kill(killed, Direction::Forward, &env);  // il lock del buffer è reso
    Ok(ELispExp::nil())
});
```

`kill-region` e `kill-ring-save` prendono la regione con `require_region`, di `region.rs`. `kill-ring-save` disattiva il mark: copiare non è una modifica, e nient'altro lo farebbe. `yank-pop` sostituisce il testo appena incollato con la voce precedente, togliendo e rimettendo attraverso le porte. Se accodare

a una voce o aggiungerne una nuova lo decide il compartimento `KillYank`, e il rettangolo ucciso sta in un campo suo (Sezione 9.5).

14.2.3 Espressioni e indentazione

I comandi sulle espressioni — `forward-sexp`, `backward-sexp`, `kill-sexp`, `backward-kill-sexp`, `up-list`, `backward-up-list`, `down-list` — leggono la tabella sintattica del modo prima del lock del buffer, perché `current_syntax_table` legge il buffer per sapere il suo modo. Poi chiedono allo scanner di `modes/sexp.rs` (Sezione 10), che riparte dal punto di ripresa più vicino della cache di scansione (`scan_resume`) invece che dall'inizio del buffer. Una parola si riconosce guardando i caratteri, un'espressione lessicando il testo con la tabella: una `)` in una stringa o in un commento è testo, e lo stesso testo si muove in modo diverso in due modi.

La colonna giusta per una riga resta una domanda sulla lingua: la primitiva `indent-line` consulta `indent-function` sul simbolo del modo e, se non c'è, usa `previous-indentation`. `indent-for-tab-command`, anch'essa Rust, consulta anche l'eventuale `indent-levels-function`: una lista di colonne alternative, ordinata e deduplicata dal nucleo. Il primo Tab applica il suggerimento; Tab consecutivi scelgono il livello inferiore e, al minimo, tornano al massimo. La cronologia dei comandi distingue un Tab consecutivo da una nuova richiesta. Le regole leggono soltanto il buffer, senza lock mantenuti durante Lisp, e il punto si ripristina anche se segnalano un errore. `indent-line-to` scrive solo spazi attraverso le porte, in un passo di undo. Return e le regioni applicano solo il suggerimento; il cycling è esclusivamente un'operazione interattiva.

14.2.4 I verbi

I verbi — `upcase-word`, `capitalize-word`, `just-one-space`, `delete-blank-lines`, `join-line`, `transpose-chars`, `zap-to-char` e gli altri — sono poche lettere di chirurgia, e trovano le parole con le funzioni di `movement.rs`: `M-t` traspone esattamente ciò che `M-f` attraversa. Ogni verbo dice nella sua docstring dove lascia il punto. I verbi delle maiuscole, con una regione, lavorano sulla regione e lasciano il punto dov'è; senza, lavorano sulla parola dopo il punto e lo lasciano dopo di essa. `join-line` chiede alla tabella sintattica se un carattere apre o chiude, per non mettere uno spazio dopo un'apertura o prima di una chiusura.

Quasi tutti passano da `replace_span`, che sostituisce un tratto e mette il punto dove il verbo vuole. `delete-trailing-whitespace` no: `delete_trailing` toglie molti tratti, dall'ultima riga alla prima perché ognuna resti dove è stata misurata, sposta il punto della lunghezza di ogni tratto che lo precede o lo porta all'inizio di quello in cui si trovava, e, per tutto il buffer con `delete-trailing-lines`, toglie prima le righe vuote in fondo, fino al primo a capo dopo l'ultimo carattere che non è uno spazio. Il modulo `whitespace` (Sezione 15.5.29) chiama questa stessa primitiva dal `before-save-hook` del suo modo minore, con gli estremi espliciti del buffer anche se c'è una regione attiva. Il Rust che salva conosce solo il protocollo degli hook (Sezione 8.7), non il nome del modo di pulizia.

La pulizia precede lo snapshot: ciò che si scrive è ciò che si vede e undo può riportare il testo com'era. Le copie periodiche restano copie del buffer. `zap-to-char` legge il suo carattere con una cattura di tasti (Sezione 5.7), armata con il nome di `zap-to-char--do`: la cattura cita ciò che riceve prima di chiamarlo, e un simbolo si cerca tra le funzioni.

14.2.5 Mark e regione

`set-mark` mette il mark e lo attiva; con INCLUSIVE i caratteri alle due estremità sono nella regione, come in una selezione modale. `region-beginning`, `region-end`, `kill-region` e `kill-ring-save` senza una regione attiva danno un errore, «The mark is not set now, so there is no region». `exchange-point-and-mark` funziona anche con un mark non più attivo, e limita la sua posizione alla lunghezza del testo: un mark inattivo non segue le modifiche, e può indicare oltre la fine. `keyboard-quit` abbandona una sequenza di tasti o un argomento prefisso a metà e disattiva il mark; non ferma un comando che gira.

14.2.6 Più modifiche in un comando

I comandi dei commenti e dei rettangoli fanno più modifiche in un comando, e ogni modifica sposta gli offset di tutto ciò che viene dopo. Le due famiglie fanno la stessa cosa: calcolano le modifiche una volta, sul testo com'è, e le applicano dall'ultima alla prima, quando gli offset di ciascuna non sono stati spostati da nessuna di quelle ancora da fare. Commentare tre righe applicando dall'alto le aperture calcolate all'inizio le mette tutte sulla prima:

Passo	Dall'alto	Dal basso
—	`ab\ncd\nef`, aperture a 0, 3, 6	lo stesso
1	a 0: `// ab\ncd\nef`	a 6: `ab\ncd\n// ef`
2	a 3: `// // ab\ncd\nef`	a 3: `ab\n// cd\n// ef`
3	a 6: `// // // ab\ncd\nef`	a 0: `// ab\n// cd\n// ef`

I commenti mettono le loro modifiche in un elenco ordinato, che `apply` applica al contrario; poi rimette il punto dove l'utente stava, calcolandolo dallo stesso elenco, perché il buffer non sa che i caratteri cresciuti davanti al punto vanno contati. La grafia viene dal modo (`set-comment-syntax`), che la colorazione, `forward-sexp` e lo scanner leggono dallo stesso posto: un modo che non ha detto nulla riceve un messaggio, e un linguaggio con soli commenti a blocco riceve le righe avvolte in uno. Decommentare toglie l'apertura e lo spazio che il comando aveva messo, non di più: `//  x` tiene l'indentazione che era dentro il commento.

I rettangoli calcolano gli span una volta (`spans`, Sezione 9.6) e `each_line` li percorre dal basso. Inserire allunga prima una riga corta fino alla colonna (`insert_at_edge`); `clear-rectangle` non allunga nulla; `yank-rectangle` aggiunge in fondo al buffer le righe che mancano al blocco, e su una riga dove il blocco ha una stringa vuota non scrive spazi. Ogni comando che cambia il buffer lascia il punto all'angolo in alto a sinistra; `copy-rectangle-as-kill` lo lascia dov'è. In un buffer in sola lettura i comandi che cambiano il testo lo chiedono prima di tutto (`refused_if_read_only`): un rifiuto a metà lascerebbe conservato per `yank-rectangle` un blocco mai tolto, che lo yank duplicherebbe.

14.2.7 I dati della corrispondenza

`string-match` ha la forma di Emacs: risponde dove comincia la corrispondenza e lascia i gruppi nei *dati della corrispondenza*, che `match-beginning`, `match-end`, `match-string` e `match-data` leggono dopo, e `set-match-data` riscrive. Il prezzo è uno stato che sopravvive alla chiamata: una funzione chiamata tra la corrispondenza e la lettura può sovrascriverlo, e la risposta è quella di Emacs, `save-match-data` (in `commands.lisp`). `string-match-p` non scrive nulla.

I dati stanno in una variabile del thread, `MATCH_DATA`: un passo di un lavoro Lisp, sul thread dell'esecutore (Sezione 11), non cambia ciò che un comando sta per leggere, mentre i lavori sullo stesso thread li condividono, e lì la risposta è di nuovo `save-match-data`. Le posizioni sono caratteri, come per `substring` e `length`: il motore dà byte, e `search` li converte. È diversa da Emacs la sintassi delle espressioni: i gruppi si scrivono `(...)`, non `\(...\)`, perché il motore è quello della ricerca (Sezione 9.2), lineare perché senza lookahead e senza backreference.

`regexp-opt`, nello stesso file, fa di una lista di parole un modello solo, che corrisponde a ognuna, con i caratteri speciali protetti, in un gruppo che non si conta; le grammatiche dei modi ne fanno le loro parole chiave.

14.2.8 `scan-buffer`

`scan-buffer` dà a Lisp `Pattern::scan` (Sezione 9.2): tutte le corrispondenze in un passaggio, per occur, grep o un linter che legge il proprio output. Risponde `(TRUNCATED ENTRIES)`, con le voci nella forma delle liste di risultati (Sezione 9.7), `(KIND SOURCE LINE COLUMN OFFSET TEXT MATCH-START MATCH-END)`, costruite dalle stesse due funzioni, `Entry::new` ed `entry_form`. Si ferma a `LIMIT` corrispondenze, 10 000 se nessuno lo dice, e allora TRUNCATED è `t`.

14.3 File, buffer, finestre e domande

Le altre primitive stanno anch'esse in un file per concetto. I comandi di shell e i lavori Lisp sono nel capitolo sul lavoro in background (Sezione 11.10.2, Sezione 11.10.3).

File	Contenuto
`files.rs`	Visitare un file e riscriverlo: `find-file`, `save-buffer`, `write-file`, `revert-buffer`, `recover-file`, con le metà che le loro domande chiamano una volta risposte; `write_buffer` (Sezione 8.7).
`file_names.rs`	I nomi dei file dal Lisp: `list-dir`, `expand-file-name` e le funzioni che smontano un nome, sopra quelle di `paths.rs` (Sezione 14.3.1); la directory di lavoro, `working-directory`, `cd` e `pwd` (Sezione 14.3.2).
`filesystem.rs`	Il disco: `file-exists-p`, `delete-file`, `rename-file`, `directory-files-recursive` con `walk_files`, `read-file-to-string`.
`environment.rs`	`getenv`, `setenv`, `path-separator`, `data-directory`.
`quit.rs`	`quit` e `save-some-buffers`, e `has_unsaved_work`, che dice se chiudere un buffer perderebbe qualcosa.
`buffers.rs`	Creare, cambiare, leggere e chiudere buffer; i valori Lisp appesi a un buffer; ciò che un buffer dice di sé.
`windows.rs`	Dividere, chiudere, scegliere e scorrere le finestre, `recenter`, le finestre flottanti.
`mouse.rs`	I comandi di un clic, di un trascinamento e della rotella (Sezione 12.10).
`ask.rs`	Le domande: `y-or-n`, `yes-or-no`, e `ask_with_keys` per chi ne fa di sue.
`args.rs`	I lettori degli argomenti, che usano tutti gli altri file.
`shell.rs`	I comandi di shell (Sezione 11.10.2).
`workers.rs`, `lisp_job.rs`	I lavori Lisp e le opzioni dello scheduler (Sezione 11.10.3).

14.3.1 I nomi dei file

Le funzioni sui nomi stanno in `paths.rs`, accanto alle directory dell'editor; le primitive che le offrono al Lisp in `primitives/file_names.rs`.

`expand_path_in` fa di un nome un percorso assoluto: espande `~` con `HOME`, o con `USERPROFILE` se `HOME` manca, lo risolve rispetto a una directory se è relativo, e toglie `.` e `..` per nome, senza chiedere al disco. Ciò che l'utente scrive passa da `EditorState::expand_path`, che la chiama con la directory di lavoro (Sezione 14.3.2): ne passano `find-file`, `write-file`, `eval-file`, le primitive del disco, `list-dir`, la ricerca nei file, la visita dei risultati e `expand-file-name`, anche per il suo secondo argomento. `expand_path_from_start` la chiama con la directory in cui l'editor è partito, per un'impostazione che non deve spostarsi con `cd`: la directory del salvataggio automatico.

Le altre funzioni smontano un nome senza toccare il disco: `as_directory` aggiunge il separatore finale, `without_trailing_separator` lo toglie (una radice lo tiene), `directory_part` e `last_component` dividono all'ultimo separatore. `directory_entries` elenca in ordine una directory, che riceve già assoluta, con il separatore dopo ogni sottodirectory, e `list-dir` può mettere prima le directory. `file_completions` ne prende i nomi che continuano ciò che è stato scritto, letto nella directory che riceve, come percorsi interi, e i file nascosti solo dopo un punto: lo usa il completamento del codice `f`, in `commands.rs`, con la directory di lavoro.

14.3.2 La directory di lavoro

L'editor ha una directory di lavoro sua, in `EditorState` (`editor/directory.rs`), che parte da quella del processo e che `cd` sposta. La directory del processo non cambia mai: un comando già partito resta dove è partito, i thread in background non la vedono muoversi, e ogni `EditorState` dei test ha la sua.

Un buffer può avere una directory propria, `Buffer::directory`; `None` segue quella dell'editor. `directory_of` sceglie tra le due, ed è l'unico punto che lo fa: un buffer che segue la directory del suo file, come in Emacs, è una modifica lì e in ciò che imposta `Buffer::directory`, non nei chiamanti. `default_directory` è la directory del buffer corrente, quella in cui `expand_path` legge un nome relativo e il completamento del codice `f` cerca i candidati; `working-directory` la restituisce al Lisp, e `pwd` la scrive nell'area messaggi. `cd` legge il suo argomento come ogni nome scritto, quindi `cd ..` sale di un livello, e rifiuta ciò che non è una directory senza cambiare nulla.

Oggi `Buffer::directory` la imposta solo `shell-command-start`, per il buffer dell'output (Sezione 11.10.2): ogni comando di shell parte nella directory del buffer corrente, e il suo output la tiene. La visita di una lista di risultati legge i suoi percorsi nella directory della lista (Sezione 9.7).

14.3.3 Il disco

Le primitive del disco espandono il percorso che ricevono. `directory-entry-count`, `delete-file`, `rename-file` e `read-file-to-string`, quando non riescono, rispondono nil e scrivono il motivo nel log. `delete-file` toglie un link simbolico e mai ciò a cui punta, e una directory piena solo con RECURSIVE; `rename-file` non sostituisce un file che c'è.

`walk_files` visita un albero in profondità, con una pila invece della ricorsione, e restituisce i percorsi relativi alla radice. Non entra nelle directory che PRUNE nomina, scarta i file che finiscono con uno dei suffissi, si ferma al limite, e prima di leggere una directory chiede se continuare. La usano `directory-files-recursive`, che paga un'unità di carburante per ogni percorso che restituisce, e la ricerca nei file, sul suo thread.

14.3.4 L'ambiente

`setenv` non cambia l'ambiente del processo: scrive in una tabella dell'editor (`EditorState::environment`) un valore, o che la variabile è tolta. `getenv` legge prima la tabella e poi il processo, e ogni programma avviato riceve la tabella sopra l'ambiente del processo (`apply_environment`, con `Command::env` ed `env_remove`). Cambiare l'ambiente del processo era `unsafe`: il lock di `std` mette in fila le sue letture e l'avvio dei processi, non il codice C che legge l'ambiente da sé, come le librerie di sistema della finestra grafica su un thread loro. `setenv` rifiuta un nome vuoto, o con `=` o un byte nullo, con cui nessun programma partirebbe più. La tabella è di un editor: `~` si espande con la `HOME` del processo, che `setenv` non tocca. `data-directory` cerca la directory dei dati accanto all'eseguibile, e un livello sopra, dove la trovano i binari dei test.

14.3.5 Uscire

`quit` offre di salvare, uno alla volta, i buffer per cui `has_unsaved_work` risponde sì: modificati, scrivibili e con un file. Poi `quit--confirm` esce, o chiede di scrivere `yes` se qualcosa è ancora da salvare. `save-some-buffers` fa le stesse domande e alla fine chiama AFTER. Ogni domanda è una keymap transitoria con quattro tasti, `y`, `n`, `!` e `q`, che chiamano `save-some-buffers--answer` con il resto della lista; prima di ogni domanda il buffer viene guardato di nuovo, perché una risposta esegue del Lisp che può averlo salvato o chiuso. Salva `write_buffer`, in `files.rs`. `close-buffer` chiede con la stessa funzione. Uscire ferma soltanto il ciclo del front-end: la sessione la scrive `main.rs`, dopo (Sezione 3.1.3).

14.3.6 Le domande

Una domanda non restituisce la risposta: la primitiva mette la domanda sullo schermo e torna, e la risposta arriva dopo, con un altro tasto, come chiamata della callback che le è stata data. `y-or-n` installa una keymap transitoria in cui rispondono solo `y` e `n` (`OnUnbound::Refuse`: un altro tasto non fa nulla e non dice nulla); ogni tasto chiama `ask--answer`, che toglie la keymap prima di chiamare la callback. `yes-or-no` apre il minibuffer e passa ciò che viene scritto a `ask--typed`, che chiede di nuovo se non è né `yes` né `no`; chiudere la domanda vale `no`. `ask_with_keys` è la stessa cosa con i tasti scelti da chi chiama: la usa `save-some-buffers`.

14.3.7 I buffer

`buffers.rs` crea, cambia, legge e chiude i buffer. `kill-buffer` è il comando, e `close-buffer` la stessa funzione sotto un altro nome: un buffer con lavoro da salvare chiede prima, con `yes-or-no`, e la risposta lo chiude con `kill-buffer-without-saving`. `buffer-put` e `buffer-get` appendono a un buffer un valore Lisp, nella tabella dei dati del buffer, sotto il nome `lisp:CHIAVE`. `major-mode`, `buffer-modified-p` e `buffer-file-name` leggono il nome del buffer allo stesso modo, quello corrente se manca.

14.3.8 Le finestre

`windows.rs` divide e chiude le finestre (`split-window-below`, `split-window-right`, `delete-window`, `delete-other-windows`, `display-buffer-at-bottom` per una striscia in fondo alla cornice), sposta il focus (`other-window`, `select-window`), scorre la vista (`scroll-up-command`, `scroll-down-command`, `recenter`) e apre e chiude le flottanti (`make-floating-window`, `close-floating-window`). Con una misura, una divisione fissa le righe o le colonne della finestra divisa (`FirstFixed`), e la nuova prende il resto. `close-floating-window` chiude sempre la flottante più recente, e senza flottanti non fa nulla. `Windows::set_focused` rifiuta una finestra che non c'è, che può essere quella da cui una flottante è stata aperta e che è stata chiusa nel frattempo; se non c'è più nemmeno quella che aveva il focus, il focus va alla prima finestra affiancata.

14.3.9 Gli argomenti

`args.rs` ha i lettori con cui le primitive leggono i loro argomenti: `text`, una stringa; `name`, un simbolo o una stringa, nil compreso, che diventa il nome `nil`, per un nome che non può mancare; `name_or_empty`, `name_or` e `optional_name`, con il valore che danno quando l'argomento manca, e per cui nil è un argomento che manca: è così che un modulo passa avanti un argomento facoltativo che non ha ricevuto. Le primitive delle liste di risultati leggono BUFFER allo stesso modo, con `named_buffer`; `movement_count` e `repeat_count`, i conteggi (Sezione 14.2); `width`, una larghezza o una colonna, che oltre `WIDTH_LIMIT`, un milione di caratteri, dà l'errore di Emacs `(args-out-of-range N)`, con `out_of_range`: una stringa così lunga si chiede al sistema, e un'allocazione che rifiuta chiude il processo invece di fallire. La usano `indent-line-to` e `kmacro-insert-counter`; `list`, gli elementi di una lista in tutte e due le forme in cui arriva, una catena di cons o la `Form` di una lista che una macro cita dai suoi argomenti; `quoted`, che mette un valore in una forma da valutare dopo, come le callback delle domande.

14.4 Comandi, ricerche e modi

Le ultime primitive stanno anch'esse in un file per concetto. Quasi tutte danno a Lisp un meccanismo che il manuale descrive nel suo capitolo, e la tabella vi rimanda; le sezioni che seguono dicono ciò che non è scritto altrove.

File	Contenuto
`mod.rs`	La macro `primitive!`, `Registry` e `install_primitives` (Sezione 14).
`commands.rs`	Il registro dei comandi, `call-interactively`, la catena dei prompt che raccoglie gli argomenti e `M-x` (Sezione 6).
`keymaps.rs`	`define-key`, `set-local-keymap`, le keymap transitorie, i tasti di ripetizione e `repeat` (Sezione 5.2).
`help.rs`	Ciò che si può chiedere all'editor di sé, per le pagine dell'aiuto (Sezione 14.4.2).
`minibuffer.rs`	Il prompt, le sue risposte, il completamento con Tab e la storia (Sezione 7).
`completion.rs`	Il completamento nel buffer: le fonti e chi le interroga (Sezione 14.4.1).
`fuzzy.rs`	`fuzzy-filter`, la corrispondenza approssimata (Sezione 14.4.1).
`isearch.rs`	La ricerca incrementale (Sezione 7.2).
`replace.rs`	La sostituzione, i suoi verbi e la vista predefinita (Sezione 9.4).
`occur.rs`	Le ricerche di occur e di grep, che producono una lista (Sezione 14.4.3).

File	Contenuto
`results.rs`	Le liste di posti: leggerle, percorrerle con `next-error`, attaccarne una da Lisp, mostrarle (Sezione 9.7).
`modes.rs`	`make-mode`, gli hook, `add-auto-mode`, i modi minori e le loro regole (Sezione 10, Sezione 10.2).
`grammar.rs`	Le regole e le regioni della grammatica (Sezione 10.3).
`syntax_table.rs`	La tabella sintattica e le domande allo scanner: `syntax-ppss`, `balance-point`, `bounds-of-enclosing-list` (Sezione 10.3).
`theme.rs`	Le facce e i loro stili (Sezione 10.5).
`overlays.rs`, `virtual_text.rs`	Overlay e testo virtuale (Sezione 8.5).
`evaluation.rs`, `diagnostics.rs`	Valutare un file o un testo, e riferire: il log, il backtrace, l'area dei messaggi (Sezione 14.4.4).
`loading.rs`	Caricare un modulo una volta, e ciò che si sa di quelli caricati e dei plugin (Sezione 15).

14.4.1 Il completamento

Una *fonte* di completamento è una funzione senza argomenti, chiamata con il suo buffer corrente e il punto dove l'utente l'ha lasciato. Risponde nil, «non è mio», o `(START END CANDIDATES)`: le posizioni del testo che si offre di sostituire e i candidati, stringhe o coppie `(VALUE . DESCRIPTION)`, la forma che `*completion-read-function*` mostra già.

`completion-at-point` chiede alle fonti del modo del buffer, poi a quelle di tutti i buffer (`add-completion-function`, `set-completion-functions`). La prima che risponde fissa il testo da completare, e quelle dopo che reclamano lo stesso testo aggiungono i loro candidati; una che ne reclama un altro si salta. Una risposta malformata costa quella fonte, e il log lo dice. I candidati si confrontano con ciò che è scritto attraverso `*completion-filter-function*`, o dall'inizio se è nil; due fonti che offrono lo stesso nome lo danno una volta, con la descrizione della prima. Un candidato solo si mette al posto del testo; più d'uno vanno al presentatore, `*completion-read-function*`, che chiama `completion-at-point-choose` con quello scelto, e senza un presentatore si mette quanto hanno in comune e se ne dice il numero.

Il presentatore può scegliere qualche tasto dopo, quando il comando che ha offerto è finito. Ciò che la scelta sostituisce sta quindi alla radice dell'ambiente, in `*completion-at-point-region*`, come `(START END LENGTH)`, con la lunghezza del buffer al momento dell'offerta: ciò che si scrive nel frattempo dentro la parola, e che restringe la striscia, sposta la fine della differenza. `*completion-at-point-start*` dice dove comincia il testo solo mentre il presentatore riceve i candidati, ed è come distingue un completamento in un buffer da uno in un prompt. `buffer-words` e `bounds-of-thing-at-point` sono i pezzi con cui sono fatte le fonti di `completion-at-point.lisp`, con una regola sola per che cosa sia un simbolo: lettere, cifre, `_`, `-` e `*`.

`fuzzy-filter` tiene i candidati in cui i caratteri del modello compaiono in ordine, anche separati, ignorando le maiuscole, e li ordina per punteggio. Ogni carattere vale uno, nove in più all'inizio di una parola (all'inizio, dopo `-`, `_`, `/`, `.`, uno spazio, `:` o `\`, o una maiuscola dopo una minuscola), otto in più accanto al precedente; il modello intero nel candidato vale quindici, venticinque all'inizio; la lunghezza toglie un quarto di sé. A pari punteggio resta l'ordine d'arrivo. Il modulo `completion.lisp` lo lega a `*completion-filter-function*`.

14.4.2 L'aiuto

`help.rs` dà le risposte di cui sono fatte le pagine dell'aiuto: `key-binding`, ciò che una sequenza esegue e la keymap che glielo dà (`transient`, `local`, `mode`, `global`), `keys-with-prefix`, `keymap-bindings`, `where-is`, `command-specs`, `help--macro-doc` e `read-key-sequence`, che arma la cattura (Sezio-

ne 5.7). Le pagine, `describe-key` e le altre, sono `help.lisp`. Ogni risposta passa dalla risoluzione dell'editor, attraverso la facciata, ed è ciò che premere i tasti farebbe. Il TARGET di un legame si dà come dati, non come la sintassi che la keymap conserva: una `Form` non è una lista per `car`, e il comando dentro non si raggiungerebbe.

14.4.3 Le ricerche nei buffer e nei file

`occur--scan` cerca nel buffer corrente, subito, sul thread dei comandi, con `Pattern::scan`, e paga un'unità di carburante per carattere. `grep--scan` cerca nei file sotto una directory su un thread dedicato (Sezione 11.10.2), che scrive nella lista man mano che trova. Tutte e due prendono il buffer della lista con `prepare_results_buffer` (Sezione 9.7), vi scrivono un'intestazione, il modello e dove si è cercato, gli attaccano un `Results` e lo rendono di sola lettura. Alla fine la lista è segnata come completa e riceve una riga di chiusura, `--- N matches ---`, che dice anche se la ricerca si è fermata prima.

La ricerca nei file non entra nelle directory di `PRUNE`, le stesse che salta `find-file-recursive` (`.git`, `target`, `node_modules` e simili), si ferma a 20 000 file e a 10 000 corrispondenze. Legge ogni file nella sua codifica, come `find-file` (Sezione 8.7.1), così una corrispondenza in un file Windows-1252 si trova e si apre dove è; salta senza dirlo un file che non si legge e uno binario, quello che `looks_binary` riconosce da un byte nullo tra i primi 8000, come fa Git: in Windows-1252 ogni byte si decodifica, e decodificare non dice più che cosa è testo. Un file aperto in un buffer con modifiche si cerca nel testo del buffer. La fermano `stop-worker` con il nome della lista e la chiusura della lista; ciò che è stato trovato resta. Occur e grep sono due produttori, ma le loro liste sono dello stesso modo, e si mostrano nella stessa striscia.

14.4.4 Valutare e riferire

`eval-file` e `eval-string-top-level` valutano alla radice dell'ambiente, come `init.lisp` (Sezione 3.1.1): ciò che il testo definisce resta, da qualunque funzione siano chiamate. `eval-string-top-level` risponde `(t VALUE)` o `(nil WHY)`, e toglie dalla pila delle chiamate le voci che un errore vi ha lasciato; è ciò che `M-:` usa, in `minibuffer.lisp`.

`log` aggiunge una riga al log senza mostrarla, `all-logs` lo restituisce, `backtrace` dà la pila delle chiamate dell'ultimo errore, senza la propria voce, e `set-echo-message` scrive nell'area dei messaggi; `message`, in `debug.lisp`, fa le due cose insieme. La pagina `diagnostics` del manuale dell'utente li descrive.

14.5 Le decisioni

1. **Un file per concetto.** Le primitive di modifica stanno in tredici file, e ognuno ha il nome di ciò che contiene: `kill-word` si trova in `killing.rs`, accanto a `yank` e a `cut_out`, e `indent-line-to` in `indentation.rs`, il nome della pagina che lo descrive.
2. **Le porte prendono il `Buffer` intero, e il loro risultato non si ignora.** Registrare una modifica richiede la storia che sta accanto al testo, e prenderli insieme rende impossibile tenere l'uno senza l'altra. Il `bool` è `#[must_use]`: un chiamante non può supporre in silenzio che la modifica sia avvenuta e dire all'utente che è andata.
3. **Un messaggio solo per il rifiuto.** «Buffer is read-only», lo stesso per ogni primitiva: il motivo è sempre quello, e il nome del comando non direbbe nulla che l'utente non sappia, perché ha premuto il tasto. Il silenzio è ciò da evitare: un tasto che non fa nulla e non dice nulla sembra un editor rotto, non un buffer protetto. Per questo il rifiuto si dice anche alla fine del comando, per quelli che non lo dicono da sé.
4. **Una modifica disattiva il mark, non lo cancella.** Ogni modifica passa dalle porte, quindi una modifica chiude la selezione, come in Emacs. La posizione resta per `exchange-point-and-mark`, e un mark attivo non ha mai avuto una modifica sotto: non va mai spostato.
5. **`changed` è una funzione sola.** Le tre porte passano dalla stessa funzione per versione, colorazione, punti di ripresa ed evento `buffer-changed`. Una modifica che incrementasse la versione senza invalidare la colorazione, o il contrario, si vedrebbe solo sullo schermo, minuti dopo, come un colore che non si assesta.
6. **`insert` unisce i suoi argomenti.** `(insert a b c)` è una modifica sola: un passo di undo, e un cambiamento solo per l'evidenziatore.

7. **L'incolla normalizza i fine riga, `insert` no.** Un terminale in modalità bracketed paste consegna i byte della selezione, e un a capo arriva spesso come CR, ciò che manda Invio; l'indice delle righe conta solo `\n`, e dieci righe incollate sarebbero una riga lunga con nove caratteri invisibili. La conversione sta alla porta da cui passa ogni incolla, anche uno yank dagli appunti di sistema; un CR scritto da Lisp di proposito resta un CR. La conversione è la stessa su ogni sistema: gli appunti di Windows danno CRLF, e l'indice delle righe conta solo `\n` anche lì.
8. **Una kill restituisce il testo, e l'anello lo riceve dopo.** L'anello è un compartimento che non si prende mai con il lock di un buffer: prenderlo nella closure aggiungerebbe all'ordine dei lock un arco che nient'altro rispetta. Per questo `cut_out` restituisce il testo e ogni kill lo consegna fuori.
9. **Una kill rifiutata non mette nulla nell'anello.** Un testo nell'anello che è ancora nel buffer verrebbe duplicato dallo yank seguente. Lo stesso per `yank`: un'incollatura rifiutata non registra una posizione, che il `yank-pop` seguente userebbe per cancellare altro testo.
10. **I nomi e la distinzione sono quelli di Emacs.** `kill-*` mette nell'anello ciò che toglie, `delete-*` lo scarta: una configurazione scritta per Emacs trova i comandi che si aspetta, e si comportano come lì.
11. **Una cancellazione toglie ciò che il movimento attraversa.** Una kill non calcola da sé dove arriva: chiede alla funzione del movimento con lo stesso nome. Due calcoli separati finirebbero per divergere, e la differenza si vedrebbe solo come «a volte si mangia il trattino basso».
12. **La colonna obiettivo sopravvive a una serie di movimenti verticali.** Riletta dal punto a ogni movimento, una riga corta in mezzo la ridurrebbe per sempre: si scende oltre una riga corta, si risale, e non si torna dove si era. Si rilegge quando il comando precedente non era un movimento verticale (`last-command`), e si chiede prima del lock del buffer.
13. **Il punto come numero: funzioni, non comandi, in caratteri.** Nessuno arriva utilmente a `point` con `M-x`, mentre ogni movimento descritto è un comando. Una posizione fuori dal buffer va al bordo più vicino invece di essere rifiutata: è quasi sempre una posizione calcolata prima di una modifica, e mettere il punto in un posto vero serve più di un errore.
14. **Le espressioni si lessicano, le parole no.** Da fuori `forward-word` e `forward-sexp` si somigliano; dentro, una parola si trova guardando i caratteri, un'espressione lessicando il buffer con la tabella del modo, perché una parentesi è un delimitatore solo fuori da stringhe e commenti.
15. **L'indentazione: il meccanismo in Rust, la politica in Lisp.** La colonna di una riga è una domanda sulla lingua, e un modo nuovo deve poterla rispondere senza ricompilare; leggere e riscrivere l'indentazione attraverso le porte è meccanismo, e sta qui.
16. **Una riga vuota è indentata fino in fondo, e il punto tiene il suo carattere.** Così riporta Emacs, e così `indent-line-to` non conserva gli spazi in coda come testo. Il punto dentro l'indentazione va alla fine di quella nuova; oltre, tiene il carattere su cui stava, spostato della differenza.
17. **I verbi sono primitive.** Sono ciò che un editor *è*, come `kill-line`: un utente che toglie un file `.lisp` dalla sua configurazione non deve trovare `M-u` senza effetto.
18. **Ogni verbo dice dove lascia il punto.** È metà di ciò che rende un tasto ripetibile: `M-u M-u M-u` sale di tre parole perché ognuno lascia il punto dopo la parola che ha cambiato. I verbi degli spazi lo lasciano dov'erano gli spazi, così si continua a scrivere da lì.
19. **Una sostituzione che non cambia nulla non è una modifica.** Quando il testo è già quello che diventerebbe, `replace_span` sposta solo il punto. Non è un'ottimizzazione: una modifica che non cambia caratteri segnerebbe comunque il buffer come modificato, aggiungerebbe un passo di undo e sveglierebbe la colorazione, e mettere in maiuscolo ciò che lo è già lascerebbe un file da salvare.
20. **Senza regione, un errore.** Un comando che non fa nulla in silenzio quando l'utente credeva di avere una selezione è peggio di uno che lo dice; i rettangoli dicono lo stesso con le loro parole.
21. **`yank-pop` è un passo di undo.** Toglie e rimette attraverso le porte, nello stesso comando; il secondo passo non si tenta se il primo è stato rifiutato, che lascerebbe il testo nuovo in un buffer da cui quello vecchio non è uscito.
22. **Le modifiche si calcolano prima e si applicano dall'ultima.** Lavorare in avanti rende sbagliato ogni offset dopo la prima modifica, e lo sbaglio sembra plausibile: la prima riga viene giusta, la terza ha

l'apertura in mezzo al testo. Ricalcolare a ogni passo funzionerebbe, con un lock e una lettura per riga. Il punto si rimette dallo stesso elenco.

23. **L'apertura va alla colonna della riga meno indentata.** Così un blocco commentato tiene la sua forma: aperture indentate con il codice fanno di un `if` annidato una scala di `//`. Le righe vuote dentro una regione restano vuote, perché un'apertura lascerebbe spazi in coda, e non contano per decidere se la regione è già commentata; una riga vuota da sola si commenta, perché è ciò che è stato chiesto.
24. **Una regione che finisce all'inizio di una riga non la prende.** Selezionare tre righe trascinando lungo il margine sinistro finisce sulla colonna 0 della quarta, che nessuno ha scelto. Vale per i commenti e per `Tab` su una regione.
25. **La grafia dei commenti la dice il modo.** Un commento è la sola sintassi che cambia con ogni lingua, e l'editor ne conosce già la grafia: niente qui decide come è fatto un commento, e un modo che non ha detto nulla lo sente dire, invece di ricevere un `#` o un `//` a caso.
26. **Il punto all'angolo in alto a sinistra.** È dove comincia il blocco appena toccato, da dove parte la prossima cosa da fargli, ed è la stessa risposta per tutti i comandi che cambiano il buffer. `copy-rectangle-as-kill` non lo sposta: il mark sopravvive a una copia, e il punto all'angolo andrebbe sopra il mark, lasciando una selezione senza larghezza; `M-w` fa lo stesso.
27. **Una riga corta si allunga per inserire, non per svuotare.** Una colonna che non esiste ancora è ciò che `open-rectangle`, `yank-rectangle` e `string-rectangle` devono riempire; `clear-rectangle` svuota ciò che c'è, e una riga corta non ha nulla da svuotare. Un blocco incollato arriva intero, con le righe che mancano in fondo al buffer, e senza spazi su una riga dove non porta nulla.
28. **`rectangle-mark-mode` spento torna una regione normale.** Premuto due volte, rende la selezione che c'era, non nessuna selezione; il flag sta sul mark, e non c'è una seconda cosa da ricordarsi di spegnere.
29. **I dati della corrispondenza hanno la forma di Emacs, in caratteri.** È la forma che ogni pezzo di Emacs Lisp si aspetta, e il codice portato funziona senza essere ristrutturato; il prezzo è `save-match-data`. Le posizioni sono caratteri, così una corrispondenza in una stringa con una `è` dà gli stessi numeri che in Emacs.
30. **I dati della corrispondenza sono per thread.** Un lavoro Lisp sul thread dell'esecutore e un comando sul thread principale non possono coordinarsi, e il primo non deve cambiare ciò che il secondo sta per leggere.
31. **Un'espressione sbagliata è un errore, non un nil.** Un'espressione che non compila è un difetto dell'espressione, e un nil la farebbe sembrare una stringa che non corrisponde. Una corrispondenza fallita invece lascia i dati com'erano.
32. **La forma di una voce è scritta una volta.** `scan-buffer` e le liste di risultati costruiscono le voci con le stesse funzioni, così una vista si scrive una volta e mostra risultati da ovunque. La lista è posizionale, come ogni risposta strutturata dell'editor; `KIND` distingue un buffer da un file, e `MATCH-START` e `MATCH-END` sono dentro `TEXT`, così una vista evidenzia senza cercare di nuovo.
33. **`scan-buffer` paga carburante per carattere percorso.** Un megabyte per un'unità permetterebbe a un ciclo un lavoro senza limite dentro un budget fatto per limitarlo; è la regola di `directory-files-recursive`, che paga un'unità per ogni percorso che restituisce.
34. **Un percorso è assoluto appena entra.** Un percorso relativo ha meno componenti di quanti sembri: l'elenco di `.` avrebbe l'intestazione `./`, e chiedere la directory sopra porterebbe alla radice, a un tasto da dove si era. Il rimedio non è insegnare a ogni chiamante a contare meglio: un percorso è ambiguo finché non è assoluto, e lo è solo nel momento in cui l'utente lo scrive.
35. **`..` si toglie per nome, senza chiedere al disco.** Attraverso un link simbolico le due risposte differiscono: per nome `/a/link/..` è `/a`, sul disco è il genitore della destinazione. Quella per nome è quella che vuole un elenco di file, dove `^` in `/a/link/` mostra `/a/`, da cui si veniva; ed è la sola possibile per un percorso che non esiste ancora, come la destinazione di una rinomina.
36. **I nomi dei file si compongono in Rust.** Che cosa sia un separatore e che cosa una radice lo dice la piattaforma: `/a/b` e `C:\a\b` hanno la stessa forma e nessun carattere in comune. Un modulo Lisp che spezza su `/` scrive la risposta di una piattaforma e la chiama regola.

37. **Un errore sul disco si riferisce, non si segnala.** Cancellare sei file non deve fermarsi al secondo perché uno è stato negato, lasciandone quattro e un elenco non aggiornato: chi chiama vuole sapere quali sono falliti, e continuare. Ogni primitiva risponde nil e scrive il motivo nel log, come `find-file` per un file che non legge.
38. **Le primitive del disco non chiedono, e non sono comandi.** Una conferma dipende dallo stato dell'editor, da una regione attiva, da un file aperto, dall'ultima risposta, e sta in Lisp con il resto dell'interfaccia; una primitiva che chiedesse non si potrebbe chiamare da uno script. `M-x delete-file` sarebbe un comando in cui solo il prompt separa un errore di battitura da un file cancellato: chi cancella usa il gestore di file (`dired.lisp`), che mostra che cosa c'è e chiede di conseguenza.
39. **La potatura sta in Rust, l'elenco arriva da Lisp.** Quali file interessino è politica, e sta in Lisp; non entrare in una directory è l'eccezione, perché non si può fare dopo: `.git` contiene migliaia di oggetti, e una visita che li raccogliesse spenderebbe il limite prima di arrivare a un file sorgente. Così anche i suffissi da scartare: filtrarli in Lisp costerebbe una valutazione per file e per suffisso.
40. **Una visita paga un'unità di carburante per percorso.** L'interprete conta un'unità per chiamata, giusto per un passo e sbagliato per una primitiva che restituisce una lista lunga quanto vuole: ventimila percorsi per tre unità. Il carburante deve limitare il tempo, e una visita che il contatore non vede fa un ciclo di visite che la guardia non ferma.
41. **Si chiede solo di un buffer modificato, scrivibile e con un file.** Un buffer senza un posto dove salvarlo non si salva rispondendo `no`: `*scratch*` e i buffer dell'aiuto e degli elenchi sono modificati di continuo, e chiedere di loro insegna a rispondere senza leggere. Uscire e chiudere un buffer usano la stessa funzione, così sono d'accordo su quali buffer contano.
42. **Due modi di chiedere.** Una risposta sbagliata non costa uguale ovunque: una domanda che sbagliando perde lavoro deve essere difficile da sbagliare (`yes-or-no`), e una fatta otto volte di fila non deve chiedere otto parole (`y-or-n`). Le domande di `save-some-buffers` sono tasti singoli; quella che perde il lavoro, alla fine di `quit`, si scrive per intero.
43. **Una domanda prende una callback.** L'editor non si blocca: un comando finisce e torna, e la risposta arriva dopo, in un altro comando, quando non c'è più nessuno a cui restituirla. In Lisp si legge come un `cond` diviso tra due lambda: è il prezzo di un editor che risponde mentre una domanda è sullo schermo.
44. **La callback sta nella forma, non in una variabile.** Ogni domanda costruisce la chiamata che le risponderà, `(ask--answer (quote CALLBACK))`, e la dà a una keymap o al minibuffer. Niente resta in un posto condiviso, e una domanda fatta dalla risposta a un'altra sono due forme, non due scritture nello stesso posto. Per lo stesso motivo `close-buffer` passa una lambda con dentro il nome del buffer.
45. **I dati appesi a un buffer stanno nel buffer.** Un modulo che deve ricordare qualcosa di un buffer avrebbe due alternative peggiori: una variabile globale con il nome del buffer per chiave, che sbaglia quando il buffer cambia nome e resta dopo che è chiuso, o il testo del buffer, che va bene solo per ciò che si mostrerebbe comunque. La chiave ha il prefisso `lisp:`, perché un valore di Lisp e uno di Rust con lo stesso nome non si scontrino.
46. **Una divisione con una misura fissa la finestra divisa.** Un elenco di file che vuole restare largo 40 colonne deve tenerle quando la cornice cambia misura, ed è il file accanto che deve crescere; ed è ciò che vuol dire `split-window-right SIZE` in Emacs.
47. **Un argomento che manca e uno sbagliato sono lo stesso errore.** Un lettore riceve un `Option`, ciò che dà `args.get(n)`, e riferisce un argomento assente come un nil del tipo sbagliato: chiamare senza niente dove va una stringa è sbagliato come chiamare con un numero, e «expected String, got nil» lo dice in una frase. Una primitiva per cui conta l'arità lo dice da sé, con `WrongNumberOfArguments`, perché è un'altra lamentela.
48. **I lettori leggono, non interpretano.** Nessun lettore sa che cosa significhi un argomento: non c'è un `pattern_arg` né un `prompt_arg`, perché un modello e un prompt sono tutti e due il primo argomento come stringa. Ciò che cambia tra i chiamanti è il messaggio d'errore, che è un parametro.
49. **Ogni modulo registra le sue primitive.** Un `Registry` è un valore, e si passa a ogni modulo: le registrazioni stanno accanto al codice che nominano, e `install_primitives` è l'elenco dei moduli e

niente altro. Due macro scritte dentro `install_primitives` si potrebbero chiamare solo da lì, e ogni registrazione starebbe a due file dalla sua primitiva.

50. **Un comando in Rust si registra con la sua specifica.** `into.command` lega il nome e registra il comando in una chiamata sola, così la specifica degli argomenti sta accanto all'implementazione e alla docstring, e non in un file `.lisp` che potrebbe divergere o non caricarsi. Un comando scritto in Lisp passa da `defcommand`, che diventa un `defun` e un `register-command`; tutti e due finiscono nello stesso registro.
51. **La raccolta degli argomenti è in Rust.** La catena dei prompt, `advance_pending` con le tre callback di `commands.rs`, è il modo in cui un comando riceve i suoi argomenti, e deve esserci anche se nessun file `.lisp` si è caricato. Le callback leggono il tipo dell'argomento dal comando in attesa, così una primitiva sola serve ogni argomento.
52. **La fonte dice anche i limiti del testo.** Solo la fonte sa che cosa sta completando: in `foo-ba|` un simbolo, in `/usr/lo|` l'ultimo componente del percorso e non la barra prima, dentro una stringa forse la stringa intera. Una fonte che desse solo i candidati lascerebbe a `completion-at-point` di indovinare quali caratteri sostituire: un'ipotesi sola per tutte le fonti, sbagliata per tutte meno una.
53. **Il confronto con ciò che è scritto sta in un posto solo.** Una fonte risponde «che cosa può stare qui», non «che cosa corrisponde a ciò che si è scritto». Un posto solo dà una risposta sola a che cosa voglia dire corrispondere, e un posto solo dove cambiarla: legare `*completion-filter-function*` rende approssimate tutte le fonti insieme, senza toccarne nessuna.
54. **Le fonti si uniscono, non si sfidano.** Emacs prende la prima che risponde, e lì basta, perché le sue fonti sono di un modo. Qui la risposta utile a `comp` in un buffer Lisp sono sia le funzioni `comp…` sia le parole già nel buffer, e una gara ne nasconderebbe una secondo l'ordine di una lista che l'utente non vede. Due fonti che completano testi diversi non possono avere ragione tutte e due, e la seconda si salta: per questo le fonti di nomi hanno una regola sola per che cosa sia un simbolo, e una che leggesse altri caratteri sparirebbe da ogni lista.
55. **Il completamento: il meccanismo in Rust, le fonti in Lisp.** Il registro delle fonti e l'unione sono il meccanismo, da cui passano ogni fonte e ogni presentatore. Ciò che una fonte sa, i nomi dei file, le parole del buffer, i nomi dell'interprete, è Lisp, in `completion-at-point.lisp`, e si sostituisce una fonte alla volta.
56. **Ciò che una scelta sostituisce sta in una variabile alla radice.** Un presentatore può essere un modulo Lisp che decide dopo qualche tasto, quando il comando che ha offerto e chi l'ha chiamato sono finiti: `indent-for-tab-command` completa dal suo corpo, e una variabile legata lì non ci sarebbe più. In una variabile lo stato si vede anche da chi cerca un errore. La fine segue ciò che si scrive nel frattempo, spostata della differenza di lunghezza del buffer: senza, scegliere `abce` dopo aver scritto `e` dietro `ab` lascerebbe `abcee`.
57. **Il punteggio premia gli inizi di parola e i caratteri vicini.** Sono ciò che fa vincere un'abbreviazione delle parti su una dispersione casuale delle stesse lettere: `cap` è fatto degli inizi di `completion-at-point`. Il modello intero vale di più, perché tre lettere ognuna all'inizio di una parola valgono quanto tre di fila, e `abc` metterebbe `a-b-c-x` alla pari con `abc`. La lunghezza conta poco, divisa per quattro: separa candidati molto diversi senza scavalcare una corrispondenza molto migliore.
58. **Ogni carattere si prende dove viene per primo.** Il miglior allineamento del modello in un candidato è un programma dinamico per candidato a ogni tasto, e `find-file-recursive` ne passa cinquemila; il cammino da sinistra è lineare, e dove dà un punteggio diverso le due corrispondenze ci sono comunque.
59. **`fuzzy-filter` paga un'unità di carburante per candidato.** È la regola di `directory-files-recursive` e di `scan-buffer`: il lavoro cresce con la lista, e un'unità per la chiamata lascerebbe un ciclo su liste lunghe oltre il budget fatto per fermarlo. Ventimila candidati sono un cinquecentesimo del budget.
60. **Con un presentatore, Tab chiede i candidati una volta per testo.** Un secondo Tab sullo stesso testo non può cambiare la risposta, e la funzione che la dà può costare: quella del manuale elenca ogni pagina

installata, sul thread che disegna. Il testo per cui sono stati chiesti si ricorda, e `completion-invalidate` lo dimentica per una lista che si sta ancora costruendo.

61. **Un prompt si apre sempre.** È come funziona `M-x`, e come l'utente eseguirebbe il comando che corregge un'impostazione sbagliata: una misura più grande del terminale diventa quella del terminale, con una colonna di margine per lato, e un valore senza senso in `minibuffer-width` o `minibuffer-height` vale quello predefinito invece di un errore.
62. **Un legame ha una forma sola.** Un simbolo che nomina una funzione diventa la sua chiamata, `(next-line)`, e il resto si esegue com'è scritto. La risoluzione esegue allo stesso modo un simbolo e una forma di un elemento, e per il tasto premuto non cambia nulla; cambia che una keymap tiene una forma sola, e chi la legge senza eseguirla, per descrivere un legame o trovare dove un comando è legato, non ne deve conoscere due. `define-key` e `set-transient-keymap` passano dalla stessa funzione: un legame scritto in un modo vuol dire ciò che vuol dire scritto nell'altro.
63. **Le risposte dell'aiuto sono in Rust, le pagine in Lisp.** Una pagina di testo in un buffer si può sostituire con una finestra con dei pulsanti senza che le risposte cambino. Le risposte no: un comando d'aiuto è creduto, e uno che un modulo mancante potesse sbagliare in silenzio sarebbe peggio di nessuno. Quale keymap vince è una regola scritta una volta in `Modes` ed eseguita a ogni tasto, e l'aiuto chiede quella: un legame riferito e uno premuto non possono essere diversi.
64. **Si cerca nel buffer, non nel file, quando il buffer ha modifiche.** Il buffer è ciò che è vero: un file aperto con modifiche non salvate non è più la sua copia su disco, e una ricerca che leggesse il disco riferirebbe testo che non c'è più, con posizioni che portano altrove. Un buffer salvato si lascia alla lettura del file, che dà lo stesso testo senza prendere il lock del buffer per ogni file.
65. **Una ricerca nei file ha un thread suo, una nel buffer no.** Una ricerca in un albero dura quanto dura: qualche migliaio di file è un attimo, una home no, e non si sa prima di cominciare. Sul suo thread l'editor continua, e i risultati compaiono man mano, che è anche come si distingue una ricerca lenta da una bloccata. Il testo di un buffer è già in memoria, e la scansione è una passata: un altro thread costerebbe più di quanto risparmia, e farebbe lampeggiare `occur`.
66. **I comandi del percorso lavorano sulla lista corrente.** `M-g n` si preme nel file che si sta correggendo, non nella lista, che quindi non si legge dallo schermo: i comandi lavorano sulla lista prodotta per ultima, finché il suo buffer vive. Sono primitive, e non funzioni di un modulo, così ogni produttore di una lista li ha senza dipendere dal modulo scritto per primo: una compilazione, una ricerca e un linter fanno una lista e la percorrono con gli stessi tre tasti.
67. **Una lista che cresce si aggiorna prima di ogni passo.** Una ricerca sa tutto ciò che ha trovato quando finisce; una compilazione no, la sua uscita arriva riga per riga. Ricostruire la lista a ogni riga sarebbe un'analisi per riga; non ricostruirla farebbe percorrere a `M-g n` ciò che il compilatore diceva trenta secondi prima. Un produttore la cui lista cresce attacca con `buffer-put` una funzione sotto `results-refresh`, chiamata con il nome della lista prima di ogni passo; uno la cui lista è completa non attacca nulla e non paga nulla.
68. **Il percorso sopravvive a una lista sostituita.** Una lista che cresce si attacca di nuovo a ogni passo, e un percorso che ricominciasse ogni volta offrirebbe per sempre il primo errore, sembrando funzionare. Si tiene l'indice e non la voce, perché una lista che cresce cresce in fondo: la terza voce è ancora la terza. Un produttore che riordina ritrova il percorso più o meno dov'era, che è meglio che in cima.
69. **Grammatica e tabella dicono cose che si sovrappongono, separatamente.** Una grammatica dice che aspetto ha il testo, una tabella che cosa è: un modo può colorare un commento di documentazione diversamente da uno normale, e sono tutti e due commenti. E lo scanner deve lavorare su un buffer la cui grammatica non è ancora stata applicata: la colorazione si calcola su un altro thread, e `C-M-f` non la può aspettare.
70. **Un modello sbagliato costa solo sé stesso.** Una grammatica è configurazione: un modello rotto in un modulo di lingua costa quel modello, che il log riferisce, e non tutto il file in cui è scritto. Così un'espressione di `add-auto-mode`, e una risposta malformata di una fonte di completamento, che costa quella fonte e non il completamento.

71. `regexp-opt` **di nessuna parola non corrisponde a niente.** Un'alternativa vuota corrisponde ovunque, e una regola fatta così colorerebbe ogni posizione. Il modello onesto non corrisponde mai: una classe di nessun carattere, `[^\s\S]`, perché il motore non ha il lookahead con cui scrivere `(?!)`.
72. **La vista predefinita della sostituzione è la più semplice che funzioni.** Una keymap transitoria di cinque tasti sul buffer, perché la sostituzione sia completa senza configurazione, come il minibuffer ha un prompt predefinito. Una vista con un'anteprima e due pulsanti lega `*replace-read-function*` e chiama gli stessi verbi; niente in `replace.rs` deve sapere che c'è.

15 I moduli Lisp

Quasi tutto il comportamento visibile è in `core/lisp/`: le associazioni dei tasti, i comandi di alto livello, i modi dei linguaggi, l'interfaccia dei completamenti. I moduli stanno in tre cartelle, per genere: `base/`, ciò con cui gli altri sono scritti; `features/`, una funzionalità ciascuno; `languages/`, un linguaggio ciascuno. I temi sono dati e non codice, e stanno fuori, in `themes/`. `build.rs` copia ogni cartella di `core/lisp` in `target/<profilo>/data/lisp/<genere>` a ogni compilazione, e un modulo nuovo arriva accanto all'eseguibile da solo. `MODULE_KINDS`, in `editor/loader.rs`, è l'elenco dei generi che `lisp-path` nomina; `loading_tests` controlla che coincida con le cartelle e che ogni modulo distribuito sia caricato dall'`init.lisp` predefinito, attraverso `default-configuration`. `tests::shipped::default_modules` legge quell'elenco dove sta: le righe `require` dell'init e, al posto di `default-configuration`, la lista `default-configuration-modules` del suo sorgente, seguita dal modulo stesso.

Genere	Modulo	Che cosa aggiunge
`base`	`commands`	`defcommand`, `save-match-data`, `with-eval-after-load` e altre macro di base.
`base`	`debug`	`message`, `report-error`, il buffer `*Backtrace*`, `debug-on-error`.
`base`	`common-keymaps`	Le associazioni di Emacs: movimento, kill, file, buffer, finestre, `C-g`.
`base`	`indent`	Tab, `indent-region`, `newline-and-indent`, `electric-indent-mode`.
`base`	`minibuffer`	`M-:`.
`languages`	`rust-mode`, `risp-mode`, `cc-mode`, `python-mode`, `toml-mode`, `bash-mode`, `cmake-mode`	I modi dei linguaggi.
`features`	`dired`	Una directory in un buffer (`C-x d`).
`features`	`completion`	La striscia dei completamenti (`*completion-read-function*`).
`features`	`preview`	Anteprima di `string-rectangle`.
`features`	`clipboard`	Copia le uccisioni negli appunti di sistema.
`features`	`electric-pair`	Chiusura automatica di parentesi e virgolette.
`features`	`buffer-list`	`C-x b` e `C-x C-b`.
`features`	`shell`	`M-!`.
`features`	`manpage`	Pagine di manuale.
`features`	`help`	`C-h f`, `C-h k`, `C-h b`, `describe-prefix-keys`.
`features`	`risp-repl`	`M-x risp-repl`: `C-j` valuta il blocco prima del punto.
`features`	`occur`	`M-s o`, `M-s g`.
`features`	`compile`	`C-c c` e `M-g n`.
`features`	`theme`	`C-c t`: scelta del tema.
`features`	`vim`	L'editing modale.
`features`	`completion-at-point`	Le cinque fonti di `C-M-i`.

15.1 Caricare una volta: `require` e `provide`

Ogni modulo comincia con i `require` dei moduli che usa e finisce con il suo `provide`. `require` carica il file solo se la feature non è già caricata; `provide` la registra in `Loader`, il compartimento di ciò che è stato caricato, con il file da cui viene: l'ultimo della pila di file che `eval_source` tiene mentre valuta. Il modello è quello di Emacs: `features`, `featurep`, `require` e `provide` hanno lì lo stesso nome e lo stesso senso, `feature-file` è ciò che Emacs ricava da `load-history`.

`EditorState::require(nome, file, env)`, in `editor/loader.rs`:

1. Se `nome` è caricato, risponde `nome` e non fa nulla.
2. Cerca il file: `file`, se dato, come percorso o come nome nudo, o `nome`; un nome nudo è `nome.lisp` nella prima directory di `lisp-path` che lo ha. Se non c'è, una riga di log e `nil`.
3. `Loader::begin_require` lo segna come in corso; se lo era già, più fuori, è un ciclo, e l'errore nomina la catena (`ping -> pong -> ping`).
4. Legge il file, che deve essere Lisp valido, e lo valuta con `eval_source`. Un errore del lettore, un errore di valutazione o un file che non ha fornito `nome` sono un errore, che finisce nel log e risale.
5. `Loader::end_require` chiude il `require`: se è fallito, `nome` diventa `failed` e le feature *fornite per la prima volta* dal suo inizio tornano a non esserlo, come in Emacs; i fallimenti registrati nel frattempo restano, per leggere il log.

`eval-after-load` consegna a `Loader` una funzione per una feature non ancora fornita, o la chiama subito se lo è; `provide` le riprende e le chiama in ordine, ognuna con il suo errore nel log, come un hook. `with-eval-after-load` è una macro di `commands.lisp` sopra la primitiva.

da `managers/loader.rs` — estratto semplificato

```
let mut index = 0;
self.known.retain(|feature| {        // dopo un require fallito
    index += 1;
    index <= mark.known || feature.state != FeatureState::Loaded
});
```

Le dipendenze di un modulo sono ciò che usa, al caricamento o dopo. Tre sono deboli e non si chiedono: `help` usa `manpage` se c'è, `manpage` usa `help-key` se c'è, `vim` usa `manpage` per `:help` e `K`. Ognuno lo controlla con `functionp` o `boundp`, e i due moduli si caricano in qualunque ordine.

15.2 Dove si cerca

`set_search_paths`, in `editor/boot.rs`, dà il primo valore alle due liste, ognuna nell'ordine in cui si cerca: prima la persona, poi i plugin, poi l'editor.

Variabile	Directory, in ordine
`lisp-path`	`<configurazione>/lisp`; la cartella di ogni plugin; `data/lisp/base`, `data/lisp/features`, `data/lisp/languages` accanto all'eseguibile.
`theme-path`	`<configurazione>/themes`; la cartella `themes` di ogni plugin; `themes` dei dati installati (`installed_data_dir`).

Un file in una directory ne nasconde uno con lo stesso nome in una successiva, e `find_module` lo scrive nel log. I moduli dell'editor si cercano solo accanto all'eseguibile, non un livello sopra come i temi e le pagine: un binario di test non ne ha accanto, e così ogni test comincia senza moduli caricati, qualunque cosa chieda il suo `init.lisp`. `theme.lisp` legge `theme-path` per `available-themes`, che elenca ogni nome una volta, in ordine, e per `select-theme`, che prende il primo file trovato.

15.3 I plugin

Un plugin è una cartella di `plugins`, nella directory dei dati, con un `plugin.lisp` che dice che cos'è:

```
(define-plugin hello :version "0.1" :requires (dired)
  :description "Says hello from dired.")
```

All'avvio `find_plugins` legge il descrittore di ogni cartella con `Parser`, come dati e senza valutarlo: il nome deve essere quello della cartella, e una parola chiave sconosciuta si salta. Una cartella senza descrittore valido si salta, con il perché nel log. I plugin trovati vanno in `Loader`, che `plugins` e `plugin-info` leggono, e le loro cartelle in `lisp-path` e `theme-path`; del loro codice non si carica nulla finché `init.lisp` non fa `(require 'hello)`. `:requires` è un dato per chi installa: i moduli del plugin chiedono da sé ciò che usano.

È la base di un gestore di pacchetti, che manca: nessun comando installa, aggiorna o toglie un plugin, e versione e dipendenze non si controllano. Un gestore aggiungerebbe una cartella e il suo descrittore; il caricamento c'è già.

15.4 Come è scritto un modulo

Un modulo è un file di `core/lisp/<genere>/`, con il nome della feature che fornisce. Comincia con un'intestazione `;;;` di poche righe — il nome e una frase, che cosa aggiunge, la sua pagina del manuale utente (`User manual:`) e la sua sezione di questo capitolo (`Design:`) —, poi i `require` di ciò che usa; finisce con una riga di log e il suo `provide`. `doc_pointer_tests` controlla che ogni modulo distribuito abbia l'intestazione, e che il suo rimando porti a un'etichetta che il manuale ha.

Dentro, le parti sono separate da intestazioni `;; ----` (lo stato, il disegno, i comandi, i tasti e le facce), nell'ordine in cui chi legge le incontra; i tasti e le facce stanno in fondo, accanto a ciò che legano. I nomi privati hanno il prefisso del modulo e un doppio trattino, `dired--parent`; quelli pubblici solo il prefisso. Un commento dice in una o due righe un perché che il codice non dice; i perché più lunghi sono le decisioni qui sotto.

Quattro moduli — `debug`, `minibuffer` e `common-keymaps` di `base/`, `clipboard` di `features/` — definiscono i loro comandi con `defun` e `register-command` invece che con `defcommand`: si caricano anche senza `commands.lisp`, come quando un test valuta il file da solo, e lì un `defcommand` sarebbe la chiamata a una funzione che non c'è.

Molti moduli di `features/` hanno la stessa forma: un modo maggiore per un buffer di servizio (`*dired*`, `*Buffer List*`, `*Themes*`), in sola lettura tranne mentre lo si riscrive, i cui comandi rileggono dallo schermo ciò su cui agiscono. Altri mostrano il loro buffer in una striscia in fondo al frame (`display-buffer-at-bottom`, o `results-display` per le liste di posti), che non prende il focus.

Forma	Moduli
buffer come stato	`dired`, `buffer-list`, `theme`, `compile` (la prima riga è il comando)
striscia in fondo	`shell`, `compile`, `occur`, `completion`
solo un hook	`preview`, `electric-pair`, `completion` (il restringimento)
Rust risponde, il modulo mostra	`help`, `occur`, `completion-at-point`, `minibuffer`, `shell`

15.5 I moduli distribuiti

Una sottosezione per modulo: che cosa fa, come, e le sue decisioni. Dove un altro capitolo descrive già il meccanismo, la sottosezione vi rimanda.

15.5.1 `commands`

Il registro dei comandi, la raccolta degli argomenti e `M-x` sono in Rust (Sezione 6); il modulo aggiunge `defcommand`, `define-minor-mode` con `minor-mode-wanted-p` (Sezione 10.2), `string-prefix-p`, `string-suffix-p`, `save-match-data` e `with-eval-after-load`.

1. **Il necessario in Rust, lo zucchero qui.** Un comando che non sa raccogliere i suoi argomenti non è un comando, e un file di configurazione che non si carica non deve poterlo togliere. Senza questo modulo un comando si definisce lo stesso, con `defun` e `register-command`.
2. **`defcommand` passa il nome come stringa e le specifiche con `list`.** Il lettore trasforma una forma citata in dati mentre legge, e un unquote dentro una citazione (`',name`) è già un dato quando il backquote si espande.
3. **`save-match-data` senza gensym.** Il Lisp non ne ha: i dati salvati stanno sotto un nome che un corpo vedrebbe solo nominandolo.

15.5.2 `debug`

`message` scrive nell'area dei messaggi e nel log; `switch-to-messages` mostra il log in `*Messages*`, come istantanea; `report-error` è ciò che l'editor chiama quando un comando fallisce, e con `debug-on-error` apre la pila delle chiamate in una flottante `*Backtrace*`.

1. **L'errore di un comando passa da una funzione Lisp.** `report-error` si può ridefinire, e il modo di riferire un fallimento cambia senza toccare il Rust.
2. **Il buffer da cui si torna è ricordato.** `close-floating-window` rende il focus alla finestra, non il buffer corrente: il modulo ricorda il buffer della prima apertura, e una seconda flottante non lo sovrascrive.

15.5.3 `common-keymaps`

I legami della keymap globale, sui tasti di Emacs, a gruppi: ogni gruppo comincia con un commento che nomina la pagina che lo descrive. I tasti di un modo stanno nel suo modulo; Return sta in `indent`, accanto al comando che esegue.

1. **Un gruppo per pagina.** Il file fa da indice dei tasti: chi cerca dove un tasto è spiegato lo trova qui, e `guide_tests` controlla le sezioni KEYS delle pagine contro questi legami.
2. **Due tasti dove i terminali ne mandano uno.** `C-/` e `C-_`, `C-M-i` e `M-<tab>` arrivano come lo stesso tasto su molti terminali: si legano entrambi.
3. **I numeri di riga senza tasto.** Costano colonne; `M-x display-line-numbers-mode` li accende, in un buffer (Sezione 15.5.13).

15.5.4 `indent`

Il modulo lascia a Rust `indent-width`, `indent-line`, `indent-for-tab-command` e `newline-and-indent`; conserva le operazioni sulle regioni, la configurazione e le regole dei linguaggi. Tab consulta `indent-function` e l'eventuale `indent-levels-function`; Return indenta la riga che apre (`electric-indent-mode`) e, da una riga vuota, conserva il livello scelto dall'utente. Il modulo ha la regola dei Lisp (`lisp-indent-line`) e i lettori comuni delle regole dei linguaggi (`prog-indent--code`, che maschera stringhe e commenti mantenendo le colonne).

1. **Una regola restituisce una colonna, non fa il lavoro.** Un modo può sbagliare solo un numero; `indent-region` può chiamarla cento volte senza cento occasioni di rovinare una riga; dove va il punto dopo è scritto una volta, in `indent-line-to`, non una per linguaggio.
2. **Il punto si rimette prima di toccare la riga.** Una regola legge il buffer muovendo il punto, e lo lascia dove ha finito. Rust lo ripristina anche quando la regola segnala un errore.
3. **Un rientro corretto è già una risposta.** Tab nel margine con alternative non inserisce spazi per il solo fatto che la riga non è cambiata. Senza alternative, `tab-always-indent` nil inserisce dopo il testo o ripetendo Tab; t indenta soltanto, complete prova anche il completamento. Return e le regioni non richiedono mai le alternative.
4. **Le regioni per numero di riga, e riselezionate.** Indentare una riga sposta gli offset di tutte quelle sotto; ogni modifica disattiva il mark, e la regione si riseleziona alla fine perché un secondo Tab agisca sullo stesso blocco. Una regione che finisce all'inizio di una riga non la copre.
5. **Uno stile sconosciuto vale `auto`.** Un refuso in `indent-region-style` dà il comportamento sensato, non un errore a ogni Tab.

6. **La regola dei Lisp legge la struttura.** `syntax-ppss` dice la profondità e dove si è aperta la lista: niente espressioni regolari né elenchi di parole, e vale per ogni modo che accoppia le parentesi. In una stringa la riga resta com'è; si allinea al delimitatore (elemento 4), non all'inizio dell'espressione, perché `'(a b)` si allinea alla parentesi.
7. **Le forme a corpo fisso sono proprietà dei simboli.** Come `lisp-indent-function` di Emacs: la forma di un linguaggio è dei suoi nomi, non di un modo che li legge.
8. **Return si lega qui.** Senza il modulo resta `insert-newline`, invece di un legame a un comando che non c'è.

15.5.5 `minibuffer`

Il prompt è in Rust (Sezione 7): `minibuffer-read`, i suoi tasti e la sua storia, perché `M-x` funzioni qualunque modulo non si carichi. Il modulo aggiunge `M-:`, che valuta l'espressione al livello più alto, perché ciò che definisce sopravviva al comando, e con l'errore catturato, perché un'espressione sbagliata si riferisca invece di chiudere il prompt.

15.5.6 `buffer-list`

`C-x b` sceglie un buffer con il completamento (`fuzzy-filter`), `C-x C-b` li elenca in `*Buffer List*`, uno per riga, con i tasti per passarvi e ucciderli.

1. **Il buffer è lo stato, come in dired.** Ogni comando rilegge il nome dalla riga, per colonna e non dividendo sugli spazi, che un nome può contenere. Un nome che non è più vivo dice che l'elenco è vecchio: lo si dice, non si agisce.
2. **L'elenco non si uccide dall'interno.** Lascerebbe il cursore in un buffer che non c'è; `q` lo chiude. Dopo `close-buffer` si torna all'elenco prima di ridisegnarlo, perché la finestra può mostrare ormai un altro buffer.

15.5.7 `clipboard`

Ogni uccisione e ogni copia va anche agli appunti di sistema: il front-end la riceve dalla fotografia (Sezione 12.2) e la consegna, con OSC 52 o con egui. Il modulo è una variabile, `clipboard-sync`, e un comando che la inverte.

1. **Un imbuto solo.** `EditorState::kill`, da cui passano tutte le uccisioni e le copie, legge la variabile: poche righe coprono tutto. Non legata vale spenta, come nei test, che non caricano Lisp e non devono accodare sequenze per un terminale che non c'è.
2. **Gli appunti non si chiedono al terminale.** I terminali disattivano quella metà di OSC 52, che lascerebbe leggere gli appunti a chiunque scriva sul terminale, e la risposta arriverebbe mescolata ai tasti. Il testo entra con l'incolla, come `insert-pasted-text`.

15.5.8 `compile`

`C-c c` esegue un comando e ne mostra l'uscita in una striscia; `M-g n` e `M-g p` percorrono i posti che nomina. L'esecuzione è `shell-command-start` (Sezione 11.10.2); il modulo è il modo del buffer d'uscita e la navigazione.

1. **Lo stato nel buffer.** La prima riga, `$ cargo build`, è il comando che `g` riesegue. Fuori dal buffer c'è solo quale buffer `M-g n` percorre, perché lo si chiede da un altro buffer.
2. **I formati sono una lista.** I compilatori non concordano, e aggiungerne uno è una riga di `compilation-patterns`; vince il primo che corrisponde, e i più specifici vengono prima.
3. **La lista di tutti.** `next-error` percorre una lista di risultati (Sezione 9.7), la stessa di una ricerca. L'uscita di un compilatore arriva ancora mentre la si percorre: `compilation--rebuild` è il suo `results-refresh`, chiamato prima di ogni passo. Ricostruire è un'analisi per riga dell'uscita, qualche centinaio; analizzare ogni riga all'arrivo farebbe lo stesso lavoro più spesso e nel momento peggiore.
4. **`M-g n` sta in `common-keymaps`.** Percorre l'ultima lista fatta, da chiunque: legato qui, ogni produttore di liste dipenderebbe da questo modulo.
5. **I segni come sfondi.** Il testo sotto ha già i suoi colori.

15.5.9 `completion-at-point`

Il comando `completion-at-point`, che chiede alle fonti e unisce ciò che offrono, è in Rust (Sezione 14.4.1); il modulo è le cinque fonti — percorsi, parole del linguaggio, nomi dell'interprete, buffer, parole dei buffer — e il loro ordine.

1. **Una fonte di nomi prende i limiti da `bounds-of-thing-at-point`.** Così rivendica lo stesso testo delle altre, e i candidati si uniscono; solo `capf-file-name` rivendica un percorso intero.
2. **Nulla, non una lista vuota.** Una lista vuota fisserebbe la regione e chiuderebbe fuori le altre fonti.
3. **La parola che si scrive non si offre.** Sarebbe l'inizio comune di tutti i candidati, e senza la striscia il tasto sembrerebbe non fare nulla.
4. **La più specifica prima.** La prima fonte che risponde fissa la regione, e di un nome offerto due volte resta la prima descrizione; le fonti di un modo vengono prima di tutte.

15.5.10 `completion`

Tab a un prompt e `C-M-i` in un buffer mostrano i candidati in una striscia in fondo al frame, che si restringe scrivendo. L'editor chiama solo `*completion-read-function*`, che il modulo imposta; senza il modulo Tab scorre i candidati nel prompt.

1. **Il modulo non sa da dove vengono i candidati.** Riceve una lista e una funzione da chiamare con la scelta.
2. **La striscia non prende il focus.** Si continua a scrivere nel prompt o nel buffer, attraverso una keymap transitoria che lascia passare i tasti che non lega: `C-n` e `C-p`, non `n` e `p`, che si scrivono; Esc e `C-g` la chiudono, perché nient'altro toglie una keymap così.
3. **Il buffer è un'immagine.** I candidati stanno nel modulo, e la striscia ne mostra una pagina intorno alla selezione. Le celle hanno larghezza fissa, tagliate e non a capo, scritte anche oltre la fine: la posizione di un candidato si calcola, non si cerca. La selezione è la regione, che si vede anche in una finestra senza focus.
4. **Si restringe da tutta la lista.** Un backspace allarga di nuovo. In un buffer si filtra la lista iniziale; a un prompt si richiede, perché un `/` scritto in un percorso vuol dire le voci di un'altra directory. Il testo si confronta con l'ultimo, perché il restringimento gira dopo ogni comando, anche dopo il `C-n` della striscia.
5. **Nessuna corrispondenza non chiude.** Un backspace riporta la lista. Un candidato solo si prende senza striscia: una finestra per una domanda con una risposta.
6. **Si chiude prima della risposta.** Chi la riceve può aprire un'altra striscia; ogni uscita chiude finestra, stato e keymap insieme.

15.5.11 `default-configuration`

L'editor come arriva: chiede con `require` ogni modulo distribuito, nell'ordine di `default-configuration-modules`, tranne quelli nominati in `default-configuration-skip`, e imposta `dired-sort` e `mouse-mode`. È la sola riga dell'`init.lisp` predefinito (`DEFAULT_INIT_LISP`, in `editor/boot.rs`).

1. **Un modulo e non un elenco nell'`init.lisp`.** L'`init.lisp` si scrive una volta sola e poi è dell'utente: un modulo aggiunto da una versione successiva non vi arrivava. L'elenco nel modulo arriva con l'editor, e l'`init.lisp` resta una riga e ciò che l'utente vi aggiunge.
2. **Togliere per nome, prima del `require`.** `default-configuration-skip` è un `defvar`, quindi un valore impostato prima che il modulo si carichi resta. Un modulo tolto che un altro chiede si carica lo stesso.
3. **Niente macro.** In un binario di test, senza `data/lisp` accanto, ogni `require` risponde nil, e una forma scritta con una macro di un modulo sarebbe una chiamata qualunque, che fallisce: il modulo deve caricarsi anche così, come l'`init.lisp` predefinito.

15.5.12 `dired`

Un file manager scritto come modulo: un modo, una keymap e pochi comandi, su primitive che non sanno di file manager — `list-dir`, `delete-file`, `rename-file`. Cerca anche un file in tutto l'albero sotto la directory, con una lettura approssimata di `.gitignore`.

1. **Il buffer è lo stato.** Nessuna variabile tiene la directory, nessuna tabella lega le righe ai nomi: la prima riga dice la directory, ogni riga porta un nome, e ogni comando rilegge lo schermo. Una directory ricordata a parte potrebbe non essere quella mostrata, e `D` cancellerebbe in una directory che l'utente non sta guardando.
2. **Un formato semplice.** L'intestazione, poi un nome per riga rientrato di due spazi: il rientro distingue una voce dall'intestazione. Colonne di dimensioni e date obbligherebbero ogni comando che legge un nome ad accordarsi con ogni comando che lo scrive.
3. **`..` si elenca.** Un elenco che non mostra la sua uscita sembra un vicolo cieco; `list-dir` lo omette, e il modulo lo rimette.
4. **Dopo ogni modifica si rilegge il disco.** L'elenco si ricostruisce da ciò che c'è, non si corregge per accordarsi a ciò che ci si aspettava.
5. **Un buffer di elenco.** Una directory aperta in un'altra finestra sarebbe un secondo elenco nello stesso buffer: si apre qui. `o` riusa la finestra in cui ha aperto l'ultimo file.
6. **Domande con risposta scritta.** Nulla nell'editor ridà un file cancellato. La risposta chiude sul percorso, in nessuna variabile, mentre l'utente fa altro; se la cancellazione è ricorsiva si decide insieme alla domanda.
7. **Un walk primitivo che pota.** `directory-files-recursive` salta `.git` e i modelli ignorati mentre cammina: `.git` da solo esaurirebbe il limite, e filtrare dopo costerebbe una valutazione per file e per modello. Un walk per prompt, ristretto a ogni tasto.
8. **`.gitignore` approssimato di proposito.** Ciò che non capisce lo salta: può offrire un file che git nasconderebbe, mai nascondere uno che mostrerebbe.

15.5.13 `display-line-numbers`

`display-line-numbers-mode`: un modo minore (Sezione 10.2) senza etichetta, spento finché qualcuno non lo accende. Il modulo lo fa con `define-minor-mode`; il margine lo disegna la composizione di ogni finestra affiancata che mostra un buffer con il modo acceso (Sezione 12.6).

1. **Per buffer, non per finestra.** I modi minori sono dei buffer: due finestre sullo stesso buffer hanno tutte e due i numeri, o nessuna. Vim li dà alla finestra; `:set number` qui li accende nel buffer.
2. **Il tipo resta una variabile.** `display-line-numbers-type` è una preferenza di chi legge, la stessa in ogni buffer; il modo dice solo dove.
3. **Senza etichetta.** Il margine si vede.

15.5.14 `electric-pair`

Scrivere un delimitatore di apertura inserisce il suo compagno, scrivere quello di chiusura lo scavalca, cancellare metà di una coppia vuota cancella entrambi. Sta su `post-self-insert-hook`, nei modi che `electric-pair-enable` nomina; la cancellazione è un comando sui tasti di `delete-backward-char` e `delete-char`.

1. **Le coppie vengono dalla tabella sintattica.** La stessa di `forward-sexp` e dell'indentazione: nessuna seconda lista da tenere allineata.
2. **Un incolla non accoppia.** `post-self-insert-hook` non gira per un incolla, e il codice incollato ha già le sue parentesi.
3. **Lo stato prima del carattere scritto.** Al punto, descriverebbe la stringa o il commento che quel carattere ha appena aperto. In un commento nulla si accoppia.
4. **Le virgolette prima della regola delle stringhe.** Davanti a una virgoletta di chiusura si è dentro la stringa che chiude: la regola le rifiuterebbe tutte. Una virgoletta apre o chiude secondo ciò che le sta accanto.
5. **Prima di una parola non si accoppia.** Una `(` davanti a `foo` sta per avvolgerlo. Emacs lo rende configurabile; qui è la regola.

6. **Una lista già chiusa non si accoppia.** Una `{` cancellata e riscritta ha ancora la sua `}` più sotto: un'altra la raddoppierebbe. `balance-point` dice se la profondità torna a zero più avanti (Sezione 10.4).
7. **Una chiusura in più va alla fine del blocco.** Se il buffer è bilanciato senza di essa e c'è una lista da cui uscire; al livello più alto è un carattere voluto. Tolta e rimessa nello stesso comando, è un passo di undo.
8. **Rust non accoppia le virgolette.** Le lifetime lo renderebbero sbagliato più spesso che giusto.

15.5.15 `fill`

`M-q` e `C-x f`, e `auto-fill-mode`: un modo minore (Sezione 10.2), con una funzione del suo `post-self-insert-hook` che, dopo uno spazio scritto, chiama `do-auto-fill` (Sezione 9.8). Lo hook è del modo minore, e quindi gira solo dove il modo è acceso; `minor-mode-enable` lo accende nei buffer nuovi di un modo maggiore.

1. **Il primo modo minore.** È quello che i modi minori servivano a fare: acceso in un buffer e spento in un altro, con un'etichetta nella riga di stato e un hook che non deve controllare da sé se è acceso.
2. **Lo spazio, e non Invio.** Il gancio è `post-self-insert-hook`, che Invio non fa scattare: `newline` è di `indent`, e spezzare lì vorrebbe dire legare i due moduli. Una riga andata oltre la colonna si spezza al prossimo spazio.

15.5.16 `hl-line`

Fa `hl-line-mode` con `define-minor-mode`, etichetta `HL`, spento in partenza. Il modulo contiene il comando e la sua docstring; l'effetto è in `ui/compose.rs`, condiviso da finestre affiancate e flottanti (Sezione 12.4). Usa la faccia incorporata `hl-line`, con il solo fondo: non ha keymap né hook da aggiornare al movimento. Il punto e il layout del fotogramma bastano per l'intera riga logica, comprese le continuazioni.

15.5.17 `help`

`C-h f`, `C-h v`, `C-h k`, `C-h b`, `C-h a` e `C-h ?`: le pagine dell'aiuto, in `*Help*`, con una pila per tornare indietro. La pagina dei tasti raggruppa i legami per la keymap da cui vengono, i modi minori compresi, e quella del modo (`C-h M`) elenca i modi minori accesi con la prima riga del loro comando (Sezione 10.2).

1. **Le risposte sono primitive.** `key-binding`, `where-is`, `function-doc` e le altre sono in Rust (Sezione 14.4.2): nessun modulo può sbagliarle. Il modulo è le pagine.
2. **Due metà per comando.** Una chiede, ed è legata a un tasto; l'altra prende il nome, per il Lisp.
3. **Una pagina è una lista di righe.** Le funzioni che le costruiscono non toccano un buffer: si provano senza, e si sostituiscono senza toccare ciò di cui sono fatte.
4. **La pila tiene come ricostruire una pagina.** Una pagina ritrovata mostra l'editor com'è ora. Tornare indietro non spinge la pagina lasciata, o `l` scambierebbe le ultime due per sempre; la pila ha un limite, `help-history-length`.
5. **`describe-key` legge come l'editor.** Passa da `read-key-sequence`: ciò che dice è ciò che sarebbe successo.
6. **Documentata e non legata è uno stato.** Una variabile documentata prima che qualcosa la imposti non vale nil, e la pagina lo dice.

15.5.18 `manpage`

`C-h m` chiede una pagina, completando su tutte quelle installate, e `K` apre quella nominata dalla parola sotto il cursore; le pagine dell'editor vengono prima. `man` formatta quelle del sistema, con `manpage-path` come `MANPATH`. Il nome arriva alla shell tra apici, con `shell-quote-argument`, e `printf(3)` diventa la sezione e il nome, `3 printf`, come in Emacs: un nome con spazi, `;` o `*` arriva a `man` e non alla shell. Una pagina dell'editor si cerca solo per un nome di lettere, cifre, `.`, `_`, `+` e `-`, senza `..`, che non può uscire dalla directory delle pagine.

1. **La variabile decide ciò che si trova.** `manpage-path` è il `MANPATH` di `man`, non solo l'elenco del completamento.
2. **Prima `man -w`.** `man` da solo, su alcuni sistemi, scrive «No manual entry» sull'uscita, che sembra una pagina; e un sistema senza manuali può avere un `man` che si scusa ed esce con 0: conta la forma della risposta, non il codice d'uscita. `-P cat` evita un pager che aspetterebbe un tasto.

3. **L'enfasi diventa overlay.** Il grassetto e il sottolineato che `man` segna sovrascrivendo diventano overlay; le intestazioni si colorano dal layout.
4. **L'elenco dei nomi si tiene, e si costruisce in background.** Una fonte di completamento si chiede a ogni Tab, e spogliare migliaia di nomi costa secondi. Durante la costruzione, ciò che si è trovato è una risposta corta ma non sbagliata, perché si aggiungono directory intere; una lista vuota invece è sbagliata, e allora si percorre subito. Si ricostruisce quando `manpage-path` cambia.
5. **Avviato al caricamento, ma nessuno lo aspetta.** È un'ottimizzazione, non una gara. Il lavoro ha un nome: ricaricare il file lo riavvia, non lo raddoppia.

15.5.19 `occur`

`M-s o` elenca le righe del buffer che corrispondono a un modello, `M-s g` quelle dei file sotto una directory.

1. **Il modulo è solo la vista.** `occur--scan` e `grep--scan` trovano le corrispondenze e le attaccano all'elenco come lista di risultati (Sezione 9.7), che `results-visit` e `next-error` leggono; qui si decide quale finestra mostra l'elenco e che cosa fanno i suoi tasti.
2. **I tasti per percorrere sono in Rust.** Nessun produttore di liste dipende da questo modulo.
3. **La riga si conta, non si legge.** Un percorso con i due punti si legge in due modi, e la voce sa già dov'è.
4. **Nominare le facce dei segni li accende.** Il percorso le cerca per nome, e con facce che nessuno ha nominato non segna nulla.

15.5.20 `overwrite`

`overwrite-mode`: un modo minore (Sezione 10.2) con l'etichetta `Ovwrt`, sul tasto Ins della keymap globale. Il modulo fa il modo e lega il tasto; che cosa succede a un carattere scritto lo decide `self-insert`, che legge il modo nel buffer (Sezione 14.2).

1. **In `self-insert`, non su `post-self-insert-hook`.** Uno hook vedrebbe il carattere già inserito e dovrebbe togliere quello dopo: due modifiche da ricucire in un passo di undo, un conteggio (`C-u 3 x`) da ricostruire, e un ordine da decidere con `electric-pair`, che sta sullo stesso hook. Nella primitiva è una sostituzione sola, fatta prima che gli hook girino.
2. **Fino alla fine della riga.** Un carattere scritto in fondo a una riga si aggiunge: sostituire l'a capo unirebbe due righe scrivendo una lettera.

15.5.21 `preview`

Mentre il prompt di un comando di `preview-commands` è aperto, ciò che si scrive si disegna nel buffer come testo virtuale, che non cambia nulla, e se ne va quando il prompt si chiude. Oggi il comando è uno, `string-rectangle`.

1. **Il solo che conosce entrambi.** `string-rectangle` e `make-virtual-text` non sanno l'uno dell'altro; il modulo, su `post-command-hook` per ogni modo, li unisce, e `pending-command` dice a quale comando appartiene il prompt aperto.
2. **Ridisegnata intera.** A ogni cambiamento si cancella e si ridisegna: poche righe, e nulla che possa divergere; un prompt invariato costa un confronto.
3. **Un aspetto riconoscibile.** Attenuato e uguale in ogni modo: chi guarda non deve mai chiedersi se ciò che vede è nel file.

15.5.22 `recent-files`

`M-g r` apre uno dei file che `recent-files` elenca: un prompt con il completamento su `fuzzy-filter`, come `C-x b`, e `find-file` sul percorso scelto. La lista e la sua memoria tra un avvio e l'altro sono dell'editor (Sezione 3.1.3); il modulo è solo il comando e il suo tasto.

1. **Un tasto sotto `M-g`.** `C-x C-r`, dove molti mettono questo comando in Emacs, è già di `dired-find-recursive`, e `C-c` seguito da una lettera è lasciato a chi usa l'editor. `M-g` è il prefisso degli spostamenti verso un posto, `M-g g` verso una riga e `M-g n` verso un errore: `M-g r` va verso un file recente.

2. **Un file che non c'è più resta nella lista.** Controllarli tutti a ogni apertura del prompt vorrebbe dire una `stat` per file, anche su un disco di rete lento; aprirne uno sparito dà un buffer vuoto, come `C-x C-f` su un percorso che non esiste.

15.5.23 `risp-repl`

`M-x risp-repl` apre `*risp*`, un buffer in `risp-mode` dove `C-j` valuta il blocco di espressioni che finisce al punto e ne scrive il valore sotto, in una riga di commento: il `*scratch*` di Emacs con `eval-print-last-sexp`. Il modulo è tutto in Lisp, sopra `backward-sexp`, `forward-sexp`, `syntax-ppss` ed `eval-string-top-level`, la stessa valutazione di `M-:` (Sezione 15.5.5).

1. **Una mappa locale, non un modo maggiore.** I modi non ereditano: un modo `risp-repl-mode` dovrebbe ripetere la colorazione, la tabella e l'indentazione di `risp-mode`, o dividerle in una funzione da chiamare per due modi. `risp-repl-map` è un modo creato solo per la sua mappa, che `set-local-keymap` dà al buffer, il cui modo resta `risp-mode`; per la stessa via `risp-repl-mode` la dà a qualunque buffer. Il prezzo è che la mappa locale è una sola: dove Vim è acceso, le sue mappe prendono il posto di questa.
2. **Il blocco si trova per espressioni, non per righe.** Da `backward-sexp` in `backward-sexp`, ci si ferma quando fra due espressioni c'è una riga vuota o un valore scritto (`;; =>`, `;; !!`): una riga vuota dentro una `defun` non la spezza, perché `backward-sexp` la salta per intero, e un valore scritto chiude il blocco di sopra senza un segnaposto nel buffer. Scartato: un marcatore della fine dell'ultimo output, come fa ielm, che non sopravvive a un taglio o a un salvataggio su file.
3. **Una forma alla volta.** `eval-string-top-level` legge una sola espressione: il modulo divide il blocco con `forward-sexp` e le valuta in ordine, fermandosi alla prima che fallisce, così il resto non gira su uno stato a metà.
4. **Il valore va nel buffer da cui si è partiti.** Se la valutazione rende corrente un altro buffer, la riga va nell'area dell'eco: tornare a `*risp*` con `switch-to-buffer` disferebbe ciò che l'espressione voleva.

15.5.24 `shell`

`M-!` esegue un comando e mostra ciò che stampa in una striscia; `C-c !` esegue la regione. L'esecuzione è `shell-command-start` (Sezione 11.10.2); il modulo è il modo del buffer d'uscita, i suoi tasti e colori, e i comandi.

1. **Un buffer per comando.** Due comandi nello stesso buffer mescolerebbero le loro righe.
2. **Una striscia, non mezzo schermo.** L'uscita si guarda di sfuggita mentre si lavora. La striscia è una, riusata finché mostra un'uscita; il focus resta dove si è premuto `M-!`.
3. **La riga d'uscita ha una forma fissa.** Così si colora secondo il numero, e un fallimento si vede senza leggere.

15.5.25 `show-paren`

`show-paren-mode`: un modo minore (Sezione 10.2) senza etichetta, senza tasti e senza hook. Il modulo lo fa con `define-minor-mode`, lo chiede per ogni modo maggiore con `minor-mode-enable` e lo accende nei buffer già aperti. La coppia la cerca e la disegna `snapshot`, nella finestra con il focus, se il buffer ha il modo acceso (Sezione 12.4).

1. **Il modulo è solo l'interruttore.** La coppia si calcola nella fotografia, sotto i suoi lock, dove Lisp non si può chiamare: il modo minore dice soltanto in quali buffer farlo.
2. **Senza etichetta.** Acceso ovunque, nella riga di stato sarebbe una parola in più in ogni finestra. `C-h M` lo elenca lo stesso.

15.5.26 `subword`

Fa `subword-mode` con l'etichetta `Subword`, spento in partenza, e definisce `subword-separators` con `defvar`, così un valore scelto prima del caricamento resta. La stringa è globale, con un override tramite `buffer-put` nel singolo buffer. Non sostituisce i tasti: le primitive di movimento e kill interrogano il modo sotto il lock del buffer e condividono i calcoli di `primitives/subword.rs` (Sezione 14.2.1). Così anche i prefissi e le chiamate Lisp usano gli stessi confini. Il modulo non impone una regola di accensione: movimento

per sottoparole e separatori sono scelte per il buffer o per un modo maggiore tramite le regole comuni dei modi minori.

15.5.27 `theme`

`C-c t` elenca i temi installati in `*Themes*`, e Return ne applica uno; `select-theme` fa lo stesso da `init.lisp` (Sezione 10.5).

1. **Un tema si applica in un modo solo.** La lista sceglie il nome e chiama `select-theme`.
2. **Un file di tema è dati.** Si legge quando il tema si applica, direttamente e non con `eval-file`, che cerca in `lisp-path`; elencare i temi legge solo le directory di `theme-path`.
3. **Ogni faccia si azzera prima.** Un tema sparso dopo uno completo non ne lascia nulla; il prezzo è che i temi non si compongono.
4. **L'elenco si legge dallo schermo**, come quelli di dired e dei buffer.

15.5.28 `vim`

L'editing modale di Vim, in un modulo. `M-x vim-mode` lo accende in un buffer, `global-vim-mode` in ogni buffer adatto.

Uno stato — Normal, Insert, Replace, Visual — è una keymap locale (`vim-normal-map`, `vim-insert-map`, `vim-replace-map`), consultata prima della keymap del modo; la keymap locale che il buffer aveva si salva e si rende quando Vim si spegne. In Normal ogni tasto stampabile è legato a `(vim--key TASTO)`, così nessun tasto scrive testo: `vim--dispatch` interpreta il conteggio, l'operatore, il registro e il tasto in attesa, che stanno in proprietà del buffer. Un movimento restituisce `(POSIZIONE INCLUSIVA PER-RIGHE)`, e un operatore lo applica come intervallo; un registro tiene la forma del testo: caratteri, righe o blocco.

Una sessione di Insert fa guardare alle porte le proprie modifiche (`track-changes-begin` all'ingresso, `track-changes-end` all'uscita, Sezione 8.3): ciò che è cambiato, con l'azione che ha aperto la sessione, è ciò che `.` ripete. Non copia il buffer: entrare e uscire costa quanto ciò che si è scritto, in un file di due megabyte come in uno di una riga. Il cambiamento è cercato nel solo tratto toccato, così una `a` scritta davanti a un'altra `a` è dove la si è scritta, e `.` la scrive lì; i comandi della sessione si uniscono in un passo di undo (`undo-join` su `pre-command-hook`). Replace fa lo stesso. I comandi `:` leggono la riga con `minibuffer-read` e passano dai comandi dell'editor; la ricerca usa lo scanner di espressioni regolari dell'editor, in offset di caratteri.

1. **Uno stato è una keymap locale.** Sintassi, indentazione, hook del modo e prompt restano del loro modo, sotto. Una keymap locale che un hook di modo installa mentre Vim è acceso si salva, e torna quando Vim si spegne.
2. **Condiviso e per buffer.** Registri, ricerche, `.` e macro sono dell'editor intero, come in Vim; lo stato modale è di ogni buffer.
3. **Quando accendersi.** `global-vim-mode`, `vim-mode-modes`, i modi esclusi — le liste e i buffer dell'editor — e `vim-mode-predicate` decidono; una scelta fatta a mano in un buffer resta finché `vim-mode-auto` non la rende. Le regole si riapplicano quando cambia il modo maggiore o il file visitato.
4. **I comandi `:` passano dall'editor.** `:w` e `:q` sono `save-buffer` e `quit`, con le loro domande.
5. **Un file solo.** Una funzionalità, una pagina, una feature: divisa in più moduli darebbe feature che nessuno chiede da sole.

15.5.29 `whitespace`

Due modi minori (Sezione 10.2) per gli spazi a fine riga. `whitespace-mode`, senza etichetta e acceso in ogni buffer come `show-paren-mode` (Sezione 15.5.25), li fa disegnare a `trailing_highlights` (Sezione 12.4). `whitespace-cleanup-mode`, con l'etichetta `Clean` e spento finché qualcuno non lo accende, registra `whitespace--cleanup` nel suo `before-save-hook`. Quel callback chiama `delete-trailing-whitespace` con `point-min` e `point-max`: una regione attiva non restringe la pulizia. Il registro seleziona il callback solo quando il modo è acceso, e il protocollo comune lo esegue prima di ogni snapshot manuale (Sezione 8.7). Togliere l'hook basta a togliere la pulizia; nessun ramo nativo la ripete.

1. **Due modi, non uno.** Vedere gli spazi e farli togliere da un salvataggio sono due scelte: in un file d'altri si vogliono vedere senza che salvare tocchi righe che non si sono modificate.
2. **La pulizia spenta in partenza, e con un'etichetta.** Cambia il testo senza che chi salva lo chieda in quel momento: va scelta, per un buffer o con una regola per un modo maggiore, e dove è accesa la riga di stato lo dice.

15.5.30 I modi dei linguaggi

`rust-mode`, `risp-mode`, `cc-mode` (C e C++), `python-mode`, `toml-mode`, `bash-mode` e `cmake-mode`: ognuno crea il suo modo, ne dichiara i due lessici (Sezione 10.3) — la grammatica che colora e la tabella che dice che cos'è il testo —, le parole che si completano, la regola d'indentazione e le estensioni dei file. `rust-mode` è il modello che la pagina `writing-modes` indica da copiare.

1. **Due lessici, e una forma in tutti e due.** La grammatica dice come appare il testo; la tabella che cosa è, ed è ciò che `C-M-f`, l'indentazione ed electric-pair leggono. Una stringa grezza di Rust si dichiara in entrambi, o lo scanner la legge come stringhe con codice in mezzo.
2. **Senza lookahead.** Il motore non ha lookahead, lookbehind né backreference: una regola prende di più e colora un gruppo, e dove servirebbe `(?<!\\)` una regione nomina un escape. Il letterale di carattere di Rust è una regola, che comincia una colonna prima della stringa e vince; non una regione, che una lifetime aprirebbe fino alla fine del file. Le stringhe grezze di C++ con delimitatore si dichiarano una per una.
3. **Il vocabolario scritto una volta.** La lista sotto la proprietà `keywords` è ciò che `capf-mode-keywords` offre, e `regexp-opt` ne fa il modello che colora: due copie divergerebbero in silenzio, con una parola che si colora e non si completa.
4. **risp-mode chiede il vocabolario.** Funzioni e macro vengono da `all-functions` e `all-macros`, al caricamento e con `risp-refresh-vocabulary`, non a ogni visita, che compilerebbe un modello su ogni nome per ogni file aperto. Si scrivono solo le forme speciali, che non hanno un legame da riportare. I confini sono scritti, non `\b`, perché un nome può finire con `+` o `<`.
5. **cmake-mode colora un comando solo davanti alla sua parentesi.** CMake legge i nomi dei comandi senza badare alle maiuscole e gli argomenti come parole qualunque: la regola ha `(?i)` e chiede la `(`, così `IF(` è una parola chiave e `set(if 1)` no. Le costanti e gli operatori delle condizioni, che CMake scrive in maiuscolo, si colorano solo così. Gli argomenti e i commenti tra parentesi quadre, `[=[ ... ]=]`, chiudono con lo stesso numero di `=`: senza backreference ogni lunghezza è una regione a sé, e se ne dichiarano tre, da zero a due `=`.
6. **L'indentazione di CMake va per comandi.** Un comando può stendere i suoi argomenti su più righe: la regola risale dalla riga di codice precedente, fuori da stringhe e parentesi, fino alla riga dove quel comando comincia, e legge il suo nome; un blocco si apre dopo `if`, `foreach`, `while`, `function`, `macro`, `block` e i rami `elseif` ed `else`, e i comandi che li chiudono tornano indietro di un livello. Dentro le parentesi vale `prog-indent--under`, come in Python e in TOML.

15.6 Le decisioni

1. **Il modello di Emacs.** `require` e `provide`, con la lista `features`, sono ciò che chi scrive Lisp si aspetta, e risolvono i due problemi del caricamento con `eval-file`: un modulo caricato due volte, e l'ordine delle righe di `init.lisp` come unica descrizione delle dipendenze. Scartato: un elenco di dipendenze in Rust, o un'intestazione da leggere come quella di package.el, che direbbero la stessa cosa in un posto in più.
2. **Un modulo che non c'è è un nil, non un errore.** In Emacs `require` di un file che non trova segnala un errore. Qui risponde nil e lo scrive nel log, come faceva `eval-file`: l'`init.lisp` predefinito deve caricarsi in un editor senza moduli, quello dei test, e un modulo tolto non deve fermare l'avvio. Un modulo che c'è e fallisce, invece, è un errore, che ferma `init.lisp` ma non l'editor (Sezione 3.1.6).
3. **Il `provide` in fondo.** Una feature è caricata quando il suo file è finito; messo in cima, un `require` dello stesso modulo durante il caricamento lo troverebbe già caricato a metà. Il `require` in cima dice al lettore che cosa il modulo usa, prima di leggerlo; può stare anche in mezzo, dove serve.

4. **Ciò che un `require` fallito ha caricato se ne va.** Come in Emacs: un modulo fermato a metà non è in vigore, e nemmeno ciò che ha caricato per strada, che un secondo `require` caricherebbe di nuovo. I fallimenti restano registrati: `feature-status` dice `failed`, e il prossimo `require` riprova.
5. **Il descrittore di un plugin si legge, non si esegue.** Sapere che cos'è un plugin non deve far girare il suo codice: un gestore di pacchetti potrà elencare, controllare e togliere senza caricare nulla.
6. **I plugin si cercano all'avvio.** La cartella dei dati si legge una volta, prima di `init.lisp`, che può così chiedere un plugin per nome. Un plugin aggiunto mentre l'editor gira si trova al prossimo avvio.

Le variabili che l'editor legge da Lisp:

Variabile	Default	Effetto
`echo-message-timeout`	5	Secondi di permanenza di un messaggio; `nil` per sempre.
`minibuffer-width`, `minibuffer-height`	60, 3	Dimensioni del prompt.
`mode-line-format`	vedi sopra	Riga di stato.
`window-separator`	`"|"`	Carattere tra finestre; `nil` per uno spazio.
`display-line-numbers-type`	`t`	I numeri di `display-line-numbers-mode`: `'relative` per la distanza dal punto, ogni altro valore per il numero della riga (Sezione 12.6).
`display-line-numbers-width`	3	Cifre minime del margine.
`mouse-mode`	`nil` (`t` in `init.lisp`)	Il mouse all'editor invece che al terminale.
`after-resize-hook`	`nil`	Funzioni chiamate con larghezza e altezza.
`frame-width`, `frame-height`	—	Dimensioni correnti, scritte dal front-end.
`lisp-path`	vedi Sezione 15.2	Dove `require` ed `eval-file` cercano i moduli.
`theme-path`	vedi Sezione 15.2	Dove si cercano i temi.
`help-key`	`"C-h"`	Il tasto che, dopo un prefisso, chiede che cosa lo segue.
`worker-fuel`	200.000	Passi per turno di un worker.
`watch-files`, `watch-file-interval`	`t`, 3	Il watcher dei file.
`auto-save`, `auto-save-interval`, `auto-save-directory`	`t`, 30, cache/autosave	L'autosalvataggio.
`history-length`	100	Voci per storia di prompt.
`save-history`, `recent-files-length`, `save-place`	`t`, 50, `t`	Che cosa la sessione tiene da un avvio all'altro (Sezione 3.1.3).
`highlight-all-matches`	`t`	Le corrispondenze di una ricerca, disegnate tutte (Sezione 12.4).
`isearch-lazy-count`	`t`	Il conteggio `3/17` nel prompt di una ricerca (Sezione 9.3.1).
`delete-trailing-lines`	`t`	Le righe vuote in fondo al buffer, tolte con gli spazi a fine riga (Sezione 14.2.4).

Variabile	Default	Effetto
`fill-column`	70	La colonna di `M-q` e di `auto-fill-mode`; anche come proprietà di un modo e nei dati di un buffer (Sezione 9.8).
`*minibuffer-read-function*`	`default-minibuffer-prompt`	Chi apre i prompt.
`*key-capture-function*`	`nil`	Chi riceve la prossima sequenza invece di eseguirla.

16 I test

I test dell'editor stanno in `core/src/tests/`, uno o più file per funzionalità (circa cento), più `tests/perf/` per le misure; quelli del front-end in `src/tests/` e, per la finestra grafica, in `src/gui/tests.rs`. Costruiscono un editor con `create_global_env`, caricano i moduli Lisp che servono con `include_str!` e valutano Lisp o simulano tasti con `handle_key_event`. `isolate_config_for_tests` punta `RSEDIT_CONFIG_DIR` a una directory temporanea del processo, una volta sola, così i test non scrivono nella configurazione di chi li esegue. Ogni file comincia con ciò che prova e con il rimando `Design:` alla parte del manuale che lo spiega.

Alcuni file meritano di essere conosciuti prima di cambiare il codice che proteggono: `layering_tests` (i confini e la mappa di `lib.rs`), `deadlock_tests` (i lock attraverso Lisp), `primitive_shape_tests` (le forme condivise dalle primitive), `edit_protection_tests` (sola lettura), `undo_tests`, `frame_snapshot_tests`. `perf::editor_performance_suite` misura tempi reali ed è esclusa dalla suite ordinaria: si esegue da sola, come dice `PERFORMANCE.md`.

16.1 I test della documentazione

Una parte dei test controlla ciò che l'editor dice di sé contro l'editor che gira: `guide_tests` le pagine del manuale utente, `doc_faithfulness_tests` le docstring e i legami, `doc_pointer_tests` i rimandi dal codice a questo manuale e le intestazioni dei moduli Lisp, `shipped_lisp_tests` i moduli distribuiti. `shipped.rs` dice a tutti che cosa è distribuito, letto da dove è definito: i moduli che l'`init.lisp` predefinito carica, le pagine di `man/`, i nomi che l'editor aggiunge al suo Lisp. L'unica eccezione è `doc_exceptions/quoted_elsewhere.txt`: i nomi che le pagine e le docstring citano come `` `nome' `` senza che siano dell'editor, come le forme speciali del Lisp; un nome che nessuno cita più fa fallire il test.

1. **Una pagina che sbaglia è peggio di nessuna.** Chi preme un tasto che la pagina promette e riceve «undefined» smette di credere anche alle parti vere; e nulla impedisce da sé che succeda, perché un tasto spostato o un comando rinominato non fanno pensare alla prosa di un'altra directory. `guide_tests` risolve ogni riga delle sezioni KEYS come l'editor risolve i tasti, e controlla le pagine nominate da SEE ALSO, l'indice della guida, i moduli di `rsedit` e i nomi delle SYNOPSIS.
2. **La prosa non si controlla.** Un tasto citato in una frase non si verifica: per questo i tasti che una pagina insegna stanno anche nella sua sezione KEYS, dove un test li legge. È un limite vero, la parte della deriva che si prende a buon mercato.
3. **Le affermazioni che possono essere false.** `doc_faithfulness_tests` controlla che una primitiva cominci la sua documentazione con la propria chiamata, che ogni comando e ogni funzione dei moduli abbiano una docstring, che ogni nome registrato come comando abbia una funzione, che ogni tasto sia legato a qualcosa che esiste, che ogni variabile dei moduli sia documentata e che i nomi citati fra un apice inverso e un apostrofo esistano. Gli esempi e lo stile no: un esempio non può essere sbagliato come una firma che nomina un'altra funzione, e un test con un'opinione sullo stile verrebbe aggirato.
4. **Un elenco solo.** I test prendono da `shipped.rs` ciò che è distribuito, invece di tenerne elenchi propri, che si allontanano dal vero.
5. **I moduli si caricano come in `init.lisp`, ma con gli errori.** Nell'editor `require` scrive nel log un modulo che non trova, perché un modulo mancante non fermi l'avvio; il silenzio è il prezzo, e `shipped_lisp_tests` carica i moduli nello stesso ordine pretendendo gli errori.
6. **Ogni rimando porta a un'etichetta.** Un'etichetta rinominata lascia un rimando che non porta da nessuna parte, e solo un test se ne accorge: `doc_pointer_tests` legge il codice, i test e i moduli Lisp.

17 Ricette

17.1 Una primitiva o un comando in Rust

1. Scegli il modulo di `primitives/`; scrivi la docstring come `const`, con la firma nella prima riga.
2. Scrivi la funzione con `primitive!`. Leggi lo stato con le closure della facciata (`with_current_buffer_mut`, `windows`, …) e *non* chiamare Lisp con un lock preso: copia, rilascia, poi chiama.
3. Modifica il testo solo con `insert_text` e `delete_range`, e controlla il loro risultato.
4. Se scorri qualcosa di grande, addebita il carburante (`ctx.consume_fuel`).
5. Registra con `into.function` o, per un comando, con `into.command` e le specifiche degli argomenti.
6. Aggiungi i test in `core/src/tests/` e aggiorna la tabella di Sezione 14.

17.2 Un comando in Lisp

```
(defcommand count-words-region (start end) ("r")
  "Report how many words the region holds."
  (message "%d words" (length (split-string (buffer-substring start end)))))
(define-key nil "M-=" 'count-words-region)
```

`define-key` con `nil` lega nella keymap globale, con un simbolo di modo nella keymap di quel modo. Un comando legato senza argomenti passa da `call-interactively`, che gli chiede la regione.

17.3 Un modo maggiore

Il modello è `rust-mode.lisp`:

```
(make-mode 'toy-mode)
(add-syntax-region 'toy-mode "\"" "\"" 'string "\\\\.")
(add-syntax-rule 'toy-mode "#.*" 'comment)
(add-syntax-rule 'toy-mode "\\b(let|if|else)\\b" 'keyword)
(set-comment-syntax 'toy-mode '(("#")))
(put 'toy-mode 'tab-width 2)
(add-auto-mode "\\.toy$" 'toy-mode)
```

Le regioni prima delle regole; ciò che può attraversare una riga è una regione, il resto una regola. Ricorda che `make-mode` sostituisce un modo esistente.

17.4 Un lavoro di sfondo

In Lisp: `background-call` con una fiber che fa `(yield)` tra un pezzo e l'altro e una callback `ON-DONE` per il risultato, oppure `define-worker` con una fiber che aspetta il suo argomento, `(yield '(wait TOPIC))`. La pagina `background` ha gli esempi, e le regole che un lavoro segue sono in Sezione 11.10.3.

In Rust:

1. Scrivi un tipo che implementa `Job<EditorState<B>>`. Ogni `step` fa una quantità limitata di lavoro: copia sotto lock, lavora senza, ricontrolla la versione prima di salvare. Dichiara le unità con `cx.worked`, e risponde `Continue` se resta altro, `Wait` o `Sleep` se no. Il modello è `Highlighter` (Sezione 11.10).
2. Dagli una `Spec`: un nome unico, la classe (`High` solo se l'utente aspetta di vederne il risultato), l'unità, gli argomenti che lo svegliano, e `Start::After` se il primo passo non serve subito.
3. Sottomettilo con `work().submit(spec, Box::new(lavoro))` dove lo stato esiste: in `EditorState::new`, o in `create_global_env` se legge impostazioni del Lisp, che allora rilegge a ogni passo.

4. Ciò che può bloccarsi non è un lavoro: va su un thread dedicato, con `spawn_work`, o con `spawn_writer` se scrive in un buffer, e con `Stop::on_cancel` per l'azione che lo sblocca.
5. Aggiungi la riga alla tabella di Sezione 11.10, e i test in modalità manuale (Sezione 11.11).

17.5 Un modulo Lisp nuovo

Scrivi `core/lisp/<genere>/nome.lisp` come dice Sezione 15.4: l'intestazione con la pagina e il rimando a una sottosezione nuova di Sezione 15.5, i `require` di ciò che usa in cima e `(provide 'nome)` in fondo; aggiungi il suo nome a `default-configuration-modules`, in `core/lisp/features/default-configuration.lisp`, la riga del modulo nella sezione MODULES della pagina `rsedit`, nello stesso ordine, e un test che lo carichi. `build.rs` lo copia da solo.

17.6 Un nuovo front-end

Un front-end deve: tradurre gli eventi nei tipi di `rsedit_core`; chiamare `handle_key_event`, `handle_paste`, `handle_mouse_event` e `resize`; comunicare `frame-width` e `frame-height`; in un ciclo, chiedere `snapshot` e disegnarlo senza tenere nulla dell'editor; aspettare al massimo `next_redraw_in` (e `drag_scroll_in`) e chiamare `tick` quando l'attesa scade; farsi svegliare da `on_background_change` quando un lavoro di sfondo cambia qualcosa; mettere `frame.clipboard` negli appunti. Tutto ciò che c'è da sapere su colori, bordi e cursore è nella fotografia; `src/tui/` è l'esempio completo (Sezione 3.2).

18 Appendice

18.1 Glossario

Buffer testo con punto, mark, storia e modo; può visitare un file.

Punto la posizione del cursore in un buffer, in caratteri.

Mark, regione una seconda posizione; la regione è tra mark attivo e punto.

Comando una funzione registrata con le sue specifiche di argomenti.

Keymap transitoria una keymap consultata prima di tutte, finché non viene tolta.

Compartimento un gruppo di campi dello stato sotto un solo lock, in `managers/`.

Facciata `EditorState`, l'unico che tiene più lock di compartimento insieme.

Fotografia il `FrameSnapshot`, ciò che il front-end disegna.

Lavoro ciò che il WorkManager esegue a passi; implementa `Job`. Nel codice è un *job*; *work* è un lavoro o un thread dedicato, e *worker* resta solo nei nomi del Lisp (`define-worker`, `stop-worker`, `running-workers`, `worker-fuel`).

Passo una chiamata a `Job::step`, cioè una quantità limitata di lavoro.

Argomento un evento con un nome e senza dati, cioè un contatore che cresce a ogni emissione.

Esecutore il thread del WorkManager che esegue i passi, uno alla volta.

Thread dedicato un thread del WorkManager per un lavoro che può bloccarsi, fermato con il suo `Stop`.

Watchdog il thread del WorkManager che osserva il passo in corso e scrive il file di log.

Callback dovuta una chiamata chiesta da un lavoro di sfondo ed eseguita dal thread dei comandi.

Epoca il timbro del comando corrente, con cui l'undo raggruppa le modifiche.

Porta uno dei due punti da cui passa ogni modifica di un buffer, che la registra nell'undo e sposta mark, overlay e testo virtuale (Sezione 8.3).

Keymap locale la keymap di un buffer, consultata prima di quella del suo modo e di quella globale; vim vi mette i suoi stati.

Finestra flottante una finestra disegnata sopra quelle affiancate, in un rettangolo suo: un prompt, il `*Backtrace*`.

Striscia una finestra bassa in fondo al frame che mostra un buffer di servizio, l'uscita di un comando o i candidati di un completamento.

Lessico uno dei due modi in cui un modo legge il testo: la grammatica, che colora, e la tabella sintattica, che dice che cosa il testo è (Sezione 10.3).

Modulo un file Lisp di `core/lisp/` che fornisce una feature con `provide` e si carica con `require`.

Feature il nome con cui un modulo si fornisce e si chiede; `features` elenca quelle caricate.

Plugin una cartella di moduli scritti da altri, nella directory dei dati, con un descrittore letto come dati (Sezione 15.3).

Front-end il programma che legge gli eventi e disegna la fotografia: la TUI o la finestra grafica.

18.2 Mantenere aggiornato questo manuale

Le regole sono in `documentazione.typ`: dove sta ogni informazione, come si scrivono il rustdoc, i commenti e questo manuale, che cosa controllano i test. Quando il codice cambia:

- Nuovo campo di `EditorState` o nuovo compartimento → Sezione 4 (tabelle e ordine dei lock).
- Cambiamento del percorso di un tasto → Sezione 5; di `call-interactively` → Sezione 6.
- Nuova primitiva o nuovo modulo di `primitives/` → Sezione 14, la tabella dei moduli e il totale.
- Nuovo lavoro dell'editor → la tabella di Sezione 11.10; nuova opzione o nuova segnalazione → Sezione 11.9.
- Nuovo campo della fotografia o cambiamento di un front-end → Sezione 12, Sezione 3.2 o Sezione 12.9.
- Nuovo modulo Lisp → una sottosezione di Sezione 15.5, con l'etichetta a cui rimanda la sua intestazione; nuova variabile letta da Rust → la tabella di Sezione 15.

- Nuovo file sorgente → la sua documentazione rimanda alla sezione che lo spiega; nuova sezione → un'etichetta, che poi non cambia.
- Difetto corretto → aggiorna la descrizione e togli il rimando al rapporto; la scheda passa tra le risolte.

Gli estratti di codice vanno riletti quando cambia la funzione da cui vengono. I diagrammi usano gli aiuti di `style.typ`. `doc_pointer_tests` controlla che ogni rimando dal codice porti a un'etichetta del manuale, che ogni rimando al rapporto nomini una scheda aperta e che ogni estratto venga da un file che esiste. Dopo ogni modifica si compila il PDF, si guardano le pagine toccate e lo si mette nello stesso commit del sorgente.